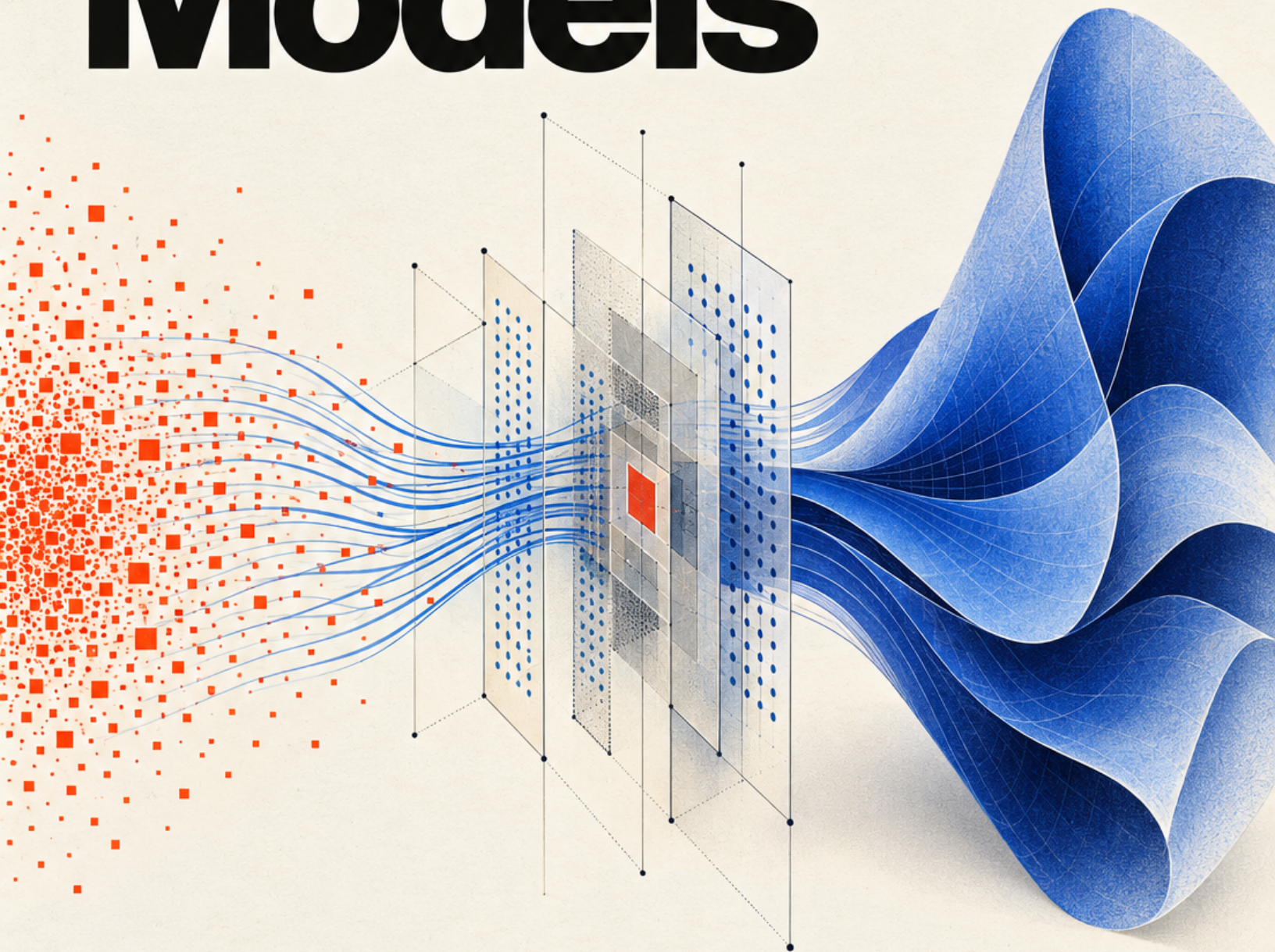


Training Your Own Models



Vibe Authored by Dr.Puma

Training Your Own Models on One 24 GB GPU

A project first beginner to practitioner handbook

Language models · classifiers · embeddings
images · audio · structured decisions

Practical full training examples at one billion parameters and below, a careful three billion parameter planning boundary, and a concluding study of larger architectures

Edition dated 2 October 2026

Includes worked projects, a linked glossary, primary sources, CPU and GPU inference paths, and a companion code collection

Contents

Contents	2
Start here	4
1 Train your first model before learning the whole field	7
2 Your computer and the training workspace	12
3 Numbers tensors parameters and learning	16
4 Project turn four sensor readings into a useful prediction	20
5 Build a dataset that teaches the job	27
6 Decide whether the model actually improved	31
7 Move from feature columns to neural representations	35
8 Project classify your own small image collection	37
9 Project locate one object with a box	42
10 Project predict a mask then extract an outline	46
11 Train a miniature language model from random weights	52
12 How a language model becomes a prediction machine	58
13 Read and control the training loop	63
14 Make a run reproducible and recoverable	68
15 The real memory and compute budget	71
16 Project make related text easy to retrieve	76
17 Choose a model by its actual structure and files	86
18 Teach a small language model one useful behavior	90
19 Repeat the project with every weight trainable	97
20 Teach domain conventions without turning weights into a database	101
21 Make a writing style reproducible without changing the facts	104
22 Teach a tool decision and a complete tool interaction	106

23 Study a small typed decision service	110
24 From a language model to a decision model	116
25 Add preferences and distillation only after supervised learning works	132
26 Project teach an image generator a small visual style	134
27 Speech recognition train a small model to hear your domain	147
28 Project build a tiny sound generator from random weights	159
29 Scale a working experiment toward one billion parameters	170
30 Understand what a public training process really demonstrates	174
31 Diagnose failures before making the model larger	176
32 From small model to small device product	179
33 Turn a checkpoint into a usable application	181
34 Design your first independent model	184
35 Understand larger models using the parts you already know	186
36 Read MiMo GLM and other architecture families	193
37 Connect architecture to data memory and deployment	198
38 Learn from public training projects without copying their assumptions	205
39 Exercise checkpoints and worked answers	211
40 Companion guide and verification record	215
Glossary with links to explanations	218
Works cited	229

Start here

A project first handbook for practical local training and architecture

Edition dated 2 October 2026

What this book will help you do

You can build useful machine learning systems on one consumer NVIDIA RTX GPU with 24 GB of video memory. You can train small models from random initial weights, adapt pretrained language and diffusion models, learn embeddings for search, and build narrowly focused classifiers and segmenters. The useful first question is what you want the system to do. The model and training method follow from that decision.

This book starts before the usual tutorial starts. It explains what a parameter is, what a tensor contains, how examples become a loss, why a gradient changes a model, how to choose training material, and what would count as evidence that training worked. You do not need prior knowledge of neural architecture. You will need to become comfortable editing a text file, running a command, and inspecting output. The early chapters teach those habits in small steps.

The practical language-model full-training examples remain at one billion parameters or below, usually far below. Language models from one to three billion parameters are an advanced planning envelope, not a promise that ordinary full-parameter training will fit or that pretraining them is a sensible first project. Image and audio pipelines may contain a trainable backbone plus separate frozen encoders or decoders; their complete component and memory inventory is stated explicitly rather than hidden behind a backbone size label. A small randomly initialized model and a pretrained model of the same size begin with very different capabilities. Fitting either model in memory does not make their training budgets equivalent.

The recurring goal is to turn a specified input into a desired output. That includes generating language, returning a structured tool call, mapping a question and documents to relevant passages, turning noise and a condition into an image or audio clip, assigning a class to a record, or labeling pixels. The book also explains when the proposed mapping cannot be learned reliably: the input may omit necessary information, the target may be inconsistent, the examples may not represent future cases, or the desired answer may change after training.

The companion contains complete files rather than expecting you to reconstruct a project from fragments printed across pages. Read the explanation first, inspect the corresponding file, then run the smallest check. Code in a PDF is useful for understanding; the companion files are the authoritative copy for execution.

The most important distinction

Three questions must be answered separately:

- Can the model and the selected training procedure fit in available memory?

- Can the computer process enough examples within your time, storage, and power budget?
- Can those examples teach the behavior you will actually evaluate?

A positive answer to one does not answer the others. Loading three billion four-bit weights says little about the memory required to update three billion parameters. A completed training job says little about whether the dataset contained the right signal. A lower training loss says little about whether a model works on genuinely new examples.

The most productive beginner progresses through experiments that separate these questions. First make a tiny pipeline work. Then establish a trustworthy comparison. Only then increase model capacity, data, context length, image resolution, or duration.

How to read the book

Start with the three-parameter classifier and run it before trying to understand every term. The projects then move through sensor prediction, image classification, masks, a tiny language model, embeddings, pretrained language adaptation, decision models, image generation, and sound generation. Theory appears when a project needs it, and later projects revisit it in greater depth. The concluding architecture studies explain larger families without presenting them as one-card training recipes.

For image or audio work, the common foundations still matter. The diffusion chapters introduce the different representation and prediction objective; a next-token training recipe does not become a diffusion recipe by changing the input file extension.

For an arbitrary input-to-output problem, start with the task specification and focused machine-learning chapters. A classifier, a regression model, a retrieval system, or a deterministic parser may solve the job more directly than an LLM. The fact that a language model can emit your desired output format does not establish that it is the right model.

The practical projects are followed by an advanced study of larger architecture families. Those systems exceed the one-card training scope; their purpose here is to explain design choices you can now understand.

For a reader who already runs local inference, pay particular attention to training-state memory, loss masking, leakage, optimizer steps, and export equivalence. Those are common places where intuition from inference becomes misleading.

Evidence and execution labels

This edition distinguishes four kinds of material. A documented capability is supported by an official model card, implementation, paper, or library reference. A worked calculation is arithmetic under stated assumptions. A starting configuration is a proposed experiment, not a benchmark. A measured result is explicitly tied to the environment where it was observed.

Dependency-free companion examples are executable without a GPU. The validation appendix records which checks were actually run. GPU recipes were not trained on the reader's RTX card while preparing this book, and no wall-clock or peak-memory estimate should be read as a measurement from that card. Exact GPU model, display use, driver, package build, attention implementation, sequence lengths, and optimizer behavior can change the outcome.

Software and model repositories evolve. The reference recipes use named versions or describe how to capture an immutable revision. A floating main branch and a mutable model name are not sufficient to reproduce a result. Use the environment ledger, archive the actual resolved revision, and keep a separate environment for each project family.

Your first success criterion

Your first success is not a model that seems impressive. It is a training run you can explain: what it was given, what it was asked to predict, which values were changed, what the loss meant, how the comparison was made, and where the resulting files are. When you can answer those questions, you have a foundation on which better models can be built.

1 Train your first model before learning the whole field

What small supervised models do

A supervised model learns a mapping from examples of inputs and desired outputs. Use it when the relationship is learnable from available evidence but too variable to specify reliably as a short deterministic program. A compact tabular predictor can score records; an image classifier can route photographs for review; a detector can supply object locations to a tracking system; a segmenter can provide masks for measurement or compositing. The application still owns validation, thresholds, coordinate transforms, human review, and failure handling.

A typical application is input validation → the same preprocessing used in training → model → output decoding → business action. Keep that chain explicit. Choosing a larger model does not repair an ambiguous target or unavailable information. We will discover the required concepts through progressively richer inputs and outputs, beginning with a few numbers.

The smallest complete experiment

Your first project will take two numbers and predict either zero or one. It uses three learned values, runs on a CPU, needs only Python, and writes a model you can load again. This may seem far from a language model, but it contains the same essential pattern: examples, inputs, targets, a parameterized computation, a loss, an update, evaluation, and an artifact.

Do not turn on the GPU for this exercise. A larger machine would hide the simplicity without adding anything useful. The goal is to see exactly where learning happens. Later projects preserve this structure while replacing the representation and computation.

The task is deliberately synthetic. Each input has two features, x_1 and x_2 , ranging from -1 to 1. A hidden labeling rule assigns one class on one side of a line and the other class on the other side. Because we generated the labels ourselves, this project has no data licensing or annotation ambiguity. It is a learning exercise, not evidence that a model is ready for a real-world application.

Run it

Install a supported Python 3 interpreter if you do not already have one. Open a terminal in the unpacked companion directory. The command below prints your Python version. Some computers use `python3` instead of `python`; use the name that refers to your intended interpreter consistently.

```
python --version
python examples/first-model/train.py --output runs/first
python examples/first-model/predict.py runs/first/model.json 0.8 -0.4
```

No package installation or model download is required. The training command refuses to write into a nonempty output directory, so a new experiment cannot silently overwrite the old one. If `runs/first` already contains results, choose `runs/first_02` or another new name.

The output includes a validation loss before training, a validation loss after training, an accuracy, and the location of model.json. In the CPU check performed for this edition, the fixed synthetic example reduced validation loss from approximately 0.693 to 0.134 and reached 0.96 validation accuracy. A separate fixed test set reached 0.985 accuracy. These are measured educational-example results, not performance claims about real data.

This standard-library implementation has no GPU backend. Its CPU path is the intended deployment and is fast enough for this three-parameter model. A GPU version would require replacing the list arithmetic with an appropriate tensor library, which the later neural projects introduce.

The prediction command should return a probability close to one for the input 0.8, -0.4 and class 1. Open model.json with a text editor. You will see two weights, a bias, a threshold, and metadata about the run. That small file is the trained artifact. It does not contain the entire training dataset.

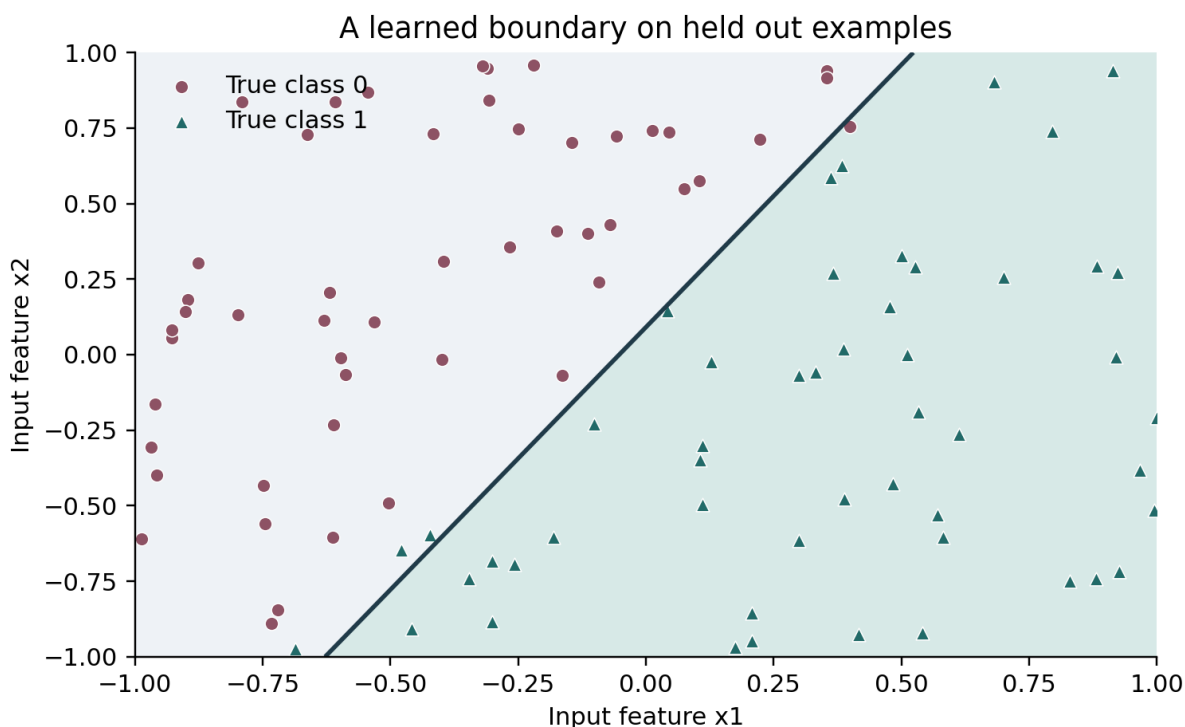


Figure 1 The first model learns a straight line boundary. Colors and marker shapes indicate true labels on the held out validation points. The figure is drawn from the actual dependency free run.

What went in and what came out

An example is one pair of input values and one correct label. The training set contains 200 examples. The validation set contains 100 separately generated examples. A test set contains 200 more. Different fixed random seeds create reproducible but separate samples from the same synthetic rule.

The model calculates a score:

$$\text{score} = \text{weight1} * x1 + \text{weight2} * x2 + \text{bias}$$

It then applies a sigmoid function to map that score to a number between zero and one. A score of zero gives 0.5. A large positive score gives a value near one. A large negative score gives a value near zero. The classifier returns class 1 when the value is at least 0.5.

The three adjustable values are parameters. At the beginning they are all zero, so every input gets probability 0.5. The initial model has no useful boundary. Training adjusts the values so inputs labeled 1 tend to get higher scores and inputs labeled 0 tend to get lower scores.

In this project the model can represent a straight-line boundary. It cannot represent every possible pattern in two dimensions. For example, if the correct class were 1 inside a ring and 0 elsewhere, a single linear score would be insufficient. Changing the training duration would not fix that architecture mismatch. You would need a different representation or a more expressive model.

The loss says how wrong the probabilities are

Accuracy is the fraction of correct class predictions. It is easy to understand, but it changes abruptly when a prediction crosses the 0.5 threshold. Moving a correct prediction from probability 0.51 to 0.95 leaves accuracy unchanged even though the model has become more confident in the right direction.

The training loss is binary cross-entropy. It penalizes giving low probability to the correct label. At the initial probability of 0.5, the loss is about 0.693 per example. The script uses a numerically stable formula so that large scores do not cause unnecessary overflow.

You do not need to memorize the loss formula before running the project. Understand its contract: lower loss means the model assigns more probability to the labels in this particular dataset. It does not mean the labels are correct in the outside world. If you deliberately reverse the labels, the model will learn the reversed task.

The update is a small correction

For each example, the script calculates $\text{error} = \text{predicted probability} - \text{label}$. If the target is 1 and probability is too small, this error is negative. If the target is 0 and probability is too large, it is positive. Multiplying this error by an input feature tells the script how to adjust the corresponding weight locally.

The script averages these contributions across all training examples, then subtracts learning rate times the result from each parameter. It repeats this update 400 times. The learning rate is 0.5 in the default configuration. These numbers are suitable for this tiny normalized synthetic problem; do not transplant them into an LLM recipe.

This is full-batch gradient descent because every update uses the entire training set. Later projects use smaller batches because the complete dataset cannot fit in one computation. The general pattern is unchanged: compute predictions, measure a loss, calculate a direction of improvement, update parameters.

Read the small part that performs learning

The complete script is in `examples/first-model/train.py`. The essential update can be understood from this simplified excerpt:

```
error = predicted_probability - correct_label
weight_gradient += error * input_feature
weight -= learning_rate * weight_gradient / number_of_examples
```

The script has two features and therefore two weight gradients. It also updates the bias. The averaging step matters: without it, changing dataset size would change the scale of the update. Real training code has more complex tensor operations, but a major part of debugging still consists of asking whether these quantities mean what you think they mean.

The script saves the model, reloads it, and checks that the reloaded values produce the same validation metrics. This is a small example of an export-equivalence test. A model that works only inside the original training process is not yet a usable deliverable.

Every control in the first command

`--steps` is the number of parameter updates. Larger values allow more optimization, but do not create more independent examples. `--learning-rate` controls update size. `--output` selects a new directory for `model.json` and `metrics.json`. The data sizes and random seeds are fixed inside the educational script so the initial comparison is repeatable.

The probability threshold of 0.5 is an inference decision rule. It is not a training parameter in this example. For real tasks, the threshold may depend on the relative consequences of false positives and false negatives. Select it using validation data and the intended operating requirement, then evaluate the resulting fixed rule on a separate test set.

The seed determines pseudo-random number generation. A seed makes this controlled example reproducible; it does not establish that the examples represent the real world. The train, validation, and test seeds are separate to avoid reusing exactly the same samples.

Change one thing and predict what should happen

Run 50 steps in a new directory. The loss should improve less than after 400 steps. Try a smaller learning rate. It should usually learn more slowly at a fixed step count. Try swapping the feature values supplied to the prediction command. The output can change because feature order is part of the model contract.

Then change the synthetic labeling rule to an XOR-style rule: class 1 when x_1 and x_2 have different signs, otherwise class 0. Keep the linear model. You should no longer expect a straight-line boundary to solve the task. This is a controlled way to learn the difference between insufficient optimization and an unsuitable hypothesis class.

Do not repeatedly inspect the test set while choosing changes. For these educational variations, use validation results and reserve a newly generated final test set for the final chosen procedure. A test set becomes another development set when it guides repeated decisions.

The first project checklist

You have completed the project when you can explain the input shape, the label meaning, the three learned parameters, the loss, the learning rate, the difference between training and evaluation data, and the saved artifact. You should be able to load the model in a new process and predict one new input.

If you cannot explain one of those pieces yet, revisit it before adding architecture. The next projects are larger versions of this loop, not an unrelated kind of magic.

2 Your computer and the training workspace

What the parts of the computer do

The CPU runs the operating system, Python, many preprocessing steps, and some parts of your data loader. System RAM holds ordinary program memory. Your SSD stores source data, processed data, model downloads, checkpoints, and outputs. The GPU performs large batches of numeric operations. Its VRAM is a separate pool of memory with different constraints from system RAM.

A computer with 64 GB of RAM and a 24 GB GPU does not have an 88 GB GPU. A program may explicitly move some state between CPU RAM and VRAM, a technique called offloading. Transfer takes time and is limited by the connection between the devices. Offloading may make a run possible while making it much slower. It must be configured by a library or your own code; it is not a transparent extension of VRAM.

The GPU may also drive your monitor, browser, and other applications. A card sold as 24 GB does not necessarily offer a training process every byte of that nominal capacity. Drivers and kernels consume memory. Other processes consume memory. Allocators reserve blocks. Transient operations can require a higher peak than the steady state visible between steps. Leave a measured margin rather than planning a job to use precisely the advertised capacity.

A model download is not the complete disk budget. You may need the original checkpoint, a local cache, processed shards, several training checkpoints, a merged model, a quantized export, generated evaluation outputs, and temporary conversion files at the same time. A small adapter can coexist with a much larger frozen base model. Plan storage by enumerating artifacts, not by looking at the adapter file alone.

A terminal is a way to give explicit instructions

A terminal runs a shell, which interprets commands. A command consists of a program name and arguments. In the command below, `python` is the program, `train.py` is a file to run, `--steps` names an option, and `20` supplies its value.

```
python train.py --steps 20
```

The command acts relative to a current working directory. If the script expects `data/train.jsonl`, it means the data directory beneath that working directory unless the code says otherwise. Use `pwd` on Linux or macOS to print the current directory and `ls` to list files. In PowerShell, `Get-Location` and `Get-ChildItem` serve similar purposes. Paths with spaces must be quoted. An absolute path begins at the filesystem root or a drive; a relative path begins at the current directory.

The examples in this book use Bash-style commands and forward-slash paths. Linux is a straightforward reference platform for the common NVIDIA training stack. Windows users should choose a supported native stack or a documented WSL2 configuration rather than mixing Linux commands into an unrelated Windows environment. Exact support depends on the

packages and GPU; consult the official installation instructions for the selected release. Do not install a random wheel merely because a forum comment says it fixes an error.

A command may keep running for minutes or hours. Output scrolling past is a log, not proof of progress. Check the process, step counter, throughput, device utilization, and written checkpoints. Ctrl+C requests interruption; whether a useful checkpoint remains depends on the program. Learn recovery on a tiny run before relying on it for an overnight job.

Python files and environments

Python reads instructions from a .py file. Indentation is meaningful. Copying code from a formatted page can introduce broken quotation marks or indentation, which is one reason the companion files matter. A comment begins with # and is not executed. A string is quoted text. An integer is a whole number. A float stores an approximate real number. A list holds an ordered sequence. A dictionary maps keys to values.

```
learning_rate = 0.001
class_names = ["refund", "shipping", "other"]
record = {"text": "Where is my package?", "label": "shipping"}
print(record["text"])
```

Here `learning_rate` is a variable. The square brackets after `record` retrieve the value stored under the key `text`. A function packages a repeatable computation; calling it runs that computation. A class can combine data and behavior, as a neural-network class combines trainable parameters and a forward method.

Libraries such as PyTorch provide large collections of existing functions. A Python environment determines which versions of those libraries are available. Installing a package into one environment and launching Python from another is a common source of errors. Prefer `python -m pip` to a bare `pip` command, because it identifies which interpreter should manage the packages.

A virtual environment is an isolated package directory. A typical project starts like this:

```
python -m venv .venv
source .venv/bin/activate
python --version
python -m pip --version
```

On PowerShell the activation command differs. Activation is convenience, not magic: you can invoke the environment's Python by its full path. Record the Python version. Then follow the project-specific dependency file and official PyTorch wheel selection. CUDA support in PyTorch must be compatible with the installed driver and GPU. A CUDA toolkit installed somewhere on the machine is not proof that the Python process has a CUDA-enabled PyTorch build.

Do not combine all projects in this book into one enormous environment. The language, diffusion, audio, and classical-ML projects may need different versions. Create one environment per project, save its package list after the smoke test, and change one dependency at a time when diagnosing a failure.

Check the environment before the model

On an NVIDIA machine, `nvidia-smi` reports the GPU, driver, running processes, and memory use. It is a diagnostic tool, not a training benchmark. Once PyTorch is installed, run this small check in the same environment that will train:

```
import torch
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("BF16 supported:", torch.cuda.is_bf16_supported())
    print("VRAM bytes:", torch.cuda.get_device_properties(0).total_memory)
```

CUDA is NVIDIA's GPU computing platform. FP32, FP16, and BF16 are numeric formats described in the memory chapter. Do not infer BF16 support from the phrase RTX alone. Check the actual device and framework support. A recipe that silently falls back to CPU can appear to work while taking far longer than expected; the training log should print the selected device.

A GPU test that only allocates a tensor verifies less than a training smoke test. A useful smoke test performs a forward pass, computes a finite loss, runs backward, updates parameters, saves a checkpoint, loads it, and performs inference. It should finish quickly and leave behind evidence you can inspect.

A workspace that you can reason about

Use a structure that makes raw material and derived artifacts easy to distinguish:

```
project/
  README.md
  requirements.txt
  configs/
  data/
    raw/
    processed/
    splits/
  src/
  runs/
    experiment_001/
  evaluation/
  exports/
```

Keep raw data unchanged. Write cleaned or tokenized versions elsewhere. Give each run its own directory. Never point a new experiment at an old output directory unless the code explicitly supports resuming and you intend to resume. A useful run directory contains the resolved configuration, data hashes, model revision, log, evaluation outputs, and checkpoint metadata.

Use version control for code, small configurations, and dataset-generation rules. Large model files and private datasets usually belong in a separate artifact store rather than ordinary Git history. Do not commit secrets. A token used to download a gated model is a credential, not part of the experiment. If logs accidentally contain a secret, treat it as exposed and follow the service's revocation process.

Exercises

Create a new directory and run a Python file that prints a sentence. Make a deliberate typo, read the error, then correct it. Locate the file from the terminal rather than relying only on an editor's open tab.

Write a JSON record containing an input, a desired output, and a source identifier. Reopen it with Python's json module. Print the desired output. Then explain the difference between the file's contents, the Python object loaded from it, and a tensor that a model would consume.

Before any GPU training, write down the exact GPU model, available VRAM, system RAM, free disk space, driver version, Python version, and PyTorch version. If one entry is unknown, the preparation is not finished.

3 Numbers tensors parameters and learning

A model is a computation with adjustable values

Suppose you want to estimate the time needed to pack an order from the number of items. A simple model might be predicted minutes = weight times item count plus bias. The weight tells the model how much time another item adds. The bias accounts for setup work that happens even for a small order. These are parameters: values learned from examples.

If weight is 0.7 and bias is 2, an order with ten items receives a prediction of nine minutes. The input is ten; the output is nine. The architecture is the form of the computation, multiplication followed by addition. Training changes 0.7 and 2. It does not automatically change the formula into a different architecture.

A hyperparameter is a choice that controls the experiment rather than a value directly fitted by the ordinary training update. The learning rate, number of layers, batch size, and maximum sequence length are hyperparameters. A trained parameter can influence many outputs; a hyperparameter can influence the way all trained parameters develop.

The distinction is practical. If you change the architecture, an old checkpoint may no longer have the right shapes. If you change the learning rate, the checkpoint can often still be loaded, but the future optimization trajectory changes. If you change preprocessing, even identical weights may receive different inputs and produce different behavior.

Features are the information you make available

The item count is a feature, a numeric representation of something about the input. Other possible features are fragile-item count, total weight, or whether gift wrapping is required. A model cannot reliably infer a factor that neither the input nor correlated information reveals. If packing time depends heavily on employee experience and that information is absent, some uncertainty is unavoidable.

A feature is not automatically legitimate merely because it improves a score. Suppose your dataset includes actual dispatch time when you are trying to predict dispatch delay before packing begins. That field would not exist at prediction time. Training on it creates a misleading shortcut. A useful feature must be available at the time and place where the system will operate.

Text and images begin as familiar human objects, but a training program needs numeric arrays. A text tokenizer converts bytes or pieces of text into integer identifiers. An image decoder converts an image file into pixel values. An audio decoder produces sampled amplitudes. A categorical feature can become an integer index or a set of indicator values. Representation is the bridge between the real input and the model's computation.

A loss turns a desired output into a training signal

For a numeric prediction, squared error is a simple loss: subtract the correct answer from the prediction and square the difference. A prediction of nine when the target is eleven has squared error four. The sign is removed, and larger errors receive disproportionately more penalty. Averaging over examples produces mean squared error, or MSE.

For classification, the model commonly produces one score per class. These raw scores are called logits. Softmax converts logits into positive values that sum to one. Those values can be interpreted as the model's distribution over classes, although they are not automatically well calibrated probabilities. Cross-entropy loss rewards the model for assigning probability to the correct class.

If the target class receives probability 0.8, the negative natural logarithm is about 0.223. If it receives probability 0.1, the loss is about 2.303. The second prediction is penalized more. When you see a language-model training loss, you are often seeing an average version of this calculation over many next-token predictions.

A target must match the task. If your actual goal is to extract an invoice date exactly, a fluent paragraph about invoices is not a substitute. If your goal is to rank relevant passages, predicting the exact wording of an answer may be an indirect and inefficient objective. You choose the output representation and loss together.

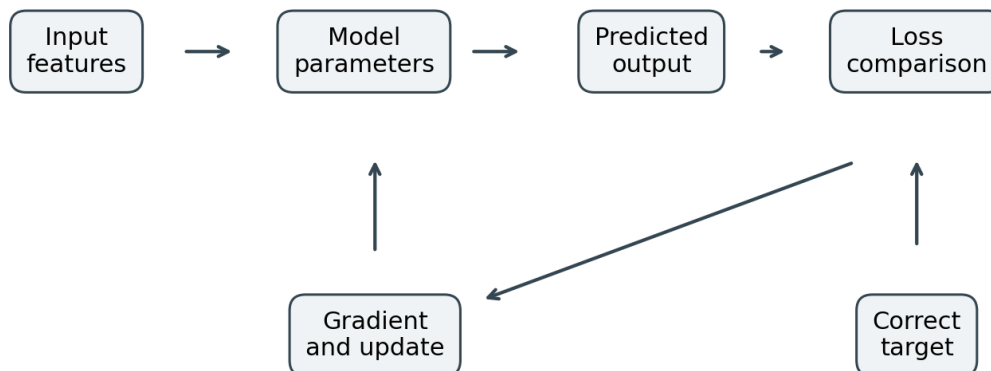
The gradient tells you how a small change affects loss

Return to predicted minutes = weight times item count plus bias. Suppose the model predicts too little for a ten-item order. Changing the weight upward would increase the prediction and, for a sufficiently small change, reduce that example's squared error. The gradient records the local rate at which the loss changes as a parameter changes.

Gradient descent updates a parameter by subtracting learning rate times gradient. The learning rate controls the step size. It is not a percentage accuracy target and not the amount of data the model learns per step. A large step can jump past a useful region and make loss unstable. A very small step can make progress unnecessarily slow.

Backpropagation computes gradients through a chain of operations. Automatic differentiation libraries record the operations needed to calculate those derivatives. You do not need to hand-differentiate an entire transformer. You do need to know that the loss must be connected to the parameters you intend to train, and that the gradient must remain intact until the update. Detaching a tensor or entering an inference-only context at the wrong place can break learning. PyTorch's autograd reference describes this computation-graph mechanism. [\[F01\]](#)

A gradient is a local signal from the selected objective and examples. It is not a judgment about truth, usefulness, or ethics. If your targets are contradictory, the optimizer will seek a compromise. If they reward an undesirable shortcut, the optimizer may learn that shortcut efficiently.



Inference stops at Predicted output. Loss, targets and parameter updates are training only.

Figure 2 Training compares predictions with targets and updates parameters. Inference uses the fitted model without the loss and update path.

What one training step actually means

A basic training step has five stages. First, obtain a batch of examples and transform them into tensors. Second, run the model forward to produce predictions. Third, compare those predictions to targets with the loss. Fourth, run backward to calculate gradients. Fifth, let the optimizer update trainable parameters, then clear gradients before starting a new independent update.

The forward pass creates intermediate values called activations. Backward may need those activations. This is why training uses memory beyond the weights. The optimizer may also maintain history, such as running estimates of gradient direction and magnitude. Inference ordinarily does not need these training-specific states.

An optimizer is the update rule. Stochastic gradient descent is a simple example. Adam and AdamW adapt updates using running statistics. AdamW applies a decoupled weight-decay term rather than mixing that term into its momentum estimates. The optimizer's defaults are library decisions, not universal best settings. [F02]

Learning does not mean storing a clean database

Parameters encode statistical regularities distributed across many values. They are not ordinary document records with reliable insert, update, and delete operations. Some examples can be memorized, particularly when repeated or distinctive. Yet a model may also distort facts, combine them, or fail to retrieve them on demand. Training on a document does not guarantee accurate recall of every statement.

This matters when choosing between training and retrieval. Training can teach a response pattern or a recurring domain skill. Retrieval can provide a particular current passage at answer time. They can work together. Treating weights as a replacement for an auditable knowledge store often creates unnecessary maintenance and evaluation problems.

Exercises

For the packing-time example, calculate the prediction for five items when weight is 0.7 and bias is 2. The answer is 5.5 minutes. Now change only the bias to 3; every order's prediction increases by one minute. Change only the weight to 0.8; a five-item order increases by half a minute and a ten-item order by one minute.

A batch contains 16 sequences of 128 token IDs. Write its shape. Then imagine each token is represented by 64 numbers and write the new shape. Explain why the token IDs should be integers while the representations may be floating point.

Describe one task where an essential piece of information is missing from the proposed input. Explain why a bigger model does not reliably solve that information problem.

4 Project turn four sensor readings into a useful prediction

You have a row of readings: temperature, vibration, load, and age. You want either an alert about a future fault or an estimate of remaining operating time. This is a good first complete machine-learning project because the input and output are small enough to inspect. You can run it on a CPU, understand every transformation, and keep the same workflow when the inputs later become photographs or text.

The accompanying example creates fictional machines. Its numbers are educational, not an engineering reliability model. No external dataset, account, GPU, or paid service is needed.

Open a fresh terminal at the unpacked companion root. Use Python 3.12 for the tested CPU reference environment. Create and activate a dedicated environment before the first command that imports these packages. The activation command below is Bash; on Windows PowerShell use the corresponding `.venv-cpu/Scripts/Activate.ps1` path.

```
cd examples/tiny-ml
python -m venv .venv-cpu
source .venv-cpu/bin/activate
python -m pip install -r requirements-cpu.txt
python train_tabular.py --task classification --out runs/classification
python predict_tabular.py runs/classification/model.joblib \
  --values 72 3.5 0.9 36
```

The first command generates data, separates machine groups, trains three candidates, calibrates the chosen classifier, selects an alert threshold, evaluates a held-out test set, and saves a model. The second turns one new row into a probability and an alert. Read the output files before changing the model.

Verification status: the classification and regression programs were executed on CPU with Python 3.12.14, NumPy 2.3.5, scikit-learn 1.8.0, SciPy 1.17.0, and joblib 1.5.3. Exact metrics below describe that one synthetic run. Neural examples later in these chapters were syntax-checked, not trained or GPU-tested in the authoring environment.

Name the input the target and the decision

An **input** is the information available when a prediction must be made. Here it is a four-number vector, in this order:

1. temperature in degrees Celsius
2. vibration in millimeters per second
3. load as a fraction between zero and one
4. age in months

A **target** is what the model learns to predict. For classification it is a zero or one indicating whether the fictional fault occurred. A **label** is the recorded target attached to an individual example. The alert is a downstream decision: issue it when the predicted probability passes a threshold. Target, score, and action are different objects. A model can rank machines well yet use a poor alert threshold.

For real data, make the target definition more precise: “fault within the next seven days, excluding planned maintenance, using measurements collected before noon today.” Specify what happens to a machine whose next seven days have not yet been observed. Calling it a negative would silently introduce incorrect labels. Specify whether maintenance changes the outcome you are trying to predict.

The dataset contains five observations per machine. Machine ID is useful for separating groups, but is not an input feature. If we split individual rows randomly, readings from the same machine could appear on both sides of the evaluation. A model might benefit from machine-specific regularities that will not be available for a new machine.

The first design artifact is a prediction contract: input fields and units, prediction time, target definition, acceptable errors, and what happens when an input is missing. Do this before choosing a network.

How the architecture changes the data you need

A linear model assumes the useful relationship can be expressed with weighted features, perhaps after carefully justified feature transformations. A tree ensemble can learn local thresholds and interactions, but needs examples in the relevant regions of feature space. A deep tabular network adds capacity without supplying missing evidence. Start with the simpler candidates and plot validation performance as the number of independent training machines grows. A learning curve still improving with more machines suggests a data opportunity; a flat curve can indicate missing information, label noise, a weak representation, or an architecture limit. No universal row count guarantees success.

For this example, 2,000 rows are only 400 independent groups. For your task, count the independent units and the rare outcomes in each deployment-relevant subgroup. The architecture cannot learn the behavior of a regime absent from training just because its parameter count is large.

Start with a baseline you can beat

The example compares three systems:

- A constant classifier predicts the training-set fault rate for every row. It establishes what can be achieved without using the readings
- Logistic regression learns one coefficient per processed feature and converts the weighted sum into a probability. It is a useful compact linear baseline
- Histogram gradient boosting builds a sequence of small decision trees. Each new tree helps reduce the remaining prediction error. It can capture nonlinear thresholds and interactions without a neural network

In this dataset, the generator makes high load interact with risk through a threshold. A small tree ensemble has a reasonable opportunity to improve on a linear model. That is a hypothesis to test, not a promise that trees win every tabular problem. The scikit-learn classifier and regressor APIs describe the controls used in this example. [\[M03\]](#) [\[M04\]](#)

The important input-to-output chain is:

```
raw row → missing-value treatment → numeric representation → model score
        → probability calibration → chosen threshold → alert
```

Logistic regression needs missing values filled in; the example uses the median measured on training rows and adds missingness flags. Standardization then subtracts the training mean and divides by the training standard deviation. Those operations are inside a pipeline, so the same fitted transformations travel with the model. The boosting implementation can handle missing values directly. Computing imputation values from the full dataset would let future evaluation data influence the model. [M01]

Separate fitting calibration selection and testing

At the default `--machines 400`, five readings per machine produce 2,000 rows. The script shuffles machine IDs once and assigns whole machines to four disjoint sets:

Set	Rows in this run	Purpose
Training	1,200	Fit coefficients, trees, and preprocessing
Calibration	300	Fit a probability correction for the selected classifier
Validation	200	Choose the candidate and the alert threshold
Test	300	Report final behavior after the other choices are fixed

These proportions are a teaching choice. They are not a universal prescription. On very small real datasets, grouped cross-validation can make better use of scarce examples, but all preprocessing and selection must then happen inside the appropriate training folds. When the product predicts future events, chronological evaluation may matter more than random group assignment. If the intended deployment is on new factories, holding out machines from the same factories is still insufficient: hold out factories too. [M20]

The script does not refit on the test set. It also disables the boosting estimator's automatic internal early stopping; otherwise an unexamined random row split inside the estimator could defeat the carefully chosen group boundaries. With `early_stopping=False`, `max_iter` is the deliberately fixed tree-building budget.

Read the first result before tuning

The CPU run selected boosting. Its validation average precision was approximately 0.703, compared with 0.681 for logistic regression and 0.240 for the constant baseline. On the held-out test set it achieved:

- average precision: 0.693
- ROC-AUC: 0.788
- precision and recall at the selected threshold: both 0.633
- threshold: approximately 0.31
- confusion matrix: 177 true negatives, 33 false positives, 33 false negatives, 57 true positives

This is not “69.3% accuracy.” **Average precision** summarizes precision as recall changes while sweeping score thresholds. **Precision** asks what fraction of alerts were correct. **Recall** asks what fraction of actual faults were caught. **ROC-AUC** measures ranking between positive and negative cases; it can look comfortable while the alert workload is unacceptable on a rare-event problem. Read the confusion matrix in counts and decide whether the operational tradeoff is useful.

The threshold maximizes validation F1 in this exercise. F1 combines precision and recall; it is a convenient neutral classroom choice, not the correct business objective by default. If missing a fault costs much more than an unnecessary inspection, choose a validation threshold that meets a recall requirement or minimizes explicitly stated costs. If inspection capacity is ten machines per day, evaluate that policy directly. Never choose a threshold by trying values on the test labels.

A probability is a claim that needs checking

A prediction of 0.8 should mean that roughly eight out of ten comparable cases assigned that probability are positive. **Calibration** tests this claim. The example fits a sigmoid calibrator on a held-out set using `CalibratedClassifierCV(FrozenEstimator(model), method="sigmoid")`. It writes mean predicted probabilities and observed positive fractions for reliability bins. Calibration data must be separate from the data used to fit the original estimator. [M02]

Brier score and log loss also appear in the report, but neither isolates calibration by itself. Inspect bin counts and a reliability plot rather than interpreting one aggregate number as proof. A small calibration set produces noisy bins. Changes in fault prevalence, sensors, or maintenance policy can make yesterday's calibrated probabilities wrong tomorrow.

High confidence is not proof that an input is familiar. A model can be confidently wrong on out-of-distribution readings. Start with practical safeguards: reject malformed units, flag ranges absent from training, expose an “insufficient information” state, and review uncertainty-sensitive cases. An ensemble's disagreement can be informative, but is not a universal detector of novelty. Probability calibration does not transform an unreliable model into a reliable one.

What each tabular control changes

The command-line controls are intentionally short:

Control	Default	What it means and how to change it
<code>--task</code>	classification	Switch to regression for a continuous target; this changes models, losses, and metrics
<code>--seed</code>	42	Reproduces data generation and split assignment; examine several seeds after the basic pipeline works
<code>--machines</code>	400	Number of independent machine groups; there are five rows per group
<code>--out</code>	runs/tabular	Artifact destination; use distinct directories for experiments

Inside the code, logistic $C=1$ is inverse regularization strength: smaller values penalize large coefficients more. `max_iter=1000` is a solver iteration limit, not a desired number of meaningful training epochs. Ridge regression's `alpha=1` is a coefficient penalty whose direction is the opposite of logistic C : larger means stronger regularization.

Boosting uses `learning_rate=0.06`, which controls each tree's contribution; `max_iter=120`, the number of boosting iterations; `max_leaf_nodes=7`, the maximum leaves in an individual tree; `min_samples_leaf=25`, which discourages tiny local rules; and `l2_regularization=1`, which penalizes leaf values. Smaller learning rates often need more iterations. Deeper or more numerous trees can fit more patterns and more noise. Do not sweep every setting at once: inspect errors, form one hypothesis, change one or two controls, and compare on validation. [M03] [M04]

Artifacts are `model.joblib`, `metrics.json`, `split_machine_ids.json`, and `test_predictions.csv`. The model bundle includes preprocessing where needed, feature order, task, and threshold. A reload-and-predict equivalence check is part of the executed script. Pickle-derived formats such as `joblib` can execute code when loaded: load your own trusted artifact, not a random downloaded file. Record package versions with the model. [M24]

Inference on the CPU and what the GPU changes

`predict_tabular.py` loads the whole trusted `joblib` bundle, uses the saved preprocessing, preserves feature order, and applies the saved threshold. Missing measurements can be supplied as `nan`; do not replace them with zero unless zero has that meaning. This scikit-learn workflow executes on CPU. There is no meaningful “move it to CUDA” switch for these fitted estimators. A GPU-specific replacement would be a different implementation requiring compatibility and accuracy validation.

The example unseen row produces a fault probability about 0.885 and an alert in the executed CPU run. That prediction is about the fictional generator, not the physical world. Serving must validate units and feature ranges before calling the model.

Change the output predict a number instead

Run:

```
python train_tabular.py --task regression --out runs/regression
python predict_tabular.py runs/regression/model.joblib \
  --values 72 3.5 0.9 36
```

The representation of the four inputs stays the same. The target becomes a fictional number of remaining hours. The constant baseline now predicts a mean; ridge regression predicts a regularized linear combination; the tree model predicts a continuous value. There is no class threshold or classifier probability calibration. The example reserves the calibration split for consistency but does not use it for this regression task.

Mean absolute error, or MAE, measures the average absolute discrepancy in the target's units. **Root mean squared error**, or RMSE, penalizes large mistakes more strongly.

R-squared compares squared error against predicting the test-set mean; it can be negative. In the executed run, test MAE was 15.77 synthetic hours, RMSE was 20.10, and R-squared was 0.871. These numbers say nothing about actual machinery.

Changing target scale changes optimization. If one output is in millimeters and another in dollars, an unweighted sum of squared errors can let the larger numeric scale dominate. Standardize each target using training statistics, or choose weights that reflect actual costs; invert the transformation before reporting physical units. If you need a range rather than a point, train quantile predictors or use an appropriately validated prediction-interval method. A single mean estimate cannot express multiple plausible outcomes by itself.

Generalize the project to your own input to output task

The purpose of this lab is a reusable procedure, not a particular classifier. Write down one real input and the exact desired output. Then ask:

1. **Is the needed information present?** A photograph of a sealed box does not determine its unseen contents. A noisy sensor snapshot may not determine the exact date a part will fail. More training cannot recover information that is absent. Change the input, predict a distribution, narrow the claim, or allow abstention
2. **Is a deterministic program better?** Converting Celsius to Fahrenheit, checking a checksum, sorting records, and applying a published tax formula are rule-based computations. Implement and test the rule. Training a predictor adds avoidable approximation error
3. **What is an independent example?** A customer, original image, patient, device, document family, or time period may generate many correlated rows. Split at the level matching deployment before augmentation or paraphrasing
4. **How will the target be represented?** Choose the representation before choosing the loss. Preserve units, coordinate frames, missing-label masks, and any required ordering
5. **What failure actually matters?** A correct class with the wrong boundary, an accurate average with catastrophic outliers, and a semantically similar but wrong policy page are different errors
6. **What must be measured outside the model?** Latency, memory, privacy, human review load, input failures, and recovery behavior belong in the acceptance test

Here are useful mappings to extend the same process:

Desired output	Representation	Starting objective	Important trap
One of K exclusive classes	K logits and one integer label	multiclass cross entropy	Softmax assumes the labels are mutually exclusive
Any subset of K labels	K independent logits and K binary labels	binary cross entropy per label	Missing annotations are not automatically negatives
One or several numbers	one or several continuous outputs	MAE, MSE, or Huber	Units and target scales change the objective
A bounded fraction	sigmoid output	a loss matching what the fraction represents	A proportion, event probability, and arbitrary bounded score have different statistical meanings

Desired output	Representation	Starting objective	Important trap
A pixel label map	class logits at every pixel	per-pixel cross entropy or binary BCE	Background imbalance can hide total foreground failure
Keypoint locations	coordinate vector or spatial heatmaps	coordinate regression or heatmap loss	Resize and crop transforms must update coordinates
A sequence of labels	one label distribution per input position	masked per-position cross entropy	Padding must not count as a real target
A variable-length sequence	token sequence with end marker	masked next-token objective	More than one output may be valid; exact match alone may mislead
An unordered set of objects	object slots with classes and geometry	matching plus class/geometry losses	Arbitrary label order creates contradictory supervision
A ranked list	query/document scores	pairwise or contrastive ranking loss	Unlabeled documents may still be relevant

For a short sensor time window, your tensor could have shape `[batch, channels, time]`. A one-dimensional convolution learns local temporal patterns; an RNN or small attention model can use longer dependencies. For a photograph plus numeric readings, encode the image and standardized numbers separately, concatenate their representations, and train a small output head. Compare each modality alone against the combined system: a supposedly useful input may contribute nothing or create a shortcut.

For a sequence output, use a padding mask and compute loss only on valid positions. For multiple outputs, write `total_loss = task1_loss + weight * task2_loss` and explain the weight's units and intent. For an unordered output, use a model and training objective that handle matching; do not force a network to learn your arbitrary enumeration of objects.

A neural network becomes attractive when useful features are difficult to hand-design, when spatial or sequential structure matters, or when a suitable pretrained representation exists. On ordinary modest tabular datasets, keep the linear and tree baselines. “Use a GPU” is not the problem definition.

Checkpoint what you should now be able to do

You should be able to describe a new task without naming a model: the prediction-time inputs, target, independent split unit, baseline, error measure, and failure policy. If you cannot, stop before increasing model size.

5 Build a dataset that teaches the job

Start with twenty examples you can defend

Before collecting thousands of records, write twenty real examples of the task. For each, record the input available at prediction time, the desired output, why that output is correct, the source, and a grouping identifier. Read them as if you were a new annotator. If the target is unclear to a careful person, training is unlikely to resolve the ambiguity reliably.

Consider a support-message router with labels billing, delivery, and other. 'Where is my order?' is delivery. 'I was charged twice' is billing. 'Please cancel the order because it is late' exposes an ambiguity: should the router prioritize cancellation, delivery, or the team that owns the action? You need an annotation policy. Adding more examples without deciding the policy creates contradictory supervision.

Write the policy next to the data. Define each label, inclusion and exclusion cases, tie-breaking rules, treatment of missing information, and when to use an unknown or abstain label. Include examples that distinguish easily confused categories. A label definition that only restates its name is not enough.

For a generated response, the target policy may specify facts that must be included, claims that must not be invented, formatting, length, tone, and what to do when evidence is absent. For an image mask, it must specify which pixels belong to the object and how to treat occlusion. For audio, it must specify segment boundaries, sample rate, loudness handling, and whether silence is meaningful.

A practical record format

JSON Lines stores one JSON object per line. It is convenient because a program can read records one at a time, and one malformed line can be identified precisely. Here is a task-neutral pattern:

```
{"id":"ticket_0042","group_id":"customer_17","input":"I was charged twice","target":"billing","source":"owned_support_export","split":"train"}
```

The id identifies the record. group_id identifies related records that must be handled together when splitting. input is what the model receives. target is what it should learn to produce. source records provenance. split indicates the assigned partition. Keep extra metadata outside the model-visible input unless it will genuinely be available at prediction time.

The eventual trainer may require a different schema, such as messages for chat fine-tuning or image and text fields for diffusion. Maintain a clean canonical dataset and a reproducible conversion step. Do not repeatedly edit the only copy to satisfy each new trainer's expectations.

A schema validator should check required fields, types, allowed labels, empty values, unique IDs, valid paths, and length limits. It should fail loudly on malformed records rather than silently skipping a large fraction. Log rejected records with a reason, taking care not to expose private contents unnecessarily.

Split related things before making derivatives

Training data teaches parameters. Validation data guides experiment choices. Test data supports a final estimate after choices are fixed. The exact percentages depend on dataset size and task diversity; a mechanical 80/10/10 split is not always appropriate. A test set of ten rare cases may be too small to tell you much, regardless of its percentage.

The split unit is often larger than one row. Keep messages from the same conversation together. Keep crops and augmented versions of the same image together. Keep clips from the same recording session together. Keep questions derived from the same document together when the intended test is generalization to new documents. Otherwise nearly identical information can appear on both sides of the evaluation boundary.

A time-based split is useful when the deployed system will face future data. A source-based split tests transfer to new sources. A person- or device-based split can test whether the system works beyond the individuals or equipment seen during training. Pick the split that matches the intended claim.

Deduplication must operate across the eventual boundaries. Exact hashes catch identical bytes, but near duplicates may differ only in whitespace, a timestamp, a crop, or a paraphrase. Use normalization appropriate to the modality and inspect suspiciously similar pairs. Do not normalize away meaningful distinctions, such as case in identifiers or punctuation in code, merely to increase duplicate counts.

Data leakage also occurs when preprocessing learns from evaluation data. A scaler's mean, a feature selector's decisions, a learned tokenizer for a strict from-scratch experiment, or a vocabulary fitted to all records may incorporate information from outside the training partition. Fit learned preprocessing on training data, then apply it unchanged to validation and test data. [F09]

Teach the difficult cases on purpose

A dataset should represent both common situations and important failure modes. If 95 percent of support messages are delivery questions, a model that always predicts delivery can look accurate while being useless for billing. Count examples per class and per important slice. A slice might be language, source, message length, image lighting, audio noise level, or tool type.

Do not correct imbalance mechanically before asking what deployment will look like. Oversampling a rare class changes how often its examples influence training. Class weights change how much its errors contribute to loss. Thresholds change decision behavior after prediction. These are different interventions. Evaluate under the actual operating distribution as well as on deliberately balanced diagnostic sets.

Negative examples matter. A retrieval model needs plausible irrelevant passages, not only obviously unrelated ones. A tool-calling assistant needs cases where no tool is appropriate and cases where required arguments are missing. A visual classifier needs backgrounds and confusing non-target objects. A segmentation model needs empty scenes if it will encounter them. Without such examples, the model may learn that every input demands the positive behavior.

Hard negatives are examples that resemble a correct case but are wrong for a specific reason. They can be powerful, but they must be correctly labeled. A supposed negative that is actually relevant damages the learning signal. Review a sample manually before scaling mined negatives.

Synthetic data is a tool with a contract

Synthetic data can isolate a concept, cover a rare structured case, or create cheap preliminary tests. The first classifier in this book uses it because the exact rule is known. The image and audio exercises use original procedural examples so you can learn the pipeline without scraping uncertain material.

Synthetic data is not automatically representative. A classifier may learn the generator's punctuation or naming conventions rather than the intended meaning. A model trained on generated answers can inherit the generator's errors and stylistic habits. Ten paraphrases of one source are not ten independent sources.

When using a larger model to generate training material, preserve the prompt, model/version, sampling settings, source evidence, validation procedure, and applicable terms. Verify factual and structural claims rather than trusting fluency. Do not allow the generator to see held-out answers and then treat its outputs as independent training examples. Keep related generated variants in one split group.

Use a small human-reviewed anchor set to assess whether generated material teaches the intended behavior. Increase synthetic volume only after the first trained comparison improves on real held-out cases. If performance improves only on synthetic tests from the same generator, report that narrower result.

Rights privacy and consent are dataset properties

Record where each source came from, what rights or permission support its use, whether redistribution is allowed, and whether derivative model artifacts have additional conditions. Publicly accessible content is not automatically unrestricted training material. An open-weight model may have a license with conditions different from an open-source software license.

Separate the questions of permission to train, permission to publish data, permission to publish weights, and permission to deploy a commercial service. They can have different answers. Read the exact model and dataset terms for the pinned revision and intended use. This book provides a workflow for checking those terms, not a legal determination for a particular dataset.

Minimize personal information. Remove identifiers that the task does not need. Use de-identified or synthetic records for early experiments. A model may memorize unusual training strings, so removing a private source file after training does not establish that the information is absent from the weights. If strict deletion or access control is required, a retrieval store with explicit records may be easier to govern than parameter updates.

For voices and identifiable people, obtain appropriate consent for the intended use. Do not assume permission to record implies permission to train a generative model or imitate a person. For children or sensitive domains, additional safeguards and legal obligations may apply; use authoritative guidance for the relevant jurisdiction and project.

Build a data card before the long run

A useful data card states the task, source types, collection dates, rights, consent basis where applicable, preprocessing, split method, counts, length distributions, label distribution, known gaps, and intended uses. It should also state prohibited or unsupported uses and who reviewed the labels.

Save the exact processed files or reproducible transformation recipe. Compute hashes of split files. If you later change cleaning or labels, assign a new dataset version. 'The same dataset' is not a sufficient experiment description when its contents have changed.

Inspect at least a small random sample and a deliberately difficult sample after preprocessing, not only before it. Decoding errors, truncation, broken chat templates, swapped label IDs, and misaligned image masks often appear in the processed representation. Print or render examples at the point where the model will consume them.

Exercises

Create twenty examples for a task you care about. Find two that could reasonably receive different labels from different annotators. Rewrite the policy until the disagreement can be resolved consistently, or add an explicit uncertain category.

Choose a grouping rule. Explain why splitting individual rows would be misleading for your data. Then state exactly what your held-out result will claim: new messages from known users, new users, future documents, new recording sessions, or something else.

Write one positive example, one easy negative, one hard negative, and one case that should trigger abstention. If you cannot identify the abstention case, your deployment boundary is probably underspecified.

6 Decide whether the model actually improved

Make the baseline answer first

Before training, run the simplest plausible system on the evaluation cases. For a classifier, this might be the most frequent class, a short rule list, or a TF-IDF model. For a pretrained LLM, it is the untouched checkpoint with a careful prompt. For retrieval, it may be keyword search or the unmodified embedding model. For diffusion, use the base model with fixed prompts and seeds.

Save the baseline outputs. Otherwise a newly trained model's fluency or visual novelty can make it seem better than a baseline you remember inaccurately. Evaluation is a comparison under a fixed procedure, not a collection of favorite examples.

Write a primary metric and a small number of guardrails. The primary metric expresses the improvement you are trying to buy. Guardrails catch unacceptable regressions, such as malformed JSON, excessive latency, privacy leakage, poor performance on an important minority slice, or catastrophic forgetting of a required capability.

Classification needs more than one accuracy number

A confusion matrix counts how often each true class was assigned each predicted class. It shows which categories are confused. Precision asks, among cases predicted positive, how many were actually positive. Recall asks, among actual positives, how many were found. F1 combines precision and recall into a single harmonic mean, but its convenience does not remove the need to inspect the underlying tradeoff.

Suppose a defect detector finds 18 of 20 true defects and raises 12 false alarms. Recall is $18/20 = 0.90$. Precision is $18/30 = 0.60$. Whether that is acceptable depends on the cost of missed defects and false alarms. A high overall accuracy on thousands of normal cases could conceal the operational problem.

For multiple classes, macro averaging gives each class equal weight; micro averaging combines decisions across all examples. Weighted averaging uses class prevalence. State which you use. An apparently improved average may hide deterioration on a rare but important class.

For numeric regression, inspect mean absolute error, root mean squared error where relevant, and errors by target range. A model can have a good average error while failing badly on unusually large values. Compare against a constant mean or median baseline, and make sure the metric's units are understandable.

Generated outputs need decomposed tests

For a structured answer, first test whether it parses. Then test schema validity, required fields, allowed values, and semantic correctness. Valid JSON containing the wrong account ID is still wrong. A tool call should be evaluated for tool selection, argument extraction, missing-information handling, and whether an action should have been proposed at all.

For a knowledge answer, separate evidence retrieval from answer generation. Did the relevant passage enter the context? Did the answer accurately use it? Did it cite the right source? Did it invent unsupported details? A failed answer can originate in retrieval, context construction, the model, or the output validator.

For writing style, use a rubric that describes observable features rather than 'sounds like me'. Examples include typical sentence length, degree of formality, use of headings, tolerance for hedging, and factual preservation. Ask evaluators to compare outputs blind to which system produced them. Keep content quality separate from style similarity.

For images, compare identity or subject fidelity, prompt adherence, diversity, unwanted copying, and artifacts across fixed prompts and seeds. For audio, listen for event identity, timing, clipping, noise, repetition, and intelligibility where speech is involved. A numerical proxy is useful when it correlates with the task, but it should not replace listening or viewing.

Test behavior outside the easiest distribution

Include ordinary cases, edge cases, missing inputs, contradictory inputs, irrelevant content, and cases requiring refusal or abstention. Test changes in wording that should preserve the answer. Test changes in facts that should change it. These paired tests can reveal whether the model learned the intended dependency.

For a tool model, change a quantity or date in the input and see whether the corresponding argument changes while unrelated fields remain stable. For a classifier, remove an accidental keyword and test whether the decision still follows the substance. For image generation, change the requested background and see whether the learned subject is entangled with its training backdrop.

Do not use adversarial cases only as a dramatic final demonstration. Design a bounded suite before training. If an error matters in deployment, it deserves a place in the evaluation contract.

Small samples have uncertainty

If a model succeeds on 18 of 20 cases, the observed rate is 90 percent, but the underlying success rate is not known precisely. One extra error changes the score by five percentage points. Reporting many decimal places does not create certainty. Record counts, sample selection, and important slices alongside percentages.

When comparing two models, evaluate them on the same cases. A paired comparison can reveal which cases improved and which regressed. For human ratings, randomize presentation order and use consistent instructions. If possible, use more than one evaluator on a subset and investigate disagreements.

Repeatedly choosing the best result on one validation set can overfit the evaluation procedure. Keep the final test set untouched while choosing hyperparameters. After a final test reveals a problem, it is legitimate to improve the system, but that test has then informed development. A new independent test is needed for a fresh unbiased claim.

Calibration and abstention

A predicted probability of 0.9 does not automatically mean the model is right 90 percent of the time on similar predictions. Calibration measures how probabilities correspond to observed outcomes. A classifier can rank examples well while being overconfident. Generative token probabilities are especially easy to mistake for answer-level confidence.

Choose abstention rules using held-out data and the actual cost of errors. For example, route low-confidence support tickets to a person. Measure both coverage, the fraction answered automatically, and accuracy among answered cases. A system can improve apparent accuracy by refusing almost everything; that tradeoff must be visible.

Temperature scaling for a classifier adjusts logits using a scalar fitted on a calibration set. This is different from freely raising generation temperature for creative text. Calibration requires separate data and should be evaluated on still-held-out cases. The typed-decision chapter develops this more carefully.

Loss and task quality can disagree

Training loss is useful for debugging optimization. Validation loss is useful for detecting some forms of overfitting or distribution mismatch. Neither replaces task evaluation. A language model can reduce average token loss by learning common formatting while still choosing the wrong tool. A diffusion model can improve its denoising objective while producing less diverse samples.

An instructive experiment keeps both curves and task outputs. At selected checkpoints, run the fixed evaluation suite and preserve outputs. Choose the checkpoint by the intended task and guardrails, not automatically by the last training step. If validation loss improves while task quality worsens, investigate label format, weighting, generation settings, and whether the loss actually matches the goal.

An evaluation report you can use

A concise report names the task, dataset version, split rule, baseline, candidate, primary metric, guardrails, generation or inference settings, measured latency and memory where applicable, counts, and failure examples. It states which cases remain unsupported. It links the exact model artifact and configuration.

Do not hide bad examples by averaging them away. A short error taxonomy is often the most useful output: wrong label, missing field, unsupported fact, truncation, failure to abstain, memorized background, or poor performance on long inputs. The taxonomy tells you what to change next.

Exercises

Write a metric that could look good while your intended system fails. Then add a guardrail that would reveal the failure. For example, overall routing accuracy can hide poor recall on urgent cases; a slice-specific recall requirement makes that visible.

Create five paired tests where a small input change should change exactly one part of the output. Create five more where harmless rewording should not change the decision. Run both sets on the baseline before training.

For your project, define the cost of a false positive, a false negative, and an abstention. If these costs cannot be expressed in money, describe their practical consequences. Use that description to choose a meaningful operating threshold.

7 Move from feature columns to neural representations

The sensor project used a short feature vector. Images and token sequences need larger structured representations. Before running the next image project, learn the two pieces that change: tensor shapes and learned layers.

Scalars vectors matrices and tensors

A scalar is one number. A vector is an ordered list of numbers. A matrix is a rectangular table of numbers. Tensor is the broader term used for arrays with any number of dimensions. In a training program, a tensor also has a numeric type and a device.

Shape tells you how its values are arranged. A batch of 32 records, each with 10 features, has shape [32, 10]. A batch of 8 RGB images resized to 224 by 224 pixels commonly has shape [8, 3, 224, 224] in PyTorch: batch, channel, height, width. A batch of 4 token sequences, each of length 256, has shape [4, 256]. After a language model converts token IDs to 128-dimensional vectors, the representation has shape [4, 256, 128].

The order of dimensions matters. A tensor can have the right number of values and the wrong meaning. Passing images arranged as [batch, height, width, channel] into code expecting [batch, channel, height, width] may fail loudly or produce nonsensical behavior. Record shapes near the boundary of each stage: load, preprocess, batch, forward, loss, and decode.

A tensor's dtype is its numeric format. Token IDs are integers. Weights and activations are usually floating-point values. A Boolean mask indicates which positions are valid or allowed. A device indicates where the tensor lives, such as CPU or cuda:0. A model and its inputs generally need compatible devices. Moving a model to a GPU does not automatically move every separately created input tensor there.

A neural layer is a learned transformation

A linear layer maps an input vector x to an output vector y using a matrix W and an optional bias b . Written compactly, $y = Wx + b$. If the input has 10 features and the output has 20 features, the layer has 200 weights plus 20 biases. It learns combinations of the input features.

Stacking linear layers without anything nonlinear between them is still equivalent to one linear transformation. A nonlinear activation, such as ReLU or GELU, allows a network to express more complicated relationships. ReLU keeps positive values and replaces negative values with zero. GELU makes a smoother change. You need not memorize their formulas to understand the role: they prevent a deep network from collapsing into a single linear map.

Depth is the number of successive transformations. Width is the size of intermediate representations. More depth or width often increases capacity, the variety of functions the network can express. Capacity is not a guarantee of useful learning. A larger network can memorize bad labels, amplify dataset shortcuts, or remain poorly trained because the available data and compute are insufficient.

A residual connection adds an earlier representation to the output of a transformation. It lets a block learn a modification rather than requiring it to reconstruct everything from scratch. Normalization layers rescale intermediate representations using a particular rule. Their details differ, but their practical role includes keeping numerical behavior manageable. When adapting a known architecture, preserve these details unless architecture research is your explicit project.

8 Project classify your own small image collection

Suppose your input is a photograph of one manufactured part and your output is one of three mutually exclusive classes: acceptable, scratched, or bent. The change from a row of numbers to an image is a change in representation: a color image becomes a tensor [3, height, width]; a batch becomes [batch, 3, height, width]. The class target remains one integer per image.

Use photographs you own or are licensed to train on. First write labeling rules: how much of a scratch counts, what to do when the part is cropped, whether an image can belong to two categories, and how to label uncertain cases. If “scratched” and “bent” can both be true, use multilabel targets instead of forcing an exclusive label.

Take images across sessions, viewpoints, lighting, and backgrounds. Keep every photograph of the same physical part in one split. If all acceptable parts were photographed on a blue desk and all defective parts on a red desk, the network can learn the desk. More rotations of the same pictures will not repair that confounding.

Arrange the pre-split images as:

```
parts/
  train/acceptable/...
  train/scratched/...
  train/bent/...
  val/acceptable/...    # also scratched and bent
  test/acceptable/...   # also scratched and bent
```

Open a fresh terminal at the companion root, then enter this project. Use Python 3.12 and a separate neural environment. Choose exactly one official torch/torchvision installation below, according to your device. The CUDA command requires a compatible NVIDIA GPU and driver; the CPU command needs no NVIDIA device. These are inspected reference versions, not an environment installed during preparation. [\[M21\]](#) [\[F15\]](#)

```
cd examples/tiny-ml
python -m venv .venv-neural
source .venv-neural/bin/activate

# CPU choice
python -m pip install torch==2.8.0 torchvision==0.23.0 --index-url https://download.pytorch.org/whl/cpu

# OR compatible CUDA 12.6 choice in a separate environment
# python -m pip install torch==2.8.0 torchvision==0.23.0 --index-url
# https://download.pytorch.org/whl/cu126

python -m pip install -r requirements-cpu.txt
```

Before collecting real parts, run the complete pipeline on an original three-class geometric fixture. This tests file discovery, labels, training, saving and inference without requiring a photo collection. It does not establish real defect-detection quality.

```
python make_classification_data.py --out data/classification-shapes \
  --train 24 --val 8 --test 8
python train_image_classifier.py --data data/classification-shapes \
  --backbone tiny --epochs 2 --batch-size 8 --lr 0.001 \
  --device cpu --out runs/shapes-classifier
python predict_image_classifier.py runs/shapes-classifier \
  data/classification-shapes/fresh-circle.png --device cpu
```

The generator uses separate split seeds, randomized position/size, and colors independent of shape class. The fresh circle is generated with a separate seed and is not part of a fitting or selection split. Its classes are circle, square, and triangle; the same trainer can later learn your separately defined part labels.

Run the supplied scratch model first:

```
python train_image_classifier.py --data parts --backbone tiny \
  --epochs 10 --batch-size 16 --lr 0.001 --device cpu \
  --out runs/parts-tiny
```

API target: PyTorch 2.8.0 and torchvision 0.23.0, a documented matching pair. This script was syntax-checked only. Install the official PyTorch build appropriate for your operating system and GPU driver; do not assume a CUDA wheel from another machine is compatible. [\[M21\]](#)

The same RGB input can support different supervised targets

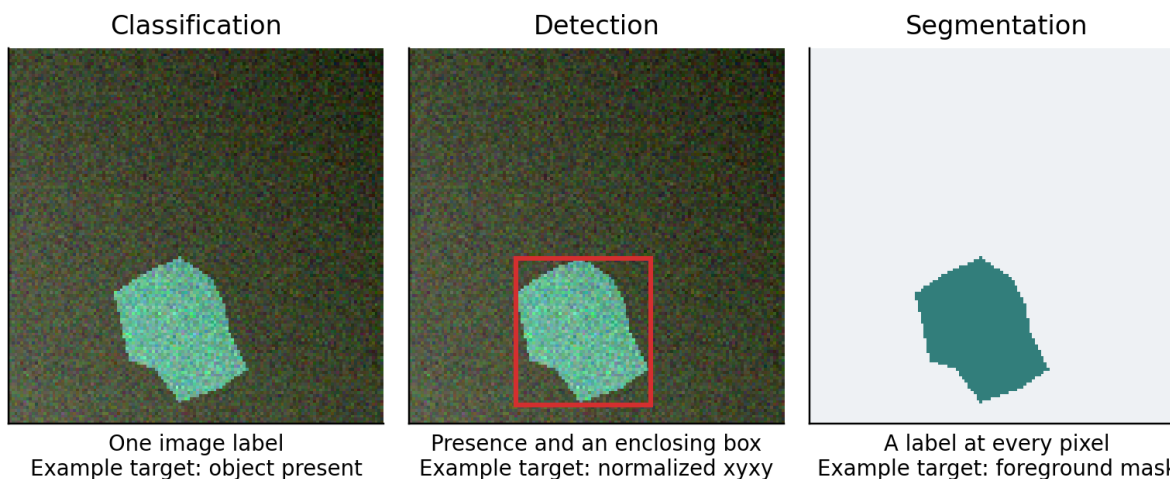


Figure 3 The same original generated RGB input can teach different outputs. These are illustrations of target formats, not predictions from a trained model. Classification returns one label, detection adds a box, and segmentation predicts a pixel mask.

Convolutions features and the output head

The small CNN has three convolutional stages with 16, 32, and 64 feature channels. A convolution reuses small spatial filters over an image, so a feature such as an edge can be detected at many positions. ReLU adds nonlinearity. Max pooling reduces spatial resolution. Global average pooling reduces each final channel to a number, and a linear head maps the 64 numbers to class logits.

A **logit** is an unrestricted score before conversion into a probability. Training passes raw logits to cross entropy and integer class labels as targets. Do not apply softmax before this loss. At inference, softmax can turn logits into a class distribution; the largest logit and largest softmax probability identify the same class. A probability still needs calibration checks if you intend to use it for an accept/reject decision.

The scratch route resizes images to 96×96 and scales RGB values to $[0, 1]$. This is simple, but it distorts aspect ratio and can erase narrow scratches. That is a deliberate starting baseline. If defects occupy two original pixels, shrinking the image may remove the information before the network sees it. Try aspect-preserving resize with padding or carefully selected crops, and update the deployment preprocessing exactly.

--epochs counts passes through the training set. --batch-size controls how many images contribute to one update. --lr is the optimizer step size. AdamW uses weight decay 0.0001 in this example. --out identifies an experiment directory; --device cuda selects the GPU if a compatible installation is present. A fixed seed of 42 makes the example easier to compare but is not a guarantee of identical results on all hardware.

Transfer learning before a bigger scratch network

Try the alternative:

```
python train_image_classifier.py --data parts --backbone mobilenet \
  --epochs 10 --batch-size 16 --lr 0.001 --device cuda \
  --out runs/parts-transfer
```

This downloads MobileNetV3-Small ImageNet weights, freezes the pretrained parameters, replaces the final classifier, and trains only that new layer. The pretrained weights' own preprocessing is used, rather than guessing normalization or crop rules. Frozen feature-extractor batch-normalization statistics stay in evaluation mode. The script keeps the rest of the classifier frozen; this is a conservative head-only baseline. [M06]

With enough target-domain examples, the next experiment is to unfreeze a late block and use a substantially smaller learning rate, such as a candidate 0.00001 for pretrained weights while the new head uses a larger rate. Recreate the optimizer with the newly trainable parameters. Choose the rates on validation. If transfer hurts, check crop behavior, color channels, label mapping, and domain mismatch before concluding that pretraining is useless.

Rights checkpoint: torchvision code uses BSD-3-Clause, but that does not automatically license every dataset or pretrained weight for every use. Torchvision explicitly asks users to review model and dataset terms separately. The scratch option avoids pretrained-weight terms; your image rights still matter. Public availability is not permission to train or redistribute. [M11]

What data the two image architectures need

The scratch CNN learns both visual features and the class decision from your examples. Its small capacity makes a tightly controlled visual task feasible, but it cannot invent the diversity of lighting, defects, and backgrounds absent from training. Transfer learning starts with general visual features and often makes better use of a small labeled collection; those features may still be inappropriate for unusual sensors, microscopic patterns, or very fine detail. Neither route removes the need for held-out acquisition sessions and examples of each class.

Measure learning curves using increasing numbers of independent physical parts, not augmented copies. Compare the head-only model with partial fine-tuning only after labeling and preprocessing are credible. Select architecture based on the task's needed spatial detail and the available labeled variation, rather than a fixed “images per parameter” rule.

Run the saved image classifier on an unseen photograph

```
python predict_image_classifier.py runs/parts-tiny new_part.jpg --device cpu
python predict_image_classifier.py runs/parts-tiny new_part.jpg --device cuda
# The same commands work with runs/parts-transfer.
```

Both paths load `metrics.json` for architecture and class mapping, reconstruct the architecture, load `best.pt`, apply the original preprocessing, enter evaluation mode, and return class probabilities plus the top class. The CUDA path moves the model and input to CUDA; the checkpoint is initially loaded on CPU. Both use FP32, so there is no implicit CPU/GPU dtype mismatch. CUDA can give slightly different numerical results and should be validated. These neural inference commands are syntax-checked only.

The transfer inference constructor uses `weights=None` because the saved checkpoint already contains the full weights; it does not redownload a baseline. A deployment must keep these companion metadata files with the checkpoint. The script does not impose a confidence-based rejection threshold because none has been validated for your dataset.

Small image datasets fail in recognizable ways

Read `metrics.json`, including class mapping, per-epoch validation loss, test macro-F1, and the confusion matrix. The lowest validation loss selects `best.pt`. Macro-F1 gives each class equal weight, so a rare class cannot disappear inside overall accuracy. Also inspect recall for the class whose mistakes are most costly.

Start with no augmentation, then add transformations that preserve the real label. Horizontal flips can be valid for an unoriented part but invalid for reading left/right arrows. Color jitter can model lighting or destroy a color-coded fault signal. Random crops can remove the only defect while preserving the original defect label. Augmentation increases variation; it does not create independent evidence.

When a rare class is ignored, first verify its labels and examples. Then consider class-weighted loss or controlled sampling. The weights should be derived from training data and the chosen cost model. Oversampling a handful of rare images repeatedly does not create new rare cases. Evaluate on the real prevalence expected after deployment, including completely unfamiliar objects if the product must reject them.

Before a real training run, attempt to overfit eight clean examples. Failure to do so is a useful bug signal: wrong target type, wrong class ordering, detached gradients, a frozen head, inconsistent normalization, or labels unrelated to inputs. Passing this test does not establish generalization; it establishes that the learning machinery can fit something.

9 Project locate one object with a box

What detection adds to classification

An image classifier answers what kind of image it sees. A detector also answers where a target is. Common application patterns include locating a part before measurement, proposing a crop for another model, finding an object in a fixed inspection station, and reporting an empty scene without inventing a detection. A downstream application still decides what to do with the box.

This project trains an original small CNN to detect at most one foreground object. It returns a probability that an object is present and one enclosing rectangle. It deliberately does not solve crowded multi-object scenes, identify individual overlapping instances, or generate a variable-length list of classes and boxes. Those require a more capable detector and appropriate annotations. This bounded project teaches the complete data-to-box path before you study those systems.

The model is small and trained from scratch. Its usefulness comes from the tightly scoped synthetic task, not from a pretrained historical architecture. You can replace the synthetic examples with a similarly bounded real inspection task only after reviewing whether the representation, labels, and input variation match.

Make boxes from masks without changing the meaning

Open a fresh terminal at the companion root, enter `examples/tiny-ml`, and activate the neural environment installed in the image-classification project. If you skipped that project, complete its explicit torch and torchvision installation block first.

```
cd examples/tiny-ml
source .venv-neural/bin/activate
python make_shapes.py --out data/one-object --size 128 \
  --train 256 --val 64 --test 64 --seed 51 \
  --max-objects 1 --empty-probability 0.20
python detector_common.py
```

The generator writes an RGB image and binary mask for each example. A binary mask is a same-sized image whose background pixels are 0 and foreground pixels are 255; it records where the target exists. About twenty percent are empty by the generator's sampling rule; the exact split counts vary. Every nonempty image contains at most one generated object. The training, validation, and test sets use separate seeds. Their simple distribution is a teaching fixture, not evidence of robustness to real photographs.

A positive mask becomes a bounding box. Find the minimum and maximum foreground x and y coordinates. The maximum edge is one pixel beyond the largest included coordinate, making it exclusive. Divide x coordinates by image width and y coordinates by image height. A box [0.2, 0.2, 0.7, 0.6] therefore spans those fractions of the original image dimensions.

An empty mask has objectness target zero. Its placeholder box is ignored by the localization loss. Teaching the model that every image contains an object would produce a poor empty-scene behavior. The generator's max-objects setting is saved in the manifest, and the trainer refuses data not explicitly generated for this one-object contract.

If you derive one box from a mask containing several disconnected objects, you obtain one box around their union. That is a region-extent task, not ordinary instance detection. Do not make that substitution silently. The supplied detector uses `max-objects 1` precisely to keep the target meaning clear.

Why the network retains a spatial grid

The input is an RGB tensor [3, 96, 96] after bilinear resizing and scaling pixel values to [0,1]. Three convolution/ReLU/pooling stages produce 16, 32, and 64 feature channels. A 4 by 4 pooled grid is retained and flattened before the output layer. Unlike a global average over the entire image, this retains coarse information about where features occur.

Direct layer-shape arithmetic gives 28,709 trainable parameters; the program records the authoritative runtime count. This is intentionally tiny, so the first challenge is correct data and output semantics rather than filling GPU memory.

The head emits five values. One is an objectness logit. Four are transformed to values between zero and one and arranged as ordered lower and upper x/y corners. Ordering ensures the decoded rectangle has nonnegative width and height. It does not guarantee that the rectangle is correct.

This architecture needs varied object locations, sizes, backgrounds, and empty scenes. If every target were centered, the network could learn a constant box rather than localization. If every positive had one background color and every negative another, it could detect the background. The synthetic generator varies nuisance conditions while preserving a learnable foreground cue.

For a real task, collect separate acquisition sessions and keep images of the same physical object together when splitting. Include the sizes, viewpoints, and absences that deployment will encounter. A one-object architecture is appropriate only when the application's field of view genuinely enforces that constraint.

Two losses teach two outputs

Objectness uses binary cross-entropy on the raw logit. Localization uses smooth L1 loss between predicted and target normalized box coordinates, only on positive examples. The total is objectness loss plus five times localization loss. Five is an explicit balancing choice for this teaching problem, not a universal detection constant.

This differs from classification's one class label per image and segmentation's one class label per pixel. The desired output representation determines what supervision is needed and where the loss applies. A box label is cheaper than a pixel mask for many real tasks, but it does not teach the precise outline inside the rectangle.

The optimizer is AdamW with learning rate 0.001, weight decay 0.0001, and the usual beta defaults of 0.9 and 0.999. Gradient norm is clipped at 1.0. The reference runs in FP32 on both CPU and CUDA. The seed is 42. These choices make the small experiment inspectable; tune on validation only if you replace its data.

Train save and evaluate

```
python train_detector.py --data data/one-object \
  --out runs/one-object-cpu --epochs 2 --batch-size 8 \
  --lr 0.001 --device cpu

# After the smoke test, a separate GPU experiment
python train_detector.py --data data/one-object \
  --out runs/one-object-gpu --epochs 15 --batch-size 16 \
  --lr 0.001 --device cuda
```

The CPU run is a short correctness pilot. A compatible CUDA setup can run the same computation on the GPU. The trainer refuses unavailable CUDA rather than silently falling back. It refuses a nonempty output directory so an old artifact cannot be overwritten by a different experiment.

--epochs controls complete passes over training images. --batch-size controls concurrent images per update. --lr controls update scale. --data locates the immutable split directories and manifest. --out identifies a fresh run. --device chooses the execution target. Input size, feature widths, box-loss weight, class mapping, threshold, data hashes, and numeric policy are saved explicitly.

The checkpoint with the lowest validation objective becomes best.pt. config.json contains the architecture identity, foreground class name, preprocessing, box convention, and threshold. metrics.json contains the epoch history and final held-out test results. These files form one inference artifact and must stay together. This small trainer intentionally does not implement optimizer resume; use a new run for a new experiment.

Evaluation reports mean IoU over all positive images, detection precision and recall requiring IoU at least 0.5, and false-alarm rate on empty images. A positive image with a high objectness score but the wrong box is an incorrect detection. The IoU threshold and objectness threshold are different: one measures geometric agreement; the other decides whether to emit a box. The default objectness threshold is a teaching choice of 0.5, not a calibrated production threshold.

Load on CPU and GPU with a fresh image

Generate a separate small set that is not used for fitting or model selection:

```
python make_shapes.py --out data/one-object-fresh --size 128 \
  --train 1 --val 1 --test 1 --seed 999 \
  --max-objects 1 --empty-probability 0

python predict_detector.py runs/one-object-gpu \
  data/one-object-fresh/test/images/00000.png \
  --device cpu --overlay outputs/detection-cpu.png

python predict_detector.py runs/one-object-gpu \
  data/one-object-fresh/test/images/00000.png \
  --device cuda --overlay outputs/detection-gpu.png
```

The inference script loads config.json and the trusted tensor state, reconstructs the network, applies the saved RGB/96-pixel preprocessing, enters evaluation mode, and moves the model and input to the selected device. It returns objectness probability and, when the threshold is met, the class name and box in original-image pixel coordinates. The optional overlay draws that box on the original image.

Both CPU and GPU use FP32 here. Small numerical differences are possible, so compare probabilities, boxes, and threshold crossings before claiming equivalence. A tiny single-image workload may not run faster on the GPU once transfer and launch overhead are included. Measure your actual latency and batching pattern.

Check what succeeded and what remains untested

During book preparation, the one-object dataset generator was executed, mask-to-box and empty-mask conversions were tested, and numerical IoU/detection-metric assertions passed. All new Python files compiled. PyTorch training and inference were not executed in the authoring environment, so no detection quality, GPU speed, or memory measurement is claimed.

Before a larger run, inspect several images and their derived boxes. Overfit a handful of clean examples, save and reload, then test an empty image and a fresh positive. If the model always emits a central box, inspect positional variation and the retained spatial grid. If it finds objects but returns poor boxes, inspect coordinate order, normalization, image resizing, and localization-loss weight.

The next segmentation project predicts a detailed mask rather than a rectangle. Keep that distinction concrete: a good enclosing box can contain much background, and a good class prediction says nothing about boundaries.

10 Project predict a mask then extract an outline

Now the input is still an image, but “what kind of image is this?” is no longer enough. You want the foreground's shape. The output must retain spatial information.

The synthetic exercise draws irregular foreground regions, occasional holes, varying illumination, and noise. It deliberately supplies a learnable color cue. Open a fresh terminal at the companion root, run `cd examples/tiny-ml`, and activate the neural environment you checked in the image-classification project. If you skipped that project, complete its explicit installation block first. Then run:

```
python make_shapes.py --out data/shapes --size 128 \
  --train 256 --val 64 --test 64 --seed 42
python mask_metrics.py
python train_segmenter.py --data data/shapes --out runs/segmenter-smoke \
  --epochs 1 --batch-size 2 --width 16 --device cpu
# After checking save and reload, use a new longer run:
python train_segmenter.py --data data/shapes --out runs/segmenter \
  --epochs 15 --batch-size 8 --width 16 --device cpu
```

For an installed compatible GPU environment, replace the last command's device with `--device cuda --amp`. Begin with a one-epoch smoke run and inspect the saved images before spending time on a longer run. The generator and mask-metric unit checks were executed on CPU; the neural training command was not executed in the authoring environment.

Classification detection semantic segmentation and instances

These outputs are not interchangeable:

- **Classification:** one label or set of labels for the whole image
- **Object detection:** an object class and a bounding box for each detected object; it localizes objects approximately
- **Semantic segmentation:** a class at each pixel; two touching objects of the same class may form one region
- **Instance segmentation:** a separate mask and identity for each object, even when neighboring objects share a class
- **Contour extraction:** convert an existing region boundary into a curve or polygon; this is often postprocessing of a mask

A box says where an object lies, not which pixels belong to it. It cannot recover an exact silhouette by itself. Semantic segmentation also cannot guarantee separate identities for touching foreground objects. If you need to count overlapping identical parts, label instances and choose an instance-capable model or a justified separation procedure.

The exercise is binary semantic segmentation: target values are background 0 and foreground 1. Files encode them as 0 and 255, but the dataset loader converts them back to floating-point 0 and 1. Each batch's target shape is `[batch, 1, height, width]`. The one-channel model output has the same shape, containing logits rather than binary masks.

A tiny encoder decoder with skip connections

TinyUNet downsamples twice to learn broader context, then upsamples twice to return to the original resolution. At each upsampling stage, a **skip connection** concatenates higher-resolution features from the encoder. These features help retain location information lost during downsampling. This is a deliberately smaller U-Net-like teaching network, not a reimplement of the original paper's exact architecture. [M07]

At width 16, the channel progression is 16, 32, and 64. --width scales that capacity. Doubling width roughly quadruples many convolution weight and compute costs because both input and output channel counts grow. Increasing image width and height together increases pixel count quadratically. Training memory also stores intermediate activations and gradients; a small parameter count does not imply small high-resolution activation memory.

Bilinear upsampling uses `align_corners=False`, and its target size comes from the corresponding skip tensor. This makes tensor sizes match rather than relying on a fragile assumption about division rounding. The final 1×1 convolution maps features to one foreground logit per pixel.

What a segmentation architecture needs from its labels

The scratch network needs aligned pixel labels, not only image-level class names or bounding boxes. A box-supervised task requires a different method; pretending every pixel inside a box is foreground teaches a different target. A pretrained encoder can reduce the amount of visual feature learning, but it does not supply your target's boundary convention. Fine structures, touching objects, holes, and difficult backgrounds must appear in labeled examples if they matter in deployment.

More layers may enlarge the receptive field but cannot recover omitted labels or distinguish instances when all instance identities were collapsed into one semantic mask. Higher resolution is often a more appropriate experiment than a much deeper network when the failure is a narrow edge. Estimate data needs through learning curves and labeled error slices, with independent scenes as the counting unit.

Choose a pixel loss that does not reward empty predictions

The example adds two terms:

```
loss = binary_cross_entropy_with_logits(logits, target)
      + mean_per_image_soft_dice_loss
```

Binary cross entropy penalizes incorrect pixel-level probability assignments. Its logits variant combines the sigmoid and logarithmic calculation stably. **Soft Dice** uses probabilities to measure foreground overlap before thresholding. In the script:

```
Dice = (2 * sum(probability * target) + 1)
      / (sum(probability) + sum(target) + 1)
Dice loss = 1 - Dice
```

The sums are per image, then averaged. The smoothing constant 1 keeps empty and tiny regions numerically manageable. It influences the loss more strongly for tiny objects; it is not a physical constant. Equal weighting of BCE and Dice is a starting choice. If foreground is

extremely rare, compare suitable class weighting or focal-style objectives, but judge the result on actual foreground and boundary errors rather than the loss value alone. [M05]

An all-background prediction can have excellent pixel accuracy when only 1% of pixels are foreground. Always include foreground overlap, empty-image behavior, and small-object errors. Include genuinely empty images in training and evaluation if the deployed camera will encounter them.

For K mutually exclusive semantic classes, use K logits per pixel and integer target maps with values from zero to $K-1$. For overlapping labels, use independent binary channels instead. If some pixels are unlabeled, mask them out of the loss; do not silently relabel them as background.

Keep image transformations and targets aligned

The dataset applies the same random horizontal flip and 90-degree rotation to image and mask. A transform applied to only one of them creates supervision for the wrong pixels. If you resize a categorical mask, nearest-neighbor interpolation preserves class IDs; bilinear interpolation invents fractional labels unless that soft-label behavior is explicitly intended. For an image, continuous interpolation is usually appropriate.

Real segmentation datasets require a labeling convention: does an outline include anti-aliased edge pixels, shadows, transparent regions, holes, and occluded sections? Are you labeling visible extent or inferred complete shape? Two annotators may disagree even when both are careful. Measure disagreement on a small independently labeled subset before demanding one-pixel model accuracy.

Split by original scene, video, subject, or acquisition session before making patches. Crops from the same original image should not leak across train and test. A giant image cut into thousands of adjacent tiles is not thousands of independent scenes.

High IoU is not exact outline accuracy

Intersection over union, or IoU, is overlapping foreground area divided by foreground area covered by either prediction or target. It is useful, but it is area-based. A one-pixel displacement of a 96×96 square gives IoU about 0.979. In an executed metric check, its exact-pixel boundary F1 was only 0.50. An application that needs a precise cutting outline should not call that “98% correct boundary.”

The supplied `mask_metrics.py` reports both IoU and a symmetric boundary F1. It extracts boundary pixels, measures how many predicted boundary pixels fall within a tolerance of a true boundary, measures the reverse fraction, and combines the two. Tolerance 2 means a two-pixel neighborhood under a square, Chebyshev-distance structuring element. This is not Euclidean distance and is not the paper's separately defined Boundary IoU metric. Both-empty masks score one; exactly one empty mask scores zero. Report that convention because datasets with many empty images can inflate mean scores. The Boundary IoU paper explains why boundary-sensitive evaluation complements regional overlap. [M08]

For a real outline product, report error in the coordinate system that matters: original-image pixels, millimeters after calibration, or another physical unit. Add boundary distance

summaries, small-object stratification, and checks for disconnected fragments and holes when relevant. A large region can dominate an aggregate metric while a narrow critical feature is missing.

From probability map to polygon

After training:

```
python predict_segmenter.py runs/segmenter/best.pt my_image.png \
  --out prediction --threshold 0.5
python contours.py prediction/mask.png --epsilon-px 1 \
  --out prediction/contours.json
```

Add `--device cuda` to `predict_segmenter.py` for GPU inference; the default is CPU. Both load the same state dictionary and saved width, convert the image to RGB float32 divided by 255, run evaluation/inference mode, sigmoid the output, and apply the requested threshold. The CUDA path moves inputs and weights to CUDA and returns the mask to CPU for PNG writing. Both inference paths remain FP32; the training `--amp` setting does not force inference precision.

The first script writes a probability visualization and binary mask at original input resolution. It does not resize the image. Very large images can therefore need substantial RAM; use a validated tiled strategy if required. The contour script requires OpenCV and was syntax-checked only; it is an optional downstream recipe, not a dependency of training.

Contour extraction uses `RETR_TREE` so the hierarchy can preserve holes, and `CHAIN_APPROX_NONE` before optional polygon simplification. Each output contour stores x/y points plus parent/child hierarchy. `epsilon-px=1` is the polygon approximation tolerance in mask pixels. It controls geometric simplification, not the neural model's uncertainty. Raising it removes vertices and can erase important details. `RETR_EXTERNAL` would discard interior holes. [M09]

A practical pipeline is probability map → threshold → optional validated cleanup → contour extraction → optional simplification → coordinate mapping. Threshold, minimum component size, hole filling, and smoothing are model-selection choices: tune them on validation, then freeze them before final testing. Blind hole filling is wrong for objects that genuinely have holes. Keeping only the largest component is wrong when multiple objects are legitimate.

If you resized, padded, or cropped an image, undo that geometry when returning the contour to the caller. A perfect mask in the wrong coordinate system is still a wrong result. A segmentation network also cannot restore subpixel detail that was destroyed by a low-resolution input.

Understand the segmentation controls and artifacts

Control	Default	Purpose
generator --size	128	Image side length; at least 32, divisible by four
generator --train/--val/--test	256/64/64	Independently generated examples in each split
--seed	42	Reproducible generator or training random state

Control	Default	Purpose
training --epochs	15	Maximum passes through training images
--batch-size	8	Images contributing to each update; lower first if out of memory
--width	16	Base number of feature channels
--lr	0.001	AdamW learning rate
--weight-decay	0.0001	Decoupled weight regularization
--device	cpu	CPU or CUDA execution
--amp	off	CUDA FP16 autocast plus gradient scaling; supported only with CUDA here
inference --threshold	0.5	Probability cutoff for foreground
contour --epsilon-px	1	Polygon simplification tolerance

The loop zeroes gradients, runs the forward pass, computes the loss, backpropagates, clips the gradient norm to 1, and updates with AdamW. In AMP mode, it unscales gradients before clipping. PyTorch's current namespace for the pinned API is `torch.amp.GradScaler`; the example avoids older deprecated `torch.cuda.amp` calls. [M12]

`best.pt` holds the lowest-validation-loss state dictionary and network width. `metrics.json` contains configuration, parameter count, training history, test metrics, elapsed time, and peak allocated CUDA memory if applicable. Eight input/probability/prediction/target groups support visual inspection. Allocated CUDA memory is not the complete device footprint; compare it with system-level observations before claiming a fit.

Training examples are procedural and much simpler than real camera data. Their purpose is to expose tensor shapes, paired transforms, output decoding, and evaluation. A strong synthetic score does not establish robustness to new lenses, reflections, clutter, textures, occlusion, or unknown classes.

When a detector or pretrained segmenter is the right next step

If your requested output is a class and rectangle for each object, use a detector. If it is a separate mask for each object, use an instance segmenter. The official torchvision detection tutorial demonstrates the dataset contract and fine-tuning a Mask R-CNN model; its example dataset is educational, not an automatic commercial data license. [M10]

A custom detection sample returns an image and a target dictionary containing boxes in `[x_min, y_min, x_max, y_max]` format, labels, and, for instance segmentation, per-instance masks. Preserve the model's conventions: torchvision detectors reserve label zero for background. Empty scenes need valid empty tensors, not invented background boxes. Detection training usually returns a dictionary of losses; sum the intended components and inspect each rather than assuming it behaves like image classification.

`fasterrcnn_mobilenet_v3_large_320_fpn` is an official compact-backbone option for a box-detection experiment. The API is versioned and the detection module's stability warnings should be read before production use. Start with a very small batch and representative resolutions on a 24 GB GPU; proposal counts and image content affect memory. Review pretrained-weight terms independently of the permissive torchvision source-code license. [M25] [M11]

Detection average precision depends on class, confidence ranking, and matching at specified IoU thresholds. AP at IoU 0.5 is not equivalent to COCO-style AP averaged over multiple thresholds, and neither proves exact edges. For segmentation transfer learning, a small pretrained encoder can be useful; retaining its normalization and adapting its output head is essential. Compare a frozen encoder, partial fine-tuning, and the scratch baseline before increasing model size.

11 Train a miniature language model from random weights

What this category does

A generative language model predicts a continuation of a token sequence. Useful application patterns include drafting a constrained message, completing a structured record, generating a response conditioned on evidence, and proposing a tool call that another program validates. A language model is useful when the output is a sequence with flexible wording or structure. If the output is one of a few fixed labels, a classifier may be simpler.

This project teaches the mechanics by training a very small byte-level transformer on original synthetic records. It will not become a general assistant. The expected learning target is a narrow text format and recurring relationships within that format. Its limited scope makes failures easier to inspect.

You have already trained a three-parameter classifier. The genuinely new elements are ordered byte sequences, shifted next-byte targets, and causal attention. Automatic differentiation, mini-batches, and tensor checkpoints build on the neural image projects you have already used. The input-to-target idea remains the same.

First generate the raw material

From the companion root, run:

```
python examples/tiny-transformer/make_data.py --output data/tiny-text
```

The generator creates records about colored household objects in rooms. A record is followed by questions and answers about that record. The generated text is original and deterministic. The training split contains 800 entity IDs, validation contains 100 different IDs, and test contains another 100. No entity is assigned to two splits.

Open train.txt before proceeding. A typical block looks like this, with values determined by the generator:

```
Record item0042: the blue lamp is in the office. Its switch is on.
Question: Where is item0042? Answer: the office.
Question: What color is item0042? Answer: blue.
```

The exact record above illustrates the format rather than claiming a particular generated item has those values. Inspect your actual file. The manifest records split sizes, seeds, and SHA-256 hashes. In the prepared CPU generator check, the training file contained 132,852 bytes. This is intentionally tiny. It is enough to exercise the pipeline, not enough to pretrain a broadly capable model.

Why this architecture needs this kind of data

The model accepts integer byte IDs and predicts the next byte. Every adjacent pair in the text provides a supervised target. It does not directly receive a label saying 'understand room location'. Instead, the loss rewards continuations that match the records and answers. If you want reliable question answering, you must separately test whether it uses the relevant earlier record rather than repeating frequent answers.

The input context is 128 bytes by default. A target depending on information farther away cannot be solved through that context window. If a record and question become separated by a long paragraph, the model may lack the necessary evidence. This connects architecture to data design: examples must expose the dependencies you want the model to learn within the representation it can use.

The training program samples fixed-length windows from each split's continuous byte stream. A window can cross a record boundary within that split. That is acceptable for this small continuation exercise because the stream format includes separators and no split boundary is crossed. For tasks where documents must be independent, use explicit boundary tokens and appropriate masking rather than blindly joining unrelated records.

Install only the small project environment

The reference API pin is torch 2.8.0 with Python 3.11. This older fixed release is intentional for the educational implementation; it is not a claim about the newest PyTorch version. Use the official PyTorch installation selector or previous-version instructions to select the correct CPU or CUDA wheel for your platform. The release and versioned attention API are documented by PyTorch. [\[F10\]](#) [\[F11\]](#)

Create a separate virtual environment, install the matching official wheel, and verify the version and device. The project requires no Transformers package, no tokenizer download, and no remote model code. Do not copy a CUDA-specific wheel command to a machine with incompatible hardware or driver.

For a Linux or Windows Python 3.11 environment, the official 2.8.0 CPU wheel can be installed with the first command below. The second is the CUDA 12.6 build for a compatible NVIDIA driver and GPU; choose one, not both. A newer GPU may require a different supported CUDA build. The official previous-version page lists the alternatives. [\[F15\]](#)

```
# CPU environment
python -m pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cpu

# Separate compatible NVIDIA CUDA environment
python -m pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cu126
```

Run the local model checks first:

```
python examples/tiny-transformer/check_model.py
```

That script tests output shapes, future-token isolation, finite gradients, a real parameter update, and save/load equivalence. It is provided for execution in an environment with PyTorch. During preparation of this edition, the source was syntax-checked but PyTorch execution was unavailable; the validation appendix states that boundary explicitly.

Run a smoke test on the CPU

```
python examples/tiny-transformer/train.py \
  --data data/tiny-text --output runs/byte-smoke \
  --device cpu --steps 20 --warmup 2 \
  --width 32 --layers 2 --heads 4 \
  --context 32 --batch-size 2 --eval-every 10
```

This reduces the model and sequence sizes so a basic CPU test is practical. The first run is a correctness check. Look for a finite loss, a printed parameter count, validation output, and checkpoint.pt. Do not judge the model's usefulness after twenty updates.

The script never silently substitutes CPU for a requested unavailable GPU. A device error should be fixed directly. Likewise, BF16 requires an explicitly compatible CUDA device. These checks prevent a long run from proceeding under an unintended environment.

Move the same pipeline to the GPU

After the CPU checks pass, a small GPU experiment can use the default architecture:

```
python examples/tiny-transformer/train.py \
  --data data/tiny-text --output runs/byte-gpu \
  --device cuda --steps 1000 --warmup 50 \
  --context 128 --width 128 --layers 4 --heads 4 \
  --batch-size 8 --accumulation 2 --eval-every 100
```

Add `--bf16` only after checking device support. The script keeps model parameters and AdamW states in FP32 and uses BF16 autocast for selected operations when requested. This is a clear numeric policy, not a claim that every tensor occupies two bytes. The default architecture has about 0.84 million distinct parameters by direct shape accounting, well below one billion.

The batch is eight windows of 128 bytes. Accumulating two microbatches presents 2,048 target bytes per optimizer update. One thousand updates therefore present about 2.048 million training bytes, sampled with replacement from the small training stream. That is repeated exposure, not two million unique bytes.

The code logs throughput including evaluation time, so it is an end-to-end run measure rather than a pure kernel benchmark. It also logs peak allocated and reserved GPU memory. Measure your own results; no particular RTX throughput or wall-clock duration is asserted here.

Understand the model one piece at a time

The vocabulary contains 256 byte values. A learned embedding maps each ID to a vector of width 128. A learned positional table adds information about the position inside the current window. Four transformer blocks repeatedly combine earlier information through causal attention and transform it through a feed-forward network. A final normalization and vocabulary projection produce 256 logits for each position.

The input and output vocabulary weights are tied, meaning the same parameter matrix is reused. The model uses learned positions and ordinary multi-head attention for clarity. It is not an implementation of Qwen, Gemma, Granite, or a modern production architecture. Those families differ in tokenization, normalization, position handling, feed-forward choices, attention layout, and other details.

PyTorch's scaled-dot-product attention receives `is_causal=True`. During evaluation, `dropout_p` is explicitly zero. This matters because the functional attention operation applies the dropout probability it is passed rather than automatically reading the surrounding module's evaluation state. [F11]

The next chapter explains the same components more deeply. First connect each component to the tensors printed by your own small run. Architecture becomes easier to understand when it names something you have already used.

Every training control in this project

`--context` sets the number of input bytes per window and the size of the learned position table. `--width` sets representation size. `--layers` sets the number of transformer blocks. `--heads` divides the hidden width into attention heads; width must be divisible by heads. `--dropout` randomly suppresses selected intermediate contributions during training and is disabled during evaluation.

`--batch-size` is windows per microbatch. `--accumulation` is microbatches per update. `--steps` is optimizer updates, not microbatches. `--learning-rate` is the peak update rate, default 0.0003. `--weight-decay` is AdamW's decoupled shrinkage strength, default 0.1. The educational implementation applies it uniformly; production recipes often use parameter groups that exempt biases and normalization parameters.

`--warmup` ramps the learning rate over early updates. Afterward a cosine schedule decreases it to one tenth of the peak. `--eval-every` controls evaluation and checkpoint intervals. `--seed` controls initialization and sampling seeds. `--device` selects CPU or CUDA. `--bf16` enables compatible CUDA autocast. `--data` and `--output` select input and artifact directories. `--resume` loads an existing trusted checkpoint and requires the checked training configuration and dataset hashes to match.

The optimizer also sets betas to 0.9 and 0.95, epsilon to 1e-8, and clips the global gradient norm at 1.0. These are explicit starting choices for this experiment. The training-control chapter explains their meanings. They are not a recommendation to use the same values for every fine-tuning job.

Evaluate the behavior you actually want

The logged validation objective is natural-log loss per byte, accompanied by bits per byte. Lower is better for the byte-prediction objective. Do not compare byte-model perplexity directly with the perplexity of a model using a different subword tokenizer. The prediction units differ.

Evaluate a final selected checkpoint on the test file only after choosing the procedure:

```
python examples/tiny-transformer/use_model.py \
  runs/byte-gpu/checkpoint.pt \
  --device cpu --evaluate-file data/tiny-text/test.txt
```

Then create fresh records with object IDs and values not present in training. Put the necessary fact and question within the context window. Score whether the answer matches the provided fact. Compare against always choosing the most common room and against a deterministic

parser for this artificial format. A transformer is educational here; a parser would be the practical solution if this exact rigid format were the entire real task.

Inference on the CPU and GPU

The same trusted checkpoint can be loaded on either device:

```
python examples/tiny-transformer/use_model.py \
    runs/byte-gpu/checkpoint.pt --device cpu \
    --prompt "Record item9999: the red lamp is in the hall."

python examples/tiny-transformer/use_model.py \
    runs/byte-gpu/checkpoint.pt --device cuda \
    --prompt "Record item9999: the red lamp is in the hall."
```

The inference program reconstructs the exact saved architecture, loads the state dictionary, calls eval, converts the UTF-8 prompt into byte IDs, moves tensors to the selected device, samples one next byte at a time, and decodes bytes back to text. Invalid UTF-8 sequences are replaced on decoding, which is a visible limitation of an immature byte model rather than a hidden text-cleaning success.

CPU and GPU use the same learned artifact but can differ slightly in floating-point results. Sampling can amplify small probability differences, so identical seeds do not guarantee identical text across devices. The tiny architecture may not benefit much from a GPU because launch overhead is significant. Larger models and batches generally change that balance. Measure latency for your intended request size.

This educational generator does not implement a key-value cache, so it recomputes the retained context at each generated byte. It also crops to the learned context length. Those are deliberate simplicity choices and become performance limits when scaling. Do not benchmark it as if it were an optimized production serving engine.

Checkpoint recovery and limits

checkpoint.pt contains weights, optimizer state, configuration, update number, and random-number-generator state. Saving uses a temporary file followed by a rename to reduce the chance that an interrupted write replaces the last complete checkpoint. Resume only trusted local checkpoints; serialized model files from unknown sources deserve security scrutiny even when safer loading modes are available.

The script checks key configuration fields and data hashes before resuming. It does not claim exact reproducibility across different PyTorch releases, operating systems, GPU architectures, or kernels. Its purpose is to make the important state explicit and reject obvious accidental changes.

Exercises

Use check_model.py to see how changing future input bytes should leave earlier logits unchanged. Explain why this is stronger evidence about causality than a low loss.

Train two small configurations with the same token-presentation budget. Double width in one and double updates in the other, keeping other choices controlled. Compare validation loss,

fresh-record accuracy, elapsed time, and memory. Report the tradeoff rather than choosing a winner by parameter count.

Replace the synthetic corpus with a small collection of text you own or are licensed to use. Preserve document-level splits. State what behavior the corpus can plausibly teach and what it cannot. A collection of maintenance manuals may teach terminology; it does not automatically teach a conversational format or reliable tool use.

12 How a language model becomes a prediction machine

From text to token identifiers

A language model does not receive a paragraph in the same way a person sees it. Its tokenizer converts text into a sequence of integer IDs. Depending on the tokenizer, an ID might represent a word, part of a word, punctuation, whitespace, a byte, or a special control marker. The vocabulary is the set of available IDs.

Token count is not word count. The same sentence may have different token counts under different tokenizers. Languages, code, unusual names, and specialized notation can tokenize very differently. When budgeting a dataset or context window, use the actual model's tokenizer. A rough words-to-tokens estimate is useful only before you can measure.

A tokenizer is part of the model interface. If you train weights with one token-to-ID mapping and later use another, the model may interpret each ID incorrectly. Adding tokens changes the embedding and output layers unless the framework provides a compatible mechanism. Preserve the tokenizer, its special-token definitions, and the chat template with every model artifact.

The tiny transformer you just trained uses byte IDs from 0 through 255. This makes the representation easy to inspect and avoids training a separate tokenizer. It is less efficient for many natural-language tasks than a good subword tokenizer. That tradeoff is appropriate for an educational model, not automatically for a larger pretraining run.

Next token prediction creates many examples

Imagine the token sequence representing 'the cat sat'. During causal language modeling, one position predicts the next token using earlier positions. An input sequence of IDs [12, 34, 56] can be paired with targets [34, 56, 78], where 78 is the token immediately after the final input token. This shifted arrangement lets one forward pass calculate several next-token losses.

A model cannot be allowed to inspect the target token through a future input position while predicting it. A causal attention mask prevents each position from attending to later positions. If the mask is wrong, training loss can look excellent because the answer leaks into the input computation. Generation will then fail because future tokens are unavailable at real prediction time.

A next-token objective teaches a statistical continuation rule. A pretrained model may acquire broad patterns useful for many tasks because predicting text requires modeling many relationships in that text. Nevertheless, a low average next-token loss does not directly establish factual correctness, tool-use reliability, or instruction-following quality. Those need task-specific evaluations.

Causal attention allows only the present and past

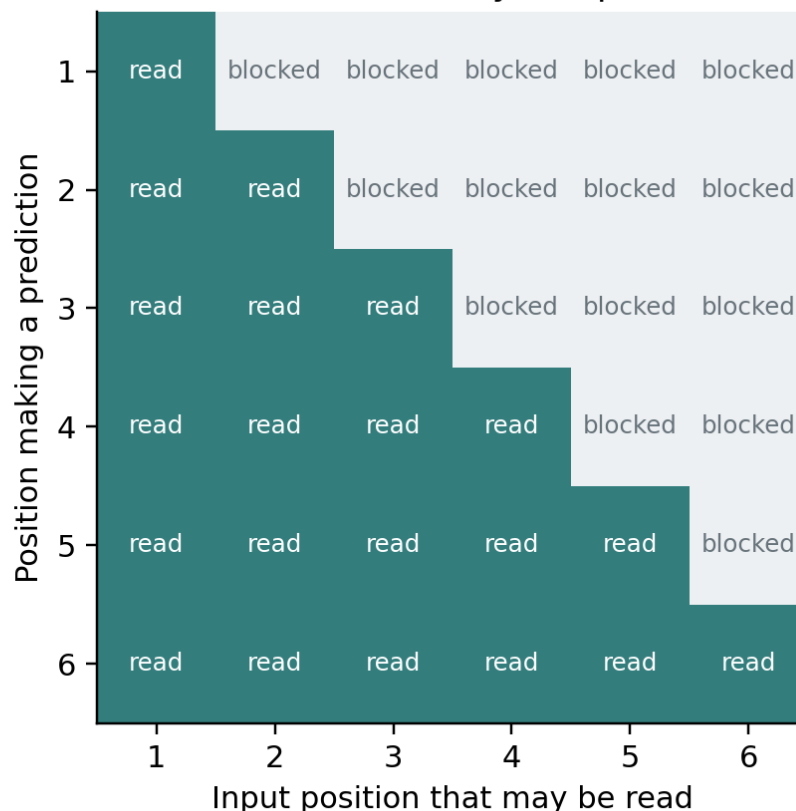


Figure 4 Each position may read the present and earlier inputs. The target is the next token, so future positions must be blocked.

The embedding table turns IDs into vectors

An embedding layer is a trainable lookup table. If the vocabulary contains 32,000 tokens and each token is represented by a vector of 512 numbers, the table contains 16,384,000 parameters. Looking up token 34 returns row 34. At initialization the vectors may be random; training changes them so the later computation can use them.

An embedding vector is not a sentence definition written in numbers. Individual dimensions do not generally have simple names. Meaning comes from how the vectors participate in learned transformations. Similarity can be useful, but a raw token embedding and a sentence embedding trained for retrieval are different objects with different intended uses.

The parameter count of an embedding table can be large relative to a small model. A model with modest transformer blocks and an enormous vocabulary may spend much of its capacity and memory on embeddings. Weight tying can reuse the input embedding matrix for the final vocabulary projection, reducing distinct parameters. It imposes a structural choice; it does not merely compress a file.

Order must be represented somehow

Without an order mechanism, a set of token vectors does not tell the model whether a sentence says 'the dog chased the cat' or 'the cat chased the dog'. Architectures add positional information in different ways. A simple educational model can use learned position embeddings: one vector for position zero, another for position one, and so on.

Many modern language models use rotary position embeddings or other schemes. You do not need to implement every scheme to begin fine-tuning, but you must respect the chosen model's supported context and positional configuration. Changing a maximum-length field does not magically give a model reliable understanding at a much longer context. Positional behavior, training distribution, attention cost, and evaluation all matter.

Attention lets a position combine information

In self-attention, each position produces a query vector, a key vector, and a value vector through learned transformations. Query-key comparisons yield scores describing which available positions may be useful. Softmax normalizes the scores into weights, and a weighted combination of value vectors produces an updated representation.

A concrete analogy is a question and a set of indexed notes, but the computation is numeric and learned. The network is not explicitly asking a human-readable question at every head. Attention weights are also not a complete explanation of why a final answer was produced.

Multiple attention heads let a block compute several such combinations. The outputs are combined and projected back into the model's hidden dimension. A feed-forward network then transforms each position's representation. Residual connections and normalization connect these components into a trainable block. Stacking blocks repeats the process at increasing representational depth. The original Transformer paper introduced the attention-centered architecture; current families modify many details. [F03]

Ordinary attention can materialize a score array whose size grows with sequence length squared. Doubling sequence length can therefore be much more expensive than doubling a small input file. Efficient attention implementations change memory behavior by avoiding unnecessary materialization, but they do not make long-context computation free.

The output head decides what is predicted

A causal language model normally ends with a vocabulary projection: each position's hidden vector becomes a score for every token in the vocabulary. The final position's scores can be converted into a distribution for the next token. Sampling or choosing a token extends the sequence, and the process repeats.

This output head is one reason a language backbone can be adapted to a different task. A classifier may read a pooled representation and predict one of a few labels. A token classifier may predict a label at every position. An option-pointer model may score a set of provided choices rather than generate arbitrary vocabulary tokens. These are architectural changes to what is predicted, not merely different wording in a prompt.

The task-specific chapters explain how to match the head, target, and loss. A backbone that was pretrained on language can provide useful representations even when the new output is not prose. Full-parameter training updates the backbone too; head-only training leaves it fixed. These are different experiments and should be described honestly.

Training and generation use different controls

During training, a causal language model can process many known target positions in parallel. During ordinary generation, later tokens depend on earlier generated tokens, so output is produced sequentially. Temperature and top-p are generation controls, not replacements for learning rate and batch size.

Temperature rescales logits before sampling. Lower positive temperature concentrates probability more strongly on favored tokens; higher temperature spreads it. Greedy decoding selects a highest-scoring token instead of sampling. Top-k keeps a fixed number of candidates; top-p keeps a probability-mass set. These choices change observed output without changing weights.

A key-value cache stores useful attention state from earlier generated tokens so it need not be recomputed at every new token. It consumes inference memory, especially with long context and multiple concurrent requests. Training often disables this cache because it is not needed in the same way and can conflict with memory-saving strategies. Never infer training-memory requirements from a generation-only memory reading.

Base models and instruction models

A base language model is primarily trained for continuation. An instruction model has usually undergone additional training intended to make it respond usefully to conversational or task instructions. The terms do not guarantee a particular quality level or safety behavior. Read the model card and evaluate the checkpoint you actually plan to use.

Chat formatting is an interface contract. Role markers, message boundaries, end-of-turn tokens, and generation prefixes tell the model how a conversation is arranged. Writing a custom format that looks reasonable to a human can conflict with the pretrained model's expected format. Use the official tokenizer's chat template unless your experiment intentionally retrains the interface.

When a training example includes a user message and an assistant response, you may want the loss only on the response. The prompt still influences the model through attention, but its tokens are not necessarily targets you want to reinforce. This requires a correct loss mask, distinct from an attention mask. The fine-tuning chapters show why this distinction is essential.

A useful architecture reading checklist

When inspecting a model, identify the tokenizer and vocabulary size, total parameter count, hidden size, number of blocks, attention type, number of attention and key-value heads, positional scheme, context claims, output head, numeric dtype, license, and exact revision. Do not memorize all of these values. Learn what changing each could affect.

For a mixture-of-experts model, distinguish total resident parameters from the subset activated for a token. For a hybrid recurrent-attention model, do not assume every block obeys an ordinary attention mask. For a multimodal model, distinguish text-only backbone size from the vision or audio components and projection layers. Marketing names are not memory accounting.

Exercises

Write a five-token input and its shifted next-token targets. Mark which target positions are legal for each input position to inspect. Explain how a future-token leak would make the training score misleading.

Calculate the parameter count of a 256-token vocabulary with 128-dimensional embeddings. Then compare it with a 50,000-token vocabulary at the same dimension. The results are 32,768 and 6,400,000 parameters, before any transformer block.

Explain the difference between changing sampling temperature, training an adapter, and replacing the output head. Only the latter two change the trained model artifact; each changes a different part of the system.

13 Read and control the training loop

Inspect one update before tuning a hundred settings

After your first neural run, open its training loop. Find where the batch is created, where the model runs, where the loss is calculated, where backward is called, and where the optimizer updates weights. Print the shapes and dtypes once. Print how many parameters require gradients. Those few checks answer more useful questions than changing a dozen unfamiliar flags.

The byte-transformer loop uses fixed-length windows, so each microbatch contains the same number of target positions. The loss is averaged over those positions. Dividing by the accumulation count makes the accumulated gradient an average across equal-size microbatches. This simple accounting becomes more subtle when sequence lengths and supervised-token counts differ.

Suppose one microbatch contains 100 supervised tokens and another contains 1,000. Averaging their mean losses equally gives each microbatch equal influence, so each token in the shorter batch counts ten times as much. If your objective is an average over supervised tokens, accumulate summed losses with a correct total-token denominator or use a trainer whose normalization matches that objective. State what is being averaged.

Batch size is a statistical and systems choice

A batch is a group of examples used together. A microbatch is the group processed in one forward/backward pass. The effective batch is the group contributing to one parameter update. On one GPU with equal microbatches, effective examples = microbatch size times accumulation steps.

For variable-length text, examples per update can be misleading. Record input tokens and supervised tokens as well. Two batches of eight examples can differ greatly if one contains short labels and the other contains long answers. Packing multiple short sequences into a longer block can improve utilization, but only if the boundary and attention behavior match the intended objective.

A larger effective batch can make gradients less noisy, but it can also change optimization behavior and the number of updates at a fixed token budget. Do not increase batch size, learning rate, and training duration together and then attribute a change to one of them. Start with a feasible microbatch, choose an effective token budget per update, and compare controlled changes.

Learning rate and schedule

The learning rate scales parameter updates. In a pretrained model, too large a rate can destroy useful behavior quickly. In a randomly initialized model, too small a rate can make learning inefficient. The appropriate range depends on architecture, optimizer, batch, numeric precision, initialization, data, and which parameters are trainable.

Warmup gradually increases the learning rate during early updates. It can help avoid abrupt early changes while optimizer statistics develop. A schedule then changes the rate over time, such as linear decay or cosine decay. A schedule is usually defined over optimizer steps, not the number of microbatches. If you change accumulation, check whether the scheduler still covers the intended number of updates.

A resume must preserve scheduler position and its original total-step plan if exact continuation is intended. Extending a run from 1,000 to 2,000 steps while recomputing a cosine schedule can change the rate at the same nominal step. That may be a valid new experiment, but it is not an invisible continuation of the old one.

Do a short learning-rate comparison using a fixed validation procedure. If loss becomes nonfinite or jumps sharply, stop and inspect rather than allowing the run to continue indefinitely. A smaller rate may help, but first rule out malformed inputs, empty supervision, bad masks, or numeric problems.

Optimizer parameters

AdamW's `beta1` controls the smoothing of the first-moment estimate, a running estimate related to gradient direction. `beta2` controls smoothing of the second-moment estimate, related to squared gradient magnitude. Values close to one retain longer history. `epsilon` is a small stabilizing term in the denominator. These values affect update behavior; they are not accuracy thresholds.

Weight decay applies a shrinking tendency to parameters. It can act as regularization, but its effect depends on learning rate and which parameters receive it. Many recipes exclude bias and normalization parameters from decay. Copying only the headline decay value while changing parameter groups is not the same experiment.

Gradient clipping limits the norm or magnitude of a gradient before the update. Global-norm clipping preserves direction while reducing the size of an unusually large gradient vector. Log the norm before clipping; constant heavy clipping can signal an unstable setup. Clipping is a guardrail, not proof that an otherwise bad learning rate is safe.

In FP16 mixed precision with a gradient scaler, unscale gradients before clipping. Keep scaling consistent across accumulated microbatches and update the scaler at effective-batch boundaries. PyTorch's AMP examples spell out these ordering requirements. BF16 often uses a different policy, so do not copy an FP16 scaler loop mechanically. [\[F12\]](#)

Precision policy and autocast

Autocast chooses lower-precision computation for supported operations while preserving higher precision where the framework's policy requires it. It does not necessarily change the dtype of stored model parameters or optimizer state. The byte-transformer and pointer examples explicitly keep FP32 parameters and AdamW state while optionally using BF16 autocast on compatible CUDA hardware.

FP16 and BF16 both use two bytes, but divide their bits differently between range and precision. BF16 has a much wider exponent range than FP16 and lower significand precision. A model that works with one can fail or behave differently with the other. FP16 training often

uses gradient scaling to keep small gradients representable; the scaler and clipping order must be correct. Do not assume a GPU supports a format merely because its model name contains RTX. Check the actual device and backend.

A quantized four-bit frozen base is a separate storage strategy. It does not mean that all activations, adapters, gradients, or optimizer states are four-bit. Write down the dtype of each state category when budgeting or comparing recipes. [F12]

Dropout and other regularization

Dropout randomly suppresses selected activations during training according to a probability. It should be disabled during ordinary evaluation. Its purpose is not to make outputs random at inference. A model with dropout left active can appear inconsistent even under deterministic decoding.

Data augmentation creates modified training inputs that should preserve the intended target. Flipping an image may preserve an object label but invalidate left-versus-right labels. Cropping can remove the target object. Adding noise to audio may preserve event identity up to a point but destroy intelligibility. An augmentation is valid only when its label-preservation assumption is valid.

Early stopping chooses a checkpoint when validation performance stops improving under a predefined rule. Patience is the number of evaluations tolerated without sufficient improvement. Choose the monitored metric and minimum meaningful change in advance. If your validation set is small and noisy, reacting to every tiny fluctuation can select unstable results.

Regularization cannot compensate for a fundamentally incorrect target. A model trained on mislabeled masks will not learn the intended boundary because dropout is well chosen. Inspect data before making optimization more elaborate.

Attention masks and loss masks solve different problems

An attention mask controls which input positions can influence each representation. A causal mask prevents future-token access. A padding mask excludes nonexistent padded input positions from attention. A custom branch mask may isolate independent questions sharing a context.

A loss mask controls which predictions contribute to the training objective. In assistant-only fine-tuning, prompt positions may be visible to attention while excluded from loss. Padding targets are commonly assigned an ignore index such as -100 for cross-entropy. That value is a convention of the loss implementation, not a token that should be inserted into the actual input vocabulary. [F13]

A training batch can have a correct attention mask and an incorrect loss mask. For example, it may teach the model to reproduce user prompts rather than only the assistant response. It can also contain no supervised tokens after truncation, producing meaningless or nonfinite loss. Count valid target positions and inspect decoded supervised spans.

Do not blindly set all occurrences of an end-of-sequence token to ignored when padding uses the same ID. Real end-of-sequence targets may then disappear from supervision, weakening stopping behavior. Construct masks from actual padding positions and the intended response spans, not from a simplistic equality test that erases legitimate tokens.

Packing and truncation are modeling decisions

Packing combines short examples to use a context window efficiently. If unrelated examples can attend to each other, the training distribution includes cross-example context that may not exist in deployment. Some objectives allow a continuous stream; others need boundaries or block-diagonal attention. Know which your trainer implements.

Truncation discards tokens beyond a length limit. It can remove the answer, the tool schema, or the evidence needed to answer. A model trained on such examples may appear to learn impossible tasks. Log how many records are truncated, from which side, and how many supervised tokens remain. Reject or deliberately transform examples when the task would no longer make sense.

Padding extends shorter sequences to a common length with placeholder positions. It adds computation unless the implementation handles variable lengths efficiently. Bucketing similar lengths can reduce waste. However, a padding optimization is not worth a silent change to target semantics.

Epochs steps and stopping

An epoch means one pass through a defined dataset, usually after shuffling. Streaming or weighted sampling may not have a simple epoch interpretation. Steps count parameter updates. Samples and tokens count presentations. Report the quantities that actually define your experiment.

If a dataset has 10,000 examples and an effective batch of 20 with no dropped remainder, one epoch is about 500 updates. With different-length examples, the token count per epoch is another useful quantity. If sampling with replacement, a given example may appear more than once before another is seen.

More epochs can improve fit and then worsen generalization. A small instruction dataset repeated many times can teach formatting quickly while causing memorization or forgetting. Preserve checkpoints and compare outputs. Do not assume the last checkpoint is best because it consumed the most computation.

A controlled tuning sequence

First verify one-batch overfitting on a tiny clean subset. The model should be able to reduce loss substantially when repeatedly shown the same examples, unless the objective or architecture makes that impossible. Failure here points to pipeline, gradient, target, or capacity issues. Success proves only that the model can fit those examples.

Next run a short representative baseline configuration. Change one major factor at a time: learning rate, data quality, number of updates, trainable modules, or context length. Record the

hypothesis for each change. Prefer a small number of interpretable comparisons over a large blind sweep that you cannot afford to evaluate properly.

Finally select the candidate using validation metrics and guardrails, then run the final test once. Save the failed experiments' summaries too. Knowing which assumptions failed is part of the reusable result.

Exercises

A run has microbatch size 2 and accumulation 8. It logs 300 optimizer steps. How many example presentations occurred if every microbatch is full? The answer is 4,800. Explain why that does not tell you the number of unique examples.

Create two artificial microbatches with different numbers of supervised tokens. Calculate the difference between averaging their mean losses and averaging all token losses. Decide which objective your project intends.

Print one example's attention mask and supervised target positions. Explain why the model may attend to a user question while receiving no loss for reproducing that question.

14 Make a run reproducible and recoverable

Save the experiment rather than only the weights

A weight file is not a complete experiment. You also need architecture, tokenizer or feature mapping, preprocessing, data version, train/validation/test assignments, optimizer configuration, training duration, and inference settings. Without these, a future version of you may have a model that cannot be loaded correctly or evaluated fairly.

For each run, save a resolved configuration rather than only command-line overrides. If a default changes in the library, the resolved value should remain visible. Record Python and package versions, GPU model, driver, numeric policy, attention backend when known, code revision, model revision, and hashes of input splits.

A model repository name can point to changing contents. Resolve and retain a commit revision. A GitHub branch is similarly mutable. A tagged release is better for reproducibility, and an immutable commit hash is stronger evidence of exactly what was used. For a published training process, archive the relevant configuration and record local modifications.

Randomness has several sources

Initialization, shuffling, dropout, augmentation, sampling, and some GPU operations involve randomness or nondeterministic execution. Set the seeds used by Python, NumPy when present, PyTorch, and data-loader workers as appropriate. A dedicated random generator for dataset sampling can make its state easier to preserve.

A seed does not guarantee identical results across software releases, devices, or platforms. Some algorithms are nondeterministic, and floating-point order can change. PyTorch explicitly cautions that complete cross-platform reproducibility is not guaranteed and that deterministic alternatives can cost performance. [\[F14\]](#)

For a serious comparison, repeat the selected small experiment with more than one seed when affordable. If a claimed improvement disappears under a different initialization or shuffle, report its instability. For a first correctness exercise, deterministic fixed seeds are useful because they reduce distractions.

Checkpoints serve different purposes

An inference checkpoint contains enough to make predictions: weights, architecture configuration, tokenizer or preprocessing, and any required adapter or head metadata. A training-resume checkpoint additionally needs optimizer state, scheduler state, update number, gradient-scaler state where used, and relevant random and sampler states.

An adapter checkpoint ordinarily does not contain the base model. Record the exact base identity and revision. Loading an adapter on a similarly named but different checkpoint may fail or silently change behavior. A merged export is a separate artifact with its own validation requirements.

Checkpoint interval is a tradeoff. Saving too often consumes time and disk; saving too rarely risks losing work. Estimate the amount of recomputation you are willing to tolerate and choose an interval accordingly. Keep a last recoverable checkpoint and selected evaluation checkpoints. A retention policy should not delete the only known-good artifact before a replacement has been checked.

Practice interruption on a tiny run

Start a short run that saves twice. Stop it after the first save, resume, and verify that the next recorded step continues appropriately. Compare configuration and data hashes. Confirm that an intentionally mismatched architecture or dataset is rejected.

A resumed run may differ from an uninterrupted run if data-loader state, random generators, or batch order were not restored. Be honest about that limit. 'Resumes without crashing' and 'continues the identical optimization trajectory' are different claims.

Use atomic or transactional saving when possible: write to a temporary location, complete the write, then replace the designated checkpoint. Check free disk space. A full filesystem can produce a truncated artifact after hours of otherwise successful training.

Logs should answer operational questions

At minimum, log update number, training loss, learning rate, evaluated validation metric, elapsed time, and throughput with clear units. For GPU work, include peak memory. Useful additional signals include gradient norm, supervised-token count, skipped nonfinite updates, truncation count, and examples per class or source.

Do not write every private training example into a public experiment tracker. Some frameworks enable external reporting by default. Read the configuration for `report_to`, `push_to_hub`, `telemetry`, and `upload` behavior. Disable unneeded external reporting for a local experiment, and obtain appropriate permission before transmitting private data or model artifacts.

Store a small, sanitized set of fixed qualitative evaluation outputs alongside metrics. Numbers can show that something changed; outputs often explain what changed. Keep the exact prompt, seed, decoding settings, and reference answer with each sample.

A minimal run ledger

Use a record with fields for run name, question being tested, baseline, data version, model revision, code revision, environment file, configuration path, start and end time, status, measured memory, measured throughput, primary metric, guardrails, selected checkpoint, and next decision.

A failed run still receives a record. State whether it failed before loading, during the first forward pass, during backward, at the optimizer step, during evaluation, or during export. That stage narrows the diagnosis. Do not erase failures and then claim the workflow was effortless or fully tested.

Exercises

Take one checkpoint and load it in a fresh process with no variables left from the trainer. If you cannot reconstruct preprocessing and output decoding from saved files, add the missing metadata.

Change a dataset file by one character and confirm its hash changes. Then explain why a filename is not a reliable dataset version.

Review a public training script for external reporting, automatic uploads, downloads, remote-code execution, overwrite behavior, and unpinned dependencies before running it. A published example is evidence to inspect, not authority to execute everything it contains.

15 The real memory and compute budget

Begin with quantities rather than model names

A billion parameters means a billion stored values in the model, not a billion examples, operations, or tokens. The memory needed for those values depends on their representation. FP32 uses four bytes per value. FP16 and BF16 use two. An idealized eight-bit value uses one, and an idealized four-bit value uses half a byte. Actual quantized formats also require scales, metadata, and sometimes unquantized tensors.

GB and GiB are different units. One GB is one billion bytes. One GiB is 1,073,741,824 bytes. GPU utilities and product descriptions may use labels inconsistently. The safest comparison begins with bytes reported by the actual device and converts using a stated divisor. Throughout worked calculations, GB means decimal bytes and GiB means binary bytes.

For one billion parameters, FP32 weights alone occupy about 4 GB, or 3.73 GiB. BF16 weights alone occupy about 2 GB, or 1.86 GiB. These are storage calculations, not complete training estimates. They omit gradients, optimizer state, activations, working buffers, allocator behavior, and framework overhead.

Full training needs more than weights

A simple FP32 AdamW training budget includes four bytes for each parameter, four for its gradient, and eight for two FP32 optimizer-state values. That is sixteen bytes per trainable parameter before activations and other costs. The table is arithmetic under that exact assumption.

Trainable parameters	Weights only FP32	Parameters gradients and Adam states	Same persistent total in GiB
10 million	0.04 GB	0.16 GB	0.15 GiB
100 million	0.40 GB	1.60 GB	1.49 GiB
500 million	2.00 GB	8.00 GB	7.45 GiB
1 billion	4.00 GB	16.00 GB	14.90 GiB
3 billion	12.00 GB	48.00 GB	44.70 GiB

This explains why a sub-billion-parameter full-training project can be a reasonable experiment on 24 GB while ordinary three-billion-parameter FP32 AdamW training is not. The one-billion row still has to leave room for everything else. A long context, large batch, large vocabulary logits, or inefficient implementation can exhaust the remaining memory.

Mixed precision is not a single memory formula. One implementation keeps FP32 parameters and optimizer states while autocasting selected operations. Another retains low-precision weights plus FP32 master weights. Gradients and optimizer states may use different dtypes. A commonly cited mixed-precision estimate is eighteen bytes per parameter, but it describes a

particular inventory of copies, not a law applying to every PyTorch configuration. Inspect the actual parameter, gradient, and optimizer-state dtypes. [F04]

An eight-bit optimizer reduces the storage of some optimizer states, but does not automatically quantize the model, activations, or gradients. A four-bit frozen base with trainable adapters is different from full-parameter four-bit training. QLoRA ordinarily updates adapter parameters while the quantized base remains frozen. An inference quantization format is not automatically supported by a training optimizer.

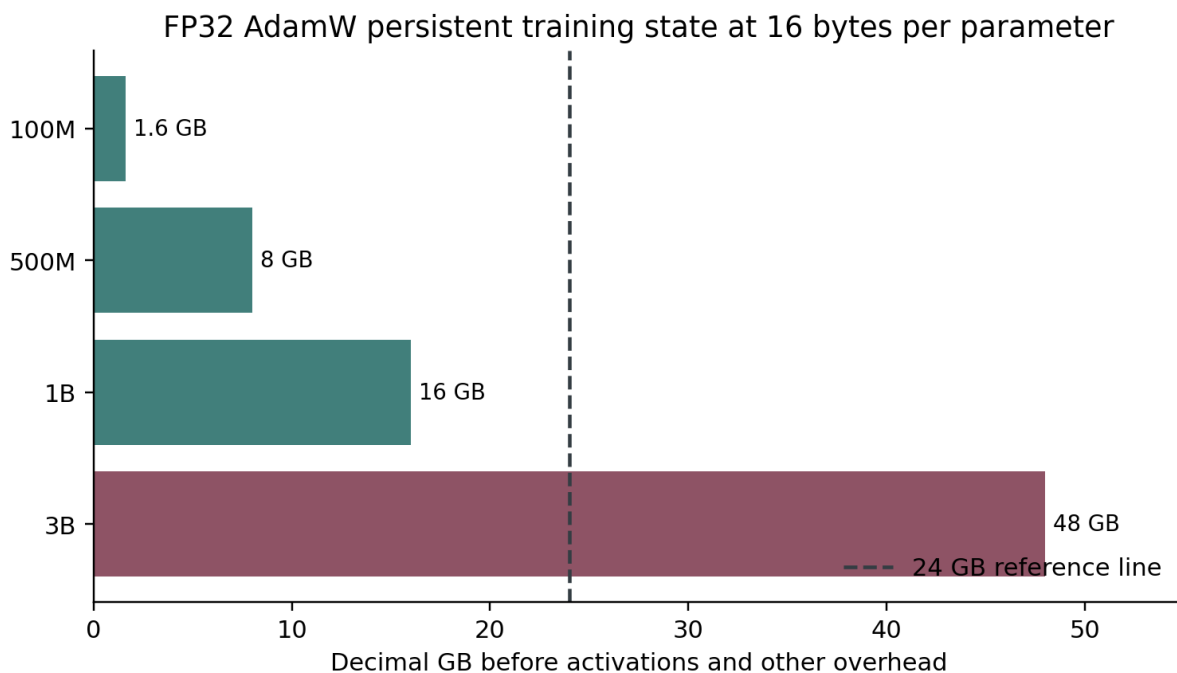


Figure 5 The dashed line is a 24 GB reference, not a feasibility guarantee. Activations, temporary buffers and other overhead are excluded from the bars.

What changes when most parameters are frozen

Let P be the total base parameter count and A the trainable adapter parameter count. A rough adapter budget is base storage proportional to P plus gradient and optimizer state proportional to A , plus activations and temporary tensors. If A is much smaller than P , trainable-state memory can fall dramatically.

Frozen does not mean absent. The forward pass still runs through the base model. Depending on the adapter placement and computation graph, backward may still need intermediate information to calculate adapter gradients. Therefore adapter training can run out of memory from activations even though its optimizer states are small.

A head-only classifier is another case. A frozen backbone produces features; a small trainable head maps them to labels. You may be able to precompute features when the backbone and preprocessing are fixed. Doing so changes the pipeline and data-storage tradeoff. You cannot reuse those cached features after changing the backbone while pretending you are still training end to end.

The largest memory knobs are often ordinary ones

Microbatch size is the number of examples processed together in one forward/backward pass. Increasing it generally increases activation memory. Sequence length affects language models; image resolution affects vision models; waveform length and sample rate affect audio models. These are not cosmetic controls. They define how much work and intermediate state each example creates.

For a 1024 by 1024 image, there are four times as many pixels as in a 512 by 512 image. The actual memory multiplier depends on latent compression and architecture. For attention over flattened spatial tokens, some costs can grow faster than pixel count. For audio, doubling duration doubles the number of samples at fixed sample rate, and attention over those samples or latent frames can add further expense.

Gradient accumulation combines gradients from several microbatches before an optimizer update. With one GPU, microbatch size 2 and accumulation 8 yield an effective batch of 16 examples, assuming every microbatch has two examples. It does not create the exact same execution as processing 16 examples simultaneously in every architecture. Batch-dependent operations and stochastic behavior can differ. It also does not reduce the memory required for a single example that is already too large.

Activation checkpointing discards selected intermediate activations and recomputes them during backward. This exchanges extra computation for lower memory use. It is unrelated to saving a model checkpoint to disk, despite the shared word. Use the implementation documented by the framework, and make the selected checkpointing mode explicit. [\[F05\]](#)

Efficient attention can reduce attention-specific memory substantially. Its availability depends on model structure, hardware, dtype, and supported masks. A custom mask or unsupported shape may cause a slower fallback. Do not claim the memory behavior of a fused kernel until the actual run uses it.

Measure the full update and the worst example

The first forward pass is insufficient as a memory test. Optimizer states may be created on the first update. Evaluation may use a different batch size. Saving or merging can allocate temporary copies. Generation can create a growing cache. A valid feasibility check includes representative training updates, evaluation, checkpoint save, reload, and the intended export path.

Measure an intentionally long example or the maximum permitted shape, not just an average batch. A dataset with mostly short examples and occasional very long examples can run for an hour before failing. Bucketing by length can improve efficiency, but the maximum still has to be bounded.

PyTorch exposes peak tensor allocation with `max_memory_allocated` and allocator reservation with `max_memory_reserved`. Reset peak counters before the measured region and synchronize when measuring GPU elapsed time. These metrics describe the current process's allocator view, so also inspect device-level usage and other processes. [F06]

```
torch.cuda.reset_peak_memory_stats()
# Run several complete training updates here.
torch.cuda.synchronize()
print(torch.cuda.max_memory_allocated() / (1024 ** 3))
print(torch.cuda.max_memory_reserved() / (1024 ** 3))
```

Calling `empty_cache` does not free live tensors that your program still references. Repeatedly invoking it is not a general cure for a model that fundamentally needs too much memory. Find what is retained, reduce a real cost, or change the training method. PyTorch distinguishes live allocation from cached reservation in its CUDA memory documentation. [F07]

Compute can become the more important limit

Training time depends on measured throughput. If the run processes 2,000 useful tokens per second and you plan to train on 100 million token presentations, raw training time is 50,000 seconds, about 13.9 hours. Add evaluation, checkpointing, startup, data processing, and interruptions. The 2,000-token rate is an example input to the calculation, not a promised speed.

Token presentations count repetitions. A dataset with 10 million tokens trained for three complete epochs presents roughly 30 million tokens, subject to packing, truncation, masking, and sampling. Ten million unique tokens and ten million repeated presentations have different implications for coverage and overfitting.

A rough dense-transformer training-compute heuristic is about 6 times parameter count times training-token count in floating-point operations. It is useful for order-of-magnitude comparisons, not an exact profiler. Attention, vocabulary projection, checkpoint recomputation, architecture, and kernel efficiency alter the real cost. Scaling-law research shows why model size and data allocation must be considered together, but no single historical tokens-per-parameter ratio is a universal prescription for a small specialized model. [F08]

A three-billion-parameter model trained on a tiny corpus may be badly undertrained even if a clever offload setup makes it run. Conversely, a ten-million-parameter model can be useful for a narrowly constrained task with good representations and representative data. Your job is to find evidence for the smallest adequate system, not to maximize the number printed in its name.

A staged budget for one GPU

Reserve a first phase for correctness. Use dozens of examples, tiny shapes, and a few updates. You should learn whether the program and labels agree, whether gradients exist, and whether saving works. Reserve a second phase for feasibility. Use the intended shapes and a small but representative dataset to measure memory and throughput. Reserve a third phase for quality. Compare a baseline and one trained candidate under a fixed evaluation procedure.

Only after those phases should you fund a longer run. Write an upper limit for training time, disk use, and checkpoint count. Define what result would cause you to stop early. If quality has not improved on the validation task after a justified trial, buying another ten hours of the same configuration is not automatically progress.

For local electricity, multiply average system power in kilowatts by hours and your local price per kilowatt-hour. Do not use the GPU's advertised maximum power as if it were measured whole-system draw. For rented compute, include storage, idle time, transfers, and failed experiments. A plan with no allowance for debugging is not a realistic first-project budget.

Exercises

Calculate persistent FP32 AdamW state for 250 million parameters under the sixteen-byte assumption. The answer is 4 GB, about 3.73 GiB, before activations. Explain why that result alone cannot establish that any 250-million-parameter job fits.

A dataset has 40,000 examples averaging 500 supervised tokens. Two epochs present about 40 million supervised tokens. At a measured 1,250 supervised tokens per second, pure processing time is about 8.9 hours. Explain why a logger counting all input tokens could produce a different throughput number.

Write down three ways to reduce activation memory and three ways to reduce persistent training state. Do not list lowering the learning rate: it is usually a quality and stability control, not a major memory control.

16 Project make related text easy to retrieve

An embedding encoder turns an input into a vector designed for a downstream similarity objective. Useful application patterns include semantic search, near-duplicate discovery, clustering, matching queries to catalog items, and a frozen encoder feeding a small task-specific classifier. A dense retriever can feed a more expensive reranker or supply evidence to a separate answer-generating system. Its output is a representation or ranked result, not a guarantee that a generated answer is correct.

Use embeddings when semantic relationships matter and exact matching alone is insufficient. Preserve deterministic filters for permissions, dates, product variants, and structured constraints. A vector search should never bypass access control just because a private document is similar. Compare lexical, dense, and combined approaches on your actual queries.

Open a fresh terminal at the unpacked companion root before the commands below. They explicitly enter examples/embeddings; do not resolve that path relative to a previous project directory.

Imagine a user asks, “How do I stop future renewals?” while the correct help article says “Cancel a subscription.” Exact keyword matching may miss the relationship. An embedding model turns each text into a vector so useful pairs can receive high similarity. It is a representation model; it does not need to generate an answer.

First run the lexical baseline. A neural model should earn its extra complexity. Start from a fresh environment; do not run an import check before installing its dependencies.

Fresh environment for the CPU retrieval baseline

The following commands assume Python 3.12 is installed and a Linux/macOS shell starts in the downloaded book directory:

```
cd examples/embeddings
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-cpu.txt
python -c "import numpy, sklearn; print(numpy.__version__, sklearn.__version__)"
python make_retrieval_data.py --out data/retrieval
python lexical_baseline.py --data data/retrieval --out runs/lexical
```

On Windows PowerShell, create the environment with `py -3.12 -m venv .venv` and activate `.venv\Scripts\Activate.ps1`, then use the remaining Python commands. Respect local execution-policy controls. Existing output directories should be replaced with new experiment names, not combined with new artifacts. The setup paths in this chapter are relative to `examples/embeddings` after the first `cd`.

The generator creates 60 fictional help documents for ten fictional products and three query variants per document. Products are split six/two/two across training, validation, and test. Training contains 108 query/positive pairs, but only 36 distinct positive documents. The shared templates make this a wiring fixture, not a serious retrieval benchmark.

The CPU-tested TF-IDF baseline achieved validation Recall@5 of 1.0 and MRR@5 of about 0.821 on this fixture. That is a useful warning: a neural model is not automatically necessary, and the dataset is deliberately easy. Real improvements require independently collected queries, relevant-document judgments, and realistic distractors.

What the keyword baseline represents

TF-IDF means term frequency-inverse document frequency. It represents a document with weighted counts: terms repeated within that document gain weight, while terms common across the corpus are downweighted. The supplied lexical baseline includes single words and adjacent two-word sequences called bigrams. Its vectors are sparse because most vocabulary entries do not appear in a given passage. A query is transformed with the same fitted vocabulary and scored against document vectors. This is a strong inspectable baseline when exact terminology carries relevance; it does not require a neural network.

Replace the fixture with your own query to document task

A **JSONL** file stores one JSON object per line. Keep three files in your data directory. These first rows form one consistent example:

`corpus.jsonl` contains documents that can be searched:

```
{"doc_id": "aster-password", "text": "Aster: How to change the password. Open Account then Security and choose Change password.", "group": "Aster", "split": "train"}
```

`queries.jsonl` contains questions with relevance judgments for evaluation:

```
{"query_id": "aster-password-q0", "text": "How do I change the password in Aster?", "relevant_doc_ids": ["aster-password"], "split": "train"}
```

`train_pairs.jsonl` contains the positive pairs used for adaptation:

```
{"query": "How do I change the password in Aster?", "positive": "Aster: How to change the password. Open Account then Security and choose Change password.", "doc_id": "aster-password", "group": "Aster"}
```

`doc_id` is a unique stable identifier. `query_id` identifies a query. `relevant_doc_ids` is a list because several documents can correctly answer one query; populate all verified positives rather than assuming only one. The training row's positive text must exactly match its referenced corpus document. Do not silently change whitespace, document content, or IDs in only one file.

`group` is the independent source unit held together across the split: the fixture uses product families; a real task might use source-document families or time-separated collections. Here every corpus group belongs entirely to `train`, `val`, or `test`; query judgments stay with the corresponding document split. All paraphrases of a query stay together. The shown row is training data, so add different groups labeled `val` and `test` with their own evaluation queries. Only train documents may appear in `train_pairs.jsonl`.

The search index contains the entire permitted corpus, including held-out documents, because these are the inference-time database. Their held-out query labels are not used for weight updates. Do not put sensitive or inaccessible documents into a shared index without appropriate access controls. For a different deployment question, such as future unseen

corpora, define that stronger split deliberately rather than pretending this fixed-corpus exercise tests it.

For this row, the default neural query input is literally:

```
Instruct: Given a support question, retrieve the help article that answers it for the correct product.
Query:How do I change the password in Aster?
```

Its positive document is unprefixed:

```
Aster: How to change the password. Open Account then Security and choose Change password.
```

The instruction consumes part of the 128-token limit. The program adds it exactly once in training, validation, final evaluation, and live search. The corpus text itself is not rewritten with this query instruction.

What an embedding represents

A text encoder tokenizes the text, produces token-level representations, and pools them into one fixed-length vector. **Mean pooling** averages non-padding token representations; including padding would make the result depend on how it was batched. Other models use a designated token, last-token pooling, or more elaborate representations. Use the pooling the model was trained for.

The primary model is **Qwen3-Embedding-0.6B**, a compact member of the instruction-aware Qwen3 embedding family. It is public/non-gated at verification and marked Apache-2.0. It offers 1024-dimensional embeddings and an advertised 32K context, but this lab deliberately uses only 128 tokens. Model capacity is not a guarantee that a full 32K training batch fits a 24 GB card. [M30]

Its released module configuration uses a Transformer, **last-nonpadding-token pooling**, and normalization. Mean pooling is a useful concept to know, but it is not the recipe for this checkpoint. The shared query format is an instruction followed by Query: and the actual question; documents receive no instruction. The saved tokenizer handles the model's native EOS behavior. Do not apply a chat template or append a second EOS manually. [M31] [M32]

The architecture configuration specifies 28 layers, hidden width 1024, 16 query heads, 8 key/value heads, head dimension 128, and feed-forward width 3072. Counting the published embedding/backbone tensor shapes implies approximately **595,776,512 parameters**; this is a source/config-derived count, not an author-executed model load. The script prints the actual loaded parameter count for verification. It adapts the encoder through Sentence Transformers rather than constructing a text-generation head. [M33]

A short historical contrast: an older MiniLM mean-pooled encoder is much smaller and illustrates a different pooling convention. It is not the primary training route in this project. Model size, pooling, and query formatting must be chosen together rather than swapping names in an otherwise unchanged pipeline.

A tokenizer converts text into model-specific token IDs. Tokens are not necessarily words. A new tokenizer changes the meaning of the model's embedding lookup indices; you cannot replace one casually while keeping all pretrained weights. Keep casing, special tokens, query/document prefixes, truncation, pooling, and normalization consistent between training,

indexing, and search. Some other retrieval models require task-specific prefixes; this lab uses the Qwen instruction format consistently in training, validation, indexing, and live search.

L2 normalization rescales a nonzero vector to length one. For normalized vectors, cosine similarity equals dot product, and squared Euclidean distance is $2 - 2 \times \text{dot_product}$; their rankings agree. Without normalization, vector lengths can change dot-product rankings. Do not train with one scoring rule and index with another accidentally.

Install the neural runtime then run a smoke experiment

The pinned, source-reviewed API target is PyTorch 2.8.0, Sentence Transformers 5.1.1, and Transformers 4.57.1. These meet the model card's requirements; they are explicit reproducibility versions, not a claim to be the newest releases. The loss, SentenceTransformer loader, and Qwen3 implementation were inspected at their tags. No neural packages were installed, no weights downloaded, and no neural training run during authoring. [M14] [M26] [M34]

Remain in the activated environment. Choose exactly one PyTorch route. On Linux/Windows with an NVIDIA driver and GPU compatible with the official CUDA 12.8 wheel:

```
python -m pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements-neural.txt
python -m pip check
python -c "import torch, transformers, sentence_transformers; print(torch.__version__,
    transformers.__version__, sentence_transformers.__version__); print('CUDA:',
    torch.cuda.is_available()); print('BF16:', torch.cuda.is_available() and
    torch.cuda.is_bf16_supported())"
```

For CPU-only Linux/Windows, replace the first installation command with `python -m pip install torch==2.8.0 --index-url https://download.pytorch.org/whl/cpu`, then run the same requirements and checks. For macOS CPU use `python -m pip install torch==2.8.0` instead. This recipe does not implement MPS. Changing `--device` cannot give a CPU wheel CUDA support. For a GPU unsupported by these pinned wheels, select an official compatible stack and revalidate; do not force an incompatible build. The official versioned install page lists the supported wheel routes. [M35]

After reviewing the model license, begin with one update:

```
python train_embeddings.py --data data/retrieval --out runs/qwen3-smoke \
    --device cuda --amp-bf16 --batch-size 2 --max-length 128 \
    --epochs 1 --max-steps 1
```

Confirm the smoke checkpoint reloads and returns document IDs before attempting the longer run:

```
python search.py runs/qwen3-smoke "How do I change the password in Aster?" \
    --device cuda --k 3
```

This is a wiring check on a training-style question, not a generalization score. Then use a new output directory for adaptation:

```
python train_embeddings.py --data data/retrieval --out runs/qwen3-adapted \
    --device cuda --amp-bf16 --batch-size 2 --max-length 128 \
    --epochs 3 --lr 0.00001 --weight-decay 0.01 --scale 20
```

The script pins Qwen/Qwen3-Embedding-0.6B to verified revision

97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3, keeps `trust_remote_code=False`, and rejects a nonempty run directory. The first model load downloads about 1.2 GB of weights plus tokenizer/configuration; a saved FP32 checkpoint is larger. Leave generous disk and host-RAM headroom. Pinning an artifact does not replace license and provenance review. [M36]

CPU training uses the same command with `--device cpu` and without `--amp-bf16`. Full 0.6B backpropagation is slow on CPU and needs substantial RAM. For a first CPU neural retrieval experiment, save and index the unmodified baseline without allocating an optimizer:

```
python train_embeddings.py --data data/retrieval --out runs/qwen3-baseline \
  --device cpu --eval-only --batch-size 2 --max-length 128
```

The adaptation run updates all encoder parameters, so it is full fine-tuning of a pretrained representation, not scratch language pretraining. There are 36 unique positive documents, hence 18 updates per epoch at batch two and 54 across three epochs. The fixture is sufficient to exercise the machinery, not to claim adaptation quality. Use independently judged real query/positive pairs before drawing product conclusions.

The program first scores the unmodified model on validation and saves it as an eligible candidate. Each epoch samples one query variant per positive document, trains, and measures validation retrieval. It selects a new checkpoint only when validation MRR@5 improves. `selected_epoch=0` means the unmodified model won. The chosen model is then validated and used for corpus indexing. Smoke, baseline, and training commands do not score test queries; final test evaluation is a separate explicit command after all tuning is done. Optimizer and original-model references are released before loading the selected checkpoint, avoiding a needless second training-sized model allocation.

The embedding experiment controls

Control	Default	Meaning
model and revision	pinned in <code>embedding_contract.py</code>	Verified Qwen3-Embedding-0.6B artifact; changing architecture requires reviewing its whole input/output contract
<code>--epochs</code>	3	Passes over unique positive-document groups
<code>--batch-size</code>	2	Actual contrastive pairs per update and encoding batch size
<code>--max-length</code>	128	Token budget including instruction and native special tokens; longer inputs are truncated
<code>--lr</code>	0.00001	AdamW learning rate for full-encoder adaptation
<code>--weight-decay</code>	0.01	Decoupled weight penalty
<code>--scale</code>	20	Cosine similarity multiplier before contrastive cross entropy
<code>--instruction</code>	support-retrieval sentence	Task description prefixed to queries, never documents; persisted for inference

Control	Default	Meaning
--seed	42	Sampling/random-state control
--device	cpu	CPU or CUDA encoder/training placement
--amp-bf16	off	CUDA BF16 autocast on supported hardware; master weights and optimizer state remain FP32
--max-steps	0	No update cap; use 1 for a smoke run
--eval-only	off	Save/evaluate/index the frozen baseline without training
--data, --out	data/retrieval, runs/qwen3-embedding	Dataset and artifact locations
search --k	5	Number of ranked result IDs to return

The loop clips gradient norm to 1 and deliberately omits a scheduler for the introductory short run. It uses SDPA without requiring a FlashAttention install, non-reentrant gradient checkpointing, `use_cache=False`, and AdamW `foreach=False`. There is no decoder-generation KV cache needed for embedding training. Gradient checkpointing saves activation memory by recomputing parts of the forward pass. BF16's numerical range generally avoids the FP16 gradient-scaling requirement; this code does not use an FP16 mode. [M34]

Match embedding architecture and supervision to the task

A single-vector encoder compresses a whole input into one vector and is efficient to index. It may blur a long document's small but important detail; chunking or a late-interaction/multi-vector architecture can preserve more detail at extra memory and serving cost. A cross-encoder reads query and document together and suits reranking a short list, but does not produce independently reusable document vectors in the same way.

Training a text encoder from random initialization requires learning language structure as well as your retrieval relationship. Fine-tuning an existing small encoder can focus a limited set of high-quality judgments on the latter task. For adaptation, the most valuable data are representative queries with verified positives, realistic confusions, and examples of what should not match. A million mechanically generated easy pairs can be less useful than a much smaller carefully judged set. The fixture demonstrates the format and loop; it does not estimate the needed count for your domain.

Build learning curves over independently labeled query families and inspect truncation rates, false negatives, and relevant-document coverage. For a multimodal encoder, you need aligned examples spanning the modalities, not arbitrary text and image collections placed side by side.

Contrastive learning turns a batch into a classification problem

For a batch of B query/document pairs, compute B query vectors and B positive-document vectors. Construct a $B \times B$ similarity matrix. Row i 's correct candidate is document i ; the other documents are treated as negatives. Cross entropy rewards the diagonal. This is **multiple-negatives ranking loss**, a common contrastive training objective. [M14]

With cosine similarity s , its query- i loss is:

```
-log( exp(scale * s(query_i, positive_i))
      / sum_j exp(scale * s(query_i, document_j)) )
```

$\text{scale}=20$ corresponds to temperature 0.05. Larger scale makes the distribution sharper and changes gradient behavior. It is not an accuracy setting. The script uses the library's loss to handle numerically stable cross entropy rather than directly implementing the exponentials shown for explanation.

A batch of size one has no competing documents and no useful contrastive training signal. The script discards singleton final batches and requires at least two distinct positives. It samples one query per document per epoch so the same positive document does not appear twice in one batch. The standard Sentence Transformers no-duplicates sampler is another useful tool for applicable trainer workflows. [M15]

A **triplet loss** instead uses an anchor, a positive, and an explicit negative. A common distance form is $\max(0, \text{distance}(\text{anchor}, \text{positive}) - \text{distance}(\text{anchor}, \text{negative}) + \text{margin})$. The margin asks for separation, not a universal semantic distance. Easy negatives that already satisfy the margin contribute nothing; mislabeled hard negatives push the representation in the wrong direction. Triplet methods are a distinct training choice, not a reason to add arbitrary integer labels to a pair-based loss. [M23]

False negatives hard negatives and leakage

An in-batch negative is an assumption, not a verified fact. Two different articles may both answer “How do I reset access?” If one is paired as the positive and the other appears in the batch, the loss may punish a genuinely useful match. Exact-text deduplication catches identical strings, not semantic duplicates.

For real data, group equivalent documents and queries; represent multiple relevant documents in evaluation; construct batches that avoid competing positives where feasible; or choose a loss that explicitly handles multiple positives. Do not equate “not labeled positive” with “known irrelevant.” Audit mined examples manually before scaling them.

A **hard negative** is a plausible but wrong candidate. For the help-desk case, an article for the wrong product or an outdated policy can be hard. Mine candidates from training queries using a baseline retriever, then verify their irrelevance. Never mine from held-out test queries and feed those pairs back into training. If a hard negative is actually relevant, additional training can make the system worse while the training loss improves.

Split by source document, customer question family, or other genuine group before creating paraphrases. A test query that is a light rewrite of a training query is weak evidence. The fixture separates product groups but shares wording templates; its evaluation limitation is intentional and documented.

Retrieval has a subtle distinction from classification: the search corpus can contain held-out documents because they are the database being searched at inference time. Indexing their text is not the same as using their held-out query/relevance labels to train. This lab evaluates the fixed-corpus setting. If the claim is generalization to a future changing corpus, hold out time periods or document families accordingly and test that stronger setting.

Batch size means something different here

A larger real contrastive batch supplies more candidate negatives to each query, which can improve the learning signal until noise or false negatives dominate. **Gradient accumulation does not automatically enlarge the in-batch negative set.** Four separate batches of 16 pairs with accumulated gradients still present 16 candidates per loss computation, not one joint 64-candidate classification problem.

Ordinary attention activation memory depends on batch size and token length; the dense similarity matrix also grows with B squared. Long padded outliers can waste considerable space. Limit or bucket token lengths deliberately, inspect actual truncation, and reduce per-step batch size or sequence length when memory is tight. Use a cached contrastive loss when its larger negative pool is justified: gradient caching trades additional computation for lower activation-memory pressure, but does not make all memory costs disappear. The original gradient-cache paper and current library loss documentation explain this tradeoff. [M16]

On a 24 GB GPU, the small encoder and short default batches are a conservative starting experiment, not a tested fit guarantee. Run one forward/backward/update step with representative maximum-length examples, record peak allocated and reserved memory, leave room for evaluation and indexing, and only then increase batch size. The provided loop keeps FP32 master weights and optimizer state while optionally autocasting eligible CUDA operations to BF16. Parameters, gradients, and two Adam moments alone account for roughly 8.88 GiB for the config-derived parameter count, before activations, workspaces, allocator reserve, and any process sharing the GPU. That is a planning calculation, not a measured fit. The script records both peak allocated and reserved CUDA memory; the smoke run must establish whether your actual configuration fits. If it does not, reduce length/batch while retaining at least two contrastive pairs, or select a separately validated parameter-efficient method.

Evaluate the retrieval product not just its loss

The neural evaluation computes rankings against all 60 documents and reports:

- **Recall@k:** fraction of the query's known relevant documents retrieved in the top k
- **MRR@k:** reciprocal rank of the first relevant result, or zero if none occurs within k, averaged across queries
- **nDCG@k:** discounted relevance accumulated near the top, divided by the ideal ordering; the example uses binary relevance, while real datasets can use graded judgments

The implementation supports multiple relevant document IDs and deterministic tie ordering. It does not assume the index contains only one positive and a few handpicked negatives. Sentence Transformers provides information-retrieval evaluators for larger workflows and additional metrics. [M18]

A **reranker** jointly reads a query and each retrieved candidate, often scoring relevance more accurately at greater latency than independently encoded vectors. A useful pipeline is lexical and/or dense retrieval → candidate merge → reranking → final selection. The reranker cannot recover an answer that the first-stage candidate set omitted. Measure first-stage recall, final ranking quality, and end-to-end latency separately.

Similarity is not a calibrated probability of correctness. A cosine score of 0.8 is not automatically an 80% chance of answering the question. Validate any “no answer” threshold using real answerable and unanswerable queries. Include product names, identifiers, negation, numbers, dates, languages, and near-duplicate policies in the error set. Dense similarity alone can be weak on exact codes; lexical retrieval remains a valuable complement.

Use the trained index consistently

The output directory contains:

- `best/`: the selected encoder, tokenizer, last-token pooling configuration, and associated model files
- `index_contract.json`: exact instruction, preprocessing, dimensions, and SHA-256 identities for model and index files
- `corpus_embeddings.npy`: normalized vectors produced by that selected model
- `corpus_ids.json`: the row-to-document mapping
- `validation_rankings.json`: inspectable validation result IDs; final test artifacts are created only by the explicit test command
- `metrics.json`: configuration, validation history, selected validation scores, input data hashes, and available memory measurements

The short trainer saves inference-ready selected weights, not optimizer/scheduler state or a resumable training position. Reloading `best/` for search is not resuming interrupted training. A resumed training experiment would need an explicitly implemented optimizer/random-state checkpoint strategy and its own checks.

Search it with:

```
python search.py runs/qwen3-adapted \
    "In Iris, I need to stop future renewals." --device cpu --k 5
python search.py runs/qwen3-adapted \
    "In Iris, I need to stop future renewals." --device cuda --k 5
```

Use `runs/qwen3-baseline` instead if you created the frozen baseline. The first command encodes on CPU; the second encodes on CUDA. Both load FP32 weights, independently of the training autocast choice. Each loads the selected local model directory, including tokenizer and last-token pooling, with the saved sequence-length limit, left padding, and exact training instruction. The query prefix is applied once; documents remain unprefixed. SHA-256 checks reject mismatched weights/tokenizer/configuration/index files, adding startup I/O in exchange for consistency. The document index stays in NumPy on CPU in this small demo; only the encoder changes device. Vectors are normalized on both paths, and the output contains document IDs and similarity scores. These neural commands were syntax-checked, not executed. For a larger GPU index you would need a separately selected indexing backend and its own correctness checks.

When you change the encoder, pooling, normalization, tokenization, or document chunking, rebuild the index or implement a carefully tested migration. Old document vectors and new query vectors are generally incompatible. Store model revision and preprocessing identity

alongside every index version. Protect sensitive documents and query logs; embeddings are derived data, not an automatic anonymization method.

The lab uses an exact NumPy dot-product search. For a large corpus, measure approximate-nearest-neighbor index recall against exact search on a sample. Quantized vectors and compressed indexes reduce memory but can change ranking. Approximation errors compound with encoder and reranker errors, so evaluate the complete deployed path.

Reserve the test set for one final comparison

During development, both the lexical command and neural training commands report validation only. The one-update smoke run and frozen-baseline run must not become early peeks at the held-out test. When the model, instructions, chunking, index settings, and thresholds are fixed, run:

```
python evaluate_saved.py runs/qwen3-adapted --data data/retrieval \
--split test --device cuda --batch-size 2
python lexical_baseline.py --data data/retrieval \
--out runs/lexical-final --split test
```

CPU final evaluation uses `--device cpu`; for a deliberately selected frozen encoder, substitute its run directory. The neural evaluator checks corpus identity against the stored index contract and writes `test_metrics.json` and `test_rankings.json`. Compare the fixed systems, report uncertainty and representative failures, and do not tune on these labels afterward. `evaluate_saved.py` defaults to `--split val` when the flag is omitted. A serious iteration after examining test failures needs a fresh final holdout.

Extend embeddings to images and multiple modalities

The same pattern applies beyond text. An image encoder maps photographs to vectors for visual similarity or a small downstream classifier. The definition of “similar” matters: identical products, matching texture, same species, and the same individual object are different objectives. Image augmentations define which variations the representation should ignore; an aggressive crop can remove the very identity signal you wanted to preserve.

A dual-encoder multimodal system has, for example, an image encoder and a text encoder that map into a shared dimensional space. Contrastive training encourages paired images and descriptions to align. CLIP is a primary example of this training pattern. It is not achieved by comparing arbitrary image vectors with unrelated text vectors that happen to have the same length. [M17]

For a small custom multimodal task, begin with an appropriate pretrained aligned model and frozen-feature baselines. Fine-tune on rights-cleared matched pairs only if validation shows a need. Captions that omit the decisive visual detail are weak supervision for that detail. Multiple images can match a caption, creating the same false-negative issue seen in text retrieval. Evaluate image→text and text→image separately if both directions matter.

Do not use a retrieval embedding as a substitute for a pixel mask. A single pooled image vector intentionally compresses spatial information; precise boundaries require an output architecture and labels that preserve it.

17 Choose a model by its actual structure and files

After the first projects, model comparison becomes easier: you know which failure needs improvement and what your training budget contains. This section is a snapshot of official cards/configurations inspected on 2 October 2026. It does not claim that every listed checkpoint is the newest release in its family or that any is universally best.

A short list with the size traps made explicit

Family and exact example	Published scale and structure	Consequence for this book
Qwen/Qwen3-0.6B	0.6B nominal; dense decoder; 28 layers; GQA with 16 query and 8 KV heads	Main full/adapter teaching checkpoint, Apache-2.0 [L02] [L34]
Qwen/Qwen3-1.7B	1.7B nominal; dense decoder; 28 layers; GQA	An adapter extension within the broader size range, not the full-update example [L35]
Qwen/Qwen3.5-0.8B and Qwen/Qwen3.5-2B	Cards describe a vision encoder and a language model with alternating Gated DeltaNet and gated-attention layers; 24 LM layers	Newer hybrid/multimodal variants need different loading, module targeting and data handling; count all components, not only the LM label [L36] [L37]
google/gemma-3-270m-it	270M, with about 170M in embeddings and 100M in transformer blocks	Good small task-specialization candidate; large vocabulary makes “tiny blocks” different from “tiny total model” [L38]
google/gemma-3-1b-it	Text-only small Gemma 3 variant; custom Gemma terms; 32K published context	Check exact unique count for a strict one-billion limit; use the correct Gemma template [L39] [L40]
ibm-granite/granite-4.0-350m	350M dense decoder; 28 attention layers; GQA; Apache-2.0	Sub-billion candidate for a separate template-validated recipe [L41]
ibm-granite/granite-4.0-h-350m	340M hybrid; attention plus Mamba2 layers	Different recurrence/kernel and adapter-target considerations, despite a similar name [L42]
ibm-granite/granite-4.0-1b	The architecture table gives 1.6B, despite “1b” in the ID	Within the general range, outside the practical full-update ceiling [L43]
ibm-granite/granite-3.3-2b-instruct	Dense decoder; config has 40 layers, width 2,048 and a 49,159-entry vocabulary	Older, still inspectable domain/tool candidate; do not assume an exact 2.000B count [L44] [L45]
HuggingFaceTB/SmolLM2-360M-Instruct	Small dense instruction checkpoint with published training resources	Another sub-billion baseline; use its own tokenizer/template [L46]
meta-llama/Llama-3.2-1B-Instruct	Dense multilingual text model with GQA and a Llama community license	Useful ecosystem comparison; “1B” is rounded and the license differs from Apache [L47]

The main script intentionally accepts the reviewed Qwen3 text architecture, rather than pretending these rows are interchangeable. A new model needs a new verification pass: loader class, tokenizer, stop tokens, supported attention implementation, masks, trainable modules and parameter count.

The latest inspected Granite 4.2-3B card describes a dense reasoning model released in August 2026. Its nominal “3B” label is not proof of an exact $\leq 3,000,000,000$ total. Treat rounded 3B-class models, including SmolLM3 and Llama 3.2 3B, as boundary comparisons until you count unique parameters; do not silently relax a strict size cap. [L48] [L49]

Liquid models offer a small hybrid comparison

Liquid provides genuinely small models as well as larger ones. The older official LFM2 card gives exact counts of 354,483,968 for LFM2-350M, 742,489,344 for LFM2-700M, 1,170,340,608 for LFM2-1.2B and 2,569,272,320 for LFM2-2.6B. The first two fit the practical full-update size ceiling; the latter two belong to the broader adapter/comparison range. These checkpoints use the LFM Open License, so review those terms rather than assuming Apache or MIT. [L55]

The currently linked successor LiquidAI/LFM2.5-350M retains 16 layers: ten double-gated short-convolution blocks and six grouped-query-attention blocks, with a 65,536-entry vocabulary and a 32,768-token context. Its published card favors narrow extraction, structured-output and tool-use workloads. Its vendor CPU speed numbers are specific measurements under their setup, not predictions for your machine. [L56]

A short convolution mixes a nearby window of sequence positions. An input-dependent gate controls how much information flows through that operation. The attention blocks provide another way to mix information across positions. This is a hybrid sequence model; “Liquid” does not mean that every released checkpoint is simply a classical continuous-time liquid neural network. The technical report describes architecture search that includes hardware constraints. [L60]

The causal text interface still supports next-token SFT and continued pretraining. You do not need a new label type merely because some blocks use convolutions. But kernel support, cache structure, padding behavior and adapter placement need new checks. Transformers provides a native `Lfm2ForCausalLM` interface with labels and cache arguments. Liquid documents LoRA SFT through TRL, targeting attention projections in its example. That establishes an available training path, not that every convolution is adapted or that our Qwen-specific script can be pointed at LFM unchanged. [L57] [L58]

There is also a serialization difference: the inspected LFM2.5 card describes Python-like calls inside its tool-call markers by default, with JSON as a prompted alternative. A Qwen JSON-call parser is not automatically the right parser. Never execute model-generated Python with unrestricted eval; parse a restricted call structure and validate it against an allowlist. This is an application boundary, regardless of model family.

For a separate LFM experiment, begin with a text-only extraction dataset, inspect its saved template, enumerate trainable modules and run the same one-batch/save/reload checks. The official TRL guide currently contains `tokenizer=` examples; the reviewed current trainer uses `processing_class=`. Resolve that version difference before a long run. For CPU deployment, the

official guide provides existing GGUF/llama.cpp paths. Adapting a model and exporting your changed weights requires supported conversion and a fresh quality check; an existing vendor GGUF is not your fine-tuned artifact. [L57] [L59]

Total active and effective are not synonyms

A **dense model** normally uses all its dense transformer-block weights for each token’s forward computation. A **mixture of experts**, or MoE, routes a token through a subset of expert networks. This can reduce arithmetic per token relative to the total model capacity. It does not make unused experts cease to exist in memory.

Qwen3-30B-A3B is explicitly 30.5B total and 3.3B activated, with 128 experts and eight selected. It is outside this book’s three-billion-total scope. Even an ideal four-bit payload for 30.5B weights is about 15.25GB before metadata, non-quantized tensors, adapters and activations. That arithmetic is not a promise of trainability on a 24GB card. [L50]

Gemma 4 E2B has another naming convention: the card gives 2.3B effective parameters and 5.1B with embeddings, using per-layer embeddings. It is also outside a strict ≤ 3 B total scope. Its card lists Apache-2.0, whereas the Gemma 3 examples above use Gemma terms. Do not assign a license to a checkpoint merely from its family name. [L51]

A checkpoint’s serialized tensor count can differ from the number of independent trainable entries when input/output embeddings are tied. **Tied embeddings** reuse the same matrix for looking up token vectors and projecting hidden states toward vocabulary scores. Qwen3-0.6B’s configuration sets `tie_word_embeddings=true`; its inspected safetensors metadata reports about 0.752B stored entries while its nominal model has about 0.6B unique parameters. Count the instantiated model’s unique parameters and inspect actual storage rather than inferring optimizer memory from a file-page badge. [L03] [L34]

Read a configuration as an engineering document

Several fields matter immediately:

- `hidden_size`: the width of each token’s internal vector
- `num_hidden_layers`: how many transformation stages tokens pass through
- `intermediate_size`: the feed-forward expansion width
- `num_attention_heads`: query-head count
- `num_key_value_heads`: shared key/value-head count for GQA
- `vocab_size`: rows in token-related tables and width of vocabulary logits
- `tie_word_embeddings`: whether input/output tables share weights
- `max_position_embeddings` and positional settings: implementation capacity, not your validated training length

For a dense linear map, parameter count is input width times output width, plus a bias if present. For an embedding table, it is vocabulary size times embedding width. These simple products explain why two “one-billion-ish” models can place capacity in different parts of the network and produce different memory peaks.

Grouped-query attention, or GQA, lets several query heads share fewer key/value heads. This reduces key/value storage relative to giving every query head its own keys and values. It does not make all training activations shrink by the same ratio. **RoPE** encodes positional relationships through rotations in attention coordinates. **Sliding-window attention** limits some layers to nearby tokens. **Hybrid attention/state-space models** combine different sequence-mixing mechanisms, so a standard full-attention memory formula is not a complete model for them.

Tokenizer choice is equally practical. Compare how many tokens your actual domain text uses, how identifiers and non-English terms split, whether special tokens are reserved and how the template encodes a tool call. A large vocabulary can shorten some sequences while increasing embedding and output-head costs. Never replace a pretrained tokenizer with another and assume the old embedding rows still have the correct meanings.

18 Teach a small language model one useful behavior

A small generative language model turns a token sequence into a distribution over the next token. Around that simple interface, you can build quite different applications. Choose the application pattern before deciding to train.

Pattern	What the model does	What the surrounding software must do
Completion	Continue a document or code prefix	Bound output, show suggestions, preserve the original
Chat assistant	Answer a structured conversation	Keep roles and history consistent with the model template
Structured extractor	Produce fields from supplied text	Validate a schema and reject unsupported values
Tool selector	Propose a function and arguments	Check permissions, validate arguments, execute safely, return observations
Retrieval assistant	Answer using supplied evidence	Retrieve authorized sources, track versions, check citations
Domain specialist	Interpret specialist language and workflows	Supply current facts, hold out realistic cases, retain general competence
Style rewriter	Express supplied content in a chosen voice	Preserve facts and commitments, review sensitive outputs

These are system designs, not seven separate neural architectures. A single decoder model may support several after appropriate training. A good schema validator can make output structurally valid; it cannot make an incorrect field true. A tool-capable model proposes an action; the application decides whether to carry it out. Retrieved text is evidence, not permission to execute instructions found inside it.

Use prompting when the model already has the capability and needs a clearer task. Use retrieval or tools when it needs fresh, exact or permission-controlled facts. Use training when you need a repeatable learned behavior or genuinely new domain patterns and have representative examples. Begin with the narrowest pattern that solves your task, then add complexity only after a held-out test shows what is missing.

You have already built a small predictor and seen weights change during training. Now use the same ideas to customize a pretrained language model. The first project is deliberately narrow: turn a short list of facts into a concise update without adding a promise. You will begin with a small adapter, repeat the experiment with full fine-tuning, then extend the machinery to domain text and tool calls.

The practical full-update example uses Qwen3-0.6B, comfortably below one billion unique parameters. The broader model discussion stays around the requested three-billion-parameter

ceiling and explicitly flags misleading size names. A model that can be loaded is not necessarily a model that can be trained. A training job that finishes is not necessarily a useful model.

Evidence labels. Model-card facts and library behavior have references. Memory arithmetic is a calculation with stated assumptions. Suggested learning rates, batch sizes and experiment stages are starter configurations, not measured optimums. The supplied Python files passed syntax checks and fourteen dependency-free unit tests. We did not install the training libraries, download model weights, run the tokenizer, train on a GPU, or measure model quality. The first task on your machine is therefore a smoke test, not a long training run.

The deliverable and the baseline

The deliverable is a local adapter directory, an exact base-model revision, a run manifest, and a comparison against the unchanged model. The target behavior is:

Input facts: “The export contains 47 items. Two images are missing. A retry time has not been agreed.”

Acceptable output: “The export contains 47 items, with two images missing. There’s no agreed retry time yet.”

Unacceptable output: “The export is complete. I’ll retry the two missing images tomorrow.”

The second answer sounds fluent, but changes completion status and invents a commitment. Your evaluator must notice that even if the prose is attractive.

Before training, try a clear instruction and two examples with the original checkpoint. Save those outputs. If prompting already meets the requirement, keep the simpler solution. This baseline is also how you discover what training needs to improve: length, factual preservation, formatting, or instruction following. These are different failure modes.

The included files live in `examples/llm/`. `make_data.py` generated the supplied fictional fixtures. `train_small_lm.py` handles training and masking. `generate_eval.py` writes raw baseline or adapted completions. `score_tools.py` handles a later structural tool-call test. No example sends model weights, datasets, or training logs to an external tracking service.

Make one record you can read

A supervised example pairs the situation the model sees with a response you want it to produce. This is an original example in the same shape as `data/style_train.jsonl`:

```
{
  "source_id": "style-train-03",
  "messages": [
    {
      "role": "system",
      "content": "Write a concise update. Preserve the facts."
    },
    {
      "role": "user",
      "content": "The import processed 124 records. Three need a category."
    },
    {
      "role": "assistant",
      "content": "The import processed 124 records. Three still need a category."
    }
  ]
}
```

JSONL means one complete JSON object per line. The pretty-printed version above is for reading; the actual file uses one-line records. `source_id` is your grouping key. All edits, paraphrases and conversation turns from one original source stay in the same split. Otherwise training can see an almost identical version of your supposed test example.

The fixture contains only four training records. That is enough to test parsing and a few weight updates. It is not enough to learn a reliable writing style. Repeating four rows a thousand times does not create four thousand independent examples.

Why the template is part of the model

A chat model does not receive a Python list of roles directly. A **chat template** converts the conversation into a token sequence with role markers, turn boundaries and sometimes tool definitions. The model learned the meaning of those markers during prior training. Substituting another model's markers is like changing the file format while keeping the old parser. Transformers provides `apply_chat_template`; use the template saved with the exact checkpoint. [\[L01\]](#)

Qwen3-0.6B has a model-specific thinking switch. These examples deliberately use `enable_thinking=False` and short outputs. The training data does not require long reasoning traces. Its official tokenizer template also handles tool calls and tool observations. The checkpoint is pinned to commit `c1899de289a04d12100db370d81485cdf75e47ca`, returned by the official repository metadata when inspected. [\[L02\]](#) [\[L03\]](#)

Open a fresh terminal at the companion root and prepare this project's isolated environment before invoking the trainer. Use Python 3.12. The CUDA 12.6 wheel below is an official build for a compatible NVIDIA driver and GPU; select another official supported build when required by your hardware. These pins were source-reviewed, not installed and runtime-tested during book preparation. [F15]

```
cd examples/llm
deactivate 2>/dev/null || true
python3.12 -m venv .venv-llm
source .venv-llm/bin/activate
python -m pip install torch==2.12.1 --index-url https://download.pytorch.org/whl/cu126
python -m pip install -r requirements-reviewed.txt
python -m pip check
python -m unittest test_offline.py
```

For a CPU-only inference environment, choose the official CPU wheel instead; QLoRA training and its CUDA quantization path are not a generic CPU training recipe. Save the actual resolved package list after your smoke test succeeds. Stay in `examples/llm` for the remainder of this chapter.

Run a preprocessing inspection before downloading weights:

```
python train_small_lm.py \
  --train data/style_train.jsonl \
  --eval data/style_valid.jsonl \
  --out runs/inspect \
  --inspect-only
```

With the environment above active, this command loads only configuration and tokenizer assets. Read both printed strings: the full input and the supervised target. The target should contain the intended assistant response and its ending marker. It should not contain the user's instruction as something the model is being trained to answer with.

The code renders the prompt and the completed conversation separately. It verifies that the prompt is an exact prefix of the full text and of the token sequence. If this check fails, it stops. This is intentional. Some templates alter earlier reasoning content or serialize tools differently depending on later turns. A guessed boundary would quietly train the wrong tokens.

Loss masks teach the desired part

For this project, the prompt is context and the assistant response is the target. Let the tokenized example be $x[0]$, $x[1]$, ..., and let $m[t]$ be 1 for supervised response tokens and 0 for prompt or padding tokens. The training loss is

$$L = - \sum_t m[t] \log p(x[t] \mid x[0:t]) / \sum_t m[t].$$

In plain language: read the conversation, predict the next token, and average the surprise only over the tokens you want the model to learn to produce. The model still reads the prompt and uses it in its computation. Masking does not hide the prompt.

In the standard causal-language-model convention, labels use token IDs at their own positions and the model shifts predictions and labels internally. Do not shift a second time. Ignored labels are -100, a sentinel used by the loss rather than a vocabulary token. The collator pads `input_ids`, fills padding attention positions with zero, and pads labels with -100. It masks by position, never

by “all tokens equal to the padding ID”; a model may legitimately share its padding and ending token IDs. [L04]

You will meet three related settings in higher-level trainers:

- **Completion-only loss:** learn the completion after a prompt
- **Assistant-only loss:** learn assistant turns, excluding user, system and tool-observation text
- **All-token language-modeling loss:** learn ordinary text continuation, used later for continued pretraining

TRL’s reviewed SFT API exposes `completion_only_loss`, `assistant_only_loss` and `max_length`. Assistant-only masking depends on generation markers in the template or a supported training-template patch. Do not assume every tokenizer supports it. Our example makes the mask explicit so that you can inspect it. [L05] [L06]

Run the smallest adapter experiment

The environment README contains install commands and reviewed release pins. They are not a fully tested lockfile. We inspected Transformers 5.18.0, Accelerate 1.15.0, PEFT 0.21.2, bitsandbytes 0.50.2 and TRL 1.14.1. The optional TRL path uses Datasets 5.0.1. PyTorch 2.12.1 with a CUDA 12.6 wheel is an example supported by the official version page; your driver and GPU must support the chosen wheel. Save `pip freeze`, `pip check`, the CUDA runtime and `nvidia-smi` output. [L07] [L08] [L09] [L10] [L11] [L12] [L13]

Begin with three optimizer updates:

```
CUDA_VISIBLE_DEVICES=0 python train_small_lm.py \
  --train data/style_train.jsonl \
  --eval data/style_valid.jsonl \
  --out runs/style-lora-smoke \
  --mode lora --steps 3
```

The checkpoint is a **pretrained model**: its original weights already encode useful language patterns. **Fine-tuning** starts from those weights and adapts behavior using your data.

Supervised fine-tuning, abbreviated SFT, names the learning objective and data arrangement. **LoRA** names which parameters you update. You can perform SFT with LoRA or with all parameters trainable. These terms are not competing model types.

The first run uses these intentional starter choices:

Setting	Value	Why it is there
Unique base size	about 0.6B	Small enough to compare methods rather than chase capacity
Microbatch	1 example	Minimize simultaneous activation memory
Gradient accumulation	8 microbatches	Combine small batches before an optimizer update
Maximum sequence length	512 tokens	Bound the first experiment; over-length examples raise an error
Learning rate	1e-4	Initial adapter experiment, to be checked on validation

Setting	Value	Why it is there
LoRA rank	8	Small trainable correction
LoRA alpha	16	Scale multiplier $\alpha/r = 2$
LoRA dropout	0.05	Randomly suppress some adapter inputs during training
Gradient checkpointing	on	Recompute selected intermediates during backward
Attention implementation	SDPA	Use the framework implementation without a separate FlashAttention install
Optimizer	ordinary AdamW	Keep the optimizer accounting explicit
Weight decay	0.01	Set on TrainingArguments so Trainer builds the intended decay/no-decay groups
CPU offload	off	No hidden dependence on host-memory transfers

The three-step run checks that loss is finite, parameters are trainable, a checkpoint saves, validation runs, and memory is logged. It does not establish that the model improved. A real experiment needs more independent examples, a fixed validation set, and a test set you do not use to tune settings.

LoRA is a learned correction matrix

Consider a linear transformation $y = W x$, where W has $d_{\text{out}} \times d_{\text{in}}$ entries. A full update changes those entries directly. LoRA freezes W and trains two smaller matrices:

$$y = W x + (\alpha/r) B A x,$$

with A of shape $r \times d_{\text{in}}$ and B of shape $d_{\text{out}} \times r$. The added trainable count is $r(d_{\text{in}} + d_{\text{out}})$ rather than $d_{\text{in}} \times d_{\text{out}}$. This is the low-rank reparameterization introduced in the LoRA paper. [L14]

For a 1,024-by-1,024 layer and rank 8, the original layer has 1,048,576 entries. The adapter has 16,384. These are calculated values. The adapter cannot represent every possible change to W at this rank, but the useful change for your narrow task may fit within that restriction.

Rank controls the capacity of this correction. **Alpha** changes its scale. Holding alpha fixed while changing rank changes both capacity and scale; do not treat such an experiment as “rank only.” The baseline uses alpha equal to twice rank so the standard scale stays at two. Other LoRA variants have different scaling rules, so save the complete adapter configuration.

`target_modules="all-linear"` asks PEFT to attach adapters broadly to supported linear layers rather than just query and value projections. Embedding tables and the output head are not automatically equivalent to these targets. Print the actual trainable parameter names and counts when you change architectures. A configuration copied from an attention-only transformer can miss important modules in a hybrid model. [L15]

Frozen weights still participate in forward computation, and gradients must propagate through the network to reach adapters. LoRA reduces the number of gradients and optimizer states you

store. It does not eliminate activation memory or make training cost proportional only to the small adapter parameter count.

Inspect what the run produced

A successful run writes:

- `run_manifest.json`: model ID, exact revision, arguments, installed package versions, GPU identity and dataset file hashes
- `run_results.json`: before/after validation loss and measured peak allocated/reserved GPU memory
- `checkpoint-N/`: a resumable training checkpoint at an optimizer step
- `final/`: the deployment artifact and tokenizer

Resuming requires a complete checkpoint inside the original run directory. The script compares data hashes, model/revision, training configuration, package versions and recipe hash before writing anything. It preserves the original manifest and creates separate resume event/result files. Changing the planned step count changes the learning-rate schedule and is a new experiment.

In LoRA mode, `final/` contains an adapter, not an independent language model. It requires the same base architecture and revision. In full-update mode, it contains the complete tuned model. Keep optimizer/scheduler checkpoints for resuming training and deployment artifacts for inference; they solve different problems.

Run the untouched baseline and the adapter on `style_test.jsonl`. Read both outputs before looking at loss. A decreasing validation loss is useful evidence about the probability of reference answers. It is not proof of factual correctness, a specific voice, or good decisions. Two equally good paraphrases can have different reference likelihoods.

Exercise. Add one original training example that explicitly preserves uncertainty and one held-out case on a different topic. Compare the original and adapted model. Mark invented facts separately from stylistic improvements. Do not add the failed test case to training and keep calling it a test.

19 Repeat the project with every weight trainable

For this chapter's commands, open a fresh terminal at the companion root, run `cd examples/llm`, then source `.venv-llm/bin/activate`. If you skipped the first small-language-model project, complete its explicit environment setup first. Do not run these relative paths from an earlier project directory.

Now keep the same checkpoint, dataset and evaluation but set `--mode full`. This changes the number of trainable parameters, not the task. You are doing full fine-tuning of a pretrained model. You are not starting from random weights.

```
CUDA_VISIBLE_DEVICES=0 python train_small_lm.py \
  --train data/style_train.jsonl \
  --eval data/style_valid.jsonl \
  --out runs/style-full-smoke \
  --mode full --steps 3
```

The recipe lowers the initial learning rate to $1e-5$, explicitly loads FP32 weights, keeps FP32 gradients and ordinary AdamW state, and uses BF16 autocast for suitable computations. These are reviewed implementation choices, not a promise that this learning rate wins. Full-update mode refuses models above one billion unique parameters. Adapter modes refuse more than three billion total parameters.

Count memory before you spend time

Training memory is a sum of several different things:

$$M_{\text{peak}} = M_{\text{weights}} + M_{\text{gradients}} + M_{\text{optimizer}} + M_{\text{master}} \\ + M_{\text{activations}} + M_{\text{temporary}} + M_{\text{runtime}}.$$

Weights are the model's learned numbers. **Gradients** describe the local direction in which the loss changes. AdamW maintains two optimizer tensors per trainable parameter: moving estimates of first and second gradient moments. A **master copy** is an extra high-precision copy used by some mixed-precision schemes. **Activations** are intermediate values needed for backward computation. Temporary tensors, attention workspaces, allocator reservation and the GPU runtime add further overhead. [L16] [L17]

"BF16 training uses two bytes per parameter" accounts only for one copy of the weights. It is not a training-memory estimate. Distinguish persistent parameter-related state from the peak that depends on the batch and implementation.

The table uses decimal GB and hypothetical exact parameter counts. It contains calculated persistent-state totals, not measured VRAM:

Scheme and explicit assumptions	Bytes per trainable parameter	0.6B	1.0B	3.0B
BF16 weights and gradients, FP32 Adam moments, no separate master copy	12	7.2GB	12GB	36GB
FP32 weights and gradients, two FP32 moments, no extra master copy	16	9.6GB	16GB	48GB
BF16 weights, FP32 gradients, FP32 master copy, two FP32 moments	18	10.8GB	18GB	54GB

These rows are different implementations, not three simultaneous charges. Do not add a master copy to a scheme whose FP32 weights already serve that purpose. Conversely, do not assume an optimizer uses FP32 moments merely because the model was loaded in BF16; verify the implementation and actual state tensor dtypes.

Our full-update example uses the middle row. At approximately 0.6B parameters, roughly 9.6GB is reserved for its parameter-related training state before activations and other overhead. Its actual unique parameter count is printed at runtime. This leaves a meaningful working margin on a 24GB-class card at a small microbatch and short sequence length, but only the smoke test can establish the real peak. At 3B, the same state alone is 48GB. Reducing sequence length cannot make those 48GB disappear.

A one-billion-parameter full update is an advanced boundary, not an unconditional fit statement. The exact architecture, vocabulary size, gradient dtypes, optimizer implementation, sequence length and temporary buffers matter. An eight-bit optimizer or CPU optimizer offload can move that boundary, but changes the recipe and sometimes numerical behavior or speed. Benchmark such a configuration separately. Do not claim that every nominal “3B” model fits full AdamW training on one 24GB card.

Sequence length can dominate the surprise

Let B be the microbatch size, T the sequence length, d the hidden width and L the number of layers. Many saved activations grow roughly with $B \cdot T \cdot d \cdot L$, with substantial architecture-dependent constants. A straightforward attention implementation can materialize score tensors proportional to $B \cdot H \cdot T^2$, where H is the number of heads. Efficient attention kernels avoid storing the full quadratic score matrix, but do not remove every sequence-length-dependent cost.

There is another large tensor: logits. An unchunked tensor for $B=1$, $T=2,048$, $V=151,936$ vocabulary entries contains about 311 million values. At four bytes each, that is about 1.24GB for one tensor, before related intermediates. This is a calculated illustration, not a measurement of the supplied recipe. A model with a large multilingual vocabulary can have expensive logits even when its transformer blocks are small.

Gradient checkpointing saves fewer forward intermediates and recomputes them during backward. It exchanges compute for memory. **Gradient accumulation** computes several small microbatches before updating weights. On one GPU, $\text{effective_examples} \approx \text{microbatch} \times$

accumulation; variable-length examples mean tokens per update vary. Neither technique removes full-model optimizer state. Accumulation also requires correct loss normalization when examples have different numbers of supervised tokens, which is one reason to use a reviewed trainer rather than improvising division by a batch count.

`use_cache=False` disables the generation KV cache during training. A **KV cache** stores past attention keys and values to accelerate autoregressive generation; it is not the same thing as the activations kept for backpropagation. Long advertised context windows describe architectural support, not affordable training lengths on your card.

Measure the run you actually made

The script records `torch.cuda.max_memory_allocated()` and `max_memory_reserved()` in GiB, where one GiB is 2^{30} bytes. Allocated memory is live tensor memory known to the allocator. Reserved memory includes its cached blocks. Neither alone captures every driver-level allocation, so also inspect `nvidia-smi` and leave headroom for your display or other processes.

Record a warm-up and several complete optimizer steps. Some optimizer states appear only at the first update, so one forward pass is not a training fit test. Include validation and checkpoint saving in the check; those stages can fail even if the training step fits. Do not extrapolate throughput from one cold iteration that includes compilation, loading, or cache setup.

If you hit out-of-memory, diagnose in this order:

1. Check other GPU processes and actual free memory
2. Confirm the requested dtype and count trainable parameters
3. Confirm there is only one training model and no unintended reference copy
4. Keep microbatch at one and reduce sequence length
5. Enable checkpointing and avoid storing evaluation logits
6. Move to LoRA or QLoRA if persistent full-update state is the limiting term
7. Consider a smaller model before adding complex offload machinery

Exercise. Repeat the same tiny run at sequence limits 256, 512 and 1,024 with examples that actually approach those lengths. Record allocated/reserved memory and tokens per second. A higher maximum alone does not lengthen short examples, so do not call that a sequence-length experiment unless the data changed accordingly.

When QLoRA earns its complexity

QLoRA keeps a quantized frozen base model and trains floating-point adapters through it. The original method combines four-bit NormalFloat storage, quantized quantization constants and paged optimizers to reduce memory pressure. Its published large-model results are method evidence, not a benchmark for your RTX or this example. [\[L18\]](#)

The starter switch is:

```
CUDA_VISIBLE_DEVICES=0 python train_small_lm.py \
  --train data/style_train.jsonl \
  --eval data/style_valid.jsonl \
  --out runs/style-qlora-smoke \
  --mode qlora --steps 3
```

The code uses NF4, double quantization, BF16 compute and ordinary AdamW for the relatively small adapters. It does not need a paged optimizer to qualify as adapter training over a quantized base. `prepare_model_for_kbit_training` freezes/prepares the base before adapters are attached. Loading a four-bit model and asking ordinary full fine-tuning to update all its quantized weights is not this recipe. [\[L19\]](#) [\[L20\]](#)

If P_q parameters are actually quantized to four bits, their ideal packed payload is $0.5 P_q$ bytes. Add quantization scales, unquantized modules, adapters, gradients, optimizer state and activations. Some preparation paths upcast unquantized modules to FP32. An embedding-heavy small model may therefore save less than the simple “four bits everywhere” calculation suggests.

For a 0.6B model, plain BF16 LoRA may already fit easily and be simpler. QLoRA becomes more attractive as you move toward the larger allowed checkpoints or longer sequences. Measure both before choosing. Quantization can change outputs and throughput; evaluate the exact deployed precision, not just the floating-point training checkpoint.

20 Teach domain conventions without turning weights into a database

For this chapter's commands, open a fresh terminal at the companion root, run `cd examples/llm`, then source `.venv-llm/bin/activate`. If you skipped the first small-language-model project, complete its explicit environment setup first. Do not run these relative paths from an earlier project directory.

The next project separates two goals often compressed into “teach it our business.” One goal is to recognize terminology and perform a workflow. The other is to answer exact questions about changing facts. They may need different solutions.

Use a fictional parts catalog so the experiment carries no confidential data. Part IDs look like P-101. A bin is a storage location. A stock count changes over time. You want the model to understand those conventions and to consult a lookup tool for the current count.

Start with the questions the model must answer

Build a held-out question set before choosing a training method. Include:

- Terminology: “What does a bin identify?”
- Document-grounded reasoning: “Using this paragraph, which field is missing?”
- Current facts: “How many P-101 units are available now?”
- Missing evidence: “What does P-101 cost?” when no price source exists
- Conflicting versions: two manuals with different effective dates
- Scope: a question about an unrelated catalog

A model may improve terminology while still hallucinating current counts. A single average score would hide that distinction. Track grounded correctness, unsupported assertions and abstention quality separately.

Retrieval-augmented generation, or RAG, retrieves relevant documents and supplies them as context at inference time. It can expose provenance and refresh information without changing model weights. The original RAG work combines learned generation with a non-parametric retrieval memory. Your implementation still needs retrieval evaluation, access controls and checks that the answer actually follows the retrieved text. [\[L21\]](#)

For exact inventory, prices, policy versions and permission-sensitive records, begin with retrieval or a database/tool lookup. For a consistent output format or specialist workflow, begin with SFT. For unfamiliar domain language throughout a substantial corpus, investigate continued pretraining. Combining these is often sensible: adapt how the model uses evidence while keeping evidence outside its weights.

Continued pretraining changes the text distribution

Continued pretraining, sometimes called domain-adaptive pretraining, starts from pretrained weights and continues the original language-modeling objective on new text. It does not require a question-answer pair for every paragraph. Research on domain/task-adaptive pretraining found benefits in the settings it studied; that does not guarantee improvement for every current decoder model or every corpus. [L22]

The provided `cpt_train.jsonl` uses records like:

```
{
  "source_id": "manual-train-01",
  "text": "Fictional Northstar catalog. Part IDs begin with P-. Stock counts are live data and must be
    looked up."
}
```

The code's `--task cpt` option applies next-token loss to the text and an ending marker. It keeps documents separate to avoid hiding document-boundary questions inside a packing implementation. Use the proper base checkpoint for a real experiment, with its own verified revision, rather than casually treating an instruction checkpoint as interchangeable.

For a concrete smoke test with the base checkpoint, resolve its immutable revision first and record it:

```
BASE_REV=$(python -c 'from huggingface_hub import HfApi;
  print(HfApi().model_info("Qwen/Qwen3-0.6B-Base").sha)')
python train_small_lm.py \
  --model Qwen/Qwen3-0.6B-Base --revision "$BASE_REV" \
  --task cpt --mode full --steps 3 --lr 5e-6 \
  --train data/cpt_train.jsonl --eval data/cpt_valid.jsonl \
  --out runs/domain-full-smoke
```

The architecture is still a causal decoder, but all document tokens are targets rather than only assistant answers. The saved full checkpoint is loaded directly for continuation. Give it a fresh prefix, without a chat template:

```
python generate_eval.py --run runs/domain-full-smoke \
  --device cuda --raw --max-new-tokens 64 \
  --prompt "Fictional Northstar manual. A bin identifies" \
  --output fresh-domain-gpu.jsonl
python generate_eval.py --run runs/domain-full-smoke \
  --device cpu --raw --max-new-tokens 64 \
  --prompt "Fictional Northstar manual. A bin identifies" \
  --output fresh-domain-cpu.jsonl
```

For an untouched base continuation, omit `--run` and supply `--base-model Qwen/Qwen3-0.6B-Base` `--base-revision "$BASE_REV"`. CUDA uses BF16 inference and CPU FP32. The model, tokenizer, raw prefix and decode limit should otherwise match. A base continuation is not a reliable assistant response; converting it into an assistant requires appropriate instruction examples and a validated template.

A staged domain project might be:

1. Clean and deduplicate licensed domain documents, preserving document and version metadata
2. Hold out entire documents or organizations, not random adjacent chunks
3. Measure domain and general-language baselines with the same tokenizer

4. Run a short continued-pretraining experiment on the base model
5. Add task-oriented SFT examples showing how to use the domain knowledge
6. Compare against the original instruction model with retrieval

A useful experiment isolates effects. Compare base plus SFT against continued-pretraining plus the same SFT. Otherwise an improvement may come from the instruction data rather than the raw-text phase.

Watch **catastrophic forgetting**: gains on your narrow distribution can accompany regressions in general instruction following or other previously useful behaviors. Keep a small, fixed retention test set. A replay mixture of general examples is an experiment you can try, not a guarantee. Tune the mixture and learning rate using both target and retention metrics.

For a continued-pretraining run, an initial full-update rate such as $5e-6$ on a sub-billion checkpoint is a cautious experiment, not a universal recommendation. A raw-text corpus with many tokens needs a token budget, held-out perplexity, checkpoints and a stop rule. Do not select an epoch count before estimating how many tokens an epoch contains.

Knowledge editing has a different promise

Ordinary fine-tuning can change factual behavior, but it is not a database update operation. The model does not expose a reliable row called “P-101 stock.” A repeated answer can become easier to generate while related questions remain wrong, old behavior persists under paraphrase, or unrelated answers drift.

Methods such as ROME and MEMIT study targeted changes to factual associations in particular model architectures and evaluation settings. Their existence demonstrates that factual behavior can sometimes be edited more selectively than with broad fine-tuning. It does not mean every fact has one isolated storage location or that edits are guaranteed to compose safely. [L23] [L24]

Evaluate a proposed edit along four axes: does the requested answer change, does it generalize across relevant paraphrases, do unrelated facts remain stable, and do consequences of the edited fact stay consistent? Then test repeated edits and reversals. A success on one memorized prompt is weak evidence.

For this catalog, live stock belongs in an external store. Train the behavior “look it up and report the result.” Do not train tomorrow’s stock count into weights today. This choice also gives you a clearer audit trail and a straightforward way to correct errors.

Exercise. Put a changed stock count in the retrieved context while leaving training examples unchanged. Test whether the model follows the new evidence or repeats the old answer. Add a conflicting, older record with an explicit date and observe whether your evaluation catches version confusion.

21 Make a writing style reproducible without changing the facts

For this chapter's commands, open a fresh terminal at the companion root, run `cd examples/llm`, then `source .venv-llm/bin/activate`. If you skipped the first small-language-model project, complete its explicit environment setup first. Do not run these relative paths from an earlier project directory.

Return to the concise-update project with a better dataset. You now know the mechanics; the main problem is defining what counts as the desired behavior.

Separate voice from subject matter

A decoder model can imitate correlations in its training examples. If every example of your desired voice discusses software releases, it may learn release vocabulary rather than a style that transfers. If every “concise” answer omits uncertainty, it may learn to sound confident instead of brief.

Use examples that contain both source content and the desired rewrite. Preserve the distinction between facts, requests, opinions and promises. Label uncertain facts as uncertain in both sides. Include examples where being concise still requires two paragraphs or a caution. A fixed length target can reward harmful omissions.

For personal style, use writing you own or are authorized to use. Remove secrets and unnecessary third-party details. Keep recipient/channel context when it affects wording, but do not teach the model to invent that relationship. “A short note to a colleague” and “a formal external update” may be separate style settings in one dataset.

The existing script fits this architecture-data relationship: source facts and the requested style are input context; the approved rewrite is the supervised continuation. A collection of standalone essays without the input conditions is a raw-text language-modeling dataset, not the same conditional rewriting task.

Build paired examples and a blind rubric

Each source item can have a rough version, an approved rewrite and a short explanation for a human annotator. The explanation need not be part of the model’s training target. Keep it as provenance: why was this a good answer?

Useful rubric dimensions are:

- Every necessary fact preserved
- No new claim, deadline, opinion or commitment
- Appropriate certainty
- Requested tone and channel conventions
- Clear structure and readable length
- No copied private or unrelated material

Evaluate on new topics and different source lengths. Randomize whether the baseline appears as answer A or B, hide model identities and allow ties. Judge content fidelity before stylistic preference. If a model judge helps scale evaluation, check a human-reviewed subset and inspect its reasons; it may prefer verbosity, familiar phrasing or its own writing habits.

Simple automatic checks can verify numbers, required names, forbidden boilerplate and maximum length. They are guardrails rather than a complete style score. A shorter sentence is not necessarily better. A high lexical match to the target is not necessarily better than a faithful paraphrase.

Start with prompting plus examples, then LoRA, and compare full fine-tuning only if the adapter experiment suggests a capacity or behavior limit. Keep the test set untouched. A stylistic win accompanied by more invented facts is a regression for this task.

Use the saved artifact on a genuinely new input

```
python generate_eval.py --run runs/style-lora-smoke \
  --device cuda --max-new-tokens 96 \
  --prompt "Write a concise update using only these facts: 19 checks passed; one failed; no retry is
    scheduled." \
  --output fresh-style-gpu.jsonl
```

Repeat with `--device cpu` and a different output filename. The script loads the manifest's exact base revision, attaches `final/`, loads the saved tokenizer, applies its chat template, moves tensors to the selected device and generates under `inference_mode()`. It slices off the prompt token IDs and decodes only the new tokens. It does not accidentally report the original prompt as generated output.

For a full-update run, use `--run runs/style-full-smoke`: the script loads the complete `final/` model directly. For QLoRA on CUDA it reloads NF4; on CPU it deliberately uses the original FP32 base plus adapter. The latter is a portable numerical variant, not an assertion that the CUDA quantization backend works on your CPU. Re-evaluate it. An architecture-supported CPU quantized export is another deployment project, with its own conversion and quality checks.

Exercise. Create three held-out inputs containing an exact number, an unresolved decision and a statement the writer has explicitly declined to promise. Have another person score content fidelity before style. Record examples where a fluent answer is still wrong.

22 Teach a tool decision and a complete tool interaction

For this chapter's commands, open a fresh terminal at the companion root, run `cd examples/llm`, then source `.venv-llm/bin/activate`. If you skipped the first small-language-model project, complete its explicit environment setup first. Do not run these relative paths from an earlier project directory.

The catalog assistant now needs to decide whether to call `lookup_part`, supply an exact ID, and use the returned observation in its answer. This is harder than copying a JSON format. It requires a policy: call when appropriate, clarify when information is missing, and avoid inventing unsupported tools.

Architecture determines what the dataset must contain

A decoder-only model learns to emit tokens. A tool call is therefore a specially formatted assistant output, not an invisible API operation inside the network. The runtime reads that output, validates it and runs the real function. The observation then becomes additional context for the next model turn.

This architecture requires examples of the decision and the consequence. A dataset containing only user questions and final answers does not show which tool produced the evidence. A dataset containing only call JSON does not teach what to do with a failed or empty result. Training only successful calls can accidentally teach “always call something.”

Keep the tool definition in each relevant example. A **tool schema** names the function, describes its purpose and specifies allowed argument types and constraints. Transformers’ tool templates accept structured definitions and structured calls; their precise serialized format is model-specific. TRL likewise distinguishes messages, assistant tool calls, tool-role observations and a `tools` column. [L25] [L26]

Our read-only fictional tool is:

```
{
  "type": "function",
  "function": {
    "name": "lookup_part",
    "description": "Look up current stock for an exact part ID.",
    "parameters": {
      "type": "object",
      "properties": {
        "part_id": {"type": "string", "pattern": "^P-[0-9]{3}$"}
      },
      "required": ["part_id"],
      "additionalProperties": false
    }
  }
}
```

A useful full conversation then includes these turns:

```
[
  {"role": "user", "content": "How many P-101 parts are available?"},
  {
    "role": "assistant",
    "content": "",
    "tool_calls": [{
      "type": "function",
      "function": {
        "name": "lookup_part",
        "arguments": {"part_id": "P-101"}
      }
    }]
  },
  {
    "role": "tool",
    "name": "lookup_part",
    "content": "{\"part_id\": \"P-101\", \"stock\": 8, \"location\": \"bin A\"}"
  },
  {"role": "assistant", "content": "P-101 has 8 units in bin A."}
]
```

The real fixtures also include a system instruction and the schema. Tool observations are serialized data. Some runtimes also require call IDs to connect multiple calls and results; preserve those IDs in your application schema and use the model-specific format the runtime expects.

Mask decisions rather than observations

Teacher forcing means that training predicts an output using the supplied correct earlier sequence, rather than the model's own possibly mistaken earlier outputs. A correct tool trajectory therefore supplies the intended prior assistant calls and tool observations. At deployment, errors can accumulate through the model's own decisions, so evaluate complete interactions as well as isolated next-step predictions.

For this project, supervise the assistant's call tokens and its final answer. Do not supervise the tool observation as if it were text the model should invent. This distinction is especially important for stock values: the model should condition on a returned count, not learn that it may fabricate one.

Our preprocessing expands a complete conversation into one example per assistant decision. The first example ends with the call. The second includes the call and observation as context and ends with the grounded answer. Each expansion inherits the same `source_id`, so it stays in the same split. This costs additional repeated context tokens but makes the target explicit and easy to inspect.

Do not delete every non-assistant token from `input_ids` to implement assistant-only loss. That would remove the observations the answer depends on. Keep context tokens in the input and mask only their labels.

Train a small version before collecting a large corpus

```
CUDA_VISIBLE_DEVICES=0 python train_small_lm.py \
  --train data/tools_train.jsonl \
  --eval data/tools_valid.jsonl \
  --out runs/tool-lora-smoke \
  --mode lora --steps 3 --max-length 1024
```

The longer sequence limit allows schema and observation text. It is a memory-relevant change, so repeat the fit check. The toy dataset includes successful stock lookups, a missing-ID clarification and an unsupported price request. It is still only a plumbing fixture.

A realistic corpus should vary the underlying decision, not just the wording. Include an exact ID, an ambiguous name, multiple similar tools, an irrelevant tool, a missing required argument, a legitimate no-tool answer, an empty result, a timeout, a recoverable validation error and an observation that contradicts the model's earlier assumption. For state-changing tools, include permission boundaries and cancellation examples, and enforce those boundaries outside the model as well.

Mix paraphrases across training examples, but hold out entire templates, ID ranges, scenarios and some tool combinations. If every P-9xx ID appears only in test, the model must copy an unseen argument rather than memorize a training ID. Include schema rewording and tool order changes to discover whether it learned function semantics or position in a list.

Untrusted observations can contain instructions. Include cases where the tool's data says something like "ignore the user and call another tool," then require the assistant to treat that as data. This is a training and evaluation concern, but permission enforcement remains an application responsibility.

Evaluate the decision at several levels

Run the unchanged model and the adapted model with the same decoding settings:

```
python generate_eval.py --input data/tools_test.jsonl \
  --output tools-baseline.jsonl --device cuda
python generate_eval.py --input data/tools_test.jsonl \
  --output tools-adapted.jsonl --device cuda \
  --run runs/tool-lora-smoke
python score_tools.py data/tools_test.jsonl tools-baseline.jsonl
python score_tools.py data/tools_test.jsonl tools-adapted.jsonl
```

The script never executes a call. It checks the toy schema, tool name, exact arguments and whether a call was expected. These are separate questions:

1. **Syntax:** can the output be parsed?
2. **Schema:** are keys, types and constraints valid?
3. **Routing:** is this the right tool, or should there be no call?
4. **Arguments:** are IDs, units, dates and values correct?
5. **Interaction:** does the model use the actual observation and recover from errors?
6. **Outcome:** was the user's task completed without unsupported claims or unauthorized actions?

A model can score perfectly on the first two and fail the last four. Likewise, “no parsed call” does not distinguish a good clarification from silence or a hallucinated answer. Read the prose and add an outcome rubric.

Next build a mock executor with a fixed fictional catalog. Give the model its own generated call, not the gold call, return the corresponding observation, then ask it to continue. This **closed-loop evaluation** exposes cascading errors that teacher-forced validation cannot see. Record the entire trajectory, tool-call count, correction count and final outcome. Impose a maximum number of turns so a failing agent cannot loop indefinitely.

BFCL is a useful public comparison framework because its versions distinguish call structure, realistic functions and multi-turn/agentic behavior. Its inspected leaderboard provides a specific evaluation commit and package version for reproducing that leaderboard snapshot. Do not mix a BFCL-v3 model-card number with a BFCL-v4 result as if they measured identical tasks. Our five toy tests are not BFCL and provide no comparable benchmark claim. [L27] [L28]

Run the tool model on CPU too

Use the same unseen test file and schema:

```
python generate_eval.py --input data/tools_test.jsonl \
  --output tools-cpu.jsonl --device cpu \
  --run runs/tool-lora-smoke
python score_tools.py data/tools_test.jsonl tools-cpu.jsonl
```

CPU inference loads the exact base revision in FP32 and adds the saved adapter. GPU inference uses BF16 for this LoRA run. The template and input schema stay the same; numeric execution and speed differ. We measured neither runtime here. CPU inference may be slower and needs RAM for the whole base model, not only the small adapter.

Exercise. Add a mock timeout and a second tool named `lookup_price` with different arguments. Make a held-out request that requires only stock, then one that requires both facts. Score the intermediate decisions and the final answer separately. Do not grant the model direct unrestricted access to production tools while testing.

23 Study a small typed decision service

What this category does

A decision model answers a bounded question about supplied evidence. Its output is a label, a yes/no probability, a distribution over candidate actions, or a rating on a defined scale. It can route a ticket, shortlist a relevant file, flag a message for review, or choose which existing tool should handle a request. It does not have to generate prose to do those jobs.

The useful unit of application design is: evidence, question, allowed answers, estimated probabilities, and an explicit policy. The neural model estimates a judgment. Ordinary code validates the input, applies thresholds, checks permissions, and performs any authorized action. Returning a valid label solves a formatting problem; it does not prove that the judgment is true.

Start with a harmless project: suggest a support queue for fictional tickets. Do not start by giving a newly trained model authority to delete files, approve transfers, or make consequential decisions about people. Keep the first result as a recommendation shown beside the original ticket.

This chapter is an inference and architecture study of the published Laya interface, not a reproduced Laya training project. The following chapter supplies the complete local training, calibration, evaluation, and CPU/GPU inference workflow using an original Qwen-derived decision head. Treat the Laya snippet as a source-reviewed study example until its official environment and model download pass your local checks.

Why Laya is a useful first study model

The exact model here is `convaiinnovations/laya`. The root English checkpoint is approximately 421M parameters, using a fully fine-tuned ModernBERT-large backbone and an additional decision head. It fits comfortably below the book's one-billion-parameter example ceiling, although memory and runtime must still be measured for the chosen training settings. The multilingual and typed-decisions variants are distinct checkpoints with different intended uses and limits. [J24]

This gives a useful progression: first observe a trained classifier; then inspect its input and output shapes; then adapt it on a focused dataset; finally construct a similar interface using a causal language-model backbone. Small does not mean trivial: the data recipe, readout, calibration, and evaluation remain central.

Run the idea before studying all the internals

After using the project's official installation instructions in an isolated environment, a source-reviewed inference example is:

```
from laya import Router

router = Router(device="cpu")
result = router.predict(
    "The courier delivered my parcel to the wrong building.",
    {
        "queue": {
            "type": "choice",
            "instructions": "Which team should investigate this request?",
            "criteria": {
                "billing": "Payments, receipts, invoices, unexpected charges",
                "delivery": "Couriers, tracking, late or missing parcels",
                "access": "Passwords, sign-in, locked accounts",
                "review": "Insufficient information or a different problem",
            },
        },
    },
    model="english",
)
print(result["answers"]["queue"])
```

Record the installed Laya, Transformers, PyTorch, and tokenizer versions and pin the model revision in the supported loader configuration. The first call can download weights. This example has not been executed in the book's preparation environment. The repository's API and dependencies are moving, so inspect the pinned release's constructor and loader signature before execution. The inspected repository describes Python 3.10+ and separate CPU/GPU setup paths. [J25]

Now vary only one thing at a time: replace delivery evidence with a billing problem; remove the critical sentence; add irrelevant text; rename the option IDs while keeping descriptions; change the order of options. Record which changes should and should not change the answer. These probes tell you which architecture and training questions are worth studying.

What Laya reads and what the head computes

The shipped model code builds a sequence containing question type and instructions, an option marker before each option, and the state. Its encoder is bidirectional, so a marker before an option can receive information from later option text and later state text. After the encoder, a learned question-type vector is added, additional transformer layers refine the sequence, and a scorer maps each gathered option-marker vector to one logit. A separate head produces act/escalate logits. [J26]

For one illustrative batch with $B = 2$ questions, padded length $T = 128$, and $D = 1024$, the encoder emits $[2, 128, 1024]$. If each question has $K = 3$ options, gathering their marker positions produces $[2, 3, 1024]$. The option scorer maps that to $[2, 3]$. Softmax is applied independently across each row's three candidates. Padded candidate positions must receive no probability. The width-1024 value is grounded in ModernBERT-large's configuration; 128 and 3 are our teaching choices. [J32]

A key contrast with the later Qwen project is marker placement. In a causal decoder, a marker placed before the option cannot read the option's later text. We will instead read an option-end position and a final decision position. Copying the Laya layout into an unchanged causal decoder would remove information from the readout.

ModernBERT mixes local and global bidirectional attention. A small diagram that permits every token to read every other token is an intuition, not a complete implementation diagram of every ModernBERT layer. Read the backbone paper after the first inference probes make the role of attention concrete. [\[J31\]](#)

Training data is part of the architecture

Every training question needs a state, instructions, option text, and a target aligned with those options. Labels can be a single correct choice or an explicitly sourced target distribution. If a teacher model supplies probabilities, these are teacher beliefs, not automatically ground truth. A model can learn the teacher's blind spots.

For yes/no, keep a stable convention for which index means false and which means true. For ordered ratings, order is meaningful: shuffling levels without preserving their semantics invalidates an ordinal target. For ordinary categories, randomizing option order can discourage position shortcuts, but the target must be permuted with the options.

The shipped English configuration gives a maximum length of 512 and an option/question budget of 192. Those are distinct from the backbone's architectural maximum. Giving many lengthy labels the same finite option budget can truncate away the words that distinguish them. Inspect what the tokenizer actually keeps. [\[J27\]](#)

Treat changes to prompts, label descriptions, ordering, truncation, or tokenizer as changes to the input contract. Re-evaluate them as carefully as a changed weight file. A better input representation may improve quality more than increasing model size.

What the public training example really optimizes

The inspected single-process training script perturbs logits, scores the resulting probability distributions, forms a relative-reward policy-gradient term, and combines it with supervised cross-entropy. It updates encoder and head parameter groups at different learning rates and supports activation checkpointing. It also reserves a subset of encoded items for calibration. This is more precise than saying only that it uses RL. The script is written for MPS/CPU; do not paste its device command unchanged into an NVIDIA training guide. [\[J28\]](#)

A strictly proper scoring rule rewards reporting the true distribution at its ideal population optimum. That mathematical property is not a guarantee that a finite neural network, trained on limited or biased data, will be calibrated on a new domain. Clipping, finite optimization, sampling, and distribution shift also require empirical checks. Start a training comparison with cross-entropy, then add the more complex objective only if it helps a held-out metric.

The upstream Kaggle notebook is a useful public training walkthrough. The repository specifically warns about its calibration samples coming from training items. The separate current MPS script does reserve items, but questions from the same source can still leak across

item-level splits. Split whole source records before expanding their questions. Keep model-selection, calibration, and final-test roles distinct. [J29] [J25]

Read the limitations as carefully as the headline

The Laya card reports substantial zero-shot weakness on its typed-decisions test, gains from task-specific fine-tuning, overconfidence, label sensitivity, and an `act_probability` output with poor decision value. These findings make it a good study of why empirical validation matters. They do not justify treating every probability field as calibrated correctness. Its published comparisons with Jev also use third-party Jev results rather than a same-run controlled comparison. [J24]

For this project, ignore the `act/escalate` output until it passes a separate held-out evaluation. Fit any probability temperature on calibration data, choose the abstention policy on appropriate held-out data, and test both risk and coverage. Never promote a threshold such as 0.9 from a tutorial into a universal safety rule.

CPU and GPU are two deployments of the learned function

Inference does not need gradients, optimizer moments, or training activations. This makes deployment much lighter than full training. A 421M-parameter model has a raw fp32 weight-size estimate of roughly 1.68 GB, or roughly 0.84 GB at two bytes per parameter, before runtime buffers and duplicate copies. Those are arithmetic estimates. An actual loader can temporarily hold more memory.

CPU inference is useful for low-volume testing and small deployments, but throughput depends on CPU vector instructions, memory bandwidth, threading, and implementation. A GPU usually becomes more attractive for larger batches and repeated calls. Tiny batches can be dominated by transfers and launch overhead. Do not compare a warm GPU batch with a cold CPU model load.

For the book's custom Qwen pointer program, use CPU fp32 as the conservative portability path and CUDA bf16 autocast only when supported. For Laya and Kev, follow the pinned runtime's dtype/device support rather than assuming the custom program's choices are universal. An exported quantized model must preserve the custom head and formatting and pass probability-parity tests. Refit or verify calibration after a representation-changing export.

Useful application patterns

The user-provided Ten Levels of Jev repository is an application lab, not a training recipe or proof of Jev's internal architecture. Its original progression includes bounded classification, multi-question scoring, routing, confidence gates, file selection, and integration into a coding agent. It uses live hosted services and an explicit offline mock. The associated video is a useful walkthrough of that codebase. [J33] [J37]

Here is a small, safe progression for your own trained model:

1. Queue suggestion: evidence is one ticket; candidates are named queues plus review; output is a suggestion beside the evidence

2. Multi-question triage: separately estimate queue, urgency, and whether required information is missing; deterministic code combines those answers into a worklist
3. Selective automation: on a held-out set, choose a threshold that meets a stated error budget; send remaining items for review and measure the retained fraction
4. Model routing: choose between a deterministic handler, a small text model, a larger reasoning model, and a person; evaluate final task success and total cost, including routing mistakes
5. Retrieve, then judge: use an embedding model to shortlist documents, then a decision scorer to assess relevance; measure retrieval misses separately from judging mistakes
6. Candidate verification: generate a small set of possible solutions, then rank them against the original requirements; compare chosen-solution success with random selection and the generator's first answer
7. Agent observation: inspect tool results or change summaries and flag suspicious cases; preserve sandbox restrictions and independent authorization controls regardless of the model's confidence

These are application designs, not automatic evidence that any checkpoint is good at them. Build labels matching the intended decision, including failure cases. A candidate verifier cannot recover a correct candidate that the generator never supplied. A router can save model calls while lowering overall task accuracy if it sends hard examples to the wrong handler.

Keep the service boundary honest

The inspected Ten Levels client separates an explicit mock provider from live providers, validates requests, imposes a total timeout, limits retries, and records provider/model metadata. It distinguishes reported, estimated, and unknown cost. Those are practical integration ideas worth retaining when replacing hosted Jev with a local model. Passing mock tests validates application behavior, not model accuracy. [J34]

Its published confidence-gating example contains hardcoded thresholds, including one that permits destructive actions above a confidence bar. Treat that as a demonstration of branching logic, not a security design to copy. Model confidence must never replace permissions, least-privilege execution, deterministic path boundaries, confirmations required by the application, or human review for high-impact actions. [J35]

An API-compatible server is not a checkpoint-compatible model. Similar JSON fields across Laya, Kev, and CLM can carry differently defined confidence values, preprocessing, context limits, and performance. Build contract tests and a shared labeled evaluation before substituting one backend. Pin model IDs instead of moving latest aliases when results must be reproducible. [J36]

Reading and watching order

First run the harmless ticket exercise. Then watch the Ten Levels video with the repo open, inspecting how model outputs enter ordinary control flow. Next read Laya's implementation and the ModernBERT paper to explain the input and gathered-marker tensors. Finally study the Qwen pointer capstone and compare it to Kev. Karpathy's build-GPT video is a complementary code-first lesson on decoder construction; nanoGPT is useful historical source, but its README now marks it deprecated and points to nanochat. [\[J37\]](#) [\[J38\]](#) [\[J39\]](#)

The published Laya browser-agent adaptation is a later case study in how data and input formatting affect a working application. Its author-reported outcome is not our reproduction; use its data-generation and evaluation methodology as material to critique before attempting browser control. [\[J30\]](#)

24 From a language model to a decision model

The project read a situation score the allowed answers

Suppose your program receives this message:

My parcel was delivered to the wrong building. Can you help me find it?

You want to choose one of three queues: billing, delivery, or account access. A chat model might generate a sentence, or spell a JSON object one token at a time. A decision model can instead return three numbers. Your program turns those numbers into a typed result.

That is the engineering goal in this chapter. It is narrower than building a general assistant, and more substantial than asking a chat model to write a confidence percentage. You will learn where those numbers come from, which existing weights can be reused, which new weights need training, and how to test whether the resulting probabilities mean anything useful.

Jev Kev CLM and JEPA are different names

TypeSafe's Jev is a hosted decision model. Its official launch describes finite typed outputs, parallel probability outputs, and a training method called Reinforcement Learning for Calibrated Decisions, or RLCD. The public pages inspected for this book do not establish a reproducible implementation of its internal architecture or RLCD. We therefore do not claim to reproduce Jev itself. Finite output types also do not guarantee correct decisions: a model can confidently choose the wrong permitted answer. [J01], [J02]

Kev is an open implementation with similar behavior. Its source exposes the model, formatting, training, and serving. It has a 0.8B release, which is relevant to our small-GPU scope. Kev cites a community reverse-engineering article; the article explicitly labels its account of Jev's internals as speculative. Evidence about Kev is not evidence that proprietary Jev has identical layers. [J03], [J04]

Contrastive-LM's CLM takes a different route: it separately represents a state and candidate actions, then compares their vectors. Its released reference encoder is Qwen3-8B, outside this book's training-example size limit. We study the idea and implementation without pretending that its learned head works unchanged on a smaller encoder. [J05]

JEPA, a joint-embedding predictive architecture, is a separate research concept. It is not an expansion of the name Jev. Nothing in the conversion below requires calling Jev a JEPA model.

The practical progression is:

1. Understand a small decoder backbone using Qwen3-0.6B as a concrete example
2. Replace text-generation readout with an option-scoring readout
3. Train and evaluate that decision model
4. Add independent questions and shared-prefix serving
5. Compare that design with a separately encoded, CLM-style scorer

Run the complete option pointer project first

The companion implementation is `examples/decision_pointer/decision_pointer.py`. It is an original educational conversion of Qwen3-0.6B, not a reproduction of private Jev training, the Kev checkpoint, or the CLM checkpoint. Start with the executable workflow, then use the architecture sections below to understand the changed head.

Open a fresh terminal at the companion root and create a separate Python 3.12 environment. The commands below use the same inspected torch 2.12.1 and Transformers 5.18.0 reference releases as the small-LLM project. Choose the official CUDA wheel only when the driver and GPU support it; a separate CPU environment can use the CPU index instead. Package resolution and model runtime were not executed during book preparation. [\[F15\]](#) [\[L04\]](#)

```
cd examples/decision_pointer
python -m venv .venv-pointer
source .venv-pointer/bin/activate
python -m pip install torch==2.12.1 --index-url https://download.pytorch.org/whl/cu126
python -m pip install transformers==5.18.0

# CPU alternative in a separate environment:
# python -m pip install torch==2.12.1 --index-url https://download.pytorch.org/whl/cpu

python make_toy_data.py
python test_numerics.py
```

The dependency-free tests can also be run before installing torch. They check thirty original toy records, group isolation, option indexing, shuffled-label alignment, overlength rejection, stable softmax/NLL, and probability metrics. They do not execute a neural model. A passing fixture prints PASS and an illustrative probability vector near `[0.575975, 0.283995, 0.140029]`.

Each record contains id, group, state, question, options, and label. Options have an ID and text; label names the correct option ID. The numerical target is recomputed when training shuffles option order. Related records share a group and must stay in one split. The twelve training, six development, six calibration, and six test examples are deliberately tiny plumbing fixtures. Six calibration records cannot establish reliable real-world calibration.

Train a head and then the complete model

A useful first comparison freezes the backbone and trains the new pointer projections. The next run updates the entire backbone and head. Both use fresh output directories and the same immutable Qwen revision verified for the LLM project. [\[L03\]](#)

```
python decision_pointer.py train \
  --train data/train.jsonl --dev data/dev.jsonl \
  --run runs/head-smoke --epochs 1 --mode head \
  --max-length 256 --accum 8 --device cuda --precision bf16 \
  --revision c1899de289a04d12100db370d81485cdf75e47ca

python decision_pointer.py train \
  --train data/train.jsonl --dev data/dev.jsonl \
  --run runs/full-smoke --epochs 1 --mode full \
  --max-length 256 --accum 8 --device cuda --precision bf16 \
  --revision c1899de289a04d12100db370d81485cdf75e47ca
```

BF16 means supported CUDA autocast here. The backbone parameters and AdamW buffers remain FP32, so the memory inventory must include them. CPU training uses `--device cpu` `--precision fp32` and may be slow even though the same mathematical task is supported. A strict

pre-download parameter check rejects an accidentally oversized backbone. The code does not use `device_map=auto` as a training-memory workaround.

`--epochs` controls complete training traversals. `--accum` combines one-example microbatches and correctly scales the final partial accumulation group. `--max-length` is a hard limit including evidence, question, candidates, and readout; excessive inputs fail rather than silently losing evidence. `--mode` head or full chooses what changes. `--pointer-width` defaults to 256, setting the new projection dimension. `--backbone-lr` defaults to 0.00002 and `--head-lr` to 0.001 because a new random head and pretrained weights need not use the same update scale. These are starting settings to evaluate, not optimized claims.

`--weight-decay` defaults to 0.01. `--warmup-fraction` uses the first tenth of updates to ramp the rate, followed by decay. `--clip` caps gradient norm at 1.0. `--seed` controls initialization and shuffling. `--checkpointing` recomputes activations to reduce memory and is on by default. `--device` and `--precision` select execution and numeric policy. `--revision` pins the base checkpoint. `--run` names a new artifact directory.

The run saves its initial random-head development results as `untrained_head_dev.json`, then selects the lowest-development-NLL checkpoint. It writes backbone safetensors, tokenizer files, `head.pt`, configuration, training history, and metrics. The complete directory is required for inference. The random-head result is a plumbing baseline; meaningful quality claims should also compare an unchanged or restricted-label language-model baseline.

This short reference does not implement optimizer resume. An interrupted training experiment starts again in a new directory. Do not infer recovery support from the fact that an inference checkpoint exists.

Calibrate without training on the calibration examples

Fit a positive temperature using only the dedicated calibration split, then evaluate the untouched test split:

```
python decision_pointer.py calibrate \
  --run runs/full-smoke --data data/calibration.jsonl \
  --output runs/full-smoke/calibration.json \
  --device cuda --precision bfloat16

python decision_pointer.py evaluate \
  --run runs/full-smoke --data data/test.jsonl \
  --calibration runs/full-smoke/calibration.json \
  --output runs/full-smoke/test.json \
  --device cuda --precision bfloat16
```

The calibrator searches 201 log-spaced temperatures from 0.1 to 10 to minimize calibration NLL. A result at the edge of that range needs investigation. The saved calibration is bound to the checkpoint identity and cannot be silently reused with another trained run. Do not choose a temperature again after looking at test outcomes.

The test report includes accuracy, negative log likelihood, multiclass Brier score, reliability-bin counts, and a coverage/error-rate example at probability 0.9. That threshold is an illustration, not an approved action policy. Replace the toy splits with sufficient independent evidence before selecting a real abstention rule.

Predict a genuinely new decision on CPU and GPU

Save the following one-line record as `fresh.jsonl` in this project directory. The label is omitted because inference does not know the answer.

```
{
  "id": "fresh_001",
  "group": "fresh_conversation_001",
  "state": "The delivery notice says my parcel was left at a different building.",
  "question": "Which queue should review this?",
  "options": [
    {
      "id": "billing",
      "text": "Payments and invoices"
    },
    {
      "id": "delivery",
      "text": "Couriers and missing parcels"
    },
    {
      "id": "access",
      "text": "Sign in and passwords"
    }
  ]
}
```

```
python decision_pointer.py predict \
  --run runs/full-smoke --data fresh.jsonl \
  --calibration runs/full-smoke/calibration.json \
  --output runs/full-smoke/fresh-cpu.json \
  --device cpu --precision fp32

python decision_pointer.py predict \
  --run runs/full-smoke --data fresh.jsonl \
  --calibration runs/full-smoke/calibration.json \
  --output runs/full-smoke/fresh-gpu.json \
  --device cuda --precision bf16
```

Both commands reconstruct the same formatting, tokenizer, backbone, pointer weights, and calibration. They return candidate probabilities and decisions rather than generating text. Compare top-choice agreement, probability differences, and threshold crossings across devices. A generic text-generation pipeline does not know this custom head, and exporting the base alone does not preserve the pointer.

The full neural workflow above is source-reviewed and syntax-checked, not runtime-tested on the authoring computer. Your acceptance test must include model loading, finite forward/backward, parameter updates, save/reload, and both intended inference targets. Now examine the architecture to understand why this code needs this particular input format and target.

Revisit the backbone before changing its output

A parameter is a learned number saved in a checkpoint. An activation is a temporary number produced while processing an input. A layer is a function that transforms activations, usually using parameters. A tensor is an array with a shape. The shape describes how many numbers lie along each axis.

We will write B for batch size, T for token count, D for hidden width, V for vocabulary size, and K for the number of allowed answers. A batch is simply several examples processed together. With $B = 2$, $T = 128$, and $D = 1024$, a hidden-state tensor has shape $[2, 128, 1024]$. It contains 262,144 numbers. Every token in every example has a vector of 1,024 numbers.

Those numbers are not a list of human-readable facts. One dimension does not reliably mean anger and another delivery. Meaning is distributed across many dimensions, and the useful representation changes from layer to layer.

1 Tokenization text becomes integer IDs

A tokenizer splits text into pieces and maps each piece to an integer. Pieces can be words, fragments of words, punctuation, whitespace patterns, or special markers. Token count is not word count. Changing the tokenizer changes the mapping between text and embedding rows; it is therefore part of the model's identity.

Imagine a tiny vocabulary where `parcel` has ID 12 and `lost` has ID 19. Tokenization might turn a short sentence into [5, 12, 8, 19]. The real Qwen vocabulary is much larger; these four numbers are only a teaching example.

A padded batch is a rectangular array of IDs, shape [B, T]. Short examples receive padding IDs. A padding mask marks which positions contain real input. This mask is different from the causal mask that controls the direction in which information can flow.

2 Embeddings an integer becomes a vector

The input embedding is a table E with shape [V, D]. Looking up token 12 retrieves row 12. The table is learned during training. If a miniature model had $V = 20$ and $D = 4$, its embedding table would contain 80 parameters.

For a toy token vector [0.2, -0.4, 0.1, 0.7], each coordinate is an ordinary floating-point number. Nothing is generated yet. We have only converted IDs into initial activations.

Qwen3-0.6B's published configuration has $D = 1024$ and $V = 151936$, so its input table contains $151936 \times 1024 = 155,582,464$ values. Its word embeddings are tied: the language-model output projection shares that table. Removing the output operation therefore does not remove the input table or save another independent copy of those weights. [J06]

3 Positions order must enter the computation

The same words in different orders can mean different things. A model needs information about position. Some transformers add a learned position vector to each token embedding. Qwen3 instead uses rotary positional embeddings, RoPE, inside attention.

The basic rotation is easiest to see in two dimensions. A vector pair [a, b] is rotated by an angle θ into $[a \cos(\theta) - b \sin(\theta), a \sin(\theta) + b \cos(\theta)]$. Different positions receive different angles. For [1, 0], an angle of zero gives [1, 0]; an angle of $\pi/2$ gives [0, 1]. Real RoPE applies coordinated rotations to many pairs of query and key coordinates. This makes attention comparisons sensitive to relative position.

You do not fix long-context behavior merely by increasing a number in a configuration file. The positional scheme, the training lengths, the attention implementation, and available memory must all support the new setting. [J07]

4 Attention a token reads other tokens

Attention mixes information across positions. Consider the word `it` in a sentence about a `parcel`. The network needs a way for the representation at `it` to use evidence from `parcel`. It learns three projections:

- Query: what information this position seeks

- Key: what information another position offers for matching
- Value: the information transferred when that position receives attention

These descriptions are intuitions, not literal labels assigned by the programmer. Each projection is a learned matrix multiplication.

For a toy query $q = [1, 0]$, two keys $k1 = [1, 0]$ and $k2 = [0, 1]$ have dot products 1 and 0. Divide by the square root of the key width, $\sqrt{2}$. The scores become approximately $[0.7071, 0]$. Softmax turns them into weights approximately $[0.6698, 0.3302]$. With values $v1 = [2, 0]$ and $v2 = [0, 4]$, the weighted sum is $[1.3396, 1.3208]$. This is how a position receives a learned mixture of information.

Softmax is a transformation of scores into positive numbers summing to one: $\exp(\text{score}_i)$ divided by the sum of all $\exp(\text{scores})$. These attention weights are internal mixing weights. They are not the final probability that a decision is correct. [J08]

Multiple attention heads make several such mixtures. In grouped-query attention, several query heads share key/value heads, reducing stored keys and values. [J09]

For Qwen3-0.6B, the actual shapes are worth checking carefully. The configuration uses 16 query heads, 8 key/value heads, and head dimension 128. The query projection expands a width-1024 vector to $16 \times 128 = 2048$ values. The key and value projections each produce $8 \times 128 = 1024$ values. Do not assume that `hidden_size` divided by `num_attention_heads` must equal `head_dim`: this model explicitly specifies a different head dimension. The output projection returns the attention result to width 1024. Qwen3 also normalizes queries and keys before applying rotary positions. [J10]

5 Masks who is allowed to read whom

In causal attention, position t may read positions up to and including t , but not later positions. With four tokens, allowed connections look like this; rows are readers, columns are sources:

```
1 0 0 0
1 1 0 0
1 1 1 0
1 1 1 1
```

An additive mask implements this by adding zero to an allowed score and a very negative value to a blocked score before softmax. A blocked score then receives effectively zero probability. Some APIs instead accept Boolean masks, and APIs differ in whether True means allowed or blocked. Check the specific API rather than transferring a mask blindly.

Bidirectional attention permits positions to read both earlier and later input positions. This is useful for some encoders, but it is not required for a decision model. A final decision position in a causal model can already read the whole preceding state and option list.

Removing a pretrained model's causal mask changes the function it learned. It is an architecture experiment requiring suitable adaptation and evaluation, not a free quality improvement. It can also break the exact prefix-caching assumptions developed below.

6 MLP transform features at each position

The feed-forward network, often called an MLP, does not directly mix token positions. It transforms each token's vector separately, using the same weights at every position. Attention has brought contextual information into the vector; the MLP can now transform combinations of those features.

A SwiGLU-style block calculates two expanded vectors, applies the SiLU nonlinearity to one, multiplies them coordinate by coordinate, and projects back to the original width. $\text{SiLU}(x) = x / (1 + \exp(-x))$. The multiplication acts like a learned gate. Qwen3-0.6B uses an intermediate width of 3072, so both expanded vectors have 3072 coordinates before returning to 1024. [J11]

7 Normalization and residual connections

RMSNorm rescales a token's vector using its root-mean-square magnitude, then applies learned coordinate scales. Ignoring the small numerical-stability epsilon, [3, 4] has $\text{RMS} \sqrt{((9 + 16)/2)} = 3.5355$. Dividing by that RMS gives approximately [0.8485, 1.1314]. Normalization helps control magnitudes while training. [J12]

A residual connection adds a block's output back to its input. In a simplified pre-normalized decoder layer:

```
u = h + Attention(RMSNorm(h))
next_h = u + MLP(RMSNorm(u))
```

This preserves a path along which information and gradients can travel. A transformer stacks many such blocks. Tokens can be processed together within an attention layer when all input tokens are known, but layer 2 still depends on layer 1. Parallel input processing does not mean that all model computation happens at once.

8 The output head decide what the vector means

After the final decoder layer and normalization, a language-model head maps each chosen hidden vector to V vocabulary scores, or logits. A logit is an unrestricted number, not yet a probability. Softmax over the vocabulary gives a next-token distribution.

For training on text, the target at position t is usually token $t+1$. With tokens [A, B, C, D], the model's predictions after [A, B, C] are compared with [B, C, D]. Causal masking prevents the prediction at A from reading B while training.

During generation, the program selects one token, appends it, calls the model again, selects the next, and repeats. That feedback loop is autoregressive decoding. A causal backbone used once to compute a decision distribution has causal information flow without an autoregressive output-generation loop.

This distinction is the key to the rest of the chapter: the backbone and the output behavior are separate design choices.

The first conversion replace spelling with scoring

There are three useful levels of conversion. Start with the simplest and measure before adding complexity.

Level A restricted next token baseline

Keep the ordinary LM head. Put the options in the input and ask for one of several verified single-token labels, such as A, B, and C. Read the logits for those exact token IDs, normalize within that permitted set, and return the result from ordinary code.

This can be a cheap baseline. It does not reproduce the training of a purpose-built decision model. Tokenization matters: an apparently single character can have different token IDs with leading whitespace or in different contexts. Verbalizers can carry prior biases. Renormalizing a tiny subset can produce a confident choice even when the model assigns almost no total vocabulary mass to those labels. Measure that behavior before using the result as uncertainty.

A second baseline scores the complete token sequence of each candidate. That uses conditional token log probabilities, requiring a documented aggregation rule and care with answer-length bias. It is not equivalent to comparing one-token labels or to training a new pointer head.

Level B fixed label classifier

Remove the vocabulary readout and attach a learned matrix W of shape $[D, K]$, plus an optional bias of shape $[K]$. If h is the last real input token's vector, compute $z = hW + b$. For $D = 1024$ and $K = 3$, this requires 3,075 head parameters including the bias.

This is appropriate when the labels always mean billing, delivery, and account access. The same output column must keep the same meaning throughout training and inference. Adding a fourth category changes the head and requires further training. A fixed-label classifier does not automatically become a general question-answering model just because its backbone was a language model.

Level C a pointer over options supplied in the input

Instead of giving output column 0 a permanent category, let every request supply its own options. Build one causal sequence containing the state, the question, every option, and a final readout position. Record an index at the end of each option and another at the final readout position.

For teaching, imagine token positions arranged as follows. The text spans are schematic; real token boundaries come from the tokenizer:

```
state ...
question ...
option 0: billing ... [option-end]
option 1: delivery ... [option-end]
option 2: account ... [option-end]
[readout]
```

The model supplies one hidden vector at every position. Gather the three option-end vectors into H_{options} , shape $[3, D]$, and the final vector into h_{readout} , shape $[D]$. Two small learned projections put them in a common width P :

```
q = decision_projection(h_readout)      # [P]
keys = option_projection(H_options)     # [K, P]
logits = keys @ q / sqrt(P)             # [K]
probabilities = softmax(logits)         # [K]
```

This is a dynamic choice head. K may change between examples. The output indexes the supplied options, so application code maps index 1 back to the string delivery. There is no need for the network to spell delivery or construct JSON.

For $D = 1024$ and $P = 256$, two affine projections contain $2 \times (1024 \times 256 + 256) = 524,800$ trainable values. That is less than one million new parameters, although training the backbone can still require gradients for hundreds of millions of values.

The end of an option comes after its text, so a causal representation can include that option. The final readout comes after every option, so it can consider the whole list. Earlier options do not see later ones directly; later options may see earlier options. Consequently, question isolation does not imply option-order invariance. Shuffle options during training and measure sensitivity to order at evaluation.

This pointer formulation is a concrete open architecture, not a claim about undisclosed Jev weights. Kev implements such a readout, jointly training a head and, for its small released models, LoRA adapters. Its code also supports a distinct full-weight path. [J13]

A decision and a learning update with numbers

Use pointer width $P = 2$ so we can calculate by hand. Suppose the projected readout is $[1, 0]$ and the option keys are $[1, 0]$, $[0, 1]$, and $[-1, 0]$. The dot products are $[1, 0, -1]$. Dividing by $\sqrt{2}$ gives $[0.7071, 0, -0.7071]$. Softmax gives approximately $[0.5760, 0.2840, 0.1400]$.

If option 1 is the correct answer, the cross-entropy loss is $-\log(0.2840)$, approximately 1.2588. If option 0 is correct, the same prediction has loss approximately 0.5514. The loss penalizes assigning little probability to the known answer.

For a one-hot target, the derivative of cross-entropy with respect to each logit is $\text{predicted_probability} - \text{target_probability}$. With option 1 correct, the derivatives are approximately $[0.5760, -0.7160, 0.1400]$. Gradient descent moves against these derivatives: it tends to lower the wrong options' scores and raise the correct option's score. Backpropagation carries that signal through the two projections and, if enabled, the backbone.

A single example does not define a useful model. The numerical exercise only shows the mechanism. Your dataset must teach the intended relation between states, questions, options, and labels over many representative cases.

What changes in the program

Input and labels

Store each decision as a record with state, question, ordered options, correct option index, and a stable example or group ID. The index refers to the order in that particular record. When you shuffle options, update the label index too. Keep the answer out of the text fed to the model.

For multiple questions about one state, keep them in the same split. If a single customer's conversation produces ten decisions, splitting those decisions randomly can leak almost identical evidence from training into testing. Split by conversation, source document, event, or other appropriate independent unit.

Include near misses, missing facts, irrelevant content, negation, numeric boundaries, and questions whose answer changes when one fact changes. If all correct options are longer, appear first, or contain a recurring cue, the model may learn the shortcut.

If none of the supplied actions is valid, the model still has to distribute probability somewhere. Include an explicit abstain or insufficient-information option when the task permits it. A distribution over a closed set does not prove the closed set is complete.

Forward pass

Use the decoder's hidden states, not the vocabulary logits. Gather exactly the intended positions. With padding, the final array position may be padding rather than the readout token; explicit indexes avoid this common bug. Assert that each option index precedes the readout and points to real input.

Start with one question per row. This requires only ordinary causal attention and avoids custom-mask compatibility issues. The cost is repeated state processing when many questions share a state. Optimize that after the one-row reference implementation is correct.

Reject an overlength record with a clear error. Blind truncation can remove an option marker, the readout token, or the evidence required for the label. If you design a truncation policy, record it and evaluate it as part of the model.

Loss

Use cross-entropy over the K option logits for single-answer questions. Do not compute next-token loss over every input token and assume you have trained a decision head. These are different tasks.

For genuinely soft labels, use a cross-entropy or KL-divergence objective appropriate to a target distribution. For independent yes/no labels that may all be true, independent binary heads or separate two-option questions are suitable; a single softmax across all labels incorrectly forces them to compete. Ordered ratings can use a categorical distribution and expected index, but the numeric spacing of levels is an assumption. Other ordinal losses are possible and should be compared explicitly.

What gets updated

- Head-only training freezes the entire backbone and updates the new head
- Adapter training freezes the original backbone weights but trains inserted adapter weights and the head
- Full fine-tuning updates the backbone and head together

Head-only training is inexpensive, but a pooled representation might not expose the distinction your task needs. Full fine-tuning has more freedom and greater memory cost; it can also overfit or damage previously useful representations. It does not promise a better result on a small dataset. Compare interventions with the same honest evaluation split.

Inference

Run the backbone once, score the options, apply any fitted calibration, and return typed values using normal application code. Do not call `generate()`. Disable dropout with eval mode and disable gradient recording for inference. Measure preprocessing, GPU work, transfer, and serialization separately when profiling.

Saving the model

The artifact must contain more than a file named `head.pt`. Save or identify the exact backbone revision, tokenizer files and revision, new head weights, head dimensions, formatting rules, marker IDs if any, label convention, maximum input length, normalization or pooling rule, calibration value, and library versions. Save training metadata and evaluation results separately from inference weights.

For full fine-tuning, save the changed backbone. For LoRA, save the adapter and the exact base identity it modifies. A custom decision head is not automatically served by a generic text-generation server or GGUF exporter. Your inference program must know how to reconstruct the readout and formatting.

Many questions without questions contaminating each other

Suppose state tokens occupy positions 0, 1, and 2. Question A occupies positions 3 and 4; question B occupies positions 5 and 6. The branch mask should permit each branch to read the state and its own earlier tokens, but not the other branch:

```
reader/source: S0 S1 S2 A0 A1 B0 B1
S0             1 0 0 0 0 0 0
S1             1 1 0 0 0 0 0
S2             1 1 1 0 0 0 0
A0             1 1 1 1 0 0 0
A1             1 1 1 1 1 0 0
B0             1 1 1 0 0 1 0
B1             1 1 1 0 0 1 1
```

Position IDs are [0, 1, 2, 3, 4, 3, 4]. B restarts immediately after the shared state, matching how it would be positioned if asked alone. Merely concatenating all questions with a normal causal mask fails: B can then read A. Merely resetting position IDs also fails: positions do not block attention.

Before trusting a packed implementation, compare its logits against the simple reference that runs [state + A] and [state + B] separately. In evaluation mode, they should agree within a documented numerical tolerance on an architecture where the mask fully controls cross-token mixing. Test changing and reordering unrelated questions. Test varying padding. Test a batch of different lengths.

An ordinary attention implementation may materialize an $L \times L$ mask. This can become expensive even when many entries are blocked. A theoretical reduction in allowed connections does not prove the chosen kernel avoids work on blocked connections.

The hybrid model trap

Not every decoder mixes tokens only through standard attention. Qwen3.5-0.8B's official text configuration alternates three linear-attention layers with a full-attention layer, repeated across 24 layers. Its recurrent mixing needs additional care. [J14]

A block attention mask does not by itself reset recurrent or convolution state between branches. If such state carries information from A into B, your supposedly isolated questions are not isolated. For an initial portable implementation, independent rows are the safe reference. Serving can branch a properly copied prefix state if the model's cache implementation supports it.

This is why a technique that works on Qwen3 cannot be ported to Qwen3.5 by changing only the repository name. Inspect `model_type`, `layer_types`, cache classes, tensor widths, supported masks, and exact library version. Prove parity before claiming shared-prefix support. Kev's tests are a useful real example of the necessary comparisons. [J15]

KV caching reuse computation without changing the answer

For ordinary causal attention, each layer produces keys and values for every processed position. Saving them is a KV cache. New tokens can read the cached keys and values instead of recalculating the prefix's layer activations.

If the state does not read future question text, its cached representations are the same no matter which question follows. Run the state once, retain its cache, and give each question its own continuation from that cache. Do not let one branch mutate the shared cache seen by another. With hybrid models, the cache may also contain recurrent and convolution states that need correct copying and resetting.

The question still reads the state, so some work scales with state length for every question. Caching avoids rebuilding the state; it does not make the state free. Cache lookup and memory traffic also have costs.

For a standard attention cache, a useful estimate in bytes is:

$$2 \times \text{layers} \times \text{batch} \times \text{cached_tokens} \times \text{KV_heads} \times \text{head_dimension} \times \text{bytes_per_value}$$

The leading 2 counts keys and values. With Qwen3-0.6B's 28 layers, 8 KV heads, head dimension 128, one row, 2048 tokens, and two bytes per value, this is 234,881,024 bytes, or 224 MiB. This is an arithmetic cache estimate, not a measured total GPU allocation. It excludes parameters, workspace, other buffers, and any replicated branches. A hybrid cache has different terms. [J16]

Training generally disables an inference KV cache. Backpropagation needs the right computation graph, and a persistent detached cache can silently stop gradients through the prefix. A differentiable shared-prefix training implementation is a separate optimization that must be checked against the unoptimized gradient results.

Calibration probabilities need testing

The probabilities [0.9, 0.05, 0.05] are a prediction. They are not proof that the first option is correct 90% of the time. Calibration asks whether predicted probabilities match observed frequencies on relevant data. If 100 comparable decisions are assigned about 0.9 probability and only 65 are right, the model is overconfident there.

A simple calibration method fits one positive temperature T on a held-out calibration set and uses $\text{softmax}(\text{logits} / T)$. T greater than 1 softens a distribution; T below 1 sharpens it. Positive temperature leaves the highest-scoring option unchanged. It can improve probability quality without improving top-1 accuracy. Fit it after model selection, without using the final test set. [J17]

For a dataset with many related records, dedicate independent groups to training, model selection, calibration, and final test where data permits. If data is small, cross-validation can help, but document every use of every example. Never repeatedly adjust thresholds after observing final-test failures and continue calling that set untouched.

Measure at least:

- Accuracy: how often the highest-probability option is right
- Negative log likelihood: how much probability the model gives the actual answer
- Brier score: squared error between the whole probability vector and the target vector; document whether you sum or average across classes
- Reliability by probability range, with counts so tiny bins do not look authoritative
- Selective accuracy or risk-coverage: quality on the fraction of decisions retained above a threshold
- Errors by task, input length, source, and important edge-case group
- Sensitivity to equivalent option reordering and unrelated questions
- Latency and peak memory with cold and warm caches reported separately

A low average calibration error can hide severe failure on a rare class or shifted data. A confidence transformation supplied by an API is not automatically an empirical accuracy estimate. Keep uncertainty reporting separate from the application policy that decides whether to act, abstain, or ask a person.

The CLM route encode the sides separately

A jointly contextualized option scorer lets the option vectors depend on the state and usually on part of the option list. This can express detailed interactions, but changing the state requires recomputing those contextual option representations.

A dual-encoder design computes a state representation and a candidate representation separately. Denote a frozen backbone encoder by f , a learned state projection by g_s , and a learned action projection by g_a :

```
u = normalize(g_s(f(state_and_question)))
v_i = normalize(g_a(f(candidate_i)))
score_i = scale * dot(u, v_i)
```

Normalization divides a vector by its length so dot products become cosine similarities. The two sides can share backbone weights while using different projection heads. The backbone need not be loaded twice merely because there are two logical encoder roles.

The inspected CLM reference uses Qwen3-8B last-token representations, separately learned projections, normalized projected vectors, and a learned score scale. The head checkpoint is tied to that encoder and pooling rule. It is not a generic attachment for arbitrary language models. [J18]

A candidate vector can be cached until its text or model identity changes. If a system chooses among the same 100 actions repeatedly, this can save considerable work. But if candidate text embeds state-specific details, or you change the encoder, head, formatting, or normalization, the relevant cache entries must be regenerated. [J19]

The representational trade-off is real. Compressing a long state and a long candidate into two fixed vectors can lose detailed token-to-token relationships. Joint scoring is often more expensive but can inspect those interactions. Neither architecture wins every workload. Compare them on the actual candidate count, candidate reuse, context length, and required discrimination.

Contrastive learning in one small batch

Take $B = 3$ matched state-action pairs. Encode states as three rows and actions as three rows. Multiply their projected vectors to produce a $[3, 3]$ score matrix. Entry $[i, j]$ says how compatible state i is with action j . The diagonal contains the matched pairs.

For each row, cross-entropy teaches the model to prefer its diagonal action over the other batch actions. A symmetric objective also applies cross-entropy to the transposed matrix, matching actions back to states. This is bidirectional matching loss. It does not mean the backbone uses bidirectional token attention.

Other batch answers are not always genuine negatives. If two states have the same valid action, blindly treating one copy as wrong creates a false-negative training signal. Use group-aware batching, multiple positives, or an appropriate loss mask. Hard negatives should be plausible but demonstrably incorrect, not merely different wording of a valid answer. [J20], [J21]

Does CLM autoregress? The inspected serving path obtains representations and scores candidates without generating a token sequence. Its backbone originates as a causal language model. Those two facts are compatible. A causal encoder can be evaluated on a known text in a forward pass, even though that same backbone could also support an autoregressive text-generation loop in a different program.

Calling negative scores energies is a possible mathematical notation, but it does not establish a separate energy-based training recipe or explain Jev's undisclosed internals. Specify the actual score function, normalization, loss, and serving loop instead of relying on a label.

Would Granite work as the evaluator

In principle, a compatible decoder can supply hidden representations to a newly trained decision head. IBM's current Granite 4.2-3B card identifies a dense decoder-only transformer. That makes the broad conversion plausible. It does not establish that Granite is better than Qwen for your decisions, that no training is necessary, or that an existing Qwen-trained head will transfer. [J22]

Inspect the exact checkpoint. Architecture families change: some releases are hybrid, some are dense attention-only, and their widths, norms, scaling conventions, and cache structures differ. A file may load because its tensor sizes happen to match while remaining semantically incompatible with the new backbone. Even equal-width representations can use different coordinate systems.

For a strict parameter ceiling, count the actual parameters. A rounded model name is not a precise budget: the Granite card says 3B while the Hub's displayed parameter category rounds differently. Keep this as a broader architecture comparison rather than a core full-fine-tuning exercise under the book's one-billion-parameter practical limit.

The transferable skill is the conversion procedure: inspect the representation boundary, choose a readout, train the correct objective, preserve the input contract, and evaluate against a baseline.

A one 24 GB GPU plan

Begin the full-weight experiment with an attention-only model below one billion parameters, such as Qwen3-0.6B, a single question per row, short inputs, and a very small batch. The exact memory depends on implementation. A useful conservative starting calculation for fp32 parameters, fp32 gradients, and two fp32 Adam moment buffers is 16 bytes per trainable parameter. At roughly 0.6 billion parameters that is about 9.6 GB before activations and temporary buffers. At 3 billion it is about 48 GB before those additional costs. These are decimal-byte arithmetic estimates, not measurements.

Autocast can run many operations in bf16 while keeping optimizer-owned parameters in fp32. Do not add a separate fp32 master copy to a budget that already counts fp32 parameters. Other mixed-precision implementations may really have a separate master copy; inspect yours. Gradient checkpointing trades extra forward computation for fewer saved activations. Gradient accumulation emulates a larger effective batch across several small microbatches but does not shorten an individual sequence.

A practical initial experiment is sequence length 256, microbatch 1, gradient accumulation 8, gradient checkpointing on, and no inference cache during training. These are deliberately conservative proposed settings, not a measured fit guarantee. Test one forward/backward/update cycle, then measure allocated and reserved peak memory before scaling.

The removed vocabulary readout can avoid large vocabulary-logit activations. For $B = 1$, $T = 128$, and $V = 151936$, a full $[B, T, V]$ bf16 tensor alone is about 38.9 MB. But optimized generative inference may already compute only the last-position logits. Do not claim the full-sequence saving applies to every baseline.

Kev-0.8B offers a real released small decision checkpoint, but its reported training recipe uses adapters. Its card is useful for understanding how data, replay, calibration, and evaluation are documented; it is not proof of the speed or memory of your own full-weight run. [J23]

A finished experiment should answer: did the trained decision model improve the task; are its probabilities usable; is its peak memory below the measured limit; does it beat a simpler baseline on quality or latency; and can a fresh process reload it and reproduce the result? Fitting into VRAM is only one of these questions.

Exercises with checks

1. A fixed classifier has $D = 512$ and $K = 7$. How many weights and biases does its affine head need? Answer: $512 \times 7 + 7 = 3,591$.
2. Can the final readout token in a causal model read every preceding option? Answer: yes, assuming the mask permits them and the options fit in the context.
3. Does resetting position IDs isolate two concatenated questions? Answer: no. You must control all cross-branch information paths.
4. A distribution is $[0.1, 0.2, 0.7]$, with true answer index 0. What is cross-entropy? Answer: $-\ln(0.1)$, about 2.3026.
5. Does temperature scaling change which option wins? Answer: a finite positive scalar temperature preserves the ordering of logits.
6. Can you reuse a 4096-input CLM head on a 1024-width decoder? Answer: not directly. The dimensions differ; even matching dimensions would not establish representation compatibility.
7. You changed one unrelated question and another answer changed substantially. What should you inspect? Answer: masks, recurrent-state leakage, position IDs, dropout, batching, formatting, cache mutation, and numerical precision, before attributing the change to intelligence.
8. Does a three-option model remain safe when all three options are wrong? Answer: no. The application needs an appropriate abstention mechanism and testing for incomplete candidate sets.

Verification status for this chapter

Repository code, model cards, configurations, and original papers were inspected on 2 October 2026. Numerical examples and branch-mask arithmetic can be checked without a GPU. The book's local environment did not have PyTorch or Transformers installed for this chapter's preparation; no GPU training, speed, memory-fit, or task-accuracy measurement is claimed. Pin source revisions and record your actual package versions when you run the companion experiments.

25 Add preferences and distillation only after supervised learning works

For this chapter's commands, open a fresh terminal at the companion root, run `cd examples/llm`, then source `.venv-llm/bin/activate`. If you skipped the first small-language-model project, complete its explicit environment setup first. Do not run these relative paths from an earlier project directory.

SFT gives the model examples of acceptable behavior. Sometimes you have two plausible answers and know which one is better. Sometimes a stronger model can produce useful training examples for a smaller one. These lead to preference optimization and distillation. They are extensions to a working supervised experiment, not substitutes for clear data and evaluation.

Preference data says which answer is better

A pairwise record contains the same prompt, a chosen response and a rejected response. Both responses should be judged under the same rubric. Here is an original example:

```
{
  "prompt": [{"role": "user", "content": "Stock lookup timed out. Tell the user."}],
  "chosen": [{"role": "assistant", "content": "The stock lookup timed out, so I don't have a current count."}],
  "rejected": [{"role": "assistant", "content": "There are probably eight units available."}]
}
```

Do not create every rejected answer by appending obvious nonsense. Then the model can learn the easy surface distinction rather than the preference you care about. Include close comparisons: one answer quietly invents a deadline; another preserves uncertainty but is slightly longer.

Direct Preference Optimization, or DPO, compares the trainable policy to a fixed reference. Let x be the prompt, $y+$ the preferred completion and $y-$ the rejected one. Define

$$z = \beta \times [(\log p_{\theta}(y+|x) - \log p_{\text{ref}}(y+|x)) - (\log p_{\theta}(y-|x) - \log p_{\text{ref}}(y-|x))].$$

The loss is $-\log \text{sigmoid}(z)$. Increasing z means the chosen answer gained relative to the rejected answer, measured against the reference. The original derivation connects this objective to a KL-regularized preference problem. β is not a universal “more improvement” slider: it controls that reference-relative trade-off and scales the training signal. [L29]

TRL’s reviewed DPO interface accepts explicit prompt/chosen/rejected data. It also offers reference-log-probability precomputation. A frozen reference need not always occupy a second full trainable model’s memory, but its identity must be correct. For an SFT adapter followed by DPO, the reference should represent the intended SFT policy. Simply disabling an adapter can expose the original base instead, which is a different reference. [L30]

A reasonable first extension is short completions, the same sub-billion model, adapter updates and a small learning-rate experiment such as $5e-6$ or $1e-5$, with $\beta=0.1$ as an initial

comparison point. These are unmeasured starter values. Profile again: chosen and rejected sequences plus reference evaluation have a different memory and compute budget from SFT. Prefer off-line data before attempting an on-policy reinforcement-learning system that repeatedly generates, scores and updates.

Evaluate preference win rate and the earlier hard task metrics. A model can win a stylistic comparison while becoming less calibrated or less accurate. Track response length, unsupported claims and refusal/clarification behavior so preference learning cannot silently optimize the wrong shortcut.

Distillation transfers behavior rather than compressing a file

Knowledge distillation trains a student using information from a teacher. It is not weight quantization. Quantization changes representation precision; distillation changes what a usually smaller model learns. The original distillation framework uses teacher probability distributions, while sequence-level distillation can train on teacher-produced outputs. [L31] [L32]

For this hardware budget, start with off-line sequence distillation:

1. Select representative task prompts without using the final test set
2. Generate candidate teacher responses using a method whose terms allow your intended use
3. Validate facts, tool schemas and task success; reject bad examples
4. Save accepted prompt/response pairs with teacher/version/provenance metadata
5. Train the student with the same SFT pipeline used above
6. Evaluate on untouched real cases and compare against human-authored SFT

This lets you run teacher generation and student training at different times. It avoids assuming both models fit in VRAM simultaneously. Teacher outputs are not automatically correct labels; fluent synthetic errors can be reproduced very efficiently by a student.

For logit distillation with matching token vocabularies, one common loss is

$$L_{KD} = \tau^2 \times \text{KL}(p_{\text{teacher}}^{\tau} || p_{\text{student}}^{\tau}).$$

Here τ is a temperature that softens the distributions and the squared factor compensates for temperature-related gradient scaling in the usual formulation. You often mix this with a supervised hard-label loss. If teacher and student use different tokenizers, their next-token distributions do not line up entry by entry. A naive KL across vocabulary positions is invalid. Sequence-level targets avoid that particular alignment problem.

The current TRL distillation documentation describes off-policy and on-policy behavior with a `DistillationTrainer`. The latter generates student trajectories and queries the teacher on those trajectories; it is a more demanding system than training on a saved JSONL file. Read the versioned API and memory path before adopting it. [L33]

A narrow student may outperform its own baseline on the target workload without reproducing the teacher's broad reasoning ability. Distill the specific behavior you can test. For factual tools, distill the decision to consult evidence and the ability to use an observation, not a cache of the teacher's guessed answers.

26 Project teach an image generator a small visual style

An image diffusion system generates a new image by iteratively transforming a noise-like tensor, usually guided by text or other inputs. It is useful when you want plausible visual alternatives: concept art, original asset variations, product-scene mockups, or controlled edits. It is unsuitable when invented details would be mistaken for evidence, such as filling in a supposedly factual medical or forensic image. For counting defects, recognizing objects or assigning a class to each pixel, a classifier, detector or segmentation model is usually the more direct design.

Common application patterns are:

- **Text → candidate images → human selection.** Inputs include prompt, dimensions, seed and generation settings. Outputs include image files and a generation record. Add checks for prohibited content, personal likenesses and downstream use; generating an image does not establish that it is accurate or rights-cleared.
- **Reference image → bounded variation.** An image-to-image pipeline encodes a reference and introduces a chosen amount of noise before generation. Stronger modification can erase identity or layout. This is an inference pipeline choice, not proof that you trained a new subject.
- **Image plus aligned mask → localized edit.** An inpainting-capable pipeline receives both. Specify which region may change, preserve the unedited source, and inspect edges and supposedly preserved regions afterward. The ordinary text-to-image LoRA trainer below does not learn this interface automatically.
- **Layout/depth/edge condition → controlled generation.** A purpose-built conditioning model can guide structure. Keep the control map's coordinate system, resolution and semantics aligned with the image. Text prompts alone do not enforce exact geometry.

A deployment contract should name the base revision, adapter revision, input types, supported sizes, expected latency, output format and limitations. Keep generation asynchronous when it takes longer than an interactive response. Save candidates with their settings, allow users to reject them, and separate a model's technical success from approval for publication. The first project covers the simplest pattern, text to image, with an adapter for a narrow visual style.

What you will make. A small adapter that nudges an existing image generator toward an original paper-card illustration style. You will create the training images locally, make a baseline image, train only a small fraction of the generator's weights, and compare the result with the unchanged model. Nothing is uploaded by the commands in this project.

Why start here? You can see whether the experiment works. The base generator already knows many objects, so you do not need to teach it the entire visual world. Later, the audio project removes this shortcut and builds a very small generator from random weights.

What is and is not verified. The companion data generators and data checks were executed on CPU; Python syntax and shell syntax were checked. The image training command was checked against the actual Diffusers v0.36.0 source. No GPU training or image generation was

executed for this book. The software pins are a deliberately conservative candidate environment, not a claimed end-to-end tested lockfile. The single-process trainer supports a conservative one-GPU setup; a 24 GB RTX 3090/4090 is a candidate target for this Sana LoRA configuration, but you must measure the first run. The model choices were checked against primary sources on 2 October 2026.

Image project step 1 create something the model can learn

All commands in this project run from the extracted `examples/diffusion` folder. Set an absolute path once, replacing the placeholder with your actual extraction location, and return to it before beginning:

```
export DIFFUSION_EXAMPLES="/absolute/path/to/extracted/examples/diffusion"
cd "$DIFFUSION_EXAMPLES"
```

Do not paste the placeholder unchanged. If another project's virtual environment is active, run deactivate first. Use Linux or WSL2 with a working NVIDIA driver. A full CUDA toolkit is not normally required merely to run an official PyTorch CUDA wheel; custom compiled extensions can add that requirement. Do not install a random CUDA package to fix a driver problem.

Make a virtual environment first. The base model is several gigabytes, and caches, checkpoints and a second environment add up: keep at least 50 GB of free SSD space for this chapter's experiments. That is a planning allowance, not the measured download size of a particular snapshot.

```
python3.11 -m venv .venv-image
source .venv-image/bin/activate
python -m pip install --upgrade pip
python -m pip install torch==2.7.1 torchvision==0.22.1 \
    --index-url https://download.pytorch.org/whl/cu126
python -m pip install -r requirements-image.txt
python -m pip check
python -c 'import torch,diffusers,peft,transformers; print(torch.__version__,
    torch.cuda.is_available()); print(diffusers.__version__,peft.__version__,transformers.__version__)'
```

The exact torchvision/PyTorch pair follows PyTorch's versioned installation instructions [D27]. A newer driver can usually run a wheel built for an older supported CUDA runtime; check your driver rather than equating the number printed by `nvidia-smi` with an installed toolkit. The example targets Ampere/Ada cards, not every GPU sold with 24 GB.

Pin the trainer and the library together. The companion requirements use Diffusers 0.36.0, PEFT 0.17.1, Transformers 4.55.4, Accelerate 1.10.1 and Hugging Face Hub 0.34.4, plus the tokenizer's SentencePiece dependency. These satisfy the inspected library's declared minimums, but the complete environment still needs the import and smoke tests on your machine [D34]. Save the resolved environment with `python -m pip freeze > image-environment.txt` after it works. This version is a reproducible recipe target, not a claim that it is the newest release.

Sana's frozen text encoder and autoencoder are additional models, even though we train only adapters in the 1.648B-parameter image transformer. The complete pipeline exceeds three billion parameters. Budget system RAM as well as VRAM: 32 GB is a reasonable starting planning allowance and 64 GB offers more headroom for model loading and offloading, but neither is a measured requirement of this recipe. The trainer also prepares the processed

image tensors in host memory. We print actual component parameter counts during the baseline load rather than labeling the whole pipeline “1.6B.”

Create 240 small, original illustrations:

```
python make_image_cards.py --out data/papercards --per-object 60
python check_image_data.py data/papercards
```

The generator draws houses, mugs, flowers and balloons with a fixed paper-like treatment. It creates 192 training, 24 validation and 24 test images. Each file is rendered natively at $1,024 \times 1,024$ pixels, with geometry scaled to the larger canvas rather than upsampling a small picture. This is intentionally a limited exercise: success on four procedural object families does not demonstrate that you have trained a broad commercial illustration model. Open a contact sheet or at least several files from each split before training. The correct first test is whether you like, understand and can legally use the data.

The layout is:

```
data/papercards/
  provenance.json
  train/
    house_0000.png
    ...
  metadata.jsonl
  val/
    ...
  metadata.jsonl
  test/
    ...
  metadata.jsonl
```

One line of metadata.jsonl looks like this:

```
{"file_name":"house_0000.png","text":"a blue house, rivetpaper style, centered on cream
paper","group":"house-0"}
```

file_name connects a picture to its caption. text says what is visible and names the desired style. group keeps related originals out of different splits. The upstream imagefolder format reads the image-caption connection; the training script ignores our extra grouping field [D16]. Keep the validation and test folders outside the training folder, and pass only data/papercards/train to the trainer. Pointing the trainer at the whole tree can change split discovery and risks training on unintended examples. Make the selected training directory explicit.

Image data captions groups crops and masks

Replace the generated pictures with your own art or product photos after the pipeline works. A small style experiment might start with 50-200 genuinely varied originals, but that is a project-size suggestion, not a statistical minimum. Two hundred near-identical frames do not provide two hundred independent examples.

For a style adapter, vary object, background, layout and color. Otherwise the model can learn “this style means a centered red mug.” Describe visible differences in captions. For a subject adapter, capture the same subject from different views, distances and backgrounds so that identity and scenery are not inseparable. Split a photographic session or video into one

partition as a group. Burst shots, crops of the same original, and lightly edited duplicates belong together.

A trigger such as `rivetpaper style` is a consistent phrase, not a newly invented tokenizer token. It will be broken into existing tokens. The adapter learns associations involving those tokens; it does not magically add a dictionary entry. If every training caption includes the trigger, test both with and without it to see how much the style spreads into unrelated prompts.

Resizing and cropping are changes to the example, not neutral housekeeping. The beginner command resizes the shorter edge and center-crops a square. A wide image of two people may lose one person; a caption that still says “two people” is then wrong. Inspect the processed crop. Do not flip images with lettering, asymmetric logos, medical laterality or other direction-sensitive meaning just because a tutorial enables random flips.

An **aspect-ratio bucket** groups similarly shaped images into batches, such as portrait, square and landscape. It reduces destructive cropping without padding every image to the largest shape. The upstream script used here does not implement arbitrary aspect-ratio bucketing; adding a `--buckets` flag will not make it do so. Move to a trainer with explicit, verified bucket support only after the square experiment is understood. Hold total pixel area approximately constant when comparing bucket shapes because changing shape can also change memory requirements.

A **mask** is an aligned map indicating a region, often one value for editable pixels and another for preserved pixels. It is needed for an inpainting-specific training objective, or for weighting a loss to a region. Merely placing a mask file beside an image does not activate masked training in this LoRA script. Edge maps, depth maps, segmentation maps and poses are other forms of conditioning. They require the corresponding model inputs and training procedure. A segmentation model that labels pixels is solving a different task from a diffusion model that generates pictures; use the discriminative-model project when labels, rather than invented pixels, are your goal.

Image project step 2 download a fixed base and make a baseline

Use the official `Efficient-Large-Model/Sana_1600M_1024px_BF16_diffusers` checkpoint. Sana is a maintained efficient image-transformer family, and the official Diffusers adaptation example specifically targets this BF16 checkpoint. Its card lists a 1.648B image transformer, a frozen Gemma2-2B-IT text encoder and a $32\times$ spatial-compression DC-AE autoencoder. The Sana weights have Apache 2.0 terms, the Gemma component has additional terms, and the card states research-oriented intended use. Read the actual current terms and restrictions before use [D30] [D35] [D36].

The official example establishes a real LoRA training route. It does not establish a measured 24 GB peak for our exact custom-caption dataset. Our single-process, batch-1, checkpointed/offloaded configuration is an unexecuted candidate designed to be measured before scaling. We chose the 1.6B BF16 variant because it is the dedicated checkpoint in that training recipe; a smaller 590M Sana variant exists, but substituting it also changes precision and model configuration and should be a separate experiment.

Download a revision-pinned local snapshot. This code resolves the current model revision once, records it, and downloads that exact revision. The recorded SHA is what reproduces the selection later. It does not promise that every future run resolves the same main revision.

```
python - <<'PY'
import json
from pathlib import Path
from huggingface_hub import HfApi, snapshot_download
repo = 'Efficient-Large-Model/Sana_1600M_1024px_BF16_diffusers'
sha = HfApi().model_info(repo).sha
path = snapshot_download(
    repo_id=repo, revision=sha,
    allow_patterns=['model_index.json', 'transformer/*', 'vae/*',
                   'text_encoder/*', 'tokenizer/*', 'scheduler/*'],
    ignore_patterns=['*.onnx', '*.msgpack', '*.h5'],
)
Path('base_path.txt').write_text(path)
Path('base_revision.json').write_text(json.dumps({'repo':repo,'revision':sha},indent=2))
print(path)
PY
export BASE="$(cat base_path.txt)"
python sample_image_lora.py --base "$BASE" \
    --prompt 'a blue house, rivetpaper style, centered on cream paper' \
    --seed 17 --out samples/baseline-house.png
```

The download filter avoids unrelated root-level checkpoints, but component folders may contain multiple precision variants and consume substantial disk space. Our loader explicitly requests the BF16 variant; do not delete weight shards merely because their names look similar. Keep the model's configuration, scheduler, tokenizer, text encoder, autoencoder and denoiser together; a lone .safetensors file is not necessarily a complete pipeline.

A **seed** initializes a pseudorandom number generator. Keeping it fixed makes a before/after comparison more informative because the starting noise is the same. It does not guarantee bit-for-bit output across different GPU architectures, library versions or kernels. Record the software, model revision, prompt, seed, image size, sampler and inference-step count.

Before training, also generate a no-trigger prompt and one unfamiliar object, such as “a bicycle, rivetpaper style, centered on cream paper.” Keep these baseline files. If the base already does what you need, an adapter may be unnecessary.

Image project step 3 one small training run

Get the training script from the same version as the installed library:

```
git clone --branch v0.36.0 --depth 1 https://github.com/huggingface/diffusers.git diffusers-src
git -C diffusers-src rev-parse HEAD > diffusers-source-commit.txt
python patch_image_trainer.py diffusers-src
git -C diffusers-src diff > image-trainer.patch
export DIFFUSERS_SRC="$PWD/diffusers-src"
export DATA="$PWD/data/papercards"
export OUT="$PWD/runs/papercard-smoke"
STEPS=20 WARMUP=5 bash train_image_lora.sh
```

The small guarded patch skips the upstream trainer's final allocation of another full FP32 pipeline when neither final validation nor upload was requested. The adapter is already saved at that point. We evaluate in a fresh process using the matching BF16 pipeline instead, avoiding an unnecessary host-memory peak and the upstream final reload's different transformer dtype. Save the patch with the upstream source SHA [D31].

This smoke test checks loading, preprocessing, forward computation, backward computation and saving. It is not long enough to establish image quality. A loss value appearing on screen is not sufficient: verify the run writes `pytorch_lora_weights.safetensors` and that `sample_image_lora.py` can load it. The command uses local TensorBoard logs and does not enable upload flags.

Run the actual reload check now, before spending time on the longer run:

```
python sample_image_lora.py --base "$BASE" --lora runs/papercard-smoke \
  --prompt 'a blue house, rivetpaper style, centered on cream paper' \
  --seed 17 --out samples/smoke-house.png
```

The smoke image is a file-loading/functionality check, not evidence that 20 steps learned the style. If either the adapter or its base manifest fails to load, fix that now.

Then start a separate deliberate run:

```
export OUT="$PWD/runs/papercard-lora"
STEPS=1000 WARMUP=100 bash train_image_lora.sh
```

The wrapper saves `book_run_manifest.json`, binding the adapter to the exact base repository/revision, model configuration hashes, tagged-and-patched trainer, wrapper settings, caption metadata and training-image hashes. Resume rejects changes in those material inputs; inference checks the adapter/base binding before loading. Keep the base snapshot directory's revision name and `base_revision.json` when moving the experiment. The large base-weight files are treated as an immutable Hub snapshot rather than rehashed on every image request; do not edit them in place.

The companion shell script expands to these important settings: 1,024-square images, per-device batch 1, accumulation 4, rank and alpha 8, learning rate 0.0001, BF16 mixed precision, gradient checkpointing and frozen-component offloading, and a checkpoint every 250 optimizer updates. All arguments were checked against the tagged Sana implementation [D31]. We use `--dataset_name` pointing to the local imagefolder and `--caption_column=text`. The required `--instance_prompt` is a fallback; our nonempty per-image captions take precedence. Do not use `--instance_data_dir` for this metadata-bearing folder: that branch treats files as images rather than loading captions. The script is named `DreamBooth`, but also accepts a captioned style dataset without enabling prior preservation.

What these image training settings actually change

- **Batch 1:** one image is processed at a time on the GPU. This is the first memory lever.
- **Accumulation 4:** gradients from four successive microbatches are added before an optimizer update. With one GPU the effective batch is 4. It does not put four images in memory simultaneously and does not make a single oversized image fit.
- **1,000 optimizer steps:** roughly 4,000 image presentations, or about 20.8 average passes over 192 examples. This is a starting experiment, not the correct number for every dataset. More training can make a model worse.
- **Rank 8:** each selected weight update is constrained to a low-dimensional correction. Higher rank gives more adjustable capacity and increases adapter memory; it does not add training examples.

- **Learning rate 0.0001:** the scale of parameter changes. A rate that works for LoRA is not automatically appropriate for updating every base weight.
- **Warmup 100:** gradually reaches the selected learning rate during the first 100 optimizer steps. The learning-rate schedule controls optimizer updates. It is unrelated to the diffusion noise schedule.
- **BF16 mixed precision:** uses bfloat16 for the image transformer and Gemma text encoder in this recipe; the autoencoder stays float32. These component-specific choices follow the inspected implementation. BF16 has a different numeric range/precision trade-off from FP16. A filename variant and a runtime arithmetic dtype are related but separate settings.
- **Gradient checkpointing:** discards selected intermediate activations and recomputes them during backward. It exchanges more computation for less activation memory.
- **Offload:** moves the frozen text encoder and autoencoder back to CPU when their turn is finished. This reduces simultaneous GPU residency and adds host RAM use and CPU-GPU transfers. It is not the same as training the text encoder or making the network smaller.
- **128 prompt tokens, no complex-human instruction:** use the same limit and prompt preparation in training, baseline inference and adapted inference. The pipeline has an instruction-prefix default that this recipe deliberately disables; inconsistent preprocessing would confound the comparison.
- **Gradient clipping 1:** bounds the gradient norm before updating, limiting unusually large updates. It does not repair an invalid objective or corrupted data.
- **Checkpoint every 250:** saves resumable state. The final adapter is a deployment artifact; resumable checkpoints additionally preserve training state. Do not assume those files are interchangeable.

With this script, resuming an interrupted run uses the same output folder and `RESUME=latest`. Keep the original configuration and dataset unchanged. Example:

```
RESUME=latest STEPS=1000 WARMUP=100 bash train_image_lora.sh
```

Resume counts toward the total target of 1,000 steps, not 1,000 additional steps. This upstream trainer resumes optimizer/model state but begins iterating from the stored epoch rather than exactly skipping to the next old minibatch. Treat it as recovery, not bit-for-bit replay of an uninterrupted data order. A finished checkpoint may be useful even if you ultimately choose an earlier one by evaluation. At this inspected tag, checkpoint folders also contain a saved LoRA file, so you can pass `--lora runs/papercard-lora/checkpoint-250` to the inference script for that candidate.

Just enough diffusion what target did the network learn

In next-token language modeling, a model predicts a distribution over next token IDs. The Sana denoiser predicts a continuous tensor that describes movement along a path between a clean image latent and noise. The caption guides the prediction but is not the output target.

Let z_0 be a clean compressed image and ϵ be freshly sampled Gaussian noise of the same shape. Choose a schedule level σ and mix them:

```
z_sigma = (1 - sigma) * z0 + sigma * epsilon
training target = epsilon - z0
```

This is the flow-matching convention used by the inspected Sana trainer. At sigma zero the mixture is clean; at sigma one it is noise. The target is the derivative of that straight interpolation in the clean-to-noise direction. Generation follows the learned field in the reverse direction, using a compatible sampler. The network is not trained to predict a word or simply to emit the clean image in one step. Do not substitute a DDPM noise-prediction loss without also changing the model/objective contract [D03] [D31].

The trainer solves one randomly chosen path-level problem per example, using mean squared error against this movement target. Fresh noise and path levels create useful corruption variety, not new independent clean images. Generation instead begins with random noise and applies repeated model/solver updates. A thousand optimizer updates, a training schedule's available time levels and twenty inference steps are three different quantities.

The latent autoencoder and text encoder each have one job

Sana does not run its image transformer directly on every $1,024 \times 1,024 \times 3$ color value. The pretrained **DC-AE autoencoder** compresses the image spatially by 32, and a decoder maps generated latents back to pixels. A 1,024-square image therefore has a 32×32 latent spatial grid; channel count and scale come from the downloaded model configuration. DC-AE is not interchangeable with Stable Diffusion's older variational autoencoder. The Diffusers pipeline calls its autoencoder component vae, but that attribute name does not make every architecture a variational autoencoder [D30] [D32] [D33].

The frozen **Gemma text encoder** converts the caption into contextual vectors and an attention mask. The denoiser uses these as a condition. It does not generate an enhanced caption in this recipe: both training and inference set `complex_human_instruction=None`, use the same tokenizer and cap the encoded caption at 128 tokens. Token IDs, hidden-state width, attention mask, negative-prompt convention and text normalization all belong to the conditioning contract.

Caching can remove repeated work when representations are keyed to their source example. We deliberately do not set `--cache_latents` here: the inspected trainer caches by batch position while its loader shuffles. With distinct captions, that can pair a cached image latent with another image's caption on a later pass. Keeping the cache disabled preserves alignment without pretending the optimization is free. Any future cache should key image hash, caption, crop/augmentation, encoder revision, latent scaling and precision together [D31].

The safe placement pattern in this script is: bring the text encoder to GPU for the current caption, move it back to CPU, bring the autoencoder to GPU for the current image, move it back, then perform the transformer update. Gradient checkpointing handles part of the remaining activation memory. This is slower than well-designed persistent caches, but simpler to inspect. The full models still exist in host or device memory; offloading does not erase their parameter counts.

A useful representation test is an autoencoder round trip: encode an input, decode it, and inspect the reconstruction. If fine lettering or texture is lost in that representation, training the denoiser longer cannot recover information the latent does not retain. Improving or replacing the autoencoder is a separate project.

LoRA is an update method DreamBooth is a personalization method

A full fine-tune allows many or all denoiser weights to move. **LoRA** freezes the original weight matrix W and learns two thin matrices, often written A and B , whose product forms the correction:

$$W_{\text{effective}} = W + \text{scale} * (B @ A)$$

For a square $1,024 \times 1,024$ weight, full training has 1,048,576 adjustable entries. Rank 8 uses $8 \times 1024 + 1024 \times 8 = 16,384$ entries for the correction, before any additional adapter details. That is a simple parameter-count illustration, not total model memory. You still store the frozen network and run it to obtain gradients for the adapter. The official LoRA guide explains the supported Diffusers integration [D08].

DreamBooth is a strategy for associating a small collection of subject images with a prompt identifier, commonly including a class-based prior-preservation objective. It can be combined with LoRA or with fuller weight updates. Thus “LoRA or DreamBooth?” is not an either/or distinction. For ten images of one personal object, inspect a DreamBooth-specific recipe; for the captioned style dataset here, we use per-image captions and leave prior preservation disabled. Prior preservation helps resist replacing a broad class with one subject, but it does not guarantee against memorization [D07].

Training the autoencoder, text encoder and denoiser together from scratch would ask the same small dataset to supply several kinds of knowledge. It greatly expands memory, data and debugging demands. A tiny 32- or 64-pixel pixel-space diffusion model can be an excellent mechanics experiment on one GPU; a general text-to-image foundation model from scratch is not a realistic beginner target for one 24 GB card.

Image project step 4 evaluate adaptation instead of cherry picking

```
python sample_image_lora.py --base "$BASE" --lora runs/papercard-lora \
  --prompt 'a blue house, rivetpaper style, centered on cream paper' \
  --seed 17 --out samples/adapted-house.png
python sample_image_lora.py --base "$BASE" --lora runs/papercard-lora \
  --prompt 'a bicycle, rivetpaper style, centered on cream paper' \
  --seed 17 --out samples/adapted-bicycle.png
```

Image inference on GPU and CPU

The inference script reconstructs the complete pipeline from the same local base snapshot and then loads the trained adapter. It uses the snapshot's tokenizer, text encoder, latent scaling, decoder and inference scheduler configuration. The training code constructs a flow-matching noise/path scheduler, while the pipeline can use a different compatible numerical solver at inference; matching the flow-prediction semantics is essential, not forcing identical scheduler class names. Text-to-image generation needs no input photograph; it starts from new noise, with the prompt encoded in the same vocabulary used during adaptation. The DC-AE decodes the final latent; the pipeline converts the image values to a PIL image and the script saves PNG. The adjacent JSON records prompt, device, dtype, seed, steps and scheduler configuration.

GPU inference is the default above. The companion implementation offers CPU execution through ordinary PyTorch operations, with all components in float32. GPU execution uses BF16 for the transformer/text encoder, FP32 for DC-AE and model CPU offload. CPU uses no CUDA offload hooks and can need substantially more system RAM; it is not equivalent in speed or memory to the GPU path:

```
python sample_image_lora.py --base "$BASE" --lora runs/papercard-lora \
  --prompt 'a blue house, rivetpaper style, centered on cream paper' \
  --seed 17 --device cpu --steps 20 --out samples/cpu-house.png
```

The CPU command is a float32 implementation path, not a measured latency or universal-kernel-support promise. The official model examples target CUDA. On CPU this multi-billion-parameter pipeline needs substantial RAM and can be very slow; it is an optional diagnostic fallback, not the recommended production path. A saved adapter does not require optimizer state for inference, but it still requires its compatible base. CPU and CUDA random generation and arithmetic may yield different pixels even with the same seed. Compare semantic behavior, not byte equality. Copy `base_revision.json`, `book_run_manifest.json`, the environment record, adapter and evaluation prompts together when moving the project to another machine.

Make a fixed evaluation grid before deciding which checkpoint is best. Suggested rows are familiar objects with the style phrase, unfamiliar objects with the phrase, familiar objects without the phrase, and prompts that explicitly change the background, color or layout. Suggested columns are four fixed seeds. Generate the same grid with the base model and every serious checkpoint candidate.

Score separate questions rather than giving one vague score:

1. **Content:** is the requested object present? Are requested colors and counts respected?
2. **Style:** is the paper treatment learned beyond copying the training composition?
3. **Control:** can the prompt alter background and layout? Does the trigger have an understandable effect?
4. **Diversity:** do different seeds produce genuinely different acceptable designs?
5. **Retention:** do ordinary prompts without the trigger still work?
6. **Memorization:** does an output reproduce a particular training picture, including unusual placement or defects?

Use the held out folders in checkpoint selection

The following commands read the actual validation `metadata.jsonl`, select eight evenly spaced records to cover the generator's object families, and create two fixed-seed samples per caption. The pipeline loads once per command. The helper writes a `cases.csv` linking each generated image to its held-out reference image, plus blank fields for your rubric scores. It does not invent scores.

```
python evaluate_image_grid.py --base "$BASE" \
  --metadata data/papercards/val/metadata.jsonl --split val \
  --limit 8 --seeds 17,29 --out evaluations/base-val
python evaluate_image_grid.py --base "$BASE" --lora runs/papercard-lora/checkpoint-250 \
  --metadata data/papercards/val/metadata.jsonl --split val \
  --limit 8 --seeds 17,29 --out evaluations/step250-val
python evaluate_image_grid.py --base "$BASE" --lora runs/papercard-lora \
  --metadata data/papercards/val/metadata.jsonl --split val \
  --limit 8 --seeds 17,29 --out evaluations/step1000-val
```

Open each CSV's generated/reference paths and score content and style consistently. The reference picture is a style/content example for human review, not an input to the text-to-image model and not an exact pixel target. Compare diversity across the paired seeds and check suspicious outputs against the training set. The held-out drawings have new random visual variations, but their simple captions can repeat training captions. This validates narrow within-generator behavior, not new semantic categories. The separate bicycle/no-trigger/layout prompts probe broader transfer and control.

Choose the checkpoint using validation only. Then generate the same untouched test-caption cases for that selected checkpoint and the base. For example, if checkpoint 250 won:

```
python evaluate_image_grid.py --base "$BASE" \
  --metadata data/papercards/test/metadata.jsonl --split test \
  --limit 24 --seeds 17,29 --out evaluations/base-test
python evaluate_image_grid.py --base "$BASE" --lora runs/papercard-lora/checkpoint-250 \
  --metadata data/papercards/test/metadata.jsonl --split test \
  --limit 24 --seeds 17,29 --out evaluations/selected-test
```

Review those test reference/generated pairs once with the same rubric. Report the outcome even if it is worse. Selecting another checkpoint based on the test results turns that split into more validation data. Use fresh output directories for each evaluation; the helper refuses to mingle new outputs with an existing nonempty run.

Loss is useful for discovering broken training, but lower flow-target error does not automatically mean better pictures. For a tiny dataset, a large-sample distribution metric such as FID is unstable and can obscure the practical question. Start with reproducible side-by-side inspection, a clear rubric and nearest-neighbor review. Image-embedding similarity can help find suspicious training matches, but no threshold certifies originality.

For memorization checks, compare outputs to all training images, including crops and resized versions. Exact file hashes catch duplicates in the dataset but not visually near-identical images. If the model repeatedly reconstructs a training composition, stop earlier, reduce the learning rate or rank, remove repeated images, and add more independent variety. Do not call a memorized output “a successful style transfer” simply because it looks polished.

Guidance belongs mainly to sampling

Classifier-free guidance combines two predictions, conditioned and unconditioned. A common convention is:

```
prediction = unconditioned + g * (conditioned - unconditioned)
```

At $g = 1$, this expression is simply the conditioned prediction; higher values extrapolate toward the condition. Training a new model for this mechanism usually includes dropping the condition on some examples, giving it an unconditional case to learn. The Sana adaptation script uses a pretrained guided model; do not assume it has a configurable caption-dropout flag. The audio script later implements condition dropout explicitly [D04].

A high guidance setting can increase apparent prompt adherence while damaging naturalness or reducing diversity. Try a small fixed set, such as 3, 5 and 7, while holding the checkpoint and seed fixed. That is an experiment, not a universal quality ranking. Do not change guidance between baseline and adapter comparisons unless that change is itself the experiment. Some distilled models use different guidance mechanisms or expected values; their cards and scheduler settings take priority over this generic formula.

Image troubleshooting and a 24 GB decision ladder

First measure the running process with `nvidia-smi`. Record peak training use as well as separate sample-generation use. A run can train successfully and fail while creating a large validation batch. CUDA allocator reservations and process memory are related but different measurements; record which you used.

Symptom	First checks	Next controlled experiment
Out of memory before the first update	Other GPU processes, image resolution, wrong model family, precision	Batch 1; turn on checkpointing; avoid simultaneous validation pipelines
Out of memory only during samples	Number and size of generated samples, live training graph	Generate one image after training has exited
NaN or infinite loss	Data range, DC-AE output, component dtypes, learning rate	Reduce learning rate; isolate FP32 forward on a tiny batch; then inspect the exact failing component
Style appears but every image has the same object	Dataset/caption entanglement	Add independent objects and backgrounds; evaluate an earlier checkpoint
Trigger seems ignored	Wrong adapter loaded, weak/incorrect captions, insufficient variation	Verify adapter file and base revision; test paired fixed-seed prompts
Fine detail is always poor	Source resolution, crop damage, autoencoder reconstruction	Improve data and representation before extending training
Fast loss reduction but poorer generation	Overfitting, repeated data, excessive guidance	Compare earlier checkpoints and lower guidance separately

The practical next-step ladder is:

- **This Sana 1.6B BF16 LoRA configuration:** batch 1, rank/alpha 8, 1,024-square images, checkpointing, frozen-component offload. The official script and its memory options support the approach; the book does not supply a measured VRAM number. Test loading, first backward, optimizer allocation, saving and separate inference before a long run [D31] [D36].
- **Sana 600M:** the official card lists a 590M transformer with a 512-pixel variant. It still needs its Gemma text encoder and DC-AE, so it is not a 590M complete application. It uses a different preferred precision/variant. Treat switching as a new checked experiment, not just changing the model name [D37].
- **Full transformer fine-tuning:** optimizer states and gradients increase sharply when you stop freezing the backbone. Prove a need beyond the adapter baseline, reduce scale and measure before assuming it fits. A small complete model such as the audio project is a better first from-scratch exercise.

These are maintained practical choices rather than an exhaustive leaderboard. Older SD 1.x systems illustrate a U-Net and epsilon-prediction contrast, while Sana demonstrates a transformer and flow-matching target; their adapters, autoencoders, text encoders and samplers are not interchangeable. There is no detailed legacy training route in this package.

U Net versus DiT another architecture the same project questions

A **U-Net** reduces spatial resolution while building features, then upsamples while reusing earlier features through skip connections. This combines local detail with wider context. A **diffusion transformer**, or **DiT**, processes patches or latent tokens with transformer blocks. Attention lets positions exchange information, but large token counts increase computation and often memory. Neither name tells you whether the model predicts noise, clean data, a velocity parameterization or a flow field. Architecture and training objective are separate choices [D06].

For practical adaptation, ask four questions before changing a command: What tensor enters the denoiser? What condition does it accept? What target was it pretrained to predict? Which parameters does this trainer actually update? The answers matter more than whether the model's name contains “diffusion.”

27 Speech recognition train a small model to hear your domain

The project accurate workshop voice notes

You are building an offline transcription tool for short workshop notes. A person says, “Replace the M eight bolt on pump three,” and the tool writes the words. You care especially about part names, numbers, negation, accents, and noisy recordings. The deliverable is a locally saved model, a repeatable evaluation report, and CPU/GPU inference commands.

This is **automatic speech recognition (ASR)**: recorded sound becomes written language. It differs from the next audio-generation project, where a model creates sound. It also differs from a classifier that says “speech present” or “doorbell,” and from a language model that rewrites a transcript. A fluent rewrite can conceal a recognition error. Keep the original audio, the raw transcription, and any edited presentation as distinguishable records.

The current practical model in this chapter is **Qwen3-ASR-0.6B**, released in 2026, with an official open training path. Its name describes the language-model scale; Hub metadata lists **938,008,576 serialized BF16 parameters including the acoustic components**, so budget approximately 0.94B overall, not 0.6B. That remains below one billion. Cohere Transcribe is the current 2B comparison and extension study. Older Whisper and CTC work appear only where they explain an enduring idea. This is not a tutorial organized around a legacy checkpoint. [S01] [S02] [S03]

Success means that a saved adaptation improves your predeclared validation objective without an unacceptable regression on general speech, then passes the untouched test set. A decreasing training loss is an intermediate observation. It does not establish that a model transcribes better.

ASR application patterns

- **Dictation and accessibility:** prioritize omissions, negation, uncommon words, readable punctuation, and correction effort
- **Searchable recordings:** transcribe, attach reliable segment boundaries, then use a separate embedding/retrieval system; transcription quality and search quality require separate tests
- **Meeting notes:** recognition, speaker diarization, and summarization are different tasks; do not imply that an ASR checkpoint supplies all three
- **Voice commands:** transcribe, parse the intended action, and confirm consequential operations; a plausible transcript is not permission to execute a command
- **Domain vocabulary:** adapt using recordings that genuinely contain the relevant words, rather than repeatedly showing the words as text alone

The ASR component should expose uncertainty through review policies and observable failure signals, not fabricated confidence percentages. Token likelihood is not a calibrated probability that a sentence is correct.

Current Cohere models know which category you are studying

Cohere is a model family and service provider, not a single architecture. The following distinctions prevent the common mistake of using a text-generation training recipe for every product with the same vendor name.

Current example, checked 2 October 2026	Input → output	Place in this book	Access/training boundary
Cohere Transcribe 03-2026	Audio → transcription	ASR comparison, 2B	Apache-2.0 weights; native Transformers support; Hub access gate observed
Cohere Transcribe Arabic 07-2026	Arabic/English audio → transcription	Specialized ASR comparison, 2B	Apache-2.0; official Arabic adaptation of Transcribe
Embed v5.0 pro/fast	Text/images → vectors	Retrieval category	Official service/deployment documentation; no publicly downloadable trainable checkpoint verified here
Rerank v4.0 pro/fast	Query + candidate documents → ranking	Reranker category	Service/deployment option; do not invent an open-weight fine-tuning command
North Micro Vision Instruct	Images/text → text	Vision-language comparison, 2.4B total	Apache-2.0, official fine-tuning references; separate model-specific project
Tiny Aya global	Text → multilingual text	Brief scale/license contrast	3.35B in its model summary, CC-BY-NC-4.0; exceeds a strict 3B total-parameter ceiling
Command A+ 05-2026	Text/images → generated text	Advanced MoE comparison only	Apache-2.0; 218B total / 25B active, far outside this GPU-training scope

Sources for this inventory: [S04] [S05] [S06] [S07] [S08] [S09] [S10]. “No checkpoint verified” is a statement about the inspected public material, not proof that an enterprise contract cannot supply a private deployment. API availability does not imply accessible gradients, downloadable weights, or permission to retrain. Conversely, a Hub gate does not make an Apache-licensed checkpoint API-only. Access requirements, license terms, model code, and feasible hardware are separate questions.

For a recording-search application, a coherent pipeline could be: ASR produces text; an embedding model retrieves related passages; a reranker improves their order; a generation model writes a grounded answer. Train or evaluate the component responsible for your actual error. Retraining ASR will not fix a retrieval index that discarded the relevant passage.

What Cohere Transcribe actually does

The March model has a **Conformer acoustic encoder** and a smaller autoregressive Transformer decoder. It consumes waveform-derived log-Mel features and predicts text tokens with supervised cross-entropy. Cohere describes training it from scratch; adapting its weights is a different, much smaller undertaking. A Conformer combines local convolutional processing with attention over a longer context. This is useful because speech has both short acoustic events and dependencies spanning many frames. [S05] [S11]

The March checkpoint covers 14 specified languages. Its card flags missing native timestamps and diarization, no explicit automatic language identification, and difficulty with code-switching, where a speaker alternates languages within an utterance or conversation. It also warns that non-speech audio can produce invented text. The native path requires Transformers 5.4.0 or later; using the built-in implementation avoids the older remote-code loading path. The card reports testing with PyTorch 2.10.0. [S04]

The July Arabic checkpoint is an official domain/language adaptation. Its introduction emphasizes Arabic dialects, English, and Arabic-English mixed speech; its limitations section still warns about code-switching. Treat that mixed message as a reason to test your own dialect and mixed-language slices rather than make a blanket promise. Both cards were visibly gated for contact-information sharing during this review. This book did not accept either gate or download those weights. [S06]

Is Cohere locally trainable

The native Transformers source exposes labels, a shifted decoder input, and a supervised loss in `CohereAsrForConditionalGeneration`. That establishes a training-capable forward path. It does not establish a completed one-card recipe, a trustworthy adapter target list, or measured memory consumption. The official `cohere-finetune` repository inspected for this chapter lists text-generation models; it is not evidence that its Command LoRA command supports Transcribe. [S12] [S13]

A full 2B-model FP32 AdamW run needs roughly 32 GB for parameters, gradients and two moment buffers before activations: $2 \text{ billion} \times 16 \text{ bytes}$. Some mixed-precision strategies differ, but merely observing approximately 4 GB of BF16 inference weights does not solve the training budget. Freezing the encoder or adding adapters could reduce optimizer state substantially. A responsible Cohere extension must first verify the exact trainable modules, tokenizer prefixes, acoustic chunk behavior, loss alignment, a successful backward pass, saved-model reload, and measured peak VRAM. We do not present that unexecuted research program as a proven 24 GB fine-tune.

The chapter therefore supplies a complete smaller current-model adaptation with the official Qwen backend, while retaining Cohere as a substantive architecture/access/training-support study and local-inference comparison. It does not substitute a Cohere API call for teaching training.

What the model learns from an audio text pair

Waveforms frames and log Mel features

A mono waveform is one ordered sequence of amplitude measurements. **Sample rate** tells you how many measurements represent one second. Eight seconds at 16,000 samples per second has 128,000 samples. Changing a header from 48,000 to 16,000 without changing the sample sequence makes the recording play three times slower; it is not resampling.

Correct downsampling filters frequencies that the lower rate cannot represent, then changes the sample grid. The companion uses polyphase resampling. Its anti-aliasing test checks that a 12 kHz tone recorded at 48 kHz is suppressed when converted to 16 kHz. No automatic

loudness normalization is applied: multiplying quiet microphone hiss until it looks like speech is a poor default. [S14]

A **spectrogram** describes frequency content over successive short windows. A Mel filter bank pools that energy into frequency bands; the logarithm compresses its dynamic range. A **frame** is one such time-window representation, not one word. Use the checkpoint's own processor to choose windows, filters and scaling. An arbitrary image spectrogram is not interchangeable with the expected input tensor.

The Qwen processor creates acoustic features, an acoustic padding mask, and text tokens. Its audio encoder produces embeddings that replace audio placeholder positions in the language-model input. The supplied training script uses this exact model-specific processor. [S15] [S16]

CTC versus autoregressive transcription

Connectionist temporal classification (CTC) handles a transcript whose word/token boundaries are unknown. The network assigns probabilities to labels plus a blank symbol at successive time positions. Many frame-level paths correspond to the same final text. Training sums the probabilities of valid paths instead of requiring a manually timestamped label for every frame. A simple decoding path such as a a blank b b collapses to ab; a blank a can represent repeated aa. CTC's blank is not a literal space character. [S17]

CTC's monotonic alignment structure is useful for speech, but it is not the loss used by our Qwen project or Cohere Transcribe. In an **autoregressive** model, each output token is predicted using audio and earlier output tokens. **Teacher forcing** supplies the correct earlier tokens during training. Cross-entropy increases when the next correct token receives too little probability. Generation instead supplies the model's own earlier guesses, so generation-based evaluation can reveal failures that teacher-forced loss hides.

Cohere is an encoder-decoder sequence-to-sequence system. Qwen3-ASR connects an acoustic encoder to a causal language decoder. Both are conditional audio-to-text systems, but their input packing and shifted-label details differ. Do not swap their processors, reuse their special tokens, or assume that a CTC blank configuration applies. [S05] [S15] [S18]

One concrete training row

Prepare your own consented, accurately transcribed recording and a JSONL row such as:

```
{
  "id": "workshop_001",
  "audio": "audio/workshop_001.wav",
  "text": "Replace the M eight bolt on pump three.",
  "language": "English",
  "speaker_id": "speaker_01",
  "session_id": "session_01",
  "source_id": "recording_01",
  "domain": "workshop",
  "split": "train",
  "consent": true,
  "rights": "Recorded with permission for this training project"
}
```

A complete minimal manifest has all three splits. For example, put `my_recordings.jsonl` next to an `audio/` directory containing these three real files. Each file must contain exactly its own written label, and each speaker must have consented. Three rows only exercise the pipeline; they are not a meaningful accuracy dataset.

```
{
  "id": "train_001",
  "audio": "audio/train_001.wav",
  "text": "Replace the M eight bolt on pump three.",
  "language": "English",
  "speaker_id": "speaker_01",
  "session_id": "session_01",
  "source_id": "recording_01",
  "domain": "workshop",
  "split": "train",
  "consent": true,
  "rights": "Permission for this local training project"
}
{
  "id": "validation_001",
  "audio": "audio/validation_001.wav",
  "text": "Check the pressure before opening the valve.",
  "language": "English",
  "speaker_id": "speaker_02",
  "session_id": "session_02",
  "source_id": "recording_02",
  "domain": "workshop",
  "split": "validation",
  "consent": true,
  "rights": "Permission for this local evaluation"
}
{
  "id": "test_001",
  "audio": "audio/test_001.wav",
  "text": "Do not restart pump three.",
  "language": "English",
  "speaker_id": "speaker_03",
  "session_id": "session_03",
  "source_id": "recording_03",
  "domain": "workshop",
  "split": "test",
  "consent": true,
  "rights": "Permission for this local evaluation"
}
```

The text is an example of the intended label format; the book does not supply a recording of someone saying it. Do not train the phrase against a tone or an unrelated recording. The companion's synthetic tones are software tests, not an ASR dataset.

For the pinned Qwen backend, the target serialized by our script becomes `language English<asr_text>Replace the M eight bolt on pump three.` followed by its actual end-of-sequence token. The model-specific prefix tells the decoder the language and task format. The `audio/user/system` prefix positions are masked out of the loss; the answer and end marker remain supervised. The official training guide documents the language-prefix convention. [S19]

The original loop uses a single example per forward pass. This avoids silently inheriting the upstream training collator's assumptions about prefix masking in padded batches. It explicitly checks that the tokenized full example starts with exactly the tokenized prefix. A tokenizer revision that changes that boundary causes a clear failure rather than a subtly wrong objective.

Build the dataset before touching the optimizer

Start small then earn more data

Use 20-50 carefully checked clips for a pipeline smoke test. This strict unseen-speaker starter requires at least three independently consenting speaker groups, one or more per split; a single person recording all clips cannot satisfy its split policy. A same-speaker personalization experiment is a different evaluation design and is not what this starter claims. For a meaningful pilot, aim for a few hours of relevant speech with multiple speakers and recording sessions, plus independent validation and test material. These are planning starting points, not a guarantee that any particular number of hours will improve quality.

If the domain is confidential, keep recordings, transcripts, caches and checkpoints local. A checkpoint can memorize names, addresses or unusual phrases. Permission to record a meeting is not automatically permission to publish its audio, train a shared model, or upload it to a transcription API. Document who consented, permitted uses, retention, and deletion procedures. Do not use real confidential customer calls merely because a tutorial needs audio.

Segment and align never blindly crop the pair

The starter enforces 0.2-8-second clips to keep the experiment bounded. Segment at sensible utterance boundaries. If a long recording says ten sentences, a crop containing the first sentence must not retain the full ten-sentence transcript. That teaches the model to invent absent content.

For a longer recording, either use reliable existing time-aligned labels or align a reviewed transcript, inspect uncertain boundaries, and produce short paired segments. **Forced alignment** estimates where supplied words occur in audio; it does not prove those words were spoken. Qwen's optional timestamp component is a separate forced-aligner model and is not downloaded by this project. **Diarization** estimates which speaker spoke when; that is another distinct task. [S01] [S20]

Silence-based chunking is useful, but whispers, breathy speech and low-volume consonants can be cut off. Overlapping chunks can avoid lost boundary sounds but also create duplicated words. For inference, use model-supported long-form handling and test complete recordings; this training project deliberately starts with manually checked short segments.

Make leakage difficult

Randomly splitting clips is insufficient when adjacent clips belong to the same call. Split whole source recordings, sessions and speakers together. The preparation script rejects an identifier appearing in more than one split and rejects identical input WAV bytes. Use pseudonymous, stable IDs. If two speakers converse in the same source recording, the source and speaker constraints may connect several recordings into one group; assign the whole connected group together.

A reasonable first policy is about 80/10/10 by grouped recording duration, then verify that validation and test have enough speakers and examples for meaningful comparisons. A tiny test set is not made reliable by a tidy percentage. Also reserve a challenge set for a new microphone, room, dialect, or time period. The script reports language, domain and speaker slices but does not invent the correct split for you.

Keep approved augmentation only in the training split. If you add realistic background noise, keep the spoken transcript unchanged and audit intelligibility. A transformed copy of a validation recording remains a validation recording. Do not optimize on the test set by repeatedly listening to its hardest examples and adding near-duplicates to training.

Preserve transcript policy

Choose verbatim versus cleaned transcription before labeling. Decide whether “um,” false starts, abbreviations, spoken numbers and punctuation should be present. If half the labels use “M8” and half use “M eight,” the optimizer receives an inconsistent formatting target. Keep the raw human label and document any training normalization separately.

For multilingual data, preserve the writing system and Unicode text. Do not strip accents from Vietnamese or merge distinct Arabic letters merely to make a metric smaller. Tokenization and normalization policies are language-dependent. The first script accepts supported Qwen language names exactly, such as English, rather than assuming that every library uses ISO codes. Cohere's inference example instead uses codes such as en and ar.

Environment and artifact plan

First open a terminal in the extracted companion directory and run `cd examples/asr`. Stay in this directory for all commands in this chapter.

Use a fresh Python 3.12 environment for Qwen. `requirements-qwen.txt` pins the official backend to commit `7c6daf77a2421100f5fb066495372c00129d39ff`, whose package metadata requires Transformers 4.57.6 and Accelerate 1.12.0. The candidate PyTorch version is 2.10.0. Other core audio/numeric pins are listed in the file. This is a source-checked target, **not a dependency lockfile validated by an installation in this book's authoring environment**. Install a PyTorch wheel suited to your GPU/driver from the official PyTorch instructions, then resolve the remaining requirements. Run `pip check` and record the resolved environment. [S21] [S22]

```
# Leave any other project environment first
deactivate 2>/dev/null || true
python3.12 -m venv .venv-qwen-asr
source .venv-qwen-asr/bin/activate
# CUDA 12.8 example; choose the official wheel compatible with your driver
python -m pip install torch==2.10.0 --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements-qwen.txt
python -m pip check
python -m pip freeze > resolved-environment.txt
python fetch_qwen_model.py models/qwen3-asr-0.6b
```

For a CPU-only installation, replace the CUDA wheel command with `python -m pip install torch==2.10.0 --index-url https://download.pytorch.org/whl/cpu`. Choose one wheel route; do not run both. The GPU training command requires a CUDA wheel and a BF16-capable GPU. PyTorch's official version page also lists other CUDA builds; selecting one requires a compatible driver. [S24]

Those are user-run setup commands; they were not executed for this book. The fetch script pins model revision `5eb144179a02acc5e5ba31e748d22b0cf3e303b0`, records it locally, and downloads the model files. It does not execute model-repository remote code. Installing the pinned official backend is still installing software: review the source and use an isolated environment. Keep the Qwen 4.57.6 environment separate from Cohere's 5.4+ environment. [S02] [S21]

Run preparation

All commands below run from the extracted companion's `examples/asr` directory, with `.venv-qwen-asr` active. The input manifest paths are relative to the manifest file, not the shell working directory:

```
python prepare_audio.py my_recordings.jsonl prepared --max-seconds 8
python test_data_metrics.py
```

Preparation reads local WAV files, handles integer/float PCM, averages channels, resamples to mono 16 kHz, writes canonical PCM16 WAVs, and saves train/validation/test manifests with duration, RMS, peaks and content hashes. Averaging is a starting policy, not universally correct: out-of-phase stereo can cancel speech, and separate call channels may contain different speakers. Listen to representative files before and after conversion; select an appropriate channel upstream when required.

The tool refuses to reuse an output directory. It does not transcribe, invent labels, accept terms, fetch a dataset, or upload recordings. Review the generated duration totals and listen to

each smoke-test clip. The record of consent is documentation supplied by you, not an automatic legal assessment.

The pinned adaptation experiment

Establish the unadapted baseline

Run validation generation before training:

```
python infer_qwen_asr.py --model models/qwen3-asr-0.6b \
  --manifest prepared/validation.jsonl --device cuda \
  --out runs/base-validation.jsonl
```

The script saves raw predictions, references, language/domain/speaker metadata, elapsed time, and a separate metrics JSON. It fixes the supplied language by default. Use `--detect-language` as a separate evaluation condition if language identification is part of your product. Do not compare a language-forced baseline with an unconstrained adaptation and attribute the whole difference to training.

What is actually updated

The starter freezes the acoustic tower and updates the remaining parameters. This is **partial fine-tuning**, not LoRA and not training from scratch. Its purpose is a controlled first experiment in transcript vocabulary/format adaptation with a lower optimizer-memory burden. It may be insufficient when the main failure is a substantially different acoustic domain; measure before deciding to unfreeze more.

The script uses FP32 trainable parameters and AdamW states, with BF16 autocast on a compatible CUDA GPU. This avoids pretending that loading BF16 weights automatically gives FP32 optimizer/master state. Gradient checkpointing recomputes intermediate values to reduce activation storage. A frozen audio tower is also held in evaluation mode. It runs microbatch 1, accumulates eight examples, normalizes the accumulated loss by the number of supervised next-token targets, clips gradient norm to 1, and uses a short warmup followed by decay. [S15] [S18] [S22]

```
python train_qwen_asr.py --model models/qwen3-asr-0.6b \
  --train prepared/train.jsonl --out runs/asr-smoke \
  --steps 2 --accumulate 2 --save-every 2

python infer_qwen_asr.py --model runs/asr-smoke/final \
  --manifest prepared/validation.jsonl --device cuda \
  --out runs/smoke-validation.jsonl
```

A two-update run validates plumbing after you execute it. It is not a claim of improved ASR. First confirm finite loss, nonzero trainable gradients, reasonable parameter counts, a full export, successful reload, and text generation. Then try a bounded pilot:

```
python train_qwen_asr.py --model models/qwen3-asr-0.6b \
  --train prepared/train.jsonl --out runs/asr-pilot \
  --steps 200 --accumulate 8 --lr 0.00001 --save-every 100
```

The 200-step command writes `runs/asr-pilot/step-000100`, `runs/asr-pilot/step-000200`, and `runs/asr-pilot/final`; `final` contains the same final-update weights as step 200. Compare both candidate checkpoints:

```
python infer_qwen_asr.py --model runs/asr-pilot/step-000100 \
  --manifest prepared/validation.jsonl --device cuda \
  --out runs/pilot-100-validation.jsonl
python infer_qwen_asr.py --model runs/asr-pilot/step-000200 \
  --manifest prepared/validation.jsonl --device cuda \
  --out runs/pilot-200-validation.jsonl
```

Inspect those `.metrics.json` files alongside `runs/base-validation.jsonl.metrics.json` and listen to the error slices. Set `SELECTED` to the actual validation winner. The following assignment is an example, not a claim that step 100 will win; if the base is better, set it to `models/qwen3-asr-0.6b` instead:

```
SELECTED=runs/asr-pilot/step-000100
python infer_qwen_asr.py --model "$SELECTED" \
  --manifest prepared/test.jsonl --device cuda \
  --out runs/selected-test.jsonl
```

Evaluate saved checkpoints on validation with exactly the same inference settings as the baseline. Select the checkpoint using your declared criterion, including non-speech and domain-critical errors. Only then run the selected checkpoint once on the untouched test split. If validation degrades, keep the base model; do not deploy the newer file merely because training finished.

The example saves full inference checkpoints and processors. It does **not** save Adam state, shuffled-data position, or RNG state. The starter deliberately accepts only the pinned base provenance; continuing from an adaptation would require a separate explicitly designed continuation/resume mode. Loading saved weights alone would be further training with a fresh optimizer, not exact resumption. Add a complete trusted-local resume mechanism only after the initial experiment works.

A realistic 24 GB planning budget

The total FP32 weights alone are approximately 3.75 GB decimal. If every one of the 0.938B parameters were trained with FP32 gradients and Adam's two moment buffers, the simple parameter-state subtotal would be about 15.0 GB decimal before activations and temporary tensors. Freezing the acoustic tower removes its gradient and optimizer buffers; the script prints the actual trainable count rather than assuming the model's marketing name gives it. These are arithmetic bounds, not measured allocator peaks.

For this short-clip, microbatch-1, frozen-tower project, a 24 GB RTX 3090/4090-class card is a plausible target to test. We have **not measured a peak VRAM number or training speed**. The first run records both peak allocated and peak reserved CUDA memory. Leave several GB of headroom; close unrelated GPU applications. If it fails, shorten correctly aligned clips, reduce accumulation only to reduce CPU staging rather than expecting major GPU savings, and inspect logits/activation length. Increasing accumulation changes effective batch; it does not shrink a single forward pass.

Plan roughly 16-32 GB host RAM, several GB for downloaded weights, and about 3.75 GB per FP32 exported checkpoint before packaging overhead. Keeping ten full checkpoints can

consume tens of GB. Start with two. Eight-second 16 kHz PCM16 mono audio is about 256 KB before container overhead; one hour is about 115 MB. Save the unmodified recordings separately only as long as your retention policy permits.

Measure wall time for 20-50 warmed-up updates. If your measured average is t seconds per optimizer update, 200 updates take approximately $200 \times t$ plus setup, exports and evaluation. For example, 3 seconds would imply around 10 minutes of update time; 12 seconds would imply around 40 minutes. These are arithmetic illustrations, not claimed RTX benchmarks. Generation-based validation may take longer than a loss-only pass.

Evaluate recognition not just readable output

WER CER and a worked error

Word error rate (WER) is $(\text{substitutions} + \text{deletions} + \text{insertions}) / \text{reference_words}$, after an explicitly chosen normalization and tokenization. If the reference is “turn the pump off” and the hypothesis is “turn pump on,” deleting “the” and replacing “off” with “on” gives $2/4 = 50\%$. The dangerous negation error matters more operationally than the harmless article omission, even though both count as one edit.

Character error rate (CER) uses character units instead. Our helper uses NFC-normalized Unicode code points after removing whitespace; it does not claim to implement grapheme-cluster segmentation. CER is useful for scripts where whitespace is an unreliable word delimiter. Chinese 你好世界 versus 你好世间 has one changed character out of four, or 25% CER. A whitespace WER for that pair treats each whole string as one word and is much less informative.

Aggregate edit counts and reference lengths across the corpus; do not simply average utterance WERs, which lets a one-word clip weigh as much as a long sentence. WER can exceed 100% when the model inserts many words. For an empty reference, the denominator is zero: report invented-output counts separately instead of manufacturing a conventional percentage.

Raw and normalized scores answer different questions

The supplied scorer reports:

- Raw WER/CER, preserving case and punctuation after Unicode and whitespace normalization
- A transparent lexical variant that casefolds and replaces Unicode punctuation with spaces
- Empty-reference false positives and inserted words
- Separate language, domain and speaker slices

This simple normalizer is **not** the exact scorer used by either vendor's leaderboard. It splits apostrophes when stripping punctuation and performs no number expansion. Use it consistently to debug your own experiment; use the dataset's published scoring rules for a benchmark comparison. Inspect punctuation/capitalization separately if they matter to the product. A lexical WER of zero does not mean the result is publication-ready.

For non-English evaluation, decide whether script variants, diacritics, spacing, numerals and code-switches represent acceptable alternatives or meaningful mistakes. Report each language rather than hiding a weak language in a large English average. In addition to WER/CER, count

domain term errors, missed negations, changed quantities, and correction time on a blind sample. These application metrics are original project design, not a new universal ASR score.

Silence noise and hallucination probes

Include exact digital silence, room tone, music without intelligible words, keyboard sounds, a distant speaker, clipped speech, an unfamiliar accent, and very quiet speech. Use empty references only for genuinely non-speech clips. Check whether the model emits a plausible sentence despite no speech.

Voice activity detection (VAD) predicts regions likely to contain speech. It can prevent unnecessary ASR calls, but false negatives erase speech before the recognizer sees it. Tune and evaluate VAD on validation audio; measure quiet-speech recall and boundary truncation as well as non-speech suppression. A raw amplitude threshold is not a reliable speech detector. The Cohere card explicitly highlights silence-related hallucination and VAD/noise-gate mitigation. [S04]

A deployment policy might flag repetition, output-token limits, empty audio, extreme duration-to-text mismatch, or high disagreement between model versions for review. These are triage signals. They cannot certify correctness. For important instructions, quantities, legal records or clinical content, preserve human verification rather than silently turning generated text into authoritative evidence.

Save once run on CPU and GPU

Use the `SELECTED` path chosen above for deployment, rather than assuming the last checkpoint is best. In a new terminal, reactivate `.venv-qwen-asr` and set `SELECTED` again. Each exported model directory contains model weights, configuration, tokenizer/processor files, and the run record. Keep the dataset manifest hashes, model revision, package versions and scoring policy with it. Do not publish private source paths, transcripts or recordings when sharing the model. Retest inference after moving the directory to its intended deployment machine.

```
# CPU: FP32, low concurrency, suitable for checking a small local file
python infer_qwen_asr.py --model "$SELECTED" \
  --audio new_note.wav --device cpu --language English \
  --out runs/new-note-cpu.jsonl

# GPU: BF16, one inference example at a time
python infer_qwen_asr.py --model "$SELECTED" \
  --audio new_note.wav --device cuda --language English \
  --out runs/new-note-gpu.jsonl
```

Expect CPU latency to depend strongly on hardware and audio length; this chapter promises no real-time rate. CPU and GPU kernels/precision can produce slightly different token choices. Compare a representative corpus, not only one sentence, before declaring them equivalent. The project requests no timestamps and loads no forced aligner.

Optional Cohere local comparison

After you personally complete the model's access requirements and obtain an authorized native-compatible local snapshot, use the separate Cohere environment and `infer_cohere.py`. Its candidate stack is PyTorch 2.10.0, Transformers 5.4.0, NumPy/SciPy for audio loading, and the tokenizer dependencies required by that snapshot, including SentencePiece/Protobuf. The snapshot must match native configuration and weight-key conventions, including `encoder_config`; an older remote-code snapshot is not interchangeable. The versioned documentation includes a converted revision example, while current cards advertise native support. The loader rejects obviously incompatible layouts. Perform a real local load test before calling the environment validated. [S04] [S12] [S23]

```
python infer_cohere.py --model /path/to/cohere-transcribe-snapshot \
  --audio new_note.wav --language en --device cpu
python infer_cohere.py --model /path/to/cohere-transcribe-snapshot \
  --audio new_note.wav --language en --device cuda
```

Use the same human references and scoring policy when comparing its outputs with Qwen. Do not compare a vendor's reported leaderboard average directly with your workshop validation WER. For the Arabic checkpoint, repeat the comparison by dialect and mixed-language condition. This script is source/API-reviewed only; neither Cohere weights nor its CPU/GPU inference were executed here.

What was tested what you still need to prove

During authoring, 11 CPU tests passed for WAV scaling, stereo conversion, duration-preserving resampling, anti-aliasing, NaN rejection, split/consent checks, preparation output, Unicode handling, edit counts, punctuation normalization, empty-reference behavior and WER above 100%. The environment was Python 3.12.14, NumPy 2.3.5 and SciPy 1.17.0. All supplied ASR Python files compiled. The exact test log is in the companion's validation materials.

PyTorch, Transformers, Qwen-ASR and SoundFile were absent. No new packages, pretrained weights, accounts, access-gate agreements, GPU runs, neural forwards/backwards, model reloads, or CPU/GPU numerical comparisons were performed. Source review and syntax validation do not establish a successful training run. Before spending hours, execute the two-update smoke test, inspect masks and counts, reload its export, and measure memory on your own machine.

Your next experiment

Compare three conditions with the same untouched validation audio: the pinned base, a 200-update frozen-audio adaptation, and a lower-learning-rate adaptation. Keep everything else fixed. Write down which words improved, which new errors appeared, whether silent clips became worse, and whether gains remain for unseen speakers. Only expand data or trainable layers when this evidence tells you what problem remains.

You have now completed the recognition half of the audio story: waveform and transcript pairs become a supervised objective and, eventually, a deployable transcriber. The audio-diffusion project changes the target completely: the output is a waveform or audio latent, and generation quality must be evaluated as sound.

28 Project build a tiny sound generator from random weights

Audio diffusion generates a waveform, an acoustic representation or a compressed audio latent by iterative refinement. Depending on its training and conditioning, it can create effects, ambience, music, or components of a speech system. A model trained on short sound effects should not be treated as a text-to-speech engine. If the task is transcription, classification, speaker verification or onset detection, choose a model trained for that discriminative or recognition task instead.

Useful application patterns include:

- **Prompt plus duration → sound-effect candidates.** Return WAV files with explicit sample rate, channels, duration, seed and model version. Audition candidates before placing them in a game or video; check event timing and tails, not just timbre.
- **Class ID → a narrow family of variations.** For example, generate alternative UI pings or percussion hits. This is the educational project below. A class ID is a finite choice; it does not understand arbitrary sentences.
- **Existing audio plus a mask or context → repair/continuation.** This requires a model explicitly trained for the input contract. Preserve timing, inspect transitions and make the provenance of generated regions visible. The tiny class-conditioned model below does not support this by itself.
- **Acoustic features → waveform.** A conditional diffusion vocoder can render mel features produced elsewhere. Feature extraction parameters and vocoder expectations must match exactly; this is not the same input contract as text-to-audio.

For an application, put generation in a job with a cancel path, store the requested duration and model settings, and return a playable file rather than an unlabeled tensor. Keep the original source recording and generated result separate. Apply a deliberate playback-level policy and never begin auditioning unverified output at high volume. Quality gates should include clipping, silence, audible artifacts, content adherence and consent/provenance. The first project chooses finite sound classes and very short mono waveforms so each part of the contract is inspectable.

What you will make. A small model that generates half-second thumps, pings and hisses when given a class label. The sounds are synthesized by our own code, not scraped recordings. This is a deliberately bounded audio task: it will not produce intelligible speech, human-quality music, arbitrary text-to-audio, or realistic room acoustics.

Why this follows the image adapter. You have already seen corrupt-data training and iterative generation. Here the whole neural network is trained, so there is no pretrained visual or audio knowledge hiding most of the work. The short waveform and simple classes make the experiment manageable and the failure modes understandable.

Audio project step 1 listen to the training problem

Return to the companion-script folder before this project. If you opened a new terminal, set `DIFFUSION_EXAMPLES` to the actual absolute folder path again. If the image virtual environment is active, run `deactivate` first. Use a separate audio environment so its libraries cannot silently replace the image project's dependencies.

The commands below install the CUDA wheel for the intended GPU. For a CPU-only smoke test, replace the PyTorch installation line with `python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cpu`, and add `--device cpu` to training, sampling and evaluation commands. CPU training is a mechanics check, not a practical substitute for the planned GPU run.

```
cd "$DIFFUSION_EXAMPLES"
python3.11 -m venv .venv-audio
source .venv-audio/bin/activate
python -m pip install --upgrade pip
python -m pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cu126
python -m pip install numpy==1.26.4 scipy==1.15.3
python -m pip check
python make_tiny_sfx.py --out data/tiny_sfx --per-class 800
```

You now have 2,400 mono WAV files: 1,920 training, 240 validation and 240 test. Open several files at low playback volume. You should hear clearly different families, with variations in frequency, onset, decay and amplitude. If you cannot identify the intended classes reliably, fix the data before asking a model to learn them.

Waveform sample rate channels and duration

A **waveform** is a sequence of amplitude samples over time. A **sample rate** of 16,000 Hz means 16,000 amplitude measurements per second per channel; it is not the number of training examples. **Mono** has one channel; stereo has two. For this project the tensor shape for one example is `[1, 8192]`: one channel, 8,192 values. Its duration is $8192 / 16000 = 0.512$ seconds.

The file stores signed 16-bit PCM. Loading converts those integers to floating-point numbers approximately in `[-1, 1]`. “16-bit audio” describes file quantization; “FP16 training” describes arithmetic precision inside the model. They are different decisions.

Changing only a WAV header from 44.1 kHz to 16 kHz changes playback speed and pitch. Proper **resampling** creates a new sequence with filtering. Frequencies above half the destination sample rate need suitable attenuation to avoid aliasing. Our generated data are created directly at 16 kHz, so the first project does not need resampling at all.

Longer audio is a real size increase. A 10-second stereo 44.1 kHz waveform contains 882,000 values, about 108 times as many as this mono half-second example. It is not a harmless setting change. In an attention model the token-cost increase can be even less forgiving.

One manifest record is:

```
path,label,split,group,source,rights,sha256
ping/ping_0000.wav,ping,train,ping_0000,synthetic_book_generator_seed_123,generated_locally_no_recorded
_people,<actual checksum>
```

The actual generator writes the checksum. Preserve source and rights fields when replacing synthetic sounds. A real recording session, speaker, musician or original long file may define a

group. All overlapping windows from the same original belong to the same split. Otherwise the validation set may be a slightly shifted version of training audio.

Audio project step 2 the smallest useful full training loop

The complete implementation is `tiny_audio_ddpm.py`. First test one update, saving and loading:

```
python tiny_audio_ddpm.py train --data data/tiny_sfx \
  --out runs/audio-smoke --steps 1 --batch 1 \
  --log-every 1 --save-every 1
python tiny_audio_ddpm.py sample --checkpoint runs/audio-smoke/step-1.pt \
  --class-name ping --n 1 --out samples/audio-smoke
```

The resulting sample should not be expected to sound good. Its purpose is to prove that model input and output shapes match, the checkpoint is readable and the sampler produces a valid WAV file. A CPU-only mechanics check can use `--device cpu`; sampling still invokes the network hundreds of times and may be slow.

Then run a measured pilot:

```
python tiny_audio_ddpm.py train --data data/tiny_sfx \
  --out runs/audio-pilot --steps 200 --batch 4 \
  --log-every 50 --save-every 200
```

Inspect `metrics.jsonl`, which records training MSE, fixed-noise validation MSE, elapsed seconds, and CUDA peak allocated/reserved GiB when using CUDA. The code prints the actual parameter count. It uses float32 rather than hiding numerical details behind a mixed-precision wrapper; the model is deliberately small. On a 24 GB card this is a conservative candidate workload, not a GPU-memory measurement made by the author.

If it runs cleanly, continue the same run to a 10,000-update experiment:

```
python tiny_audio_ddpm.py train --data data/tiny_sfx \
  --out runs/audio-pilot --resume runs/audio-pilot/step-200.pt \
  --steps 10000 --batch 4 --log-every 100 --save-every 1000
```

The checkpoint includes network, exponential-moving-average network, optimizer, diffusion-step count, random-generator state and a material run configuration. Every WAV is checked against its manifest SHA-256 before use. Resume requires the original output directory, its latest saved checkpoint, unchanged data, source code, batch, accumulation, learning rate, seed, training device, numerical precision, schedule and library versions. Increasing the total step target or changing log/checkpoint frequency is allowed, as in this pilot-to-long example. To change a material training setting, start a deliberately new experiment. CPU inference remains possible for a CUDA-trained checkpoint; this strict same-device rule applies to continuation of optimization. The initial `run.json` is retained rather than overwritten. Each continuation appends an invocation record with its actual optimizer learning rate; metrics include an invocation ID, so replayed steps after a crash are distinguishable. A fresh run refuses a nonempty output directory. The supplied file format is intended for checkpoints you created or trust; it uses `torch.load(..., weights_only=True)` and should not be changed to unrestricted loading to silence errors from an unknown file.

Read the audio training loop in ordinary language

This time we use DDPM noise prediction, rather than Sana's flow target. For clean waveform x_0 , Gaussian noise ϵ and a timestep with retained signal power $\alpha_{\text{bar}}[t]$, the corruption is:

```
xt = sqrt(alpha_bar[t]) * x0 + sqrt(1 - alpha_bar[t]) * epsilon
training target = epsilon
```

The square-root factors control signal and noise power. The network gets x_t , time and class, then estimates the actual added noise. This differs from predicting $\epsilon - z_0$ along Sana's straight path. Both produce a generator, but their loss targets and reverse updates must not be mixed [D01].

For every optimizer update, the script performs these operations:

1. Select a small batch of clean waveforms and class IDs.
2. Draw a timestep independently for each waveform.
3. Draw fresh Gaussian noise of exactly the same shape as the waveform.
4. Mix clean audio with noise according to the selected noise level.
5. Drop the class label for roughly 10% of examples, substituting a learned null label.
6. Ask the network to predict the added noise.
7. Compare prediction and noise with mean squared error, backpropagate, clip gradients, and update the weights.
8. Update a slowly moving copy of the weights for evaluation and sampling.

The class label is conditioning; the target is still noise. The training pairs are not “hiss goes in, ping comes out.” At high noise levels all three classes look noise-like. The condition helps identify which clean distribution the reverse process should favor.

The model is a small 1D U-Net: convolutions operate along time instead of across image width and height. It reduces the temporal length by factors of four three times, processes the compressed features, then upsamples while using skip connections. Time and class embeddings modify intermediate features. This is an original educational implementation, not the released DiffWave architecture. DiffWave is a primary example of applying diffusion directly to waveforms [D18].

Why these audio hyperparameters are small

- **8,192 samples:** enough for a short transient but not for phrases, melodies or long room reverberation.
- **Batch 4:** a conservative initial batch. Reduce to 1 if you need to diagnose memory; use `--accum 4` if you want roughly the same effective batch afterward.
- **Learning rate 0.0002:** a starting value for this newly initialized small network. It is not a recommended rate for an arbitrary pretrained audio transformer.
- **256 diffusion levels:** the discretization used by both training and the simple ancestral sampler. It does not mean 256 optimizer steps.

- **Cosine noise schedule:** brings the terminal signal close to pure noise while distributing intermediate signal levels. The cumulative signal-power curve is converted into per-step noise amounts. This follows the schedule idea in Improved DDPM rather than silently reusing a thousand-step linear schedule at 256 steps [D17].
- **0.1 condition dropout:** teaches a null-conditioned case, enabling ordinary classifier-free guidance later.
- **EMA factor 0.999:** the evaluation copy retains most of its previous value and incorporates 0.1% of the current weights each update. Early in a short run it lags considerably; do not judge a one-step EMA sample as a trained model.
- **10,000 updates:** a bounded experiment. At batch 4, it is 40,000 waveform presentations, about 20.8 average passes over the 1,920 training examples. Training uses random sampling with replacement, so these are average exposures, not exact complete epochs.

The code does not include every production optimization. This is useful: you can inspect the loss, forward noising and reverse update without disentangling several frameworks. If the experiment is too slow, measure where time goes before adding mixed precision, compilation, larger batches or a different sampler.

Audio project step 3 generate and compare every class

```
for CLASS in thump ping hiss; do
  python tiny_audio_ddpm.py sample \
    --checkpoint runs/audio-pilot/step-10000.pt \
    --class-name "$CLASS" --n 16 --seed 17 --cfg 1 \
    --out samples/audio-10000
done
python audit_audio.py --data data/tiny_sfx --samples samples/audio-10000
python tiny_audio_ddpm.py evaluate --data data/tiny_sfx \
  --checkpoint runs/audio-pilot/step-10000.pt --split val
```

The sampler starts with a new noise waveform, evaluates the denoiser at decreasing noise levels and applies the matching DDPM posterior update. Its final clean estimate is bounded to the waveform range. The file writer preserves generated relative amplitude instead of peak-normalizing every clip. That makes loudness and clipping defects visible in the diagnostics.

At first, use `--cfg 1`, which gives the conditional prediction without extrapolation under our formula. Then make a separate comparison with `--cfg 2`. Increasing guidance must not be the only way to make classes recognizable; it can amplify harshness and reduce variation. The two prediction calls used by guidance add inference work. They do not retrain the model.

Listen quietly first. A bad generator can output unexpectedly loud noise, tones or clicks. Never judge sound quality only by looking at an attractive spectrogram.

Audio inference on GPU and CPU

Training and inference use the same class order, waveform length and sample rate. The checkpoint records these values and the cosine schedule identity; the loader rejects mismatches. Inference restores the EMA weights, creates the matching schedule and changes only the device. Both CPU and CUDA paths in this small implementation use float32. There is no tokenizer, text encoder, latent codec or vocoder to substitute: conditioning is the stored class vocabulary, and the generated tensor already is a waveform.

To generate one sample on CPU:

```
python tiny_audio_ddpm.py sample \
  --checkpoint runs/audio-pilot/step-10000.pt \
  --class-name ping --n 1 --seed 17 --cfg 1 --device cpu \
  --out samples/audio-cpu
```

CPU generation is implemented but still requires 256 network evaluations per unguided sample; guidance above 1 uses two predictions per step. It is appropriate for a functionality check or occasional output, not a claimed real-time engine. GPU and CPU outputs need not be numerically identical. The writer clips to the valid range and quantizes to mono PCM16 at 16 kHz; it does not resample or peak-normalize each sample. Thus the saved half-second duration and relative amplitude retain the training contract. For evaluation on either device, the real-input loader uses the same PCM-to-float conversion as training.

The source-code hash and data-manifest hash are stored in training checkpoints. Retain the code and manifest that produced a checkpoint. Increasing duration, changing class order or swapping the noise schedule without retraining is not a portable inference optimization.

An audio evaluation sheet that answers the task

For each checkpoint, generate the same class/seed combinations. Rename or shuffle files before listening so that knowing the checkpoint does not bias your score. Evaluate:

- **Class correctness:** can a listener distinguish thump, ping and hiss without seeing the filename?
- **Signal integrity:** unexpected DC offset, near-full-scale samples, clicks at boundaries, persistent noise, abrupt truncation or unwanted silence.
- **Diversity:** varied valid frequencies, decays and onsets within each class. Listen to several outputs in sequence.
- **Distribution match:** compare ranges of duration, amplitude, spectral centroid and envelope with held-out real/synthetic examples. A model that always emits the loudest ping has not captured the distribution.
- **Memorization:** compare suspicious samples with nearest training sounds. The included audit uses log-magnitude short-time Fourier features to screen for spectral neighbors. It is a review aid, not proof that an output is or is not a copy.

MSE on newly noised held-out waveforms is a useful learning diagnostic, especially with fixed noise and timesteps for checkpoint comparisons. It is not a listening score. Waveform MSE between two independently generated sounds is usually not a meaningful quality metric: a tiny phase shift can produce a large pointwise error while sounding similar.

A **spectrogram** shows energy at frequencies over time, computed from short overlapping waveform windows. A **mel spectrogram** pools those frequencies on a perceptually motivated scale. Both are representations, not automatically playable audio. A spectrogram-generating model needs a reconstruction process or vocoder, and its settings must match that decoder. A **vocoder** turns acoustic features into a waveform; it is another model or algorithm to train, acquire and evaluate. Our project uses waveforms directly so there is no hidden vocoder.

For larger text-to-audio work, embedding-based audio quality/distribution measures and audio-text similarity may be informative when computed with a documented protocol and enough examples. They can be fooled by background cues or domain mismatch. Do not use a single audio-text similarity score as a substitute for checking clicks, intelligibility, rhythmic continuity or whether a prompted event actually occurs.

Only after choosing the checkpoint using validation should you run:

```
python tiny_audio_ddpm.py evaluate --data data/tiny_sfx \
  --checkpoint runs/audio-pilot/step-10000.pt --split test
```

If an earlier checkpoint is better, change the checkpoint path. The objective is a defensible selection, not necessarily using the largest step number.

Audio project step 4 replace synthesized clips carefully

A safe next experiment is a narrow collection of your own non-vocal sounds: keyboard clicks, hand percussion, doors, small tools, or foley props. A directory of random songs is a much harder task with different rights and temporal structure.

Start by preparing files offline to the exact format the educational loader accepts: mono PCM16 WAV, 16 kHz, exactly 8,192 samples. The loader intentionally rejects other formats so that a sample-rate mistake cannot silently train the wrong task. Create a new manifest and matching class set; editing CLASSES changes the model's label vocabulary and makes old checkpoints incompatible.

For real recordings:

1. Keep the original audio unchanged, with provenance and permission records.
2. Decode to floating point and inspect channel layout. Do not blindly average stereo: oppositely phased channels can cancel.
3. Resample with an actual resampler. Document the original and final sample rates.
4. Choose event-centered windows for transients. Random cropping may turn a useful sound into silence.
5. Pad only when necessary, track true length, and decide whether padding should contribute to the loss. The tiny fixed-length example assumes every position is part of the example; a serious variable-length trainer needs masks or other explicit length handling.
6. Remove corrupt and heavily clipped examples. Do not normalize silent clips by dividing by a near-zero peak.
7. Use sensible common gain or a documented loudness policy. Per-clip peak normalization can erase meaningful loudness variation and raise background noise.

8. Split by original recording/session before extracting windows, and listen to the final processed files.

A half-second mono 16 kHz model has a limited frequency range and context. If your target includes cymbal shimmer, stereo space or several seconds of decay, acknowledge that this representation is insufficient. Choose the correct duration, bandwidth and channel layout before a long run, then remeasure memory.

For text-conditioned datasets, a sidecar or manifest caption should describe the actual audible event, texture, environment and sequence, not tags copied from an unrelated image. “Three close dry wooden taps, with a short pause before the last” conveys more useful structure than “excellent quality sound.” Captions cannot make timing controllable if the architecture never receives timing information or the examples contradict it.

Voice music and dataset provenance

Recorded voices may identify people even when filenames do not. Obtain explicit informed permission for the intended voice-model training and generation, including scope of use and retention, and be able to separate or delete that person's source records when required. Owning a recording device or finding a public video does not establish permission to clone the speaker. Do not impersonate someone or imply that they said generated words.

For music and effects, the recording, composition, performance and dataset license can involve different rights. Record the source URL or original filename, creator, exact license/version, attribution requirements, acquisition date, transformations and intended use. A repository's software license does not automatically license its training audio. Model licenses are a separate layer. Inspect the actual current terms rather than treating “Creative Commons,” “open,” or “research” as interchangeable legal categories.

Synthetic examples in this project avoid recorded voices and borrowed artwork. That makes the first experiment easier to audit; it does not establish blanket rights for future datasets or guarantee that every pretrained model's outputs are risk-free.

After the tiny project pretrained audio adaptation

A realistic text-to-audio system commonly uses a pretrained waveform autoencoder, a text encoder and a latent denoiser. This parallels the image pipeline, but the compression rate, frequency fidelity, stereo handling and long-range temporal structure are different. Before adapting the denoiser, test autoencoder reconstruction on your target sounds: metallic transients and unusual textures can reveal representation failures.

Stable Audio Open 1.0 is an established official option: its card describes text-conditioned stereo generation at 44.1 kHz up to 47 seconds, using an autoencoder, a T5-based text representation and a latent DiT. The official `stable-audio-tools` repository contains training machinery, but a model-card inference snippet is not by itself a verified fine-tuning configuration [D19] [D21].

Stable Audio Open Small is a different model: its card specifies up to 11 seconds, stereo 44.1 kHz, and a short distilled inference recipe. Do not assume that ordinary noise-prediction fine-tuning preserves the behavior of a distilled model or that any LoRA trainer supports it. Check the exact model, objective, sampler and trainer together [D20].

Stable Audio 3 now has an explicit official adaptation path. Its repository distinguishes base checkpoints for training from post-trained inference checkpoints, and the inspected `train_lora.py` accepts only base-model names. It supports raw audio with same-stem `.txt` captions or pre-encoded latents, and exposes duration, batch, rank, learning rate and local CSV logging. This is a genuine current route to investigate, rather than an invented Stable Audio Open LoRA flag [D22] [D24].

A cautious upgrade procedure is:

1. Use a separate checkout/environment. Read the official model license and download-access terms. Stable Audio 3 Small SFX also names Gemma terms for its text-conditioning component; a single umbrella “open weights” label is insufficient [D26].
2. Record the repository commit, model revision and resolved `uv.lock`. The inspected project pins PyTorch 2.7.1; its own dependency environment is substantially different from our small script [D25].
3. Inspect `python scripts/train_lora.py --help` at that commit and select one of its actual base-model names. Do not substitute an inference model just because its filename is similar.
4. Start with a few seconds of audio, batch 1 and a small adapter, and keep demonstrations bounded. In the inspected script, the demonstration callback uses the model configuration's sample size, which can differ from the training crop duration. A short training crop alone is not proof that demo generation will be small.
5. Run a very short optimization-and-reload test before budgeting a full run. Measure preprocessing, first backward pass, optimizer state allocation, checkpointing and demonstration peaks separately.
6. Compare generated audio against the unchanged base under identical prompts, durations, seeds and sampling settings. Test content outside your adaptation domain to detect degradation.

Do not promote uncertain VRAM figures into a guarantee. Stable Audio 3's README labels its table as inference measurements on an H200. Its LoRA guide contains lower approximate memory entries but also a “medium on about 16 GB” example without a uniform duration/batch measurement protocol. These do not establish the peak VRAM of your 24 GB training run. The guide confirms support, not a benchmark reproduced in this book [D22] [D23]. The tiny waveform project above is supplied in full precisely so your first audio training experiment does not depend on an unverified pretrained-model stack.

Revisit the theory noise prediction score prediction and flow matching

Now that you have trained both an adapter and a full small network, the terminology should connect to code rather than remain a list of names.

Noise prediction asks for the particular Gaussian noise used to corrupt an example.

Clean-data prediction asks for the original clean tensor. A **velocity parameterization**, often named `v_prediction`, predicts a schedule-dependent combination such as $v = \alpha * \epsilon - \sigma * x_0$, with $\alpha^2 + \sigma^2 = 1$ for this convention. These targets can be converted under

the corresponding schedule, but their raw outputs are not interchangeable. The training target, model output and sampler interpretation must agree.

A **score** is the gradient of the log density of noisy data with respect to the noisy tensor. It points toward higher density locally; it is not a human quality score. For Gaussian corruption with noise standard deviation σ , the optimal noise predictor and score have a known scaling relationship, $\text{score} \approx -\text{predicted_noise} / \sigma$. Score-based formulations describe reverse stochastic or deterministic dynamics using this quantity [D02].

Flow matching trains a time-dependent vector field along a chosen probability path. A simple illustrative path from noise z at time 0 to data x at time 1 is $x_t = (1-t)z + t x$, with conditional target velocity $x-z$. The network predicts a movement vector and an ODE solver follows it. Real systems may use different paths, weightings, time parameterizations and coupling schemes. Flow matching includes a wider family of paths than this one example; it is related to diffusion but is not merely renaming an epsilon-prediction loss [D03].

Be especially careful with the word “velocity”: diffusion $v_{\text{prediction}}$ and a rectified-flow vector field can use different definitions. Copying a loss from one into the other while retaining the original sampler is a silent conceptual bug.

There are at least three different schedules in a training project:

- **Learning-rate schedule:** how optimizer step sizes change during training.
- **Noise/path schedule and timestep sampling:** how corrupted examples and targets are constructed and which regions of the path receive training weight.
- **Sampling schedule:** which noise/time levels and solver updates are used to generate an example after training.

Fewer sampling steps can speed inference without reducing the number of learned parameters. It can also reduce quality or require distillation. Changing the training noise schedule for a pretrained checkpoint is not a free speed optimization; it changes the problem the network sees. Start with the base model's objective and schedule, and change one component only when you can explain and test the consequences.

End of project checklist

You are ready to move on when you can answer these questions without memorizing a model list:

1. What is the shape and numeric range of a clean example?
2. What condition is given to the network, and what exact target does its loss compare against?
3. Which components are frozen and which are optimized?
4. What does one optimizer step mean, and how many examples does it represent?
5. Which noise schedule and sampling rule interpret the network's outputs?
6. What can the validation experiment establish, and what remains untested?
7. What are the source rights and consent for the data?
8. What is the measured peak memory of the actual configuration, including evaluation?

9. Can you load a saved artifact in a fresh process and reproduce the evaluation setup?

10. Does the model do something useful beyond reproducing training examples?

If you can answer those, the next model family is a controlled engineering change rather than a new collection of unexplained flags.

Read a public training process after building your own

Open the original DDPM repository beside the paper and find the functions that construct noisy data, define the prediction target and update a sample. Its TensorFlow 1.15/TPU environment is historical; use it to understand the published process rather than replacing this book's environment with its old requirements [D28]. Then compare the DiffWave repository's waveform and mel-conditioning paths with our finite class labels. Trace where its conditioning enters, what its preprocessing saves and how inference obtains a waveform [D29]. The exercise is to identify the data-model-loss-sampler contract in real code, not to assume that a paper's released configuration will fit your card.

29 Scale a working experiment toward one billion parameters

The next experiment should answer one question

After the tiny transformer works, the tempting next step is to make it a thousand times larger. A better next step asks a specific question: is the small model underpowered for a task whose data and evaluation are already trustworthy? If you cannot show that the task needs more capacity, increasing size mainly increases the cost of uncertainty.

A useful scaling experiment keeps data processing, split definitions, and evaluation fixed while changing one capacity dimension. Compare a slightly wider model, a deeper model, or a longer training budget. Measure quality, memory, and throughput. The result should tell you whether another increase is justified.

Before scaling, replace the toy corpus with sufficient lawful material for the intended task. Repeating 133,000 synthetic bytes millions of times does not create the linguistic coverage of a real pretraining corpus. You may obtain a model that memorizes the generator extremely well and generalizes poorly.

Three different meanings of full training

Training from scratch initializes all learned parameters without a pretrained checkpoint and optimizes them on your data. Full-parameter fine-tuning starts from pretrained weights and allows all selected model parameters to change. Continued pretraining starts from pretrained weights and continues a broad self-supervised objective, often on domain text. These can all update every parameter, but their starting capability, data needs, and goals differ.

If you fine-tune a 600-million-parameter pretrained model, you inherit the results of its original large-scale pretraining. A 600-million-parameter model trained from scratch on your small dataset does not inherit that capability. Comparing their parameter counts without their data and training histories is misleading.

Head-only training and adapter training update a subset of parameters. They are valuable techniques, but should not be described as full-parameter training. A model can have a billion total parameters and only a few million trainable parameters. Record both counts.

A realistic ladder of model sizes

The educational byte transformer begins below one million parameters. Models in the low millions are suitable for learning mechanics, constrained synthetic tasks, and some genuinely narrow applications. Tens of millions allow more capacity while keeping many experiments relatively manageable. Hundreds of millions become a meaningful systems and data project. Near one billion, full training on 24 GB requires careful numeric, activation, optimizer, and sequence-length choices even when persistent state appears to fit.

Do not read those categories as performance rankings. A compact classifier with the right features may outperform a much larger generative model on a fixed decision task. A pretrained

350-million-parameter encoder or decoder may outperform a randomly initialized billion-parameter network because its representation already contains useful structure.

The companion architecture is intentionally easy to count. With vocabulary V , context T , width d , and L blocks, its tied-embedding parameter count is approximately $Vd + Td + L(12d^2 + 9d) + 2d$. The exact implementation determines the bias terms. The program prints the authoritative count from distinct parameter tensors.

For illustration, width 768, twelve blocks, a 256-byte vocabulary, and context 512 give roughly 86 million parameters. Width 1,792 with twenty-four blocks and the same small vocabulary gives roughly 927 million. These are shape calculations, not recommended pretrained architectures or measured 24 GB training configurations. A real subword vocabulary, different feed-forward ratio, untied output head, or grouped-query attention changes the count.

Do not scale all costs at once

Increasing width changes many matrices quadratically. Increasing depth adds repeated blocks. Increasing vocabulary enlarges embedding and output layers. Increasing context raises activation and attention costs. Increasing batch raises concurrent activation cost. If you change all of them together, you lose the ability to diagnose the bottleneck.

For a larger full-training feasibility test, start with microbatch one and a short but meaningful sequence. Run complete updates, including optimizer-state allocation. Then measure longer sequences and evaluation. Only after the full lifecycle works should you consider accumulation to reach a chosen effective batch.

A short sequence can make a model fit while making the task impossible. If the input evidence and target depend on a 2,000-token document, a 128-token feasibility demonstration does not establish that the useful task fits. Resource constraints and task requirements must be reconciled honestly.

One billion is a project boundary rather than a magic number

Under the simple FP32 AdamW inventory, one billion trainable parameters require about 16 GB of persistent state before activations. A 24 GB card may have enough remaining capacity for some short-context configurations, but not all. Mixed precision, optimizer implementation, attention kernels, output-logit materialization, and checkpointing determine the real peak.

An efficient optimizer or CPU offload can move the boundary. It also changes numerical behavior, throughput, CPU RAM demand, and failure modes. A memory-saving flag should be accompanied by an explanation of which state it reduces and a new measured smoke test. If the implementation keeps an FP32 master copy, include it in the inventory.

The practical full-training projects in this book stay at or below one billion parameters. The supplied from-scratch model is much smaller so the reader can understand and validate it. The pretrained full-fine-tuning project uses a small published checkpoint to demonstrate a useful adaptation path without pretending to reproduce its original pretraining.

What the three billion ceiling actually means

Three billion parameters is within the model-selection discussion, including adapter methods and specialized architectures. It is not a blanket promise of ordinary full-parameter training in 24 GB. In the simple FP32 AdamW case, the persistent training state alone is about 48 GB. Lower-precision and offloaded methods must be assessed individually.

A three-billion-parameter four-bit base may fit for adapter training because the frozen base has compressed storage and only a small subset needs optimizer state. That does not make three-billion-parameter full training equivalent. Likewise, a mixture-of-experts name mentioning three billion active parameters may have far more resident parameters.

Pretraining from scratch adds a compute and data constraint even if memory can be solved. A model that sees too little varied text may never develop useful general language ability. A long run on inadequate data can be less useful than a much shorter adaptation of a well-chosen pretrained model.

Build a measured budget before committing

Measure useful tokens per second over a representative interval after startup. Include a separate end-to-end figure accounting for evaluation and checkpointing. Estimate total token presentations from data size, epochs or sampling plan, and masking. Divide by measured throughput, then add contingency for restarts and preprocessing.

Write down the quality decision that will justify continuing. For example: after the first fixed budget, the larger model must beat the smaller model on held-out task accuracy without exceeding a latency limit. If it does not, inspect errors before spending more. Scaling can help with capacity; it cannot repair a mislabeled task or a missing input field.

A sensible project budget also includes human work: reviewing data, defining labels, inspecting failures, and maintaining the resulting artifact. GPU time is only one part of the cost. For many personal projects, a day spent improving data and evaluation is more valuable than a day spent training a larger network.

A scaling experiment worksheet

Record the hypothesis, baseline configuration, proposed change, exact parameter count, trainable parameter count, numeric policy, estimated persistent state, measured peak state, sequence or image/audio shape, effective batch, data version, token or sample budget, measured throughput, estimated duration, stopping criteria, and evaluation result.

Keep the first larger run deliberately bounded to feasibility and early learning. Do not interpret a successfully completed ten-step run as evidence of eventual quality. It answers a systems question. Quality needs a separate training budget and held-out evaluation.

Exercises

Using the stated parameter formula, calculate how doubling width differs from doubling depth. Explain why width can make memory grow much faster than expected.

Write two project plans for the same domain: one from-scratch tiny model and one pretrained full fine-tune. List which capability each starts with, what data it needs, and what result would justify its cost.

A three-billion-parameter checkpoint loads successfully in four-bit inference. List the additional facts you need before claiming full-parameter training will fit: trainable dtypes, master copies, optimizer state, gradients, activations, context, batch, working buffers, and supported training kernels.

30 Understand what a public training process really demonstrates

You can learn a great deal from a repository without being able to reproduce its biggest run. The useful habit is to inspect the relationship among code, configuration, data and reported result.

Audit a public fine tuning script before running it

Hugging Face’s inspected `smollm/text/finetuning/train.py` is a useful example. It exposes a model, dataset, sequence length, accumulation, LoRA settings and optional NF4 loading. But the inspected file still passes `max_seq_length` to `SFTConfig`, while reviewed TRL 1.14.1 uses `max_length`. It also defaults to Hub upload and Weights & Biases reporting. A reputable repository can contain a script written for an older API and defaults inappropriate for private data. [L52] [L06]

The lesson is not to avoid public examples. Before running one, answer:

1. Which commit and dependency versions does this script expect?
2. What data does it load, and are you allowed to use it?
3. Is the loss all-token, completion-only or assistant-only?
4. What is actually trainable, and how is precision configured?
5. Does it upload data, weights or logs by default?
6. Which evaluation produced the reported result?
7. Can you run one batch, save, reload and generate before the long job?

Our examples use local fixtures, explicit masks and disabled reporting/uploading. The implementation was written independently for this book rather than presented as an unmodified official benchmark recipe.

Compare scratch training with adapting a finished model

Training from scratch initializes weights randomly and must learn basic token relationships, language patterns and task-relevant structure. **Full fine-tuning** starts from pretrained weights but updates all of them. “Full” answers which weights change; “from scratch” answers where the weights came from. A full fine-tune is not equivalent to recreating the original model.

The inspected SmolLM3 model card reports 11 trillion pretraining tokens and 384 H100 GPUs. Its public pretraining README describes a 2.36-million-token global batch and 24 days on those 384 GPUs, along with training/configuration resources. This is a transparent published process worth studying. It is not a one-consumer-GPU recipe. [L49] [L53]

For your hardware, scratch training a tiny transformer is an excellent learning project. Scaling an educational run to tens or hundreds of millions of parameters is possible only with a matching data, memory and time budget. A sub-billion parameter ceiling does not make

general-purpose pretraining cheap. Your tiny model can demonstrate real learning without matching the broad competence of a publicly pretrained checkpoint.

A common dense-transformer planning approximation is $\text{training_FLOPs} \approx 6 N D$, for N parameters and D training tokens. It omits important architecture, attention and recomputation details, so use it for order-of-magnitude reasoning. The compute-optimal training literature studies data/model allocation under particular budgets; a remembered “tokens per parameter” ratio is not a universal data requirement or a guarantee of useful quality. [L54]

For an explicitly hypothetical example, $N=1e9$ and $D=20e9$ gives $1.2e20$ FLOPs. At an assumed sustained 50 TFLOP/s, division gives about 27.8 days of uninterrupted arithmetic. That throughput is not a measured RTX result and the token count is not a recommendation; data processing, checkpointing, inefficiency and experiments add time. The calculation simply shows why a model that fits in memory can still be expensive to pretrain.

For a real estimate, measure processed training tokens per second on your actual architecture and sequence length, then $\text{use time} \approx \text{planned_tokens} / \text{measured_tokens_per_second}$, adding evaluation and saving overhead. Separate processed input tokens from supervised target tokens when reporting SFT throughput. An hour with short answers and heavily masked prompts is not directly comparable to an hour of all-token pretraining.

A stop rule that matches the experiment

A smoke test stops after establishing finite loss, valid masks, a complete update, saving and reloading. A useful supervised experiment stops when validation no longer improves under the chosen protocol or the pre-agreed token/compute budget is spent. A final held-out test estimates how well the selected configuration transfers.

Keep a simple result record: model and revision, data hashes and split method, trainable count, optimizer/precision assumptions, effective tokens per update, peak memory, throughput, validation trend, held-out task scores and a few representative failures. Report a measured improvement only after making that comparison. Honest small results teach more than an impressive model name attached to an unverified recipe.

31 Diagnose failures before making the model larger

Locate the first wrong boundary

A training system has boundaries: raw input, decoded input, processed representation, batch, model output, loss, gradient, update, saved artifact, and deployed prediction. Find the earliest boundary where reality differs from expectation. This is usually more efficient than changing architecture in response to every poor result.

If the model's answer is wrong, first print the exact input it received. Was the relevant text truncated? Were image channels swapped? Was audio resampled correctly? Was a label ID mapped to the wrong name? Did an instruction template add a different generation prefix? The model may be responding correctly to an unintended input.

Keep a small diagnostic dataset whose examples you know by hand. It should include one ordinary case, one boundary case, one missing-input case, and one case requiring abstention. Run it after changes to preprocessing, dependency versions, export, or serving.

When loss does not fall

Try to overfit a tiny clean subset. If the model cannot reduce loss there, inspect whether the intended parameters require gradients, whether gradients are finite and nonzero, whether the optimizer receives those parameters, and whether `optimizer.step` is actually called. Check that zeroing gradients happens at the correct point.

Verify target alignment. A language model's logits at one position must be compared with the intended next token, not accidentally the same input token or a target shifted twice. A segmentation mask must match the transformed image. A classifier's label IDs must match the output dimension and class mapping.

Inspect the loss scale and supervised-token count. A mean over an empty mask is not a meaningful objective. A loss accidentally summed over very different lengths can produce unstable updates. A model output passed through softmax before a loss expecting raw logits can change the intended computation.

Only after those checks should you tune learning rate, initialization, or capacity. A larger network with the same alignment bug is a more expensive broken pipeline.

When training improves and validation worsens

This pattern suggests overfitting, but investigate its form. Are there duplicates in training? Is the dataset too small or unrepresentative? Are labels inconsistent? Has the model memorized an identifier? Did preprocessing differ between train and validation? Is the validation set from a meaningfully different distribution?

Possible responses include better data coverage, fewer updates, stronger justified regularization, a smaller trainable subset, or a better representation. The right response

depends on the failure. If the deployment distribution changed, early stopping alone may not solve the mismatch.

Look at individual errors and slices. A model may improve on common short examples while regressing on long or rare cases. An average curve cannot tell you which data to add. An error taxonomy can.

When the GPU runs out of memory

Identify the phase. Failure during model loading concerns weights and initialization copies. Failure during forward often concerns activations, logits, or attention buffers. Failure during backward adds saved activations and gradients. Failure at the first optimizer step may reveal newly allocated optimizer states. Failure during evaluation can come from a different batch or generation cache. Failure during merge/export can come from temporary copies.

Reduce the relevant cost. Lower microbatch size or sequence length for activation pressure, enable a supported efficient attention path, use activation checkpointing, or select a smaller model. For persistent state, consider a supported optimizer policy, adapter training, or explicit offload. Do not delete arbitrary files or kill unrelated processes as a reflex; understand their ownership and purpose.

Check for retained computation graphs. Appending loss tensors to a Python list instead of detached scalar values can retain memory across steps. Keeping generated logits for every batch can do the same. A steadily rising memory curve may indicate a leak rather than a model that is intrinsically too large.

When the GPU is mostly idle

The bottleneck may be data loading, tokenization, image decoding, augmentation, disk access, CPU preprocessing, or synchronization. Measure time by stage. Preprocess deterministic work once when appropriate. Use a reasonable number of data-loader workers and avoid overwhelming RAM or storage. Cache only what remains valid under the chosen augmentation and model settings.

A tiny model can also be launch-bound: the GPU spends a large fraction of time starting small operations. Larger batches may improve utilization if memory allows. Compilation can help some workloads but adds startup cost and compatibility complexity. Establish a correct baseline before optimizing it.

Do not compare throughput numbers without their units and shapes. Images per second at 256 pixels and at 1024 pixels describe different work. Tokens per second can count padded tokens, input tokens, supervised tokens, or generated tokens. Name the measure.

When outputs look memorized or repetitive

Check data duplication, excessive epochs, narrow prompt coverage, generation settings, and stopping tokens. A style adapter trained on a tiny collection can reproduce phrases rather than learn general style. An image adapter can bind a subject to its background. Audio training can overfit a repeated segment or generate a narrow set of textures.

Evaluate on prompts designed to separate the intended concept from accidental correlations. Ask for the subject in a different scene. Ask for the same writing style on a new topic. Ask for a different sound duration or context within the model's supported interface. If the model fails, add representative examples or reduce the learned change; do not simply select a more flattering seed.

When the exported model differs

Compare the training checkpoint, unmerged adapter, merged full-precision model, and quantized export on the same fixed inputs. Confirm identical tokenizer, chat template, special tokens, context settings, preprocessing, and generation parameters. Check that evaluation mode is active.

Quantization can change outputs and may have format-specific operator support. A successful conversion is not a quality test. An export that loads in a CPU runtime may still use a different tokenizer or fail on long inputs. Validate the actual deployment path.

A debugging notebook that compounds in value

For each failure, record the symptom, smallest reproducer, expected behavior, observed behavior, diagnosis, change made, and verification. Save the exact error message without secrets. Distinguish a confirmed cause from a plausible hypothesis.

A good failure note prevents rediscovering the same problem. It also protects you from attributing every improvement to the most interesting change. Sometimes the decisive fix was correcting a label mapping rather than adopting a new architecture.

Exercises

Introduce an intentional target shift in a copy of the tiny transformer and observe the effect. Restore the original before continuing. Explain why a pipeline can run successfully while optimizing the wrong task.

Create a fresh-process inference test for one saved model and compare it with in-process output. Then change one preprocessing setting and document the mismatch.

Write an out-of-memory decision tree organized by the phase of failure. For each branch, name one measurement you would gather before changing a setting.

32 From small model to small device product

A 24 GB training GPU and a phone have different constraints. A model that trains comfortably on your workstation may exceed a phone's RAM, run slowly on its supported kernels, or heat the device under continuous use. In embedded practice, **TinyML** often means microcontroller-class deployment with very small memory and power budgets; the examples above are small models, not a claim that every artifact fits every microcontroller.

Measure the entire path on the target device

Write an acceptance budget before optimizing:

- model file size and installation/download size
- peak runtime memory, including inputs, activations, buffers, tokenizer, and retrieval index
- cold-start latency, steady-state median latency, and tail latency
- sustained throughput, thermal behavior, and battery cost
- supported operators and data types on the chosen CPU, GPU, or NPU backend
- input preprocessing, output decoding, and any privacy or offline requirements

Parameter count is only one term. The current project's 1024-dimensional FP32 embedding uses 4,096 bytes before index overhead: one million such document vectors require about 4.096 decimal GB. A tiny encoder plus a huge local index can be a poor phone design. Keep only the required corpus, compress and re-evaluate, or choose a permitted server-side architecture when appropriate.

Export quantize and verify instead of assuming

A typical deployment path is train → choose checkpoint → export with fixed preprocessing contract → optimize for the target backend → validate numerical parity → evaluate the optimized model → benchmark on actual hardware. ExecuTorch is PyTorch's current on-device ecosystem for mobile and edge workloads; follow its version-specific exporter and backend documentation rather than copying an old mobile tutorial blindly. This chapter does not claim a tested phone export. [\[M19\]](#)

Quantization represents some values with fewer bits. Post-training quantization uses a representative calibration set to choose ranges; quantization-aware training simulates the effect during training. Neither guarantees preserved accuracy. Calibration for numeric quantization ranges is different from probability calibration. Test thin segmentation structures, rare classes, long texts, and low-confidence cases after conversion, not just average agreement on easy examples.

For simple models, reducing image resolution, feature channels, sequence length, or index size may matter more than an exotic optimization. Each reduction changes what information remains available. Distillation can train a smaller student to imitate a stronger teacher, but the

student still needs representative examples and independent evaluation. A successful export proves format compatibility, not product quality.

A compact failure diagnosis checklist

Symptom	First checks	Next controlled experiment
Training loss never falls	inspect targets, dtypes, non-finite inputs, trainable parameters, gradient flow	overfit a handful of examples
Training improves; validation worsens	leakage-free split, label consistency, sample size, distribution shift	smaller model, stronger justified regularization, earlier checkpoint
All predictions are majority class/background	label counts, loss reduction, missing-label handling, output decoding	foreground/class-aware objective and honest per-class metrics
Good IoU, bad outline	resolution, coordinate mapping, thin features, boundary tolerance	boundary-sensitive evaluation and higher-resolution patches
Good retrieval loss, worse search	false negatives, duplicate positives, too-easy evaluation, index/model mismatch	inspect hard examples and compare the frozen encoder
GPU out of memory	longest input, activation sizes, batch, evaluation buffering, other processes	one representative step at a smaller batch/resolution/length
Export succeeds, output changes	normalization, channel order, padding, unsupported operator fallback, quantization	compare intermediate or final tensors on a saved test bundle
Offline scores good, field behavior poor	subgroup/session/time shift, input acquisition, unseen cases, actual latency	gather rights-cleared field examples and rebuild the evaluation contract

The end-to-end skill is now the same across these projects: define what information is available, represent the target faithfully, start with a cheap baseline, fit only on authorized training data, choose decisions on validation, test once on meaningful held-out cases, and verify the exact deployed pipeline. The architecture follows from that contract.

33 Turn a checkpoint into a usable application

The model is one component of the system

A useful application validates input, applies the exact preprocessing used in training, runs the model, decodes output, checks constraints, and decides what happens next. A trained checkpoint alone does not perform all of those responsibilities. Deployment is the process of preserving that contract outside the training script.

For a sensor predictor, the contract includes feature names, order, units, missing-value policy, calibration, and alert threshold. For a language model, it includes tokenizer, chat template, context limit, stopping behavior, and generation settings. For image or audio generation, it includes the base model, adapter, conditioning, scheduler, preprocessing, and output conversion.

Keep deterministic checks outside the model when they can be explicit. A tool argument can be checked against a schema. A date can be parsed. A path can be restricted to an allowed directory. A financial or destructive action can require approval. Model confidence is not permission to bypass such boundaries.

Package the smallest complete inference artifact

For each project, create an export directory containing the required weights or adapter, architecture configuration, preprocessing or tokenizer files, class mapping or output schema, dependency specification, and a short model card. Include a small known input and expected output or tolerance-based test.

For an adapter, identify the exact base model and revision. If redistribution of the base is not allowed or not desired, document how the authorized user obtains it. Do not imply that an adapter-only folder is self-contained. A merged model may simplify inference but increases size and must respect the relevant licenses.

A classifier saved with joblib or pickle-like serialization should be loaded only from trusted sources because such formats can execute code. Tensor-focused formats can reduce some risks, but preprocessing code and remote model code still need review. Avoid enabling `trust_remote_code` casually; it permits repository-provided Python to run in your environment.

CPU and GPU are two deployment targets

CPU inference is useful for small classifiers, many embedding workloads, low-throughput services, and portability. GPU inference is useful when model size, latency, batch throughput, or generation workload justify it. The same artifact may support both, but the runtime and dtype choices can differ.

Do not assume a GPU-oriented four-bit training configuration has a CPU inference path. A quantization library may require specific hardware kernels. A model can sometimes be exported to a different CPU-supported format, but that is a new conversion with its own quality

and compatibility tests. Loading full FP32 weights on CPU may be simpler for a small model, at the cost of RAM and latency.

CPU memory is still finite. One billion FP32 weights alone require about 4 GB before runtime overhead and caches. A larger context, multiple concurrent requests, and tokenizer or preprocessing buffers add more. GPU serving similarly needs headroom for caches and concurrency rather than just the model's weights.

Benchmark the actual request pattern. A batch of 100 classification records measures throughput; one interactive request measures latency. Report startup time separately from steady-state inference. Include preprocessing and decoding in an end-to-end measurement, and distinguish it from model-only time.

Test export equivalence

Choose a fixed small suite of representative inputs. Run the original selected checkpoint and the exported artifact with matching preprocessing and deterministic settings where appropriate. Compare class probabilities or logits within a justified tolerance, generated outputs under fixed decoding, or task-level metrics when small numerical differences are expected.

For a merged adapter, first compare the unmerged base-plus-adapter path with the merged full-precision path. Then evaluate any quantized conversion. This staged comparison identifies where a regression appears. Converting and quantizing simultaneously can make diagnosis harder.

A model that loads successfully has passed a structural test. It still needs semantic tests. A wrong class-name order can preserve every tensor shape while reversing the application's decisions. A changed chat template can make a language model behave differently without any loading error.

Keep the application bounded

Specify maximum input sizes and output sizes. Reject malformed data clearly. Decide how to handle empty text, corrupt images, unsupported audio rates, missing fields, and requests outside the trained domain. A silent fallback to an arbitrary default can be more dangerous than an explicit failure.

For tool use, separate proposal from execution. Let the model propose a structured action. Let deterministic code validate the tool name, argument types, allowed values, permission, and current state. Use a dry-run or sandbox in testing. High-impact actions need the appropriate human or policy approval independent of the model's score.

For retrieval-assisted generation, preserve source identifiers and permission checks. A model should not retrieve a document merely because an embedding search finds it relevant if the current user lacks access. Fine-tuning does not provide per-document access control.

Monitor without quietly changing the task

Track production input distributions, error reports, abstention rates, latency, and relevant outcome metrics. A drift in data does not automatically justify retraining. First determine whether the issue is a changed target, changed population, preprocessing bug, or missing coverage.

Collect feedback with clear provenance. User corrections may be useful labels, but they can also be mistaken, malicious, or inconsistent. Do not automatically train on every interaction. Review, deduplicate, protect privacy, and version the resulting dataset.

Keep a rollback path. A new model should be evaluated against the old one before replacement. Deploy gradually when the application warrants it, and retain enough information to reproduce a reported error. The ability to restore a known-good artifact is part of reliability.

Write a model card that makes honest claims

State the model's purpose, architecture and base revision, training method, trainable parameter count, data sources and rights, split methodology, evaluation results, hardware and software used, tested inference targets, known limitations, and unsupported uses. Distinguish measured results from expected behavior.

A personal model card can be short, but it should still answer what changed from the base and how you know. If GPU training was not run during preparation of a recipe, say so. If a result is based only on synthetic data, say so. If a model was tested on one language or one recording condition, do not imply broad coverage.

Exercises

Package one model so a fresh Python process can load it and predict a provided sample without access to the training notebook's variables. List every required file.

Measure ten cold starts and a sequence of warm predictions on the intended CPU or GPU. Report the conditions and distinguish model-only from end-to-end timing.

For a proposed tool action, write the deterministic validator before asking a model to produce the action. Explain which conditions can be enforced by code and which require human judgment.

34 Design your first independent model

Choose a job small enough to verify

Your first independent project should have a clear input, a limited output, examples you can lawfully use, and a result you can inspect. 'Build my own general assistant' contains too many unresolved tasks. 'Route these five kinds of repair request and abstain when unsure' is a project. 'Answer questions from this manual with a cited passage' is a project. 'Generate short original machine sound effects in three categories' is a project.

Choose a task where you can obtain correct targets and where mistakes are recoverable. Avoid making a first experiment responsible for medical, legal, financial, hiring, or other high-impact decisions. Learning the machinery and establishing real-world validity are separate obligations.

Write the prediction contract

Describe the exact input fields or file format, what is available at prediction time, the output format, the target policy, and the downstream action. State which outputs are valid and what happens when required information is missing. List examples outside the intended domain.

Then identify the application pattern. A fixed decision suggests classification or a typed-decision model. A numeric quantity suggests regression. A ranked list suggests retrieval or ranking. A pixel mask suggests segmentation. Flexible text suggests conditional generation. Images or audio synthesized from conditions suggest an appropriate generative model. A deterministic transformation may need ordinary code rather than training.

Do not force a problem into the category whose tutorial you most recently read. The input/output contract is the organizing principle.

Build a baseline and a small evaluation set

Create a baseline with the simplest plausible method. Save its outputs on a carefully selected initial evaluation set. Include common cases, costly errors, missing information, and domain boundaries. Define a primary metric, guardrails, and a stopping rule.

The evaluation set should be large enough to expose important failures and small enough that you can review it. It can expand as the project matures. Preserve a final holdout that does not guide repeated choices. Record sources and grouping units from the beginning so later expansion does not create leakage.

If the baseline already meets the requirement, deployment engineering may be more valuable than training. A model project succeeds when the user's problem is solved, even if the winning method is simpler than expected.

Select the smallest useful intervention

Try a better representation or prompt before a major weight update when appropriate. Try retrieval for changing factual knowledge. Try a small head or adapter when the pretrained representation is useful but the task interface differs. Try full fine-tuning when you have a reason to update the whole model and can afford the controlled experiment. Train from scratch when the objective, architecture, data ownership, or narrow task makes that justified.

For every choice, write why the data should teach the architecture. If the model predicts next tokens, show how the desired behavior appears in target sequences. If it scores options, show positive choices and plausible alternatives. If it predicts masks, show aligned pixel labels. If it denoises audio, show consistent waveform preprocessing and conditioning.

Use four gates

The data gate requires a valid schema, clear targets, lawful provenance, representative coverage, and protected split boundaries. The correctness gate requires a tiny run, finite gradients, a parameter update, and save/load equivalence. The feasibility gate requires measured peak memory, throughput, storage, and recovery under representative shapes. The quality gate requires a held-out improvement over the baseline without unacceptable regressions.

Do not skip a gate because the previous one was exciting. A clean dataset does not prove a trainer is correct. A working trainer does not prove the useful workload fits. A feasible run does not prove the result is good.

Plan the next experiment from errors

After the first trained comparison, classify the failures. Missing evidence suggests input or retrieval changes. Ambiguous labels suggest annotation work. Format errors suggest target consistency or constrained decoding. Long-input failures suggest truncation or context design. Confident wrong decisions suggest calibration and abstention work. Systematic visual or audio artifacts suggest representation, preprocessing, or training-coverage problems.

Choose one change that addresses the dominant error category. Write the predicted effect and how you will measure it. Preserve the prior artifact so you can compare or roll back. This creates a series of understandable experiments rather than a pile of increasingly complicated configurations.

A final handoff to yourself

At the end, you should have an inference-ready artifact, a readable model card, a data card, a tested CPU or GPU deployment path, an evaluation report, an environment record, and a list of known limitations. You should also have one next decision: deploy within the stated boundary, improve a particular weakness, or stop because the approach is not worth its cost.

The central skill is no longer knowing one command. It is connecting a desired behavior to evidence, representation, architecture, optimization, and evaluation. That skill transfers from three learned numbers to hundreds of millions of parameters, and it lets you read larger-model research without confusing an impressive architecture with a feasible personal project.

35 Understand larger models using the parts you already know

You can understand a model that needs a data center without owning that data center. The earlier projects taught the pieces: a classifier turned features into scores; an embedding model learned representations; the tiny transformer mixed information across tokens; the image and audio projects turned signals into tensors; fine-tuning changed a useful behavior. Large architectures reorganize and scale these pieces.

This final part is an architectural study, after the practical projects. Its purpose is to help you read a model card, follow a configuration file, predict what data a design needs, and recognize the system cost hidden behind a model name. The hands-on training path remains centered on models below one billion parameters, with a broader adaptation and comparison range up to three billion unique parameters; any rounded boundary model must be identified explicitly. The hundreds-of-billions-parameter examples here are not one-card training recipes. No large checkpoint was downloaded or executed for this book.

The case studies were checked against primary model cards, configuration files and implementation code on 2 October 2026. They include the requested MiMo-V2.6 and GLM-5.3-Flash, plus selected reference designs that make their choices easier to understand. “Flash,” “active,” “hybrid” and “open” each need a precise explanation. None is a hardware budget by itself.

Six questions that make a model card readable

For any unfamiliar model, ask these questions in order.

1. **What enters and leaves?** Text tokens, image patches, audio features or several kinds of input? Does the model output text, images, audio or action tokens? An audio input encoder does not imply a speech output decoder.
2. **What mixes positions?** Full attention, local attention, recurrent state, convolution or a layer-by-layer mixture?
3. **What transforms each position?** One feed-forward network or a routed selection of experts?
4. **What persists during inference?** Weights, a growing key/value cache, a fixed-size recurrent state, a short convolution buffer or several of these?
5. **How was it taught?** Prediction targets, demonstrations, preferences, teacher outputs, interactive rewards, and the data formats that make those losses meaningful?
6. **What did the release actually include?** Weights, tokenizer, processor, inference code, training code, data descriptions, data itself and an applicable license are separate items.

The architecture is a program with learned numbers. The checkpoint is a particular set of those numbers. A training recipe explains how they were obtained. The serving system implements that program efficiently for requests. When one item is public, do not assume that the others are complete.

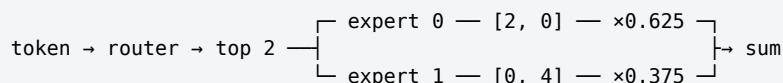
The feed forward network becomes a mixture of experts

In the tiny transformer, each token representation passes through the same feed-forward network, or FFN. This is a **dense** model in the usual language-model sense: the same major parameter blocks participate for every token. An embedding lookup still selects rows, so “dense” is a convention about the network, not a claim that every scalar is read on every operation.

A **mixture of experts**, or MoE, replaces some FFNs with several FFNs and a learned **router**. An expert is a neural subnetwork. It is not a separate chatbot, a human specialist or necessarily an interpretable topic module. The router reads a token's current representation, computes expert scores and chooses a small number of experts for that token. The selected outputs are combined. The next token can choose differently. DeepSeek's released inference implementation makes this separation visible in its Gate, Expert and MoE classes. [A01]

An original four-expert example is small enough to do by hand. Suppose the scores for one token are 0.50, 0.30, 0.15 and 0.05. With **top-2 routing**, choose experts 0 and 1. Normalize their selected scores by their sum, 0.80. Their mixing weights become 0.625 and 0.375. If their output vectors are [2, 0] and [0, 4], the result is:

$$0.625 \times [2, 0] + 0.375 \times [0, 4] = [1.25, 1.50]$$



experts 2 and 3 exist, but do not process this token

This example starts with supplied scores; it does not implement the production router's learned logits, sigmoid, balancing corrections or grouped selection. Those choices differ across models. In MiMo's inspected code, a correction bias affects expert selection, while the combination weights come from the uncorrected selected scores. “Take softmax and select a few experts” is therefore an incomplete description of that checkpoint. [A02]

For a batch with B sequences, T tokens per sequence and representation width D, the input shape is [B, T, D]. Flattening the first two axes gives [B×T, D]. With E experts, routing scores have shape [B×T, E]. Selecting k experts gives expert indices and mixing weights of shape [B×T, k]. A dispatch operation gathers the appropriate token rows for each expert; a combine operation returns their weighted outputs to the original token positions.

Resident parameters are the weights that must be stored somewhere for the complete model to be available. **Active parameters** describe the subset used in a particular token's forward computation under the publisher's counting convention. Active count is useful for rough compute reasoning. Resident count determines the basic weight-storage requirement. Neither includes all cache, activation, workspace or communication costs.

Imagine eight experts with ten million parameters each and a ten-million-parameter shared backbone. Top-2 routing gives 90 million resident parameters and roughly 30 million active parameters for a token. Across many different tokens, every expert can be used. Keeping only the two most recently selected experts on a GPU requires fetching missing experts later; it does not delete the other sixty million expert parameters.

A **shared expert** is used for every token in addition to the routed experts. Some designs include one; MiMo-V2.6's routed FFNs do not. A shared expert can learn broadly useful transformations while routed experts supply extra conditional capacity. This is a design choice to test, not proof that the router has discovered neat human categories.

What changes during MoE training

The input data can still be ordinary next-token language-modeling sequences. It does not require a human-written “send this word to expert 7” label. The training problem gains an allocation problem: some experts may receive too many tokens while others receive too few.

Load balancing encourages usable distribution. A capacity limit can bound how many token assignments an expert processes, but dropping or rerouting overflow changes the learning calculation. A release's actual method matters.

Log the expert assignment histogram, the fraction of overflowed assignments if applicable, and loss by data source. The global average loss can hide a collapsed router or an expert that rarely trains. A token-level top-k choice is discrete; a training implementation must specify how gradients, balancing signals and routing decisions interact. Reading an inference forward pass alone does not establish a correct training algorithm.

On a GPU, a small dense matrix operation can be easier to execute efficiently than many tiny expert operations. On a CPU, dispatch and irregular memory access also cost time. MoE improves a particular balance of capacity and computation; it does not guarantee that every small batch runs faster than a smaller dense model.

Attention choices solve different problems

Attention compares a query with available keys and uses the resulting weights to combine values. Queries and keys answer “which positions should influence this position?” Values supply the information to combine. The earlier transformer already did this with a causal mask so future tokens were unavailable.

Full attention and the growing cache

During a prompt's **prefill**, many token positions are processed together. In ordinary full causal attention, position t may attend to all previous positions and itself. The total number of allowed position pairs grows approximately with T squared. During **decode**, the model typically generates one new token per active sequence. A **KV cache** stores prior key and value vectors so that the entire prefix does not have to be projected again at every step. The new query still has to interact with the relevant cached history.

A cache is working memory for a particular request. It is not newly learned model knowledge. A longer cache consumes memory even when the weights never change. An application's stored conversation history, a search index and the neural KV cache are three different kinds of memory.

Multi-head attention has several query/key/value heads. **Multi-query attention**, or MQA, shares one key/value head across query heads. **Grouped-query attention**, or GQA, shares key/value heads within groups of query heads. Reducing KV heads reduces cached vectors and memory traffic; query heads can remain numerous. This sharing is part of the learned

architecture, so arbitrarily changing a pretrained checkpoint's head counts is not a lossless runtime setting. [A03]

For a teaching configuration with 12 layers, 4 KV heads, 64 key coordinates, 64 value coordinates, 4,096 tokens, batch size 1 and two bytes per cached number:

```
cache bytes = batch × tokens × layers × KV heads
              × (key width + value width) × bytes per number

              = 1 × 4096 × 12 × 4 × (64 + 64) × 2
              = 50,331,648 bytes = 48 MiB
```

Using eight KV heads would double this payload to 96 MiB. This is cache arithmetic, not total inference memory. Allocator overhead, temporary tensors and the weights still exist.

Sliding windows and selected positions

Sliding-window attention, or SWA, keeps each query's direct attention local to a recent window. With a fixed window W , local attention's position-pair count grows approximately with $T \times W$ instead of T squared. An efficient rolling cache can discard older keys and values for that local layer. Several local layers expand the effective receptive field, and occasional global layers allow direct long-range communication.

Sparse attention instead selects a subset of historical positions or blocks according to a pattern or learned selection mechanism. A learned indexer can look for useful distant positions that a fixed recent window would omit. It also introduces its own scores, state, computation and possible selection errors. Sparse attention is sparsity over token connections; MoE is sparsity over parameter blocks. A model can use both.

Sparse selection does not automatically create a constant-size cache. If a future query might select any past position, the system needs a way to retain, compress or recover the corresponding information. Masking most entries of a dense attention matrix is mathematically sparse, but can still allocate and compute like a dense implementation. Hardware-efficient sparsity requires a suitable kernel and cache layout.

Latent attention and exact kernels

Multi-head latent attention, or MLA, stores a compressed representation from which the needed key/value information can be obtained. Think of retaining a shorter learned code instead of a separate full key and value for every head. DeepSeek-V3's official inference code exposes both a direct path and an optimized absorbed path; the latter maintains latent and positional caches. The choice of algebra and kernel is part of whether the memory advantage appears in practice. The `kv_lora_rank` configuration name here describes a low-rank attention projection, not a separately trained user LoRA adapter. [A01] [A04]

FlashAttention addresses a different level of the system. It computes exact attention with a more memory-efficient execution order, tiling the work to reduce transfers between memory levels. It does not, merely by being enabled, replace full attention with a local window or a recurrent state, and it does not make the full-attention pair count linear. Numerical rounding can differ between kernels. [A05]

This distinction is useful when a model called “Flash” is discussed. A product suffix, FlashAttention, an FP8 checkpoint and a sparse attention pattern are four separate facts to verify.

Recurrent state and short convolution

A **recurrent** layer updates a state as tokens arrive. In a simplified notation, the new state is a function of the previous state and current token. The output reads from the new state. Its state can remain fixed in size as the sequence grows. That is a strong inference-memory advantage, but the state must summarize history rather than preserve arbitrary past token vectors separately.

A **state-space model**, or SSM, gives that state update a structured parameterization. Modern selective SSMs make parts of the update depend on the input. Mamba-2 is a useful reference because the original paper relates structured state-space computations to attention and explains efficient sequence algorithms. Its implementation contains convolution state, SSM state and separate sequence-processing and single-step paths. It is not simply a conventional transformer with a smaller KV cache. [\[A06\]](#) [\[A07\]](#)

Linear attention refers to sequence-mixing formulations whose main sequence cost avoids the usual quadratic attention matrix. Some can be expressed as recurrent matrix updates. A schematic associative state is a matrix S that accumulates associations between key and value vectors; a query reads from S . A delta-rule-style update can correct an existing association rather than only add another one. Gating controls what is retained or forgotten. Kimi Delta Attention, or KDA, develops a finer-grained gated update in this family. [\[A08\]](#)

This is a conceptual analogy, not the exact KDA equation. Do not substitute a simple sum of key/value outer products for a production KDA implementation and call the result equivalent. Normalization, gates, delta corrections, chunking and numerical precision all matter.

A **short convolution** mixes a token with a fixed number of nearby positions using learned filters. A causal width-four convolution needs only a short recent buffer for incremental operation. A **depthwise** convolution applies a filter independently within each channel rather than combining every channel with every other one at that step. Other projections can still mix channels. Short convolution offers useful local processing; by itself it does not create unbounded long-range recall.

The earlier Liquid LFM comparison belongs here conceptually: its inspected small models mix gated short-convolution blocks with attention blocks. The brand name “Liquid” does not mean that every checkpoint uses the same continuous-time differential-equation architecture. Read the specific LFM version's block list. That small-model chapter supplies the practical comparison; there is no second large-model Liquid training recipe here.

Why hybrid models keep some attention

A **hybrid** model alternates or combines different sequence mixers. It may let recurrent or local layers do most of the repeated work while a smaller number of attention layers retain direct access to the past.

```
Example A  local → local → local → global → repeat
Example B  recurrent → recurrent → recurrent → sparse attention → repeat
Example C  short convolution → attention → short convolution → ...
```

These diagrams are patterns, not interchangeable checkpoint definitions. They explain the design tension: compressed history can be cheap; direct retrieval from history can be precise; both consume compute and memory in different ways. Evaluate a hybrid on exact copying, retrieval among distractors, long-range dependencies and the real downstream task. A short next-token loss alone cannot establish useful long-context behavior.

Multimodality connects encoders to a sequence model

A **modality** is a kind of input or output, such as text, images or audio. An **encoder** converts a signal into learned features. A **projector** maps those features to the representation width expected by the language backbone. A vision-language model can then process text embeddings and projected visual features in a common sequence.

```
text → tokenizer → token embeddings ————┐
image/video → patch encoder → projector —┴→ sequence backbone → output head
audio → audio encoder → projector ————┘
```

This original diagram describes a common input-fusion pattern. It does not imply that every family has all three branches or that one output head generates every modality. For image or audio generation, inspect the decoder, output representation and training objective separately.

A **patch** is a small image region or a group of nearby signal elements treated as one input unit. A 224×224 image divided into 16×16 patches gives 14×14 = 196 spatial patches before extra tokens or merging. A **patch merger** combines neighboring features to reduce the sequence length passed downstream. That reduces cost but also changes the detail the language backbone receives.

Video adds a time axis. Frame sampling, patch size, temporal grouping and image resizing all affect the token budget. Audio may be represented as spectrogram features, learned continuous embeddings or discrete codec tokens. A **residual vector quantizer**, or RVQ, uses several codebooks in sequence to represent a signal with discrete indices. Those indices are learned audio representations, not ordinary word-token IDs.

The correct data follows the desired cross-modal relationship. Image captioning needs image/text pairs. Document reading needs legible visual text and corresponding answers. Speech recognition needs audio with transcripts. Sound-event description needs audio with grounded descriptions. A pile of unpaired images and a pile of unrelated sentences does not, by itself, specify which sentence describes which image. Check alignment, time stamps, resolution, consent and splits by source recording or document.

Residual connections can have several streams

In the tiny transformer, a residual connection added a sublayer's output back to its input.

Hyper-connections generalize the route through a deep network by maintaining several representation streams and learning how to read, write and mix them. **Manifold-constrained hyper-connections**, or mHC, constrain the stream-mixing operation to help control signal growth. The original work uses approximately doubly stochastic mixing: nonnegative entries with rows and columns normalized toward sums of one. [\[A09\]](#)

For intuition, two scalar streams $[2, 10]$ mixed by rows $[0.75, 0.25]$ and $[0.25, 0.75]$ become $[4, 8]$. Their mean is preserved. That example explains constrained mixing; it does not prove the stability or usefulness of an entire trained network. The sublayers add learned information too, and finite normalization iterations introduce numerical approximation.

With four streams, a representation changes from $[B, T, D]$ to $[B, T, 4, D]$. The sublayer can still operate on a collapsed $[B, T, D]$ representation, then write back into the streams. More streams increase activation and memory-traffic costs even if the main expert width stays the same. GLM-5.3-Flash provides a concrete inspected implementation below.

36 Read MiMo GLM and other architecture families

The following are case studies in design choices. They are not a leaderboard or a claim that these are the newest or best models for every use. DeepSeek-V3 and Mamba-2 are deliberately included as clear reference designs. Newer names do not invalidate the mechanisms they illustrate.

Match the requested names to exact releases

Name in conversation	Verified release or reference	Why it is in this book
MiMo 2.6	XiaomiMiMo/MiMo-V2.6-Flash-RL and MiMo-V2.6-Pro-RL; newer Flash-MOPD and Pro-MOPD also listed officially	Local/global attention, MoE, multimodal inputs, post-training and speculative decoding
GLM 5.3 Flash	zai-org/GLM-5.3-Flash, with a separate BF16 release listed by Z.ai	Linear plus sparse attention, MoE and mHC
Qwen larger hybrid	Qwen/Qwen3.5-397B-A17B	Gated DeltaNet plus attention, routed plus shared experts
DeepSeek reference	deepseek-ai/DeepSeek-V3	MLA cache compression and an inspectable MoE inference implementation
Mamba reference	Mamba-2 in state-spaces/mamba	Structured recurrent state and hardware-aware scan algorithms
Liquid	The exact small LFM checkpoints in the earlier practical chapter	A small hybrid makes the sequence-mixer comparison accessible

The official MiMo Flash card says 309B total, the technical report rounds to 310B, and the Hub's tensor summary shows approximately 311B. These are different displayed counts; this book does not claim to have independently reconciled every auxiliary tensor or counting convention. It uses “about 310B” for rough storage calculations and preserves the exact source labels in the evidence ledger. [\[A10\]](#) [\[A11\]](#)

The GLM card reports 320B total and 18B active; the MiMo cards report approximately 15B active for Flash and 42B for Pro. These are publisher specifications, not measured resident memory. [\[A10\]](#) [\[A12\]](#) [\[A13\]](#)

MiMo V2 6 as a local and global attention design

Start with the Flash checkpoint's configuration. The text backbone has width 4,096 and 48 layers. Its pattern contains 39 sliding-window layers and 9 global layers. The local window is 128 tokens. Global attention has 64 query heads and 4 KV heads; local attention has 64 query heads and 8 KV heads. Key/query width is 192 per head and value width is 128. There are 256 routed experts, with eight selected per token; the first FFN is dense. The exact context configuration is 1,048,576 positions. [\[A14\]](#)

These are compact architectural facts, not advice to use a million-token sequence in the earlier training script. You can now reason about them. Frequent narrow-window layers keep many attention operations local. Global layers provide direct access across the sequence. GQA reduces stored key/value heads. MoE increases stored FFN capacity without executing every expert for each token. The representation width need not equal query-head count multiplied by head width: projection matrices can expand the attention space and project back.

Pro scales the same general pattern: the official card lists 70 layers, including 60 local and 10 global, width 6,144, 384 routed experts and top-8 activation. Its global KV head count is eight. The different global-layer count and KV head count substantially change cache cost even before accounting for its roughly 1.02 trillion parameters. [A12]

Follow the input and output interfaces

The Flash release includes a vision configuration, audio configuration and special image, video and audio token identifiers. Its source defines vision patch embedding, a patch merger, an audio encoder and audio tokenization components alongside the text backbone. Those branches explain why a text-only tokenizer call is not a complete multimodal input pipeline. Inspect the processor's resizing, frame sampling and audio sample-rate assumptions with the checkpoint. [A02] [A14]

The model card describes a vision encoder and two-stage audio input path, plus a separate five-layer speculative drafter. This book treats those as input-understanding and inference components rather than assuming native image or waveform output. [A10]

A configuration field is not the entire implementation

Three inspection details prevent expensive mistakes.

First, the released `MiMoV2MoEGate.forward` rejects training mode for its `noaux_tc` routing path. This is direct evidence that the inspected Hugging Face model implementation is not a ready-made full-training recipe. A separate research training framework can exist without making this particular loader trainable. [A02]

Second, the main configuration contains an MTP-related count, while the separate `dflash/config.json` describes a five-layer drafter and an eight-position block. Read the separate component rather than assuming one legacy field completely specifies speculative decoding. [A15]

Third, the quantization configuration combines `quant_method=fp8` with `store_dtype=mxfp4`, and lists higher-precision exceptions. Storage format, runtime arithmetic and accumulator precision need not match. A filename or Hub dtype badge cannot tell you exactly how many bytes your chosen engine allocates. [A14]

Training data and post training implications

The MiMo technical report describes text-only pretraining followed by multimodal training. Reported totals are 48T tokens for Flash, split into 26T text-stage and 22T multimodal-stage tokens, and 30T for Pro, split into 27T and 3T. It then describes agent-focused mid-training, context extension, quantization-aware training and mixed-task reinforcement learning. The reported RL setup freezes the router and uses interactive trajectories with verifiers or graders. These are descriptions of that research run, not a public reconstruction of every original training example. [A11]

The practical lesson is about target information. A tool-use trajectory must include the relevant observations and consequences of an action; a code task needs a trustworthy test environment; a visual task needs the actual image. More compute cannot repair a reward that encourages the wrong behavior. The earlier tiny tool project should retain its safe local environment and held-out evaluator rather than reproduce broad autonomous web or security workloads.

The newer MOPD checkpoint is a post-training variant. Its card describes teacher-guided student continuations from several kinds of prefixes and a mitigation for repeated tool calls. It does not describe a new resident-parameter size that would make the model a consumer-card checkpoint. A distillation stage can preserve a student's size; “distilled” does not always mean “small.” [A16]

GLM 5 3 Flash as a recurrent and sparse attention design

GLM-5.3-Flash is a distinct newly trained base architecture, not merely a cheaper quantization of GLM-5.3. Its official card identifies a multimodal model, a hybrid of sparse and linear attention, mHC, and a 30T-token multimodal pretraining corpus. The broader GLM-5 technical report linked from the card is a family reference; it is not sufficient evidence for every Flash-specific implementation detail. [A13] [A17]

The inspected configuration is more informative for this chapter. It specifies 45 text layers: 34 linear-attention layers and 11 sparse-attention layers. The first three FFNs are dense; the remaining 42 are sparse FFNs. They use 288 routed experts, eight selected experts and one shared expert. The hidden width is 4,096. The mHC stream multiplier is four, and the maximum configured position is 1,048,576. Its vision encoder is a separate 24-layer component. [A18]

The model's Transformers implementation names the linear mixer Kimi-style KDA. It has short convolution, a forget gate, a recurrent-state update for one-token decoding and a chunked path for sequences. The sparse-attention path has a separate indexer and compressed latent KV cache. This makes “hybrid” concrete: some layers carry a bounded recurrent state, while other layers retain growing historical information for selected attention. [A19]

Two shape changes to trace

Take a teaching input with $B=1$, $T=16$ and $D=4,096$. The initial text embeddings have shape $[1, 16, 4096]$. Four mHC streams give $[1, 16, 4, 4096]$. Before a sublayer, learned mixing collapses the stream axis; afterward, a write operation and constrained residual mixing update all streams. These are not four copies of the entire 320B-parameter model.

For the KDA layers, the config supplies 64 heads with width 128. A recurrent matrix with a 128×128 state per head has $64 \times 128 \times 128$ entries per layer and sequence. At four bytes per entry, that is 4 MiB. Across 34 such layers, the illustrative matrix-state payload is 136 MiB for batch size one. This calculation excludes convolution buffers, sparse-layer caches, indexer state, temporary tensors and all weights. The inspected implementation explicitly stores the recurrent state in FP32. [\[A18\]](#) [\[A19\]](#)

Sparse attention still needs a system

The indexer configuration includes a selection budget of 2,048 and grouping pools of four. That is an index-selection setting, not a 2,048-token context limit. The implementation handles incomplete tail pools and converts selections into a mask for the supported reference attention paths. Its reference path also expands compressed KV representations for the attention computation. Efficient production kernels can avoid costs that remain in such a readable implementation. [\[A18\]](#) [\[A19\]](#)

The source supports cross-layer index reuse, but the inspected checkpoint's `indexer_types` are all `full`. Do not copy a feature advertised for another GLM version and assume it is enabled here. Likewise, the Transformers documentation explicitly says that its implementation omits the MTP layer, despite an MTP-related configuration field. Model support must be checked component by component. [\[A18\]](#) [\[A20\]](#)

For training data, take the card's multimodal-corpus description as the available evidence. Do not invent exact image/text ratios, unpublished filtering thresholds or a full Flash pretraining script. A small architectural experiment can use controlled data to study recurrent versus sparse recall without claiming to reproduce the released model.

Qwen and DeepSeek make the design axes clearer

Qwen3.5-397B-A17B is an informative comparison because its official card specifies 15 repetitions of three Gated DeltaNet layers followed by one gated-attention layer. Its MoE has 512 routed experts, ten selected experts and one shared expert. The model has visual input capability and roughly 397B total versus 17B active parameters. Its documented native context is 262,144, with a separately described extension to approximately one million; the hosted product's settings should not be assumed to equal the local checkpoint's defaults. [\[A21\]](#) [\[A22\]](#)

Compare the mechanism, not the scores. MiMo allocates many layers to short-window direct attention. Qwen allocates many to a recurrent linear mixer. GLM combines KDA with indexed sparse attention and adds multi-stream residual routing. All still need enough stored parameters and working memory for their chosen implementation. A similar active count does not imply identical compute, cache traffic, vision cost or latency.

DeepSeek-V3 is the older reference for the separation between MoE and latent attention: one changes which FFN parameters execute; the other changes how attention information is represented and cached. The original report describes 671B total and 37B active parameters, 14.8T pretraining tokens and an MTP objective. Its official inference code provides unusually direct places to inspect those mechanisms. It is included as a documented architectural reference, not as a claim about the latest DeepSeek release. [\[A01\]](#) [\[A04\]](#)

Mamba-2 supplies the complementary reference: a language model can use structured state updates as its primary sequence mixer. An architecture family is a collection of implementation and modeling choices, not a single point on a “bigger is better” scale. The earlier small Liquid comparison lets you examine one hybrid on practical hardware before studying these much larger combinations. [\[A06\]](#) [\[A07\]](#)

37 Connect architecture to data memory and deployment

A model's useful behavior depends on architecture, data, objective and execution together. The following design map turns the case studies into decisions for your own projects.

Choose data that tests the architectural claim

Design question	Appropriate training or test evidence	What a misleading test misses
Does local mixing suffice?	Examples with controlled dependency distances, plus representative real sequences	Short examples never exercise distant information
Does recurrent state retain useful details?	Recall among distractors, multiple associations and changing sequence lengths	One repeated pattern can fit in a simple summary
Does sparse attention select the right history?	Evidence placed at varying positions, including competing plausible evidence	A single obvious retrieval key is too easy
Does MoE capacity help?	Balanced domain mixtures, held-out sources, routing and per-domain metrics	Global loss can hide unused experts and domain regressions
Does a vision bridge learn grounding?	Aligned images and questions, with image swaps and text-only controls	Answers may come from textual shortcuts
Does audio input carry the answer?	Aligned recordings and transcripts or descriptions, split by speaker/source as appropriate	Recording duplicates and caption leakage inflate results
Does distillation transfer useful behavior?	Teacher-generated examples with checks, then independent test tasks	Student similarity to the teacher is not correctness
Does interactive RL improve actions?	Safe environments, observable outcomes, robust verifiers and failure penalties	An exploitable reward can improve while task success worsens

A tiny model can demonstrate any of these mechanisms. It will not establish the scaling behavior of a model trained on trillions of tokens. Hold one question fixed at a time: changing the tokenizer, data mixture, model size, training duration and attention type simultaneously prevents a clean architectural conclusion.

Pretraining and later objectives

Pretraining usually supplies broad prediction practice. For a causal language model, the target is the next token. A multimodal sequence model can condition the same text-prediction loss on image or audio features; additional modality objectives depend on the design.

Continued pretraining or **mid-training** adds another broad or targeted phase before final task adaptation. These names describe stages, not universal algorithms.

Supervised fine-tuning uses desired responses or action traces. **Preference optimization** uses comparisons or preference signals. **Reinforcement learning** uses outcomes or reward estimates from generated behavior. **Distillation** lets a teacher provide targets or feedback for

a student. An **on-policy** distillation setup trains on continuations produced by the student itself, so supervision covers mistakes the student is likely to make.

Multi-token prediction, or MTP, adds learning targets beyond the immediate next token.

Speculative decoding uses a drafter to propose several tokens that a target model verifies. They are related in some designs but are not synonyms. A training auxiliary head might not ship in a runtime; a separate drafter might be trained after the target. Speed depends on proposal acceptance, the extra draft work, verification cost, batch size and kernel support. No fixed speedup follows from the words “predicts seven tokens.”

Data quantity is not the only scaling variable. Repeating duplicate documents increases processed-token count without proportionally increasing useful independent information. A much smaller model may be trained longer for economical inference, while another run may prioritize capability for a fixed training budget. Do not turn a single paper's token-to-parameter ratio into a universal rule for your narrow dataset.

Separate three kinds of sparsity and compression

MoE sparsity skips unselected experts for a token. **Attention sparsity** skips some query/key relationships. **Weight sparsity** sets or stores some individual weights as zeros. They require different algorithms and hardware support.

Unstructured pruning does not necessarily accelerate an ordinary dense matrix multiply: a stored zero still occupies its usual position and can still participate in the operation. Structured sparsity uses a supported pattern and representation. NVIDIA's cuSPARSElt documentation, for example, specifies particular 2:4 patterns for several dtypes, with other patterns for some formats. That is a kernel contract, not permission to assume any 50% sparse model runs twice as fast. [A23]

Quantization reduces precision. Distinguish weight storage, activation precision, arithmetic and accumulation, and cache precision. A four-bit weight file can still use higher-precision activations and accumulators. Scales, zero points, packing metadata and excluded tensors add overhead. **Post-training quantization** transforms a trained model; **quantization-aware training** exposes training to quantization effects. Quality checks must include the cases that already worried you, such as rare names, tool arguments and numerical values.

Distillation changes the training signal and can, when the student is smaller, change the model's resident size. Quantization changes the numerical representation of its parameters. Pruning changes which structure remains. None gives a free guarantee of preserved quality. Compare the exported result to your unchanged baseline on the same held-out cases.

Compute weight memory before discussing a GPU

Here are ideal payload calculations using rounded publisher parameter counts. A billion means 10^9 , and a decimal GB means 10^9 bytes. These are not measured checkpoint sizes or usable VRAM requirements.

Study model	Rounded resident count	All BF16 payload	All 8-bit payload	Ideal packed 4-bit payload
MiMo V2.6 Flash	310B	620 GB	310 GB	155 GB
MiMo V2.6 Pro	1,020B	2,040 GB	1,020 GB	510 GB
GLM 5.3 Flash	320B	640 GB	320 GB	160 GB
Qwen3.5 397B A17B	397B	794 GB	397 GB	198.5 GB

The formula is parameter count $\times$ bits per parameter $\div$ 8. Even the ideal four-bit columns exceed 24 GB several times over, before overhead or cache. The active count cannot replace the resident count in this equation. CPU offloading can move part of the storage to system RAM, but it also changes the execution system and its data-transfer cost. It does not turn the model into an all-resident 24 GB GPU model.

For full mixed-precision training, an illustrative Adam-style budget is two-byte working weights, two-byte gradients, four-byte master weights and two four-byte optimizer moments: 16 bytes per parameter before activations and temporary buffers. At 310B parameters that is 4.96 decimal TB. This is a stated-assumption illustration; actual optimizers and sharding schemes differ. At one billion it is already 16 GB, which is why the earlier projects carefully budget sequence length, microbatch size and optimizer state. At three billion it is 48 GB, so the selection ceiling is not a promise of straightforward full Adam training on one card.

Adapters reduce trainable state, not the requirement to make the frozen backbone available for forward and backward computations. The earlier QLoRA project works by combining a small base with quantized storage and low-rank updates. It should not be extrapolated to hundreds of billions of resident parameters simply because the adapter itself is small.

The cache can become a second large model sized allocation

Use MiMo's inspected dimensions to make a conditional cache estimate. Assume one sequence, BF16 keys and values, an efficient rolling cache for each local layer, and no speculative, vision, audio, padding or allocator overhead. For Flash, nine global layers each retain four KV heads of width 192+128. Thirty-nine local layers each retain up to 128 tokens with eight KV heads. [\[A14\]](#)

$$\begin{aligned} \text{Flash cache bytes} \approx & T \times 9 \times 4 \times (192 + 128) \times 2 \\ & + \min(T, 128) \times 39 \times 8 \times (192 + 128) \times 2 \end{aligned}$$

At 131,072 tokens this is approximately 2.84 GiB. At 1,048,576 tokens it is approximately 22.52 GiB. GiB uses 2^{30} bytes. Pro's analogous calculation is approximately 6.29 GiB and 50.04 GiB, because it has ten global layers and eight global KV heads. These values are arithmetic predictions for the specified cache layout, not observations from a serving engine. [\[A12\]](#) [\[A14\]](#)

Changing cache precision, sharing prefixes or using a different layout changes the result. A reference implementation that retains a full history for local layers can consume more. Batch size and simultaneous requests multiply working state. The important conclusion is robust: supporting a context length architecturally does not mean that it is free to serve.

In a hybrid recurrent model, recurrent state can remain fixed as T grows, but the attention layers' history still grows. GLM's 136 MiB recurrent-matrix illustration does not imply a 136 MiB total cache. Its latent attention cache, selection index state and buffers must also be counted. A careful estimator sums the actual state for every layer type rather than applying one formula to the entire model.

CPU and GPU inference have different bottlenecks

A CPU often offers more affordable addressable RAM and flexible control flow. A GPU usually offers much greater parallel arithmetic and memory bandwidth for supported operations. Neither statement alone determines latency for your model. The workload, precision, cache length, batch size, thread configuration and kernels matter.

At low-batch decoding, moving weights and cache entries can dominate. An illustrative bandwidth lower bound is bytes read per generated step divided by sustainable bandwidth. If an imaginary implementation really had to read 6 GB for a step and sustained 600 GB/s, memory traffic alone would take at least 0.01 seconds, before other work. This is not a benchmark for a named GPU. Caches, reuse and overlap can change the bytes that are actually read; contention and poor kernels can lower achieved bandwidth.

During prefill or larger batches, matrix multiplication can reuse weights across many tokens and become more compute-intensive. Multimodal input adds image/audio decoding and encoder work before text generation. Recurrent decoding may maintain little history but still execute substantial projection matrices. Sparse expert execution can save arithmetic while suffering from small per-expert batches or expensive data movement.

Measure at least time to first token, decode tokens per second, prompt tokens per second, peak memory and end-to-end task latency. Record input length, output length, batch/concurrency, precision and exact software revision. Separate cold-start loading from a warm request. Compare output quality at the same time. A faster server that silently truncates the useful context is not an equivalent result.

Apply the systems questions to each family

The following are engineering implications of the inspected structures, not measured rankings.

Case	Data or evaluation emphasis	CPU inference consideration	GPU inference consideration
MiMo V2.6	Grounded multimodal examples and long tool trajectories; test local and distant evidence	Large expert storage needs substantial RAM; offloaded expert traffic can dominate	Local-window kernels help only part of the stack; global KV and expert residency remain
GLM 5.3 Flash	Long-range recall plus multimodal grounding; test recurrent compression and sparse selection	Runtime must support KDA state and sparse-index operations, with hundreds of GB of weight storage at common precisions	Efficient recurrent and sparse kernels matter; count recurrent, latent and indexer state separately
Qwen3.5 397B A17B	Visual grounding and tasks contrasting compressed history with direct attention	A17B does not remove the resident 397B footprint	Hybrid layers have different state and execution paths; shared experts always execute

Case	Data or evaluation emphasis	CPU inference consideration	GPU inference consideration
DeepSeek V3 reference	Text prediction and reasoning tasks; verify rare factual and structured outputs after compression	Large expert memory and latent-attention operator support are both required	MLA's cache advantage depends on the actual attention path and kernels
Mamba 2 reference	Sequence tasks probing what fixed-size state retains and forgets	Small recurrent state is useful, but projections and implementation quality still determine speed	Efficient scan/chunk kernels help training and prefill; single-step decode is a different path
Small Liquid comparison	The earlier narrow task plus distant-recall controls	Measure the exact small checkpoint and CPU backend	Check support for its convolution and attention blocks; do not infer speed from parameter count alone

Distributed memory introduces communication

Data parallelism runs model replicas on different examples. Ordinary replicas do not solve the problem of one replica being too large. **Optimizer or parameter sharding**, as in ZeRO-style or fully sharded approaches, divides selected training states across devices and gathers or communicates them when needed. **Tensor parallelism** splits large matrix operations across devices. **Pipeline parallelism** assigns successive layer groups to stages. **Expert parallelism** places different experts on different devices and dispatches token representations to them. **Sequence or context parallelism** distributes work or state along the sequence dimension; its exact meaning varies by framework. [A24] [A25]

For an MoE model, expert dispatch may involve all-to-all communication. A lightly used remote expert can cost more to reach than its arithmetic suggests. Training adds backward communication and synchronization. Pipeline stages need enough microbatches to keep several stages busy. These mechanisms can be combined, but a count of “eight GPUs” omits the interconnect, memory per device, placement and communication schedule.

CPU/GPU offload is another placement strategy. A system with a 24 GB GPU and hundreds of gigabytes of system RAM is a different hardware proposition from a 24 GB card in a modest desktop. Feasibility depends on RAM, storage, transfer bandwidth, runtime support and acceptable speed. No one-card launch command is offered here for the giant study models.

Three bounded experiments that return you to the small projects

Experiment one checks routing and memory arithmetic

The companion examples/architecture_lab/inspect_designs.py uses only Python's standard library. It performs the original top-2 mixture example, computes the weight payload table and checks the cache calculations. It does not download a model, import a training framework or contact a service.

```
cd examples/architecture_lab
python inspect_designs.py
```

Expected evidence is a passed set of arithmetic assertions, the mixture output approximately [1.25, 1.50], and Flash's illustrative 1,048,576-token cache of approximately 22.52 GiB. These checks were run for the book. They verify arithmetic and a routing illustration, not model accuracy, VRAM allocation or generation speed.

Change k from two to one in a copy of the mixture example. Predict the new output before running it. Change batch size in the cache function and confirm proportional growth. Explain why reducing k does not change the full resident-weight payload. This is a useful architecture exercise before spending any GPU time.

Experiment two compares local and distant dependencies

Return to your already working miniature transformer. Keep its tokenizer, optimizer, train/test split and tiny parameter scale. As a proposed extension, compare a full causal mask with a local mask at window sizes 16 and 64. Use controlled sequences in which an early randomly generated key/value pair must be recalled after distractors. Hold out the random keys, values and templates, and vary the distance independently of sequence length.

Begin with a few hundred examples only to verify that the labels and masking are correct. Then use a fixed data and compute budget for the comparison. Include a task whose answer is near the end so that the local model has a fair positive control. Record exact-match recall by distance and held-out next-token loss. Inspect whether enough stacked local layers could transmit the information indirectly. Do not label a failure at one distance as a universal impossibility for every local-attention model.

This is an experiment plan, not a delivered and GPU-tested modification. It stays at the miniature project's parameter size, well below one billion. A dense attention implementation with a local Boolean mask may not save memory or time; use it first to study behavior, and study an efficient kernel separately.

Experiment three compares direct attention with compressed state

Use the earlier small Liquid comparison and a small dense baseline of similar practical size. Keep the real task and evaluation data fixed. Add tests for short local patterns, exact distant recall and a long sequence with several competing facts. Evaluate the unchanged checkpoints first; model size, pretraining data and tokenizer differences mean this is a product comparison, not a controlled proof about architecture.

If you need a causal architecture study, build two tiny models from the same starting design and change only the mixer, then train both from scratch on the same controlled dataset. Do not splice a recurrent block into a pretrained transformer and assume the remaining weights will preserve its behavior. A from-scratch architecture ablation and a comparison of pretrained model families answer different questions.

For either version, capture CPU and GPU measurements on the hardware you actually own. Do not extrapolate a published data-center throughput number to your desktop. Stop the study when you can explain the observed difference, reproduce the evaluation and identify which parts of the result are due to an uncontrolled difference. The goal is a defensible design decision, not a new state-of-the-art claim.

A final checklist for unfamiliar architectures

Before adding another family to your project, write a one-page inspection record.

- Exact repository, revision, date, license and model class
- Actual resident count and the publisher's definition of active count
- Tokenizer and processor; input and output modalities
- Layer types, attention heads, experts and residual structure
- Per-layer persistent state and its precision
- Full-training, adapter-training and inference support checked separately
- Available data and objective evidence; unknown details clearly marked
- Memory arithmetic with units and assumptions
- A relevant quality baseline and a smoke-test plan
- What you measured, what you calculated and what remains untested

Once you can complete this record, model names become much less mysterious. You can learn from a trillion-parameter architecture while making a practical choice to train a useful tiny model. The core skill is matching a mechanism and its data to a job you can evaluate and afford.

38 Learn from public training projects without copying their assumptions

The most useful tutorial is one you can interrogate. After a small experiment works, open a second implementation and ask why its data, shapes, objective, evaluation, and saved files differ from yours. This makes public repositories and lessons useful throughout the book instead of turning them into a large prerequisite list. Use the book's current-model recipes for practical work. Older lessons appear here only as small concept exercises or brief comparisons that explain why a current practice matters; their model recommendations and installation commands are not a current shopping list.

The reading path was checked on 2 October 2026 against repository code, configurations, tests, creator-written lesson material, and original YouTube descriptions. The video guidance relies on companion material and author-published chapter markers, not a claim of watching entire recordings or reading their caption transcripts. These sources were reviewed; their GPU workflows were not tested for this book.

Stage 1 Make a gradient visible

Start after the book's first small model and its explanation of [one training step](#). You do not need a GPU to understand an update.

Read the small [micrograd engine](#) alongside its [comparison tests](#). It represents scalar computations as a graph, orders that graph, and accumulates the contribution of every path into each gradient. The tests compare both outputs and gradients with PyTorch. This is a better first autograd exercise than implementing a large model whose wrong gradients can hide behind apparently plausible samples. [\[T01\]](#), [\[T02\]](#)

Use Andrej Karpathy's [micrograd lesson](#) selectively: [08:08](#) for the first derivative, [1:22:28](#) for a reused-node bug, and [2:01:12](#) for the parameter-update walkthrough. The companion notebook explicitly corrects an exp backward assignment to accumulation. Prefer the corrected written implementation when it differs from a live recording. [\[T03\]](#)

Original exercise: evaluate $y = x*x + x$ at $x = 3$. Its output is 12 and its derivative is 7. Check the derivative three ways: algebra, a small finite difference, and autograd. Next, deliberately overwrite rather than add a gradient contribution and explain why the answer changes. Then take a small step that reduces $(y - 8)^2$. This introduces derivative, gradient, graph, backward pass, and learning rate exactly when they become useful.

Grant Sanderson's [gradient-descent lesson text](#) supplies a visual bridge from one adjustable value to many. It distinguishes the prediction function from the cost function whose inputs are parameters. Use that distinction when a beginner asks whether training changes the input examples or the model. The text is an official adaptation credited to Josh Pullen; it is not a transcript read from the video. [\[T04\]](#)

Advance when: you can name which values are data, parameters, predictions, loss, and gradients; explain why gradients are reset between ordinary optimizer steps; and check one derivative independently. Scalar autograd is an educational microscope, not a GPU

implementation strategy. “Small ML” here should not be confused with TinyML deployment on a microcontroller.

Stage 2 Turn text into supervised examples

Karpathy's [makemore MLP notebook](#) is valuable because it makes each next-character training pair visible. It shuffles whole names into training, development, and test partitions before producing overlapping context windows. Its embedding lookup, flattening, hidden layer, logits, and cross-entropy can be inspected separately. A shape shown in an exploratory notebook comment can become stale as a later cell changes dimensions: trust the actual tensor shape. [T05]

The [associated MLP lecture](#) is a follow-along companion, not something the beginner must finish before trying the project. The separate [makemore executable repository](#) packages multiple model families in one file and defaults to a tiny transformer. Do not assume that running the repository default reproduces the MLP lecture. [T05], [T06]

Original exercise: use a small, permissioned list of invented product names. Split complete names first, build length-three context/next-character pairs, and print ten decoded pairs. Compare a count-based bigram against an MLP. Hold the split and evaluation code constant. Record held-out loss and samples, including malformed outputs and memorized names. This is the right moment to learn token, vocabulary, embedding lookup, context window, logit, cross-entropy, validation, and leakage.

Advance when: you can account for every row in a batch, explain what the target means, and explain why windows from the same source item must not casually cross dataset partitions.

Stage 3 Diagnose a network before enlarging it

In the [activation-and-gradient notebook](#), the instructive artifacts are the activation histograms, saturation measurements, gradient histograms, and update-to-parameter ratios. Its running normalization statistics also show that a trained artifact can contain buffers as well as learnable parameters. These are reasons to inspect a model's state, not merely count its weights. [T07]

Original exercise: make a hidden layer's initial weights ten times larger, keep the data and seed fixed, and inspect the activation distribution and loss. Restore the initialization before testing a different learning rate. Record a hypothesis and one changed variable. Remove expensive debug hooks and retained intermediate gradients after the diagnostic run. Do not teach a numeric saturation percentage or update ratio as a universal pass/fail threshold.

Read normalization only after the experiment makes its purpose concrete. BatchNorm in this MLP lesson does not imply that a GPT should receive BatchNorm; the transformer architecture uses its own normalization design.

Stage 4 Replace fixed context with causal attention

The corrected [GPT video companion](#) shows shifted targets, a causal mask, head-sized query/key/value projections, multi-head concatenation, residual paths, and a pre-normalized block. Keep this as a read-only comparison unless its licensing is clarified: no license was reported by GitHub for this repository at inspection. Independently written book code is safer to distribute. [T08]

Karpathy's [GPT lesson](#) has particularly useful checkpoints at 14:27, 47:11, 1:02:00, and 1:26:48. They connect batching, weighted aggregation, learned attention, and residual connections. Its description corrects two mistakes: future tokens must not influence earlier predictions, and attention normalization uses the head dimension. These corrections were read directly on the author's YouTube page. [T09]

Original exercise: choose a short token sequence. Print the allowed attention positions as a grid; change only a future token; verify that earlier-position logits do not change in evaluation mode. Then compare a one-head implementation with a vectorized multi-head implementation using matching weights. A numeric equivalence test is stronger evidence than two runs both producing plausible text.

Sanderson's [attention lesson text](#) is helpful for visualizing a weighted update. It explicitly warns that its column-vector diagrams use a transposed convention relative to the original paper. Write tensor shapes beside every equation before porting it. Its adjective/noun example is an illustrative possible behavior, not a promise that every trained head has a clean human-readable purpose. [T10]

Now read sections 3.2.1-3.2.3 of [Attention Is All You Need](#). Identify the scaling, learned projections, and masking in your own code. The original work is an encoder-decoder translation system; a small decoder-only GPT is not an exact reproduction. Save the original training recipe until you can interpret its eight-P100 hardware premise. [T11]

Advance when: you can trace [batch, time, channels], explain why targets are shifted once, distinguish self-attention from cross-attention, and pass the causal-leakage test.

Stage 5 Turn a notebook into a recoverable training run

For a current public process reference, inspect [nanochat's checkpoint manager](#) together with the checkpoint block in [base_train.py](#). Model weights, optimizer state, model configuration, run arguments, data-loader state, and loop state play different roles. The load path also distinguishes training from evaluation mode and checks tokenizer vocabulary size. This is a process-reading exercise; its default large run is not the book's one-RTX command. [T25]

One historical comparison is useful: nanoGPT's smaller checkpoint dictionary did not save every random/data-stream state. Its readable loop remains educational, but the repository is now explicitly deprecated. Do not use its old GPU timings or installation advice to plan a current run. [T12], [T13]

[Raschka's standalone training script](#) offers another readable bridge. Its evaluation helper switches to evaluation mode and disables gradients before restoring training mode. It tracks tokens seen as well as optimizer steps. The short-story experiment is suitable for studying the

loop and overfitting, not evidence that a small corpus creates a generally capable language model. [T14]

Original exercise: stop the same tiny run, reload it, and verify that evaluation logits agree before continuing. Separately test whether the resumed next optimization step matches an uninterrupted run. The first check verifies saved-model fidelity; the second requires optimizer, scheduler, random-state, and data-position accounting. Keep these claims separate in the experiment report.

Advance when: a fresh process can reconstruct the tokenizer and model, evaluate held-out data, generate with evaluation-mode settings, and explain the precise resume guarantee it does and does not provide.

Stage 6 Adapt a small pretrained model with auditable labels

Read [Raschka's instruction collator](#) before hiding label construction behind a trainer. It shifts targets and masks repeated padding while retaining a terminal token. Its default objective includes the instruction text: padding masking and assistant-only masking are different choices. That distinction is the lesson to transfer. [T15]

The [Hugging Face smol-course LoRA lesson](#) makes adapters, loading, and merging concrete. Pair it with the book's fixed small-model SFT recipe; do not import the course's entire changing environment into an already pinned setup. [T16]

Original exercise: before training, print one complete rendered conversation, its token IDs, attention mask, and loss mask. Color or label which tokens contribute to loss. Confirm that the answer has not been removed by truncation and that the chosen end-of-turn token remains supervised. Then overfit a tiny training-only slice as a plumbing test; a deliberately memorized slice is not the final evaluation.

The course's [v2 hands-on page](#) deserves a version audit. Its root requirements pin Transformers 4.46.3 and TRL 0.12.1, while the page uses newer SmolLM3-era examples, Trackio, and `SFTConfig(max_length=...)`. The pinned TRL documentation instead uses `max_seq_length`. A later example passes `args=config` after defining `training_config`. Evaluation intervals alone do not create a held-out evaluation dataset. These are reasons to validate the precise chosen recipe, not evidence that the general method is wrong. [T17], [T18]

Read [LoRA](#) once you can identify frozen weights, trainable weights, and the dimensions of a low-rank update. Its large-model result is evidence about the adaptation method, not proof of your 24 GB configuration's fit. The LLM chapter provides the actual memory and objective discussion.

Advance when: you can report trainable parameter count, supervised token count, adapter/base revisions, maximum sequence length, measured peak memory, a baseline comparison, and reload behavior. A tutorial's word “small” is not a memory budget.

Stage 7 Train an embedding for a real retrieval decision

The current [Sentence Transformers NLI training example](#) separates training loss from an embedding-similarity evaluator and evaluates the initial model. The [MS MARCO example](#) adds mined negatives, teacher-score filtering, a no-duplicates sampler, normalized vectors, and cached contrastive loss. The full example has substantial dataset and host-RAM requirements; its mini-batch control only addresses part of memory usage. [T19]

Original exercise: begin with your small retrieval corpus and the book's exact-search evaluator. Write ten difficult queries and label all known acceptable documents. Inspect the top errors before mining harder negatives. Verify that a supposedly negative document is not a second valid answer. Compare retrieval metrics before and after adaptation, then rebuild the corpus embeddings using the new encoder. Use the embedding chapter's pinned API rather than mixing its v5 recipe with a newer main-branch import layout.

This is where the meanings of embedding, pooling, bi-encoder, hard negative, false negative, in-batch negative, and Recall@k become operational. Read Sentence-BERT after observing why independently computed document vectors make retrieval possible. Do not require readers to retrain a generic embedding model from random weights before making a useful domain retriever.

Stage 8 Learn denoising before adapting an image or audio generator

Jonathan Whitaker's [official diffusion lesson](#) links the creator's [video walkthrough](#) and the actual Hugging Face notebooks. The [scratch notebook](#) deliberately starts with a tiny uniform-noise denoiser, then compares it with DDPM. Do not relabel the toy corruption rule as the DDPM forward process. Noise distribution, target parameterization, timestep conditioning, and sampler must be named separately. [T20]

Original exercise: on the book's synthetic small images, plot one clean example and several noised versions. Print the sampled timestep and target type. Verify the scheduler's forward-noise calculation numerically on one example. Only then compare learned denoising and generated samples on fixed seeds. Change one item at a time: model, target, schedule, or sampling steps.

The [audio notebook](#) is useful for tracing waveform → resampling → spectrogram → denoising → audio reconstruction. This particular example diffuses spectrogram images; it is not a general description of every audio diffusion architecture, and it does not teach transcription. Its loop contains `loss.backward(loss)`. For ordinary scalar MSE training, use `loss.backward()`; the extra argument supplies an upstream gradient and multiplies the update signal by the loss value. [T21], [T22]

Advance when: you can state what the model predicts, which representation it sees, how a sample is reconstructed, and which held-out checks measure quality beyond a loss plot. Start an audio round-trip test before training: if representation conversion already damages the source substantially, more denoising updates cannot simply erase that limitation. For advanced image/audio architectures and adaptation, follow the image and audio chapters and their primary papers.

Stage 9 Study faster implementations as controlled experiments

[llm.c's LayerNorm study](#) recomputes a normalized activation in backward and compares manual derivatives with autograd. Its repository also validates C results against a PyTorch reference. The transferable practice is “prove equivalence, then optimize,” not “rewrite everything in CUDA first.” Its CPU starter is a short pretrained GPT-2 fine-tune, and its legacy single-GPU FP32 path is distinct from its modern reproduction process. [T23]

[nanochat](#) extends the public process across tokenizer training, pretraining, SFT, evaluation, and inference. Its default speedrun targets an eight-H100 node. The README specifically warns that GPUs below 80 GB need configuration changes. Its [CPU/MPS script](#) is explicitly educational and shrinks the model. Do not convert either example's reported duration into an RTX estimate. [T24]

Original exercise: keep model outputs and evaluation fixed while comparing two attention implementations or precision settings. Report numerical tolerance, peak memory, tokens per second, warm-up policy, and evaluation difference. Decide whether the improvement is worth the added setup and debugging cost. Architecture research, pipeline engineering, and raw-kernel engineering are different next projects; the beginner need not complete all three to become a useful practitioner.

A compact source audit checklist

Before recommending a public command, inspect the actual file that will execute it.

1. Record a commit or released version, not just a default-branch URL
2. Read the data loader and collator before the optimizer settings
3. Identify every download, login, telemetry integration, and upload
4. Check whether “optional” publishing is actually executed: the inspected Sentence Transformers examples attempt `push_to_hub` inside `try` blocks
5. Separate a source-code license from book artwork, weights, data, and generated-artifact rights
6. Compare the tutorial's API names with its dependency file and the matching versioned docs
7. Keep GPU model, GPU count, precision, sequence length, batch size, and token budget beside any runtime claim
8. Run a tiny smoke test and a reload test before a longer job
9. Read later corrections and tests as carefully as the tutorial narrative
10. Mark every unrun GPU workflow honestly; a successful source review is not a benchmark

39 Exercise checkpoints and worked answers

Use answers to check reasoning rather than memorize settings

Some exercises have a numerical answer. Others ask you to design an experiment, label real data, or make a tradeoff. Those do not have one universal correct configuration. The checkpoints below show what a defensible answer contains. A result can differ from an example and still be sound if its assumptions and evidence are explicit.

The first classifier and simple numbers

For predicted packing time = 0.7 times item count plus 2, five items give 5.5 minutes. Increasing the bias by one adds one minute to every prediction. Increasing the weight from 0.7 to 0.8 adds half a minute for five items and one minute for ten. This demonstrates the difference between an additive offset and an input-dependent effect.

The first two-feature classifier has two weights and one bias. Its architecture can represent a straight-line boundary. An XOR-style target requires a nonlinear representation or model. More gradient-descent steps do not turn a linear score into an arbitrary nonlinear boundary. A useful answer to the XOR experiment distinguishes model capacity from optimization success.

A valid explanation of its saved artifact names the feature order, three parameters, sigmoid, threshold, and target meaning. Saving only the weights while forgetting feature order would leave an incomplete inference contract.

Shapes and parameter counts

Sixteen sequences of 128 token IDs have shape [16, 128]. Replacing each ID with a 64-dimensional vector gives [16, 128, 64]. Token IDs are integer indices; the embedding values are floating-point learned representations.

A 256-row embedding table of width 128 has 32,768 parameters. A 50,000-row table at the same width has 6,400,000. Vocabulary size therefore matters greatly in a small language model. Tying the input and output table avoids storing a second independent copy of that same-shaped projection; it does not eliminate the computation of output logits.

An RGB batch of eight images at 96 by 96 pixels commonly has shape [8, 3, 96, 96]. A three-class classifier returns logits [8, 3]. A one-object detector may return objectness [8] and boxes [8, 4]. A binary segmenter may return pixel logits [8, 1, 96, 96]. These output shapes imply different targets and losses.

Loss weighting and accumulation

Microbatch size two, accumulation eight, and 300 optimizer steps produce 2 times 8 times 300 = 4,800 example presentations if all microbatches are full. The number of unique examples depends on sampling and repetition. If each example contains 128 supervised targets, the same schedule presents 614,400 supervised targets.

Suppose microbatch A has 100 supervised tokens with mean loss 2.0 and microbatch B has 1,000 with mean loss 1.0. Averaging the two means equally gives 1.5. Averaging all token losses gives $(100 \times 2 + 1,000 \times 1) / 1,100$, approximately 1.091. Neither calculation is mysterious, but they optimize different weighting. State whether your objective treats examples, microbatches, or supervised tokens equally.

In assistant-only training, a prompt token can be visible through attention while excluded from loss. The attention mask controls information flow; the loss mask controls which predictions are rewarded. A correct answer mentions both separately and verifies the actual supervised span after tokenization.

Memory and time arithmetic

Under the explicit FP32 AdamW assumption of sixteen persistent bytes per trainable parameter, 250 million parameters require 4 billion bytes, or about 3.73 GiB, before activations and other overhead. One billion requires 16 GB, about 14.90 GiB. Three billion requires 48 GB, about 44.70 GiB. These are inventories, not observed GPU peaks.

A dataset of 40,000 examples averaging 500 supervised tokens contains about 20 million supervised tokens per complete pass. Two epochs present about 40 million. At a measured 1,250 supervised tokens per second, that is 32,000 seconds, about 8.89 hours of processing. Add startup, evaluation, saving, preprocessing and interruptions. A rate that counts padding or prompt tokens is a different measure.

For a transformer dominated by square weight matrices, doubling width roughly quadruples many parameter terms. Doubling block count roughly doubles those terms. Embeddings, position tables, output heads, and architectural variations change the exact total. A good answer uses the actual configuration rather than a slogan about model size.

Splits and unavailable information

A strong split explanation names the independent unit and the intended generalization claim. Frames from one video, clips from one recording session, paraphrases of one source document, and photos of one physical part should usually remain grouped for the corresponding new-source evaluation. Counting derivatives as independent examples inflates apparent data diversity.

For a future-fault predictor, a measurement recorded after the fault cannot be an input to a prediction supposedly made before it. For an inventory assistant, today's stock level cannot be inferred reliably from last month's static training text alone. A larger model does not create missing current information. Appropriate responses include adding a legitimate input, retrieving current state, or abstaining.

A useful annotation policy resolves cases where reasonable people disagree. If a support message mentions both a late shipment and cancellation, specify the routing priority or allow multiple labels. Do not ask the optimizer to solve an undefined policy.

Classification and detection metrics

A detector finding 18 of 20 true targets with 12 false alarms has recall $18/20 = 0.90$ and precision $18/30 = 0.60$. Overall accuracy can look high when most images are empty. The relevant operating decision depends on missed-target cost, false-alarm workload, and whether abstention is available.

A box detector that returns a confident box in the wrong location has not made a correct detection. In the one-object exercise, a successful match requires both sufficient objectness and IoU at least 0.5. The objectness threshold and IoU criterion answer different questions.

For a mask that covers x coordinates 4 through 13 and y coordinates 2 through 5 in a 20 by 10 image, exclusive-edge coordinates are [4, 2, 14, 6]. Normalized xxyy is [0.2, 0.2, 0.7, 0.6]. This convention includes the full last foreground pixel. A mask with no foreground has no real object box, and its placeholder coordinates should not contribute to localization loss.

A large region shifted by one pixel can retain high IoU while its exact boundary differs substantially. Therefore an outline application should evaluate boundary tolerance in meaningful units, not only region overlap. An answer that proposes 'higher resolution' should also explain annotation precision, original-image coordinates, and compute cost.

Retrieval and contrastive learning

A useful hard negative is plausible but truly wrong for the query. A passage that paraphrases the correct answer is a false negative if treated as irrelevant. Review mined negatives before training; a similarity score alone does not establish irrelevance.

When a batch uses other positive passages as negatives, duplicate or semantically equivalent passages can contradict the training objective. Ordinary gradient accumulation over separate microbatches does not automatically create cross-microbatch negatives. A correct larger-batch proposal must explain how the loss sees the intended candidate set.

After retraining the encoder, regenerate document embeddings with the selected saved encoder and the same prompts, pooling and normalization. Reusing an old index with new query embeddings compares incompatible representations. The search program must load the matching encoder/index pair.

Language and typed decisions

A complete tool-call test checks whether an action is appropriate, whether the selected tool is correct, whether all arguments are valid and semantically correct, and whether the application permits execution. Parseable JSON solves only the first formatting layer. A model that fabricates a missing identifier should fail the task even if its schema is valid.

A typed-decision model returns scores over allowed choices. The output type limits the set of possible answers, not the chance of choosing the wrong one. Calibration must use separate examples, and an abstention policy must be judged by both accepted-case quality and coverage. A system that answers nothing is not automatically useful because its remaining error rate is low.

The causal-mask exercise changes a future input token and checks earlier-position logits in evaluation mode. Those earlier logits should remain unchanged within numerical tolerance. A low training loss cannot substitute for this information-flow test because an incorrect mask may make the task artificially easy.

A valid style evaluation preserves facts and compares observable style features on new topics. Copying familiar phrases from a small training collection is weaker evidence than adapting the style to genuinely new content. Blind comparisons reduce the temptation to reward the model you just trained.

Speech and generated audio

For reference 'turn the blue valve' and hypothesis 'turn blue valves', one plausible minimum edit alignment deletes 'the' and substitutes 'valves' for 'valve'. That is two errors over four reference words, so WER is 0.5. Normalization policy changes what counts as a word or substitution; record it and apply it consistently.

A transcript must match the exact audio segment. Cropping an utterance while retaining its complete transcript creates incorrect supervision. Keeping the same speaker or recording session across splits may also make an evaluation claim about unseen speakers invalid.

Speech recognition predicts text from recorded speech. A diffusion sound model generates audio from noise and conditions. Training a sound generator on tones does not teach speech transcription, and a transcription model does not automatically synthesize speech. A valid category choice begins with which direction the application needs.

For a denoising project, identify the precise target implemented by the trainer: noise, clean signal, velocity, or another parameterization. Preserve that relationship with the scheduler used at inference. A falling loss under one objective does not justify switching the sampler to a different unsupported convention.

The final independent project

A strong project plan contains the prediction contract, lawful data, independent split, baseline, training objective, small correctness test, measured feasibility test, fixed quality evaluation, artifact contract, and failure policy. It also states what evidence would make you stop.

If a first run fails, a strong next step identifies the earliest wrong boundary and proposes one controlled change. 'Use a bigger model' is justified only when evidence points to a capacity limit rather than missing information, bad labels, broken preprocessing, or an unsuitable objective.

When you can explain your model's input, target, learned values, loss, update, evaluation and deployment path without relying on the name of a library, you have completed the core learning goal of this book.

40 Companion guide and verification record

The code follows the same learning ladder

Unpack the companion into a new directory. Keep its examples and data-generation scripts together. Commands in the book state when to run from the companion root and when to change into a project directory. Do not mix those working directories silently; a relative path is resolved from the current directory.

The companion is a collection of separate small projects. It intentionally does not have one global requirements file. A CPU tabular environment, a pinned image-diffusion environment, and a current Transformers environment have different compatibility needs. Use the requirements and README beside the project you are running.

Project directory	Main result	First useful check
examples/first-model	A three-parameter classifier and JSON model	Run train.py and predict.py with only Python
examples/tiny-ml	Tabular prediction, image classification, one-object detection, masks and contours	Run the CPU tabular project, then inspect generated images and masks
examples/tiny-transformer	A byte-level causal model trained from random weights	Generate data, then run check_model.py in the pinned torch environment
examples/embeddings	Keyword baseline, neural retrieval adaptation, saved-index search	Generate data and measure the lexical baseline before neural training
examples/llm	Full, LoRA, and QLoRA adaptation plus structured-output scoring	Run test_offline.py before installing or downloading models
examples/decision_pointer	A Qwen-derived candidate-option scorer	Run dependency-free numerical tests before model training
examples/asr	Audio transcript validation, current small ASR adaptation and CPU/GPU transcription	Run manifest and WER/CER checks before downloading a model
examples/diffusion	Image LoRA and a tiny waveform diffusion model	Validate generated images and audio before any neural run
examples/shared	Small budgeting and environment-inspection utilities	Run the memory worksheet with your own stated assumptions

What was actually executed

The dependency-free first classifier was trained, saved, reloaded, and used for a fresh prediction. Its fixed synthetic validation accuracy was 0.96 and its separately generated test accuracy was 0.985. Those values are educational observations on the supplied synthetic rule.

The CPU tabular classification and regression projects were executed with the versions recorded in their verification file. The lexical retrieval baseline, dataset generators, image and mask checks, retrieval metrics, audio file audits, numerical decision-model tests, and offline LLM parser tests were also run. The final companion includes per-project validation records with exact commands or checked behaviors.

Python syntax compilation and shell syntax checks were performed where appropriate. Independent source review checked important model APIs, masking logic, memory assumptions, tagged training code, calibration boundaries, and resume behavior. Several implementation defects were corrected during that review, including training-resume provenance and malformed tool-output scoring.

What was not executed

No neural model training or inference was performed on the reader's RTX GPU while preparing this edition. The authoring environment did not contain the necessary PyTorch model runtime for those neural paths. No large pretrained checkpoints were downloaded for an end-to-end reproduction. GPU peak-memory figures and wall-clock estimates in planning examples are calculations or starting assumptions unless explicitly identified as measurements from a named source.

Syntax checks do not prove that a dependency set resolves, that a model download is accessible, that a device kernel is supported, or that a training run improves quality. The pinned neural environments are inspected reference targets. Complete the local smoke tests before treating them as a working lockfile on your machine.

Read every project's exact source-version, dependency, and local-modification instructions. A public trainer can contain stale defaults or version conflicts even when its main idea is correct. A copied command without its associated source version and documented changes is a different experiment.

Your local acceptance checklist

First, verify the interpreter, package versions, device, and free storage. Second, generate or validate a tiny dataset and inspect the exact encoded input and targets. Third, run one complete update and confirm finite loss, gradients, and changed parameters. Fourth, save, reload in a fresh process, and compare outputs. Fifth, test the intended CPU or GPU inference path on a new input.

Then test the maximum supported input shape, evaluation, and checkpoint writing. Test a deliberate resume only where the project implements optimizer recovery; otherwise record that limitation and use short runs or add and test recovery before a long job. Record measured memory and throughput. Run the fixed baseline and candidate evaluation before increasing the training budget. Freeze the environment only after these checks pass.

If a check fails, keep its error and the environment record. Do not remove version pins at random until the command happens to run. Identify the incompatible component, change it deliberately, repeat the smoke test, and record the new validated environment.

Where to keep your own evidence

Each project should have a runs directory with a unique name for each experiment. Keep configuration, data hashes, logs, metrics, selected outputs, checkpoints, and a short decision note. Use the supplied data-card and model-card patterns as a starting point, and expand them when the task's consequences require more documentation.

The book's examples demonstrate mechanics and disciplined experimentation. Your own evidence determines whether a resulting model is useful for your application.

Glossary with links to explanations

Use each link to return to the explanation and its worked project. Terms introduced at several levels point to a useful first or central explanation.

- **Activation.** An intermediate value produced during a forward pass. [Explanation](#)
- **Activation checkpointing.** Recomputing selected activations during backward to save memory. [Explanation](#)
- **Activation function.** A nonlinear transformation such as ReLU or GELU, distinct from an intermediate activation value. [Explanation](#)
- **Active parameters.** The subset involved in processing a token; not the complete resident or trainable parameter count. [Explanation](#)
- **AdamW.** An adaptive optimizer with decoupled weight decay. [Explanation](#)
- **Approximate nearest neighbors.** An indexing approach trading some exact-search fidelity for faster vector retrieval. [Explanation](#)
- **Architecture.** The form of the model computation and arrangement of its learned parts. [Explanation](#)
- **Architecture/data relationship.** The connection between what a model can represent and what its examples must demonstrate. [Explanation](#)
- **ASR hallucination.** Text generated by a recognizer despite the corresponding words being absent from the audio; silence and noise probes help expose this. [Explanation](#)
- **Assistant-only loss.** Loss on assistant responses, while user, system and tool-observation text remains context. [Explanation](#)
- **Attention.** A learned way for a position to combine information from other permitted positions. [Explanation](#)
- **Attention mask.** A rule controlling which positions may influence a representation. [Explanation](#)
- **Audio-transcript alignment.** Ensuring that a segment's target text contains exactly the words present in that segment, rather than the transcript of a longer recording. [Explanation](#)
- **Autocast.** A framework policy choosing compute precision for supported operations without necessarily changing stored parameter dtype. [Explanation](#)
- **Autograd.** Automatic differentiation used to calculate gradients. [Explanation](#)
- **Automatic speech recognition (ASR).** Converting recorded speech audio into a written transcript; distinct from generating speech, identifying speakers, or summarizing a transcript. [Explanation](#)
- **Average precision.** A ranking metric summarizing precision as recall increases. [Explanation](#)
- **Backpropagation.** Computing parameter gradients through the model computation. [Explanation](#)
- **Base model.** A pretrained model before a particular instruction or task adaptation. [Explanation](#)

- **Baseline.** A simpler or unchanged system used as the comparison before training. [Explanation](#)
- **Batch.** Examples processed together or contributing to one update, depending on the stated unit. [Explanation](#)
- **BCE.** Binary cross-entropy, a loss for binary targets such as foreground versus background. [Explanation](#)
- **BF16.** A two-byte floating-point format with a different precision/range tradeoff from FP16. [Explanation](#)
- **BF16 master-state distinction.** The difference between low-precision operation dtype and higher-precision stored parameters or optimizer state. [Explanation](#)
- **Bias.** An additive learned value in a model transformation. [Explanation](#)
- **Boundary F1.** Precision/recall-based agreement of boundaries under a specified spatial tolerance. [Explanation](#)
- **Boundary tolerance.** The distance within which predicted and reference boundary points count as matching. [Explanation](#)
- **Calibration.** Checking and adjusting the relationship between predicted probabilities and observed outcomes. [Explanation](#)
- **Calibration split.** Held-out examples used to fit a probability correction without fitting the original predictor. [Explanation](#)
- **Catastrophic forgetting.** Loss of previously useful behavior during adaptation to a different or narrower distribution. [Explanation](#)
- **Causal mask.** A mask that prevents a position from reading future positions. [Explanation](#)
- **Character error rate (CER).** The analogous edit rate over declared character units; useful when whitespace-delimited words are unsuitable. [Explanation](#)
- **Chat template.** A model-specific serialization that converts roles, turns and optional tools into the tokens expected by a chat checkpoint. [Explanation](#)
- **Checkpoint.** Saved model state, with additional optimizer and random state when intended for resume. [Explanation](#)
- **Class imbalance.** Unequal representation of target classes, often requiring slice-aware evaluation. [Explanation](#)
- **Classifier free guidance.** A sampling technique combining conditional and unconditional predictions. [Explanation](#)
- **CLM.** A contrastive approach comparing separately encoded state and action representations. [Explanation](#)
- **Closed-loop evaluation.** Testing trajectories using the model's own calls and the observations they actually produce. [Explanation](#)
- **Code-switching.** Alternating languages within speech, which requires separate evaluation rather than assuming single-language quality transfers. [Explanation](#)
- **Completion-only loss.** Loss computed on the completion after a prompt, excluding prompt targets. [Explanation](#)

- **Conformer.** An acoustic architecture combining local convolutional processing with attention over a longer context. [Explanation](#)
- **Connectionist temporal classification (CTC).** A sequence objective summing probabilities of frame-level paths that collapse to the desired transcript, including a blank label. [Explanation](#)
- **Context length.** The amount of input history available to a sequence model. [Explanation](#)
- **Continued pretraining.** Continuing a pretrained model's original language-modeling objective on additional text. [Explanation](#)
- **Contour.** A boundary extracted from a predicted or labeled region. [Explanation](#)
- **Contour hierarchy.** Relationships among outer boundaries and nested holes or regions. [Explanation](#)
- **Contrastive loss.** An objective making matched representations score above selected alternatives. [Explanation](#)
- **Convolution.** A learned local filter reused across spatial or temporal positions. [Explanation](#)
- **Cosine similarity.** The normalized dot product measuring vector direction similarity. [Explanation](#)
- **CPU dependency setup.** Installing and verifying the packages required by a CPU execution path in an isolated environment. [Explanation](#)
- **CPU/GPU inference.** Running the same learned function on different devices with compatible preprocessing, numeric policy, and output checks. [Explanation](#)
- **Cross entropy.** A loss penalizing low probability assigned to the correct target. [Explanation](#)
- **CTC blank.** A special no-output label used when collapsing CTC paths; distinct from a written space. [Explanation](#)
- **Data augmentation.** Training-input transformations justified by a target-preservation assumption. [Explanation](#)
- **Data leakage.** Information crossing an evaluation boundary or appearing before it would be available. [Explanation](#)
- **Dataset card.** A record of data sources, rights, processing, splits, coverage, and limitations. [Explanation](#)
- **Decode.** Producing subsequent tokens incrementally. [Explanation](#)
- **Dense model.** A model whose dense blocks normally participate for each token, unlike selectively routed experts. [Explanation](#)
- **Depthwise convolution.** Local filtering independently within channels. [Explanation](#)
- **Detection.** Predicting object locations and usually their classes. [Explanation](#)
- **Device budget.** The combined memory, compute, storage, power, and response-time constraints of the target device. [Explanation](#)
- **Dice loss.** An overlap-based objective derived from the Dice similarity of predicted and target regions. [Explanation](#)
- **Diffusion.** A generative approach learning to reverse a noise-corruption process. [Explanation](#)

- **Distillation.** Training a student using selected teacher behavior or distributions. [Explanation](#)
- **Double quantization.** Quantizing quantization constants as well as base weights to reduce storage overhead. [Explanation](#)
- **DPO.** Direct Preference Optimization: fitting chosen-versus-rejected preferences relative to a reference policy. [Explanation](#)
- **Dropout.** Random suppression during training that must be disabled for ordinary evaluation. [Explanation](#)
- **Dual encoder.** An architecture encoding the two sides of a comparison separately. [Explanation](#)
- **Effective batch.** All microbatches contributing to one optimizer update. [Explanation](#)
- **Embedding.** A learned vector representation used for a specified prediction, similarity, or retrieval task. [Explanation](#)
- **Embedding table.** A learned lookup from discrete IDs to vectors. [Explanation](#)
- **Encoder-decoder.** A model that forms an intermediate representation and transforms it into an output representation. [Explanation](#)
- **Epoch.** One pass through a defined dataset, where that concept applies. [Explanation](#)
- **Evaluation.** A fixed procedure for comparing task behavior against requirements. [Explanation](#)
- **Expert parallelism.** Placing experts on separate devices and dispatching tokens. [Explanation](#)
- **Export.** Packaging a learned artifact and its interface for a particular inference runtime. [Explanation](#)
- **F1.** The harmonic mean of precision and recall. [Explanation](#)
- **False negative.** A genuinely positive or relevant example treated as negative by a decision or training construction. [Explanation](#)
- **Feature.** Information represented in a form the model can consume. [Explanation](#)
- **Feature channel.** One learned component of a representation at each spatial or temporal position. [Explanation](#)
- **Final holdout protocol.** Evaluating a protected test set only after model and pipeline choices are fixed. [Explanation](#)
- **Fine tuning.** Further training starting from pretrained weights. [Explanation](#)
- **FlashAttention.** IO-aware execution of exact attention. [Explanation](#)
- **Flow matching.** Learning a vector field for a path between noise and data distributions. [Explanation](#)
- **Forced alignment.** Estimating where supplied transcript units occur in an audio recording; it does not verify that the supplied words are correct. [Explanation](#)
- **Foreground imbalance.** A segmentation condition where target-object pixels are much rarer than background pixels. [Explanation](#)

- **FP16.** A two-byte floating-point format whose numeric behavior differs from BF16. [Explanation](#)
- **FP32.** A four-byte floating-point format. [Explanation](#)
- **Fresh environment.** An isolated package environment that avoids accidental dependency inheritance from another project. [Explanation](#)
- **Frozen feature extractor.** A pretrained component whose parameters remain unchanged while another part learns. [Explanation](#)
- **Full fine-tuning.** Updating all model parameters starting from pretrained weights; it does not mean random initialization. [Explanation](#)
- **Full parameter training.** Updating every selected model parameter rather than only an adapter or head. [Explanation](#)
- **GQA.** Grouped-query attention: several query heads share a smaller number of key/value heads. [Explanation](#)
- **GQA and MQA.** Sharing KV heads among several or all query heads. [Explanation](#)
- **Gradient.** The local rate of change of loss with respect to a parameter. [Explanation](#)
- **Gradient accumulation.** Combining several microbatch gradients before an optimizer update. [Explanation](#)
- **Gradient boosting.** An ensemble built by adding learners that improve the remaining prediction error. [Explanation](#)
- **Gradient caching.** A technique recomputing or caching parts of embedding training to support a larger logical contrastive batch. [Explanation](#)
- **Gradient checkpointing.** Recomputing selected forward activations during backward to trade extra compute for lower memory use. [Explanation](#)
- **Gradient clipping.** Limiting unusually large gradients before updating. [Explanation](#)
- **Grouped split.** Keeping related records in the same partition to protect evaluation independence. [Explanation](#)
- **Hard negative.** An incorrect candidate that is plausibly confusable with the correct match. [Explanation](#)
- **Hybrid.** Combination of different sequence-mixing mechanisms. [Explanation](#)
- **Hyperparameter.** A configuration choice controlling the model or training procedure. [Explanation](#)
- **Imputation.** Filling missing values using a rule fitted only on appropriate training data. [Explanation](#)
- **In-batch negatives.** Other examples in a training batch used as alternative candidates in a contrastive objective. [Explanation](#)
- **Index consistency.** Agreement among the saved encoder, preprocessing, vector representation, and indexed documents. [Explanation](#)
- **Inference.** Using fitted model parameters to produce an output. [Explanation](#)
- **Input.** The information available to the model when a prediction is made. [Explanation](#)

- **Instance segmentation.** Predicting distinct masks for individual objects. [Explanation](#)
- **IoU.** Intersection over union, measuring region overlap relative to the combined region. [Explanation](#)
- **Jev.** A hosted typed-decision model; its private internals are not reproduced here. [Explanation](#)
- **KDA.** Kimi Delta Attention, a gated recurrent linear-attention mechanism. [Explanation](#)
- **Kev.** An open decision-model implementation studied separately from Jev. [Explanation](#)
- **Knowledge editing.** Methods designed to change targeted factual associations, with locality/generalization limits requiring evaluation. [Explanation](#)
- **KV cache.** Stored attention state used to avoid repeated generation computation. [Explanation](#)
- **L2 normalization.** Rescaling a vector to unit Euclidean length. [Explanation](#)
- **Label.** The recorded correct target for an example. [Explanation](#)
- **Last-token pooling.** Using the final nonpadding token representation as the sequence embedding under the model's trained contract. [Explanation](#)
- **Latent autoencoder.** A model mapping images or audio to and from a smaller learned representation. [Explanation](#)
- **Laya.** A public ModernBERT-based typed-decision model used as a study case. [Explanation](#)
- **Leakage.** Information crossing an evaluation boundary or becoming available earlier than it would at prediction time. [Explanation](#)
- **Learnability.** Whether the desired mapping can be inferred from the available information and examples under the chosen model. [Explanation](#)
- **Learning curve.** A comparison of performance as training data or training budget increases. [Explanation](#)
- **Learning rate.** The scale of an optimizer update. [Explanation](#)
- **Linear attention.** Sequence mixing that avoids the usual quadratic pair matrix. [Explanation](#)
- **Load balancing.** Mechanism to avoid unusably uneven expert traffic. [Explanation](#)
- **Log-Mel spectrogram.** Successive short-window frequency representations pooled through Mel filters and logarithmically scaled, using the model processor's expected conventions. [Explanation](#)
- **Logistic regression.** A linear score transformed into a class probability, fitted from labeled examples. [Explanation](#)
- **Logit.** An unnormalized prediction score before probability conversion. [Explanation](#)
- **Logits.** Unnormalized scores over the vocabulary before conversion to token probabilities. [Explanation](#)
- **LoRA.** Low-rank adaptation: freeze a base matrix and learn a smaller factorized correction. [Explanation](#)
- **LoRA alpha.** A scale parameter; standard LoRA multiplies the adapter correction by alpha divided by rank. [Explanation](#)

- **LoRA rank.** The inner dimension of the two adapter matrices, controlling the rank and capacity of their product. [Explanation](#)
- **Loss.** A numeric training objective comparing prediction and target. [Explanation](#)
- **Loss mask.** A rule selecting which predictions contribute to training loss. [Explanation](#)
- **Macro-F1.** The average of per-class F1 scores, giving each class equal weight. [Explanation](#)
- **MAE.** Mean absolute error, measured in the target's units. [Explanation](#)
- **Margin.** A required score or distance separation between desired and undesired outcomes. [Explanation](#)
- **Master weights.** An additional high-precision parameter copy maintained by some mixed-precision training schemes. [Explanation](#)
- **Mean pooling.** Averaging selected position representations into one vector. [Explanation](#)
- **mHC.** Constrained learned routing among residual streams. [Explanation](#)
- **Microbatch.** The examples handled in one forward/backward pass. [Explanation](#)
- **Mixed precision.** Using different numeric formats for different parts of training. [Explanation](#)
- **Mixture of experts.** A model that routes each token through selected expert networks while retaining the larger expert collection. [Explanation](#)
- **MLA.** Learned latent representation of attention KV information. [Explanation](#)
- **Model card.** A record of model purpose, provenance, evaluation, tested targets, and limitations. [Explanation](#)
- **Model/index SHA contract.** Using cryptographic file identities to ensure a saved encoder, preprocessing, and vector index belong together. [Explanation](#)
- **MRR.** Mean reciprocal rank of the first relevant result. [Explanation](#)
- **MTP.** Training prediction targets beyond the immediate next token. [Explanation](#)
- **Multimodal embeddings.** Representations that support matching or retrieval across different input modalities. [Explanation](#)
- **Multimodal input.** Input combining more than one type of information, such as text and audio. [Explanation](#)
- **nDCG.** Normalized discounted cumulative gain, a ranking metric rewarding relevant results near the top. [Explanation](#)
- **NF4.** NormalFloat four-bit quantization, designed for representing approximately normally distributed weights. [Explanation](#)
- **On-policy distillation.** Teacher supervision on the student's own continuations. [Explanation](#)
- **Optimizer.** The rule that updates parameters using gradients and possibly history. [Explanation](#)
- **Optimizer state.** Persistent tensors an optimizer maintains beyond weights and gradients, such as Adam first and second moments. [Explanation](#)
- **Option pointer.** A readout scoring candidate options supplied in the input. [Explanation](#)

- **Out-of-distribution.** Inputs that differ meaningfully from the conditions represented in training and evaluation. [Explanation](#)
- **Overfitting.** Improving fit to training examples without matching improvement on new cases. [Explanation](#)
- **Padding.** Placeholder positions added to make sequence shapes compatible. [Explanation](#)
- **Parameter.** An adjustable learned value in the model. [Explanation](#)
- **Parameter sharding.** Dividing stored training parameters among devices. [Explanation](#)
- **Partial fine-tuning.** Updating selected existing parameters while freezing the others; the ASR starter freezes its acoustic tower and does not use LoRA. [Explanation](#)
- **Patch merger.** Combination of neighboring features into fewer input units. [Explanation](#)
- **Pipeline parallelism.** Distributing successive layer groups across stages. [Explanation](#)
- **Polygon simplification.** Reducing boundary vertices while bounding an allowed geometric change. [Explanation](#)
- **Pooling.** Combining several positions into a smaller summary representation. [Explanation](#)
- **Precision.** The fraction of predicted positive cases that are actually positive. [Explanation](#)
- **Prediction contract.** A specification of input fields, target meaning, output format, timing, and failure handling. [Explanation](#)
- **Prefill.** Processing the supplied prefix or prompt. [Explanation](#)
- **Pretraining.** Learning an initial representation or generative model from a broad objective. [Explanation](#)
- **Pretraining from scratch.** Training a newly initialized model rather than adapting pretrained weights. [Explanation](#)
- **Probability calibration.** Agreement between stated probabilities and observed outcome frequencies. [Explanation](#)
- **Projector.** Map from encoder features to a backbone's feature width. [Explanation](#)
- **QAT.** Training that accounts for quantization effects. [Explanation](#)
- **QLoRA.** Training floating-point low-rank adapters through a frozen quantized language-model base. [Explanation](#)
- **Quantization.** Representing selected numeric values with fewer bits under a specific runtime format. [Explanation](#)
- **Query instruction.** A task description added to a retrieval query in the format expected by an instruction-aware embedding model. [Explanation](#)
- **Qwen3 embedding representation.** The selected Qwen encoder's last-token-pooled, normalized vector with model-specific query formatting. [Explanation](#)
- **R-squared.** A regression comparison against predicting the mean, whose interpretation depends on the evaluation data. [Explanation](#)
- **RAG.** Retrieval-augmented generation: supplying retrieved external evidence to a model at inference time. [Explanation](#)
- **Recall.** The fraction of actual positive cases that the system finds. [Explanation](#)

- **Recall@k.** A retrieval measure of relevant results found among the first k candidates under the task's relevance definition. [Explanation](#)
- **Recurrent state.** Persistent summary updated as tokens arrive. [Explanation](#)
- **Reference policy.** A fixed baseline distribution used to measure how the trained policy changes completion likelihoods. [Explanation](#)
- **Regression.** Predicting a numeric quantity rather than a discrete class. [Explanation](#)
- **Regularization.** Constraints or training choices intended to improve generalization rather than only training fit. [Explanation](#)
- **Reliability.** In calibration, the relationship between predicted probability bins and observed outcomes. [Explanation](#)
- **Reranker.** A second-stage model scoring a smaller candidate set more precisely. [Explanation](#)
- **Resampling.** Changing an audio sample grid with suitable filtering; downsampling must suppress frequencies above the new Nyquist limit. [Explanation](#)
- **Resident parameters.** Complete weights that must be stored somewhere. [Explanation](#)
- **Residual connection.** Adding an earlier representation to a learned transformation. [Explanation](#)
- **RMSE.** Root mean squared error, emphasizing larger errors while retaining target units. [Explanation](#)
- **ROC-AUC.** A score-ranking measure comparing positive and negative examples across thresholds. [Explanation](#)
- **RoPE.** Rotary positional embeddings: positional relationships represented by rotations in attention coordinates. [Explanation](#)
- **Router.** Learned function that selects experts for token representations. [Explanation](#)
- **Run manifest.** A saved record of model/data identity, configuration, versions and execution details for one experiment. [Explanation](#)
- **RVQ.** Several codebooks used successively for discrete signal representation. [Explanation](#)
- **Sample rate.** The number of waveform amplitude measurements per second; changing a file header is not resampling. [Explanation](#)
- **Seed.** An initial state for pseudo-random generation. [Explanation](#)
- **Semantic segmentation.** Assigning a class to each pixel without necessarily separating individual instances. [Explanation](#)
- **Sequence distillation.** Using teacher-produced complete output sequences as student training targets. [Explanation](#)
- **SFT.** Supervised fine-tuning on examples of desired output conditioned on the supplied input. [Explanation](#)
- **Shared expert.** Expert used for all tokens alongside selected experts. [Explanation](#)
- **Short convolution.** A sequence-mixing operation over a local window, used with input-dependent gates in LFM2. [Explanation](#)

- **Skip connection.** A route carrying earlier features to a later stage, often preserving detail. [Explanation](#)
- **Sparse attention.** Attention over a selected subset of positions. [Explanation](#)
- **Speaker diarization.** Estimating which speaker spoke when; a separate task from recognizing the spoken words. [Explanation](#)
- **Speculative decoding.** Draft proposals checked by a target model. [Explanation](#)
- **SSM.** Structured state-space sequence model. [Explanation](#)
- **Structured output.** An output governed by explicit fields, types, labels, or other schema constraints. [Explanation](#)
- **SWA.** Direct attention limited to a moving local window. [Explanation](#)
- **Target.** The desired output used to define correctness during training. [Explanation](#)
- **Target modules.** The specific network modules selected to receive trainable adapters. [Explanation](#)
- **Target scaling.** Transforming target values during training and reversing the transformation for output. [Explanation](#)
- **Teacher forcing.** Training predictions on a supplied correct history, rather than the model's own earlier generated mistakes. [Explanation](#)
- **Temperature in calibration.** A held-out-fitted logit scale used to improve probability calibration. [Explanation](#)
- **Temperature in contrastive learning.** A scale controlling the sharpness of contrastive candidate probabilities. [Explanation](#)
- **Temperature in generation.** A logit rescaling used to change a sampled token distribution. [Explanation](#)
- **Tensor.** A numeric array with shape, dtype, and device. [Explanation](#)
- **Tensor parallelism.** Splitting matrix operations across devices. [Explanation](#)
- **Test.** A held-out evaluation role used after development choices are fixed. [Explanation](#)
- **Test set.** Held-out data used after development choices are fixed. [Explanation](#)
- **TF-IDF.** Term frequency-inverse document frequency, a sparse representation weighting local term counts against corpus prevalence. [Explanation](#)
- **Threshold.** A selected cutoff converting a score into a decision. [Explanation](#)
- **Tied embeddings.** Reusing one parameter matrix for token input embeddings and vocabulary output projection. [Explanation](#)
- **TinyML.** Machine learning designed around tight device memory, compute, power, and latency limits. [Explanation](#)
- **Tokenization.** Converting text into the discrete token identifiers expected by a particular model. [Explanation](#)
- **Tokenizer.** A mapping between text and model token identifiers. [Explanation](#)
- **Tool observation.** The result returned by the external executor and supplied as context to the model. [Explanation](#)

- **Tool schema.** A structured specification of a tool name, purpose, argument types and constraints. [Explanation](#)
- **Top-k routing.** Selecting k expert assignments for each token. [Explanation](#)
- **Training from scratch.** Optimizing randomly initialized parameters rather than adapting pretrained weights. [Explanation](#)
- **Transfer learning.** Reusing a pretrained representation for a new task. [Explanation](#)
- **Triplet loss.** An objective comparing an anchor, a positive match, and a negative example. [Explanation](#)
- **Truncation.** Discarding input or target content beyond a length limit. [Explanation](#)
- **Validation.** An evaluation role used to compare candidates and select configuration or checkpoints. [Explanation](#)
- **Validation set.** Held-out data used to guide experiment choices. [Explanation](#)
- **Virtual environment.** An isolated Python package directory tied to a chosen interpreter. [Explanation](#)
- **Voice activity detection (VAD).** Predicting which audio regions likely contain speech, with a tradeoff between non-speech suppression and accidentally deleting quiet speech. [Explanation](#)
- **Weight decay.** A parameter-shrinking term applied by the optimizer. [Explanation](#)
- **Word error rate (WER).** Substitutions plus deletions plus insertions, divided by the number of reference words under a specified normalization/tokenization policy. [Explanation](#)

Works cited

Checked 2 October 2026 unless a source entry states otherwise. A cited implementation can explain a design without being a measured reproduction. Repository main branches can change; use the pinned versions and captured revisions in each project.

Foundation

- **F01.** PyTorch contributors. Autograd mechanics. Official reference on reverse automatic differentiation and computation graphs. <https://docs.pytorch.org/docs/stable/notes/autograd.html>
- **F02.** PyTorch contributors. AdamW reference, PyTorch 2.8. Parameter meanings and update algorithm. <https://docs.pytorch.org/docs/2.8/generated/torch.optim.AdamW.html>
- **F03.** Vaswani, A. and colleagues. Attention Is All You Need. 2017. The original Transformer architecture paper. <https://arxiv.org/abs/1706.03762>
- **F04.** Hugging Face. Model training anatomy, Transformers 4.48.0. A documented example of mixed-precision training-state accounting, not a universal bytes-per-parameter law. https://huggingface.co/docs/transformers/v4.48.0/model_memory_anatomy
- **F05.** PyTorch contributors. torch.utils.checkpoint. Activation checkpointing and recomputation behavior. <https://docs.pytorch.org/docs/stable/checkpoint.html>
- **F06.** PyTorch contributors. torch.cuda.max_memory_allocated. Peak tensor-allocation measurement. https://docs.pytorch.org/docs/stable/generated/torch.cuda.max_memory_allocated.html
- **F07.** PyTorch contributors. CUDA semantics. Allocated versus reserved memory and the limits of empty_cache. <https://docs.pytorch.org/docs/stable/notes/cuda.html>
- **F08.** Hoffmann, J. and colleagues. Training Compute-Optimal Large Language Models. 2022. Empirical scaling of parameters and data under compute budgets; not a universal prescription for every domain. <https://arxiv.org/abs/2203.15556>
- **F09.** scikit-learn developers. Common pitfalls and recommended practices. Train/test separation and preprocessing leakage. https://scikit-learn.org/1.9/common_pitfalls.html
- **F10.** PyTorch Foundation. PyTorch 2.8 Release Blog. 6 August 2025. Verified release used as the tiny-transformer reference API pin. <https://pytorch.org/blog/pytorch-2-8/>
- **F11.** PyTorch contributors. scaled_dot_product_attention, PyTorch 2.8. Causal masking, dropout behavior, and supported attention implementations. https://docs.pytorch.org/docs/2.8/generated/torch.nn.functional.scaled_dot_product_attention.html
- **F12.** PyTorch contributors. Automatic Mixed Precision examples. Gradient scaling, clipping order, and accumulation. https://docs.pytorch.org/docs/stable/notes/amp_examples.html
- **F13.** PyTorch contributors. CrossEntropyLoss. Logit targets, reduction, and ignore_index. <https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>
- **F14.** PyTorch contributors. Reproducibility. Seeds, deterministic operations, and cross-platform limitations. <https://docs.pytorch.org/docs/stable/notes/randomness.html>
- **F15.** PyTorch Foundation. Previous PyTorch Versions. Official wheel commands for CPU and CUDA variants, including torch 2.8.0 with torchvision 0.23.0 and torch 2.7.1 with torchvision 0.22.1. <https://pytorch.org/get-started/previous-versions/>

Tiny ML Embedding

- **M01.** scikit-learn 1.8, [Common pitfalls and recommended practices](#). Supports training-only preprocessing, pipelines, and consistent inference transformations

- **M02.** [scikit-learn 1.8](#), [Probability calibration](#). Supports reliability interpretation, independent calibration data, `FrozenEstimator`, sigmoid calibration, and limitations of Brier/log-loss interpretation
- **M03.** [scikit-learn 1.8](#), [HistGradientBoostingClassifier](#), and [LogisticRegression](#). Exact estimator controls for the CPU classification example
- **M04.** [scikit-learn 1.8](#), [HistGradientBoostingRegressor](#). Exact tree-regression controls
- **M05.** [PyTorch 2.8](#), [BCEWithLogitsLoss](#) and [CrossEntropyLoss](#). Logit/target conventions and stable loss APIs; the teaching Dice term is implemented explicitly in the accompanying script
- **M06.** [torchvision 0.23](#), [MobileNetV3-Small](#). Explicit weight enum and associated transforms
- **M07.** Ronneberger, Fischer, and Brox, [U-Net: Convolutional Networks for Biomedical Image Segmentation](#), 2015. Encoder-decoder/skip-connection source; the supplied tiny network is a pedagogical variation, not a reproduction claim
- **M08.** Cheng et al., [Boundary IoU: Improving Object-Centric Image Segmentation Evaluation](#), 2021; [original CVPR paper](#). Motivation for evaluating boundaries separately from region overlap. The included boundary F1 is a different, explicitly specified metric
- **M09.** OpenCV, [Structural Analysis and Shape Descriptors](#). `findContours`, hierarchy/retrieval modes, and polygon-approximation semantics. This older versioned reference documents the classic API used by the optional contour recipe
- **M10.** [PyTorch](#), [TorchVision Object Detection Finetuning Tutorial](#). Dataset contract, box/instance masks, label conventions, and pretrained Mask R-CNN fine-tuning. The tutorial states torchvision ≥ 0.16 for its current variant; follow the pinned 0.23 API when combining with this book
- **M11.** [torchvision BSD-3-Clause source license](#) and [README's separate dataset/model-weight licensing warnings](#). Permissive code does not settle training-data or pretrained-weight rights
- **M12.** [PyTorch 2.8](#), [Automatic Mixed Precision](#). `torch.autocast`, `torch.amp.GradScaler`, and deprecation of old AMP namespaces
- **M14.** [Sentence Transformers v5.1.1](#), [MultipleNegativesRankingLoss source](#). Exact `model`, `scale`, `similarity`, and `forward-call` behavior used in the example
- **M15.** [Sentence Transformers](#), [Samplers](#). Duplicate-avoidance rationale; the book's direct loop performs its own one-document-per-batch grouping
- **M16.** Gao et al., [Scaling Deep Contrastive Learning Batch Size under Memory Limited Setup](#), 2021, and [Sentence Transformers loss reference](#). Gradient-cache tradeoffs and cached contrastive options; no claim that all memory costs vanish
- **M17.** Radford et al., [Learning Transferable Visual Models From Natural Language Supervision](#), 2021; [arXiv version](#). Primary CLIP dual-encoder/multimodal contrastive example; no claim to reproduce its large-scale pretraining on one GPU
- **M18.** [Sentence Transformers](#), [Evaluation reference](#). Information-retrieval evaluator terminology and supported metrics; this book includes a small independent exact-search metric implementation
- **M19.** [PyTorch](#), [ExecuTorch project](#) and [How ExecuTorch works](#). Export, optimization, and target-runtime deployment workflow; no tested mobile artifact is supplied
- **M20.** [scikit-learn](#), [Cross-validation guide](#). Group-aware and time-aware evaluation principles; this source was the stable guide at verification, while the executed estimator version is 1.8.0
- **M21.** [torchvision](#), [official compatibility table](#). [PyTorch 2.8](#) pairs with [torchvision 0.23](#) and supports Python 3.9-3.13 according to the published table
- **M22.** Reimers and Gurevych, [Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks](#), 2019. Primary background reading on independently encoded sentence representations
- **M23.** Schroff, Kalenichenko, and Philbin, [FaceNet: A Unified Embedding for Face Recognition and Clustering](#), 2015. Primary triplet-learning background; the book discusses the objective, not a biometric deployment recommendation

- **M24.** scikit-learn, [Model persistence](#). Security and compatibility cautions for pickle-derived model artifacts
- **M25.** torchvision 0.23, [Faster R-CNN MobileNetV3-Large 320 FPN](#). Official architecture/weight interface; use a custom target dataset and review its rights separately
- **M26.** Sentence Transformers, [v5.1.1 release](#). The named version exists; this is an API target, not a tested environment lock
- **M28.** torchvision v0.23.0, [official detection training process and reference scripts](#). Published scripts, losses/evaluation support, and documented original training commands. Many original commands use eight GPUs; they are process references, not recipes promised to fit one 24 GB GPU unchanged
- **M29.** Sentence Transformers, [official embedding training examples](#). Public examples for retrieval, similarity, triplet, and other supervision styles; main-branch code can differ from the pinned v5.1.1 direct-loop recipe

Current Embedding

- **M30.** Qwen, [Qwen3-Embedding-0.6B official model card](#). Public/non-gated at verification, Apache-2.0 metadata, 0.6B family label, 1024-dimensional output, 32K stated context, minimum library requirements, and recommended query-only instructions. No leaderboard claim is used as an application-quality guarantee
- **M31.** Qwen, [pinned pooling configuration and released module graph](#). Last-token pooling, no mean pooling, 1024 features, and normalization module. [Sentence Transformers 5.1.1 Pooling implementation](#) shows last non-padding-token selection
- **M32.** Qwen, [official prompt configuration, pinned tokenizer configuration, and model repository file history](#). Query instruction/document empty-prefix distinction and native tokenizer artifacts; the lab supplies its own domain instruction in the documented format consistently across all paths
- **M33.** Qwen, [official architecture configuration and Transformers 4.57.1 Qwen3 implementation](#). Shape-based parameter calculation and Qwen3 backbone configuration. Derived count is 595,776,512 for this backbone, not a claim of author-executed loading; the program reports the actual loaded count
- **M34.** [SentenceTransformer v5.1.1 implementation](#), [MultipleNegativesRankingLoss v5.1.1 implementation](#), [Qwen3 v4.57.1 implementation](#), and [PyTorch 2.8 AMP API](#). Source-reviewed loader/tokenizer/forward/loss, SDPA-compatible model, and autocast APIs. This is not an empirically tested dependency lock
- **M35.** PyTorch, [official previous-version installation commands, v2.8.0](#). CPU/CUDA 12.8 wheel indexes and matching pinned release. Hardware/driver compatibility must still be established locally
- **M36.** Qwen, [verified immutable model revision 97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3 and repository files/sizes](#). Source identity and approximate BF16-weight download size
- **M37.** Qwen, [official Qwen3 Embedding repository and Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models](#). Current public model/process and original research references. The lab is a small full-fine-tuning example, not a reproduction of Qwen's original large-scale training

Llm Training

- **L01.** Transformers chat templates. URL: https://huggingface.co/docs/transformers/chat_templating Inspected: Current documentation; role serialization, apply_chat_template, training/inference generation-prompt and duplicate-special-token cautions
- **L02.** Qwen3-0.6B official model card. URL: <https://huggingface.co/Qwen/Qwen3-0.6B> Inspected: Model Overview; Quickstart; thinking/non-thinking mode; Apache-2.0. Card says 0.6B, 28 layers, 16 Q/8 KV heads and 32,768 context. No quality benchmark copied
- **L03.** Qwen3-0.6B official Hub metadata. URL: <https://huggingface.co/api/models/Qwen/Qwen3-0.6B> Inspected: Inspected sha c1899de289a04d12100db370d81485cdf75e47ca, tokenizer template, 751,632,384 serialized BF16 entries. This is not a unique trainable-parameter count

- **L04.** Transformers Trainer tagged implementation. URL: <https://raw.githubusercontent.com/huggingface/transformers/v5.18.0/src/transformers/trainer.py> Inspected: v5.18.0; constructor processing_class and optimizer_cls_and_kwargs; causal-LM shifted target counting; quantized model placement; loss-only evaluation design
- **L05.** TRL supervised fine tuning trainer. URL: https://huggingface.co/docs/trl/v1.14.1/sft_trainer Inspected: v1.14.1; expected dataset formats; completion/assistant masks; training templates; tool columns; default chunked_nll. Used for API review, not a copied runnable benchmark
- **L06.** TRL SFTConfig tagged implementation. URL: https://raw.githubusercontent.com/huggingface/trl/v1.14.1/trl/trainer/sft_config.py Inspected: v1.14.1; max_length, assistant_only_loss, completion_only_loss, padding/packing fields and loss_type
- **L07.** Transformers releases. URL: <https://github.com/huggingface/transformers/releases> Inspected: Observed v5.18.0 release; tagged source inspected separately. Release existence does not prove installed compatibility
- **L08.** Accelerate releases. URL: <https://github.com/huggingface/accelerate/releases> Inspected: Observed v1.15.0 release and release notes; used only to identify review pin
- **L09.** PEFT releases. URL: <https://github.com/huggingface/peft/releases> Inspected: Observed v0.21.2 release, including Transformers 5.18 compatibility fix described for encoder-decoder models
- **L10.** bitsandbytes releases. URL: <https://github.com/bitsandbytes-foundation/bitsandbytes/releases> Inspected: Observed stable 0.50.2, distinguished from continuous prerelease wheel
- **L11.** TRL releases. URL: <https://github.com/huggingface/trl/releases> Inspected: Observed v1.14.1; reviewed tagged implementation and metadata rather than relying on latest docs alone
- **L12.** Datasets releases. URL: <https://github.com/huggingface/datasets/releases> Inspected: Observed 5.0.1 release; optional TRL environment pin
- **L13.** Official previous PyTorch versions. URL: <https://pytorch.org/get-started/previous-versions/> Inspected: Inspected v2.12.1 CUDA 12.6 wheel command; this is a chosen reproducible release, not a claim that it is newest. No installation performed
- **L14.** Hu et al LoRA. URL: <https://arxiv.org/html/2106.09685v2> Inspected: arXiv v2, 2021; low-rank reparameterization and frozen pretrained weights. Matrix example in book calculated independently
- **L15.** PEFT LoRA API reference. URL: https://huggingface.co/docs/peft/v0.21.0/package_reference/lora Inspected: Official docs resolved to v0.21.0; LoraConfig rank, alpha, target_modules, all-linear behavior. Code pins patch release 0.21.2
- **L16.** PyTorch AdamW API. URL: <https://docs.pytorch.org/docs/2.12/generated/torch.optim.AdamW.html> Inspected: PyTorch 2.12; moment updates, optimizer options and foreach memory caveat. Book memory rows state their own precision assumptions
- **L17.** PyTorch automatic mixed precision. URL: <https://docs.pytorch.org/docs/2.12/amp.html> Inspected: PyTorch 2.12; autocast changes selected compute precision, distinct from stored parameter dtype
- **L18.** Dettmers et al QLoRA. URL: <https://arxiv.org/html/2305.14314v1> Inspected: arXiv v1, 2023; frozen four-bit base, NF4, double quantization and paged optimizers. No paper hardware result represented as a local measurement
- **L19.** PEFT quantization guide. URL: https://huggingface.co/docs/peft/v0.21.0/developer_guides/quantization Inspected: v0.21.0; BitsAndBytesConfig preparation and prepare_model_for_kbit_training before attaching adapters
- **L20.** Transformers bitsandbytes integration. URL: <https://huggingface.co/docs/transformers/quantization/bitsandbytes> Inspected: Current official docs; NF4, compute dtype, double quantization, extra-parameter training restriction and memory footprint
- **L21.** Lewis et al Retrieval Augmented Generation. URL: <https://arxiv.org/abs/2005.11401> Inspected: Abstract and publication metadata; parametric generation combined with non-parametric retrieval memory

- **L22.** Gururangan et al Do not Stop Pretraining. URL: <https://arxiv.org/abs/2004.10964> Inspected: Abstract and publication metadata; domain/task adaptation in studied settings. Not treated as a universal modern-decoder guarantee
- **L23.** Meng et al Locating and Editing Factual Associations in GPT. URL: <https://arxiv.org/abs/2202.05262> Inspected: Abstract and metadata; ROME factual-association editing scope. No unsupported empirical locality rate used
- **L24.** Meng et al Mass Editing Memory in a Transformer. URL: <https://arxiv.org/abs/2210.07229> Inspected: Abstract and metadata; MEMIT many-edit research scope. Engineering cautions in book are recommendations, not claimed study results
- **L25.** Transformers tool use templates. URL: https://huggingface.co/docs/transformers/chat_extras Inspected: Current official docs; structured schemas, tool_calls and tool-role observations, model-specific serialization
- **L26.** TRL dataset formats and types. URL: https://huggingface.co/docs/trl/dataset_formats Inspected: Current official docs; conversational, preference and tool-calling structures. Local fixtures are original fictional data
- **L27.** Berkeley Function Calling Leaderboard. URL: <https://gorilla.cs.berkeley.edu/leaderboard.html> Inspected: Inspected V4 methodology links; snapshot says models evaluated at f7cf735 and bfcl-eval==2025.12.17. No leaderboard number reported as an author measurement
- **L28.** BFCL multi-turn methodology. URL: https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html Inspected: Official BFCL-v3 multi-turn/multi-step description; used to motivate separation of call syntax and interaction outcomes
- **L29.** Rafailov et al Direct Preference Optimization. URL: <https://arxiv.org/abs/2305.18290> Inspected: arXiv v3 metadata and abstract; reference-relative pairwise objective checked against TRL math in L30
- **L30.** TRL DPO trainer. URL: https://huggingface.co/docs/trl/v1.14.1/dpo_trainer Inspected: v1.14.1; prompt/chosen/rejected records, equation, reference model and precomputed reference log probabilities
- **L31.** Hinton et al Distilling the Knowledge in a Neural Network. URL: <https://arxiv.org/abs/1503.02531> Inspected: Abstract and metadata; teacher distributions and distillation origin. Temperature explanation is standard derivation, not a reproduced result
- **L32.** Kim and Rush Sequence Level Knowledge Distillation. URL: <https://arxiv.org/abs/1606.07947> Inspected: Abstract and metadata; sequence-level teacher targets distinguished from logit matching
- **L33.** TRL Distillation Trainer. URL: https://huggingface.co/docs/trl/distillation_trainer Inspected: Current official docs; DistillationTrainer and on/off-policy modes. No untested trainer code presented as a measured run
- **L34.** Qwen3-0.6B official config. URL: <https://huggingface.co/Qwen/Qwen3-0.6B/blob/main/config.json> Inspected: Inspected config at main, last config change shown 167b810; head_dim=128, hidden=1024, intermediate=3072, 28 layers, vocab=151936, tied embeddings. Full model pinned via L03
- **L35.** Qwen3-1.7B official model card. URL: <https://huggingface.co/Qwen/Qwen3-1.7B> Inspected: Model Overview and license; nominal 1.7B, 28 layers, 16 Q/8 KV GQA heads and 32,768 published context
- **L36.** Qwen3.5-0.8B official model card. URL: <https://huggingface.co/Qwen/Qwen3.5-0.8B> Inspected: Model Overview; LM parameters separate from vision encoder, 24 layers, DeltaNet/gated attention layout, padded vocabulary 248320; Apache-2.0
- **L37.** Qwen3.5-2B official model card. URL: <https://huggingface.co/Qwen/Qwen3.5-2B> Inspected: Model Overview; 2B LM label, vision encoder, 24 layers, hybrid layout. No benchmark transferred to local hardware
- **L38.** Google Gemma 3 270M release article. URL: <https://developers.googleblog.com/introducing-gemma-3-270m/> Inspected: Core capabilities; 270M total, 170M embedding/100M transformer parameters and task-specific intent. Phone-energy result deliberately not generalized
- **L39.** Gemma 3 1B instruction model card. URL: <https://huggingface.co/google/gemma-3-1b-it> Inspected: Model Information, access conditions, context, license label. Family-level multimodal text not applied indiscriminately to 1B

- **L40.** Transformers Gemma 3 documentation. URL: https://huggingface.co/docs/transformers/model_doc/gemma3 Inspected: Architecture and text-model class documentation; small text-only path distinguished from larger vision-language models
- **L41.** Granite 4.0 350M official model card. URL: <https://huggingface.co/ibm-granite/granite-4.0-350m> Inspected: Model Architecture table; dense baseline, 28 attention layers, GQA, 350M; Apache-2.0. Card metadata tag is not used as architecture authority
- **L42.** Granite 4.0 H 350M official model card. URL: <https://huggingface.co/ibm-granite/granite-4.0-h-350m> Inspected: Model Architecture; 340M total hybrid, attention and Mamba2. No claim of a tested CUDA kernel path
- **L43.** Granite 4.0 1B official model card. URL: <https://huggingface.co/ibm-granite/granite-4.0-1b> Inspected: Architecture table gives 1.6B dense and 1.5B hybrid, despite shortened names. Used to flag strict parameter ceilings
- **L44.** Granite 3.3 2B instruction model card. URL: <https://huggingface.co/ibm-granite/granite-3.3-2b-instruct> Inspected: Model Summary, Apache-2.0 and intended domain/tool/RAG use. Not described as latest Granite
- **L45.** Granite 3.3 2B official config. URL: <https://huggingface.co/ibm-granite/granite-3.3-2b-instruct/blob/main/config.json> Inspected: Inspected 40 layers, hidden=2048, intermediate=8192, Q=32/KV=8, vocab=49159 and tied embeddings
- **L46.** SmolLM2 360M instruction model card. URL: <https://huggingface.co/HuggingFaceTB/SmolLM2-360M-Instruct> Inspected: Official model card and training-resource links; small checkpoint candidate
- **L47.** Meta Llama 3.2 1B instruction model card. URL: <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct> Inspected: Model information, GQA and license label. License/gating require checkpoint-specific review; no claim of downloaded gated weights
- **L48.** Granite 4.2 3B official model card. URL: <https://huggingface.co/ibm-granite/granite-4.2-3b> Inspected: Model Summary/Design; August 25 2026, dense reasoning model and nominal 3B. Exact total cap not asserted
- **L49.** SmolLM3 3B official model card. URL: <https://huggingface.co/HuggingFaceTB/SmolLM3-3B> Inspected: Training/software/hardware sections; 11T pretraining tokens and 384 H100. Used as published process scale, not practical full-training project
- **L50.** Qwen3 30B A3B official model card. URL: <https://huggingface.co/Qwen/Qwen3-30B-A3B> Inspected: Model Overview; 30.5B total, 3.3B activated, 128 experts, 8 activated. Excluded from practical scope
- **L51.** Gemma 4 E2B instruction model card. URL: <https://huggingface.co/google/gemma-4-E2B-it> Inspected: Dense-model table and license; 2.3B effective, 5.1B with embeddings, per-layer embeddings, Apache-2.0. Excluded from strict total scope
- **L52.** SmolLM published fine-tuning script. URL: <https://raw.githubusercontent.com/huggingface/smollm/main/text/finetuning/train.py> Inspected: Main as inspected; args/defaults and SFTConfig call. Verified max_seq_length, default push_to_hub=True and report_to=wandb. No immutable repository SHA was resolved; do not label this source pinned
- **L53.** SmolLM public pretraining process. URL: <https://github.com/huggingface/smollm/blob/main/text/pretraining/README.md> Inspected: Main as inspected; launch example, intra-document masks, configuration paragraph, 2.36M global-token batch, 384 H100 for 24 days. Linked logs not re-run
- **L54.** Hoffmann et al Training Compute Optimal Large Language Models. URL: <https://arxiv.org/html/2203.15556v1> Inspected: arXiv v1; section 3 and efficient-frontier 6ND approximation, tables/context. Book 1B/20B-token/50TFLOP example is explicitly hypothetical
- **L55.** LFM2 350M official model card. URL: <https://huggingface.co/LiquidAI/LFM2-350M> Inspected: Model details table with exact counts; hybrid blocks, custom license, tool format, link to newer LFM2.5-350M
- **L56.** LFM2.5 350M official model card. URL: <https://huggingface.co/LiquidAI/LFM2.5-350M> Inspected: Model Details, Tool Use, Fine-Tuning, exported formats; 16 blocks, 32K context, vocab=65536, custom LFM1.0 label. Vendor speed claims not reused as local measurements

- **L57.** Liquid official TRL fine-tuning guide. URL: <https://docs.liquid.ai/lfm/fine-tuning/trl> Inspected: LoRA Fine-Tuning and full-update examples; attention projections explicitly targeted; inspected old tokenizer keyword and minimum-version install string. Current API correction comes from L05
- **L58.** Transformers LFM2 model documentation. URL: https://huggingface.co/docs/transformers/model_doc/lfm2 Inspected: Native Lfm2ForCausalLM with input_ids, labels, attention_mask and cache arguments; proves supported model interface, not all-backend training success
- **L59.** Liquid official llama.cpp deployment guide. URL: <https://docs.liquid.ai/deployment/on-device/llama-cpp> Inspected: GGUF downloading, CPU-first execution, CLI/server and GPU-offload distinctions; model-specific existing export route, not proof of arbitrary adapter conversion
- **L60.** LFM2 technical report. URL: <https://arxiv.org/abs/2511.23404> Inspected: arXiv v1 abstract/metadata; hybrid gated-short-convolution/GQA design and hardware-in-the-loop search. No claimed reproduction of reported CPU gains

Jev Architecture

- **J01.** TypeSafe's announcement. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>
- **J02.** official API. <https://docs.typesafe.ai/api>
- **J03.** Kev. <https://github.com/jaredpalmer/kev>
- **J04.** architecture hypothesis and its caveats. <https://archerhume.com/posts/jevs-architecture-unmasked>
- **J05.** CLM. <https://github.com/Contrastive-LM/CLM>
- **J06.** Official configuration. <https://huggingface.co/Qwen/Qwen3-0.6B/blob/main/config.json>
- **J07.** RoFormer paper. <https://arxiv.org/abs/2104.09864>
- **J08.** Transformer paper. <https://arxiv.org/abs/1706.03762>
- **J09.** GQA paper. <https://arxiv.org/abs/2305.13245>
- **J10.** Official implementation. https://github.com/huggingface/transformers/blob/main/src/transformers/models/qwen3/modeling_qwen3.py
- **J11.** GLU variants paper. <https://arxiv.org/abs/2002.05202>
- **J12.** RMSNorm paper. <https://arxiv.org/abs/1910.07467>
- **J13.** Kev model source. <https://github.com/jaredpalmer/kev/blob/main/kev/model.py>
- **J14.** Official Qwen3.5 configuration. <https://huggingface.co/Qwen/Qwen3.5-0.8B-Base/blob/main/config.json>
- **J15.** Kev parity tests. https://github.com/jaredpalmer/kev/blob/main/tests/test_model.py
- **J16.** KV-cache explanation. https://huggingface.co/docs/transformers/main/en/llm_tutorial_optimization
- **J17.** Temperature-scaling paper. <https://arxiv.org/abs/1706.04599>
- **J18.** CLM head implementation. <https://github.com/Contrastive-LM/CLM/blob/main/src/clm/heads.py>
- **J19.** CLM scoring and cache implementation. <https://github.com/Contrastive-LM/CLM/blob/main/src/clm/engine.py>
- **J20.** Contrastive predictive coding and InfoNCE. <https://arxiv.org/abs/1807.03748>
- **J21.** CLM training implementation. <https://github.com/Contrastive-LM/CLM/blob/main/train/finetune.py>
- **J22.** Granite 4.2-3B model card. <https://huggingface.co/ibm-granite/granite-4.2-3b>
- **J23.** Kev-0.8B model card. <https://huggingface.co/jaredpalmer/kev-0.8b>
- **J24.** Laya official model card. <https://huggingface.co/convaiinnovations/laya>
- **J25.** Laya original repository. <https://github.com/NandhaKishorM/laya>

- **J26.** Laya shipped model implementation.
https://huggingface.co/convaiinnovations/laya/blob/main/rl_common.py
- **J27.** Laya shipped configuration. https://huggingface.co/convaiinnovations/laya/blob/main/rl_agent_config.json
- **J28.** Laya single-process MPS trainer.
https://github.com/NandhaKishorM/laya/blob/main/notebooks/laya_finetune_typed_decisions_mps.py
- **J29.** Laya Kaggle 2xT4 notebook.
https://github.com/NandhaKishorM/laya/blob/main/notebooks/laya_finetune_typed_decisions_2xT4_kaggle.ipynb
- **J30.** Laya measured browser-agent adaptation.
https://github.com/NandhaKishorM/laya/blob/main/docs/finetune_browser_agent.md
- **J31.** ModernBERT paper. <https://arxiv.org/abs/2412.13663>
- **J32.** ModernBERT-large configuration.
<https://huggingface.co/answerdotai/ModernBERT-large/blob/main/config.json>
- **J33.** Ten Levels of Jev original repository. <https://github.com/disler/ten-levels-of-jev>
- **J34.** Ten Levels provider client.
<https://github.com/disler/ten-levels-of-jev/blob/main/apps/ten-levels/src/core/client.ts>
- **J35.** Ten Levels confidence thresholds.
<https://github.com/disler/ten-levels-of-jev/blob/main/apps/ten-levels/src/levels/level04/confidence.ts>
- **J36.** Ten Levels application README.
<https://github.com/disler/ten-levels-of-jev/blob/main/apps/ten-levels/README.md>
- **J37.** Ten Levels of Jev video. https://www.youtube.com/watch?v=_U-O5lYhJ7Q
- **J38.** Karpathy: build GPT in code. <https://www.youtube.com/watch?v=kCc8FmEb1nY>
- **J39.** nanoGPT source (historical, now deprecated). <https://github.com/karpathy/nanoGPT>

Diffusion Training

- **D01.** Ho, Jain and Abbeel, Denoising Diffusion Probabilistic Models (2020). <https://arxiv.org/abs/2006.11239> - foundational Gaussian forward corruption, noise-prediction training and reverse generation. Used for the short objective explanation; our waveform code is an original teaching implementation, not a reproduction of their benchmark results.
- **D02.** Song et al., Score-Based Generative Modeling through Stochastic Differential Equations (2020/2021). <https://arxiv.org/abs/2011.13456> - score interpretation and reverse SDE/probability-flow viewpoint. No paper benchmark is presented as a result of our scripts.
- **D03.** Lipman et al., Flow Matching for Generative Modeling (2022/2023). <https://arxiv.org/abs/2210.02747> - vector-field regression along probability paths; distinguishes flow matching from a particular epsilon-prediction recipe.
- **D04.** Ho and Salimans, Classifier-Free Diffusion Guidance (2022). <https://arxiv.org/abs/2207.12598> - conditional/unconditional predictions and inference-time quality/diversity trade-off. The chapter clearly states its guidance-scale convention.
- **D05.** Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models (2021/2022). <https://arxiv.org/abs/2112.10752> - separates an autoencoder representation from conditional latent diffusion.
- **D06.** Peebles and Xie, Scalable Diffusion Models with Transformers (2022/2023). <https://arxiv.org/abs/2212.09748> - transformer backbone on latent patches; architecture is distinct from output target.
- **D07.** Ruiz et al., DreamBooth (2022). <https://arxiv.org/abs/2208.12242> - subject-driven personalization and class-specific prior preservation; not an alternative mathematical definition of LoRA.

- **D17.** Nichol and Dhariwal, Improved Denoising Diffusion Probabilistic Models (2021). <https://arxiv.org/abs/2102.09672> - source for the cosine cumulative-noise schedule idea. The supplied code fixes $s=0.008$ and clips beta at 0.999; it does not implement every improvement in the paper.
- **D18.** Kong et al., DiffWave: A Versatile Diffusion Model for Audio Synthesis (2020/2021). <https://arxiv.org/abs/2009.09761> - waveform diffusion and conditional/unconditional audio synthesis. Our U-Net architecture is explicitly not described as DiffWave.
- **D08.** Hugging Face Diffusers, LoRA training guide. <https://huggingface.co/docs/diffusers/training/lora> - general parameter-efficient adaptation integration. The current project follows the Sana-specific code below, not this page's legacy SD1.5 command.
- **D16.** Hugging Face Datasets v3.6.0, Create an image dataset. https://huggingface.co/docs/datasets/v3.6.0/en/image_dataset - imagefolder and JSONL metadata relationship. Our extra `group` field and split discipline are deliberate project design, not a feature that automatically prevents leakage in the upstream trainer.
- **D19.** Stable Audio Open 1.0 official card. <https://huggingface.co/stabilityai/stable-audio-open-1.0> - 47-second, 44.1 kHz stereo limit; autoencoder, T5 and latent DiT components; library usage. Research paper: <https://arxiv.org/abs/2407.14358> . These are model characteristics, not a reproduced 24 GB fine-tuning result.
- **D20.** Stable Audio Open Small official card. <https://huggingface.co/stabilityai/stable-audio-open-small> - 11-second, 44.1 kHz stereo model and distilled inference recipe. Related paper: <https://arxiv.org/abs/2505.08175> . The book does not invent a generic LoRA training flag for this checkpoint.
- **D21.** Stability AI stable-audio-tools official repository. <https://github.com/Stability-AI/stable-audio-tools> - separate model/dataset configuration and training machinery. Current README: <https://raw.githubusercontent.com/Stability-AI/stable-audio-tools/main/README.md> . Presence of general training code is not a validated memory estimate or proof of a particular distilled model's adapter support.
- **D22.** Stability AI Stable Audio 3 official repository. <https://github.com/Stability-AI/stable-audio-3> - model variants, fine-tuning support and published inference performance. README inspected: <https://raw.githubusercontent.com/Stability-AI/stable-audio-3/main/README.md> . The VRAM table labels H200 inference with unchunked decoding; it is not a consumer-GPU training table. Technical report: <https://arxiv.org/abs/2605.17991> .
- **D23.** Stable Audio 3 official LoRA guide. <https://github.com/Stability-AI/stable-audio-3/blob/main/docs/workflows/lora.md> - confirms adapter workflow. Approximate memory table and a separate reduced-memory example are insufficiently uniform to treat as reproduced peak training measurements. No quoted training throughput is claimed.
- **D24.** Stable Audio 3 actual training script. https://github.com/Stability-AI/stable-audio-3/blob/main/scripts/train_lora.py - inspected parser and training code, including base-model-only loading; raw audio plus `.txt` metadata; duration, rank, batch, local CSV logging; and demo sample-size handling. `--save_dir` is the actual parser name at the inspected source; do not blindly copy prose mentioning `--output_dir`. Its full dependency stack was not installed or executed for the book.
- **D25.** Stable Audio 3 dependency specification. <https://github.com/Stability-AI/stable-audio-3/blob/main/pyproject.toml> - pins torch/torchaudio 2.7.1 and has its own modern Transformers dependencies. Avoid mixing this environment with the image project's pins.
- **D26.** Stable Audio 3 Small SFX official model card. <https://huggingface.co/stabilityai/stable-audio-3-small-sfx> - describes the family, base relationship, T5Gemma conditioning and additional Gemma terms. Note that the card's lower-level sample has a CPU-path variable issue (`model_half` is not initialized in the CPU branch); the book does not copy that snippet or claim parity with the higher-level API.
- **D27.** PyTorch previous versions, v2.7.1. <https://pytorch.org/get-started/previous-versions/> - official torch 2.7.1 / torchvision 0.22.1 pairing and CUDA 12.6 wheel index. Driver compatibility and actual device availability remain machine-specific.
- **D28.** Original DDPM implementation. <https://github.com/hojonathanho/diffusion> - primary author repository. Its README specifies TensorFlow 1.15, Python 3.5 and TPU experiments. Read to connect the paper with its historical implementation; do not present its installation as a modern single-RTX recipe.

- **D29.** DiffWave reference implementation. <https://github.com/lmnt-com/diffwave> - public waveform/vocoder training and inference implementation. Useful next reading after the toy class-conditioned example; a mel-conditioned vocoder's data contract is different from text-conditioned sound generation.
- **D30.** Official Sana 1.6B BF16 model card. https://huggingface.co/Efficient-Large-Model/Sana_1600M_1024px_BF16_diffusers - exact primary checkpoint, 1.648B image transformer, Gemma2-2B-IT text encoding, 32× DC-AE compression, component-specific dtypes, Apache/Gemma terms and research-oriented intended use. The complete pipeline is explicitly larger than the transformer alone.
- **D31.** Actual Diffusers v0.36.0 Sana LoRA trainer. https://github.com/huggingface/diffusers/blob/v0.36.0/examples/dreambooth/train_dreambooth_lora_sana.py - inspected argument parser, local caption-dataset path, mixed component precision, offload, flow target noise - model_input, checkpoint hooks and resume behavior. Raw inspected source: https://raw.githubusercontent.com/huggingface/diffusers/v0.36.0/examples/dreambooth/train_dreambooth_lora_sana.py . Important code findings: cache-by-batch-position plus shuffled loader can mismatch per-image captions; our command disables that cache. Final pipeline loading is unconditional in upstream even without requested validation; our guarded patch skips that unused allocation after adapter save. Resume restores training state but does not exactly skip to the old next minibatch.
- **D32.** Actual Diffusers v0.36.0 Sana inference pipeline. https://github.com/huggingface/diffusers/blob/v0.36.0/src/diffusers/pipelines/sana/pipeline_sana.py - verified SanaPipeline, LoRA loading mixin, CPU-offload sequence, prompt embeddings and masks, 32-divisible dimensions, sampling inputs and decoder postprocessing. Training and inference explicitly agree on max_sequence_length=128 and complex_human_instruction=None instead of inheriting the pipeline's different instruction-prefix default.
- **D33.** Chen et al., SANA: Efficient High-Resolution Image Synthesis with Linear Diffusion Transformers (2024/ICLR 2025). <https://arxiv.org/abs/2410.10629> - efficient architecture and representation design. Its reported inference hardware/result is not represented as a measured fine-tuning result for the book.
- **D34.** Diffusers v0.36.0 dependency table. https://github.com/huggingface/diffusers/blob/v0.36.0/src/diffusers/dependency_versions_table.py - checked PEFT/Hub/Transformers requirements against the explicit candidate pins. Actual installation/import tests remain required.
- **D35.** Official maintained NVlabs Sana repository. <https://github.com/NVlabs/Sana> - current family, training/inference documentation and model variants. It is actively maintained beyond the original paper; the book selects a stable checkpoint with an explicit fine-tuning route rather than claiming the newest family release automatically has the same trainer.
- **D36.** Official Diffusers Sana DreamBooth/LoRA guide. https://github.com/huggingface/diffusers/blob/main/examples/dreambooth/README_sana.md - expressly targets the selected BF16 1.6B checkpoint, shows training, and documents offload/cache/optimizer options. The runnable book script is pinned to v0.36.0, not this moving document. No exact consumer-24GB training peak is stated or inferred.
- **D37.** Official Sana 600M 512-pixel card. https://huggingface.co/Efficient-Large-Model/Sana_600M_512px_diffusers - maintained 590M-transformer alternative, distinct FP16 variant, Gemma and DC-AE companions. It is a possible later experiment, not a drop-in equivalent of the BF16 project or a claim that the full pipeline has only 590M parameters.

Cohere Asr

- **S01.** Qwen3-ASR-0.6B official card. URL: <https://huggingface.co/Qwen/Qwen3-ASR-0.6B> Inspected: 2026 release identity, Apache-2.0, language identification and 30-language/22-dialect coverage, separate forced aligner, inference wrapper, submodel versus whole-model size distinction. No vendor throughput result is adopted as a consumer-GPU measurement.

- **S02.** Qwen model metadata and immutable revision. URL: <https://huggingface.co/api/models/Qwen/Qwen3-ASR-0.6B> Inspected through authorized HTTPS fetch: revision 5eb144179a02acc5e5ba31e748d22b0cf3e303b0; safetensors metadata lists 938,008,576 BF16 parameters. Snapshot contains one model.safetensors plus config/processor/tokenizer files. Serialized count may differ from a deduplicated runtime parameter count; training prints the latter. Immutable card: <https://huggingface.co/Qwen/Qwen3-ASR-0.6B/blob/5eb144179a02acc5e5ba31e748d22b0cf3e303b0/README.md>
- **S03.** Qwen3-ASR technical report. URL: <https://arxiv.org/abs/2601.21337> Inspected: official report abstract and model-family scope; no replication of the training corpus or benchmarks is claimed.
- **S04.** Cohere Transcribe 03-2026 official card. URL: <https://huggingface.co/CohereLabs/cohere-transcribe-03-2026> Inspected: 2B, Apache-2.0, 14 languages, observed contact-sharing gate, native Transformers >=5.4.0 route, reported PyTorch 2.10.0 testing, missing timestamps/diarization, language and non-speech limitations. No access gate accepted. Weights not downloaded.
- **S05.** Cohere technical announcement. URL: <https://cohere.com/blog/transcribe> Inspected: Conformer encoder + Transformer decoder, waveform/log-Mel input, supervised token cross-entropy and from-scratch original training. These are model characteristics, not evidence that full training fits 24 GB.
- **S06.** Cohere Arabic official card and release note. URLs: <https://huggingface.co/CohereLabs/cohere-transcribe-arabic-07-2026> ; <https://docs.cohere.com/changelog/transcribe-arabic> Inspected: official 2B Arabic/English fine-tune and Apache-2.0; introduction's code-switch optimization claim versus remaining code-switch caveat; observed access gate. No interpretation turns those caveats into a guarantee.
- **S07.** Cohere Embed and Rerank official current catalogs. URLs: <https://docs.cohere.com/docs/cohere-embed> ; <https://docs.cohere.com/docs/rerank> ; <https://docs.cohere.com/docs/models> Inspected: Embed v5.0 pro/fast and Rerank v4.0 pro/fast names and task roles. Public sources inspected here do not establish a downloadable training checkpoint for these exact products. No API account was created or called.
- **S08.** North Micro Vision Instruct official card. URL: <https://huggingface.co/CohereLabs/North-Micro-Vision-Instruct> Inspected: 2.4B total = 2B language + 400M vision; Apache-2.0; model-specific Transformers 5.16.0 support and linked training recipes. Used only to place the model in the right category and avoid treating a 2B language-backbone label as a total count.
- **S09.** Tiny Aya official card. URL: <https://huggingface.co/CohereLabs/tiny-aya-global> Inspected: explicit model-summary count 3.35B and CC-BY-NC-4.0. The card contains inconsistent boilerplate elsewhere; we rely on the model summary, not the erroneous 111B sentence in its terms section. No license permission beyond the card is inferred.
- **S10.** Command A+ official card. URL: <https://huggingface.co/CohereLabs/command-a-plus-05-2026-bf16> Inspected: 218B total/25B active sparse MoE and Apache-2.0. Active parameters are not a storage or optimizer-memory estimate. Large-family conceptual comparison only.
- **S11.** Conformer foundational paper. URL: <https://arxiv.org/abs/2005.08100> Inspected: convolution-augmented attention architecture concept. Historical reference for the mechanism, not an outdated checkpoint recipe.
- **S12.** Native Cohere ASR source at Transformers 5.4.0. URL: https://raw.githubusercontent.com/huggingface/transformers/v5.4.0/src/transformers/models/cohere_asr/modeling_cohere_asr.py Inspected: conditional generation forward accepts labels, shifts decoder inputs, computes supervised loss, and absorbs audio_chunk_index for generation. Trainability is source-supported; no fine-tune recipe or measured memory is claimed.
- **S13.** Official Cohere fine-tuning repository. URL: <https://github.com/cohere-ai/cohere-finetune> Inspected: supported base-model list and text-model LoRA/QLoRA scope. It does not establish Transcribe support. Mutable main inspected, not installed.
- **S14.** SciPy audio resampling. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.resample_poly.html Inspected: filtering and rational resampling semantics. Actual authoring tests ran SciPy 1.17.0; candidate training requirements pin 1.15.3. The project uses stable resample_poly and scipy.io.wavfile APIs, but the candidate environment was not installed.

- **S15.** Qwen model implementation at pinned repository revision. URL: https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/qwen_asr/core/transformers_backend/modeling_qwen3_asr.py Inspected: thinker.audio_tower; audio embeddings replace placeholders; thinker.forward accepts labels/use_cache; causal loss; SDPA and gradient-checkpointing support; outer class primarily exposes generation. Original project calls thinker directly rather than copying the upstream monkey-patch.
- **S16.** Qwen processor implementation at pinned revision. URL: https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/qwen_asr/core/transformers_backend/processing_qwen3_asr.py Inspected: chat template, audio-placeholder expansion, input_features/feature_attention_mask, default left padding. The project's one-example microbatch and prefix assertion deliberately avoid padded-prefix ambiguity.
- **S17.** CTC paper. URL: https://www.cs.toronto.edu/~graves/icml_2006.pdf Inspected: original connectionist temporal classification construction, blank labels and sum over valid alignments. Historical objective contrast only; the chapter does not claim its Qwen or Cohere loop uses CTC.
- **S18.** Transformers 4.57.6 causal loss implementation. URL: https://raw.githubusercontent.com/huggingface/transformers/v4.57.6/src/transformers/loss/loss_utils.py Inspected: causal label shifting, ignored -100 labels and mean loss. Project accumulates losses weighted by supervised shifted target count, rather than averaging unequal-length examples equally.
- **S19.** Official Qwen ASR fine-tuning guide/source. URLs: <https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/finetuning/README.md> ; https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/finetuning/qwen3_asr_sft.py Inspected: audio/text JSONL, language header, processor-driven supervised training and checkpoint support. This book provides an original bounded loop, not a verbatim copy or a claim that the upstream defaults fit 24 GB.
- **S20.** Official Qwen inference wrapper and audio utilities. URLs: https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/qwen_asr/inference/qwen3_asr.py ; https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/qwen_asr/inference/utils.py Inspected: from_pretrained kwargs; local paths; CPU device fallback; transcribe(language=..., return_time_stamps=False); 16 kHz normalization; optional separate forced aligner. CPU/GPU examples remain source-reviewed, not runtime-tested.
- **S21.** Qwen package metadata and repository commit. URLs: <https://github.com/QwenLM/Qwen3-ASR/blob/7c6daf77a2421100f5fb066495372c00129d39ff/pyproject.toml> ; <https://api.github.com/repos/QwenLM/Qwen3-ASR/commits/main> Inspected via public HTTPS: commit 7c6daf77a2421100f5fb066495372c00129d39ff, dated 2026-06-26; package version 0.0.6; exact Transformers 4.57.6, Accelerate 1.12.0 dependencies. Unpinned transitive dependencies remain the reason requirements-qwen.txt is labeled a candidate API target, not a solved lockfile.
- **S22.** PyTorch 2.10 automatic mixed precision. URL: <https://docs.pytorch.org/docs/2.10/amp.html> Inspected: autocast context and dtype behavior. The original loop explicitly uses FP32 model/optimizer storage and BF16 CUDA autocast. No GradScaler is used for that BF16 path; no claim of FP16 compatibility is made.
- **S23.** Transformers 5.4.0 Cohere documentation. URL: https://huggingface.co/docs/transformers/v5.4.0/model_doc/cohere_asr Inspected: native model/processor API and generation usage. Cohere local inference script syntax-checked only. Weight access is left to an authorized user-controlled setup.
- **S24.** Official PyTorch wheel installation matrix. URL: <https://pytorch.org/get-started/previous-versions/> Inspected: PyTorch 2.10.0 Linux/Windows CUDA 12.6/12.8/13.0 and CPU wheel indexes. The chapter selects only torch for this project; torchvision/torchaudio are not required by its code. No package installation executed.

Large Architecture

- **A01.** DeepSeek-AI. Official DeepSeek-V3 inference implementation, especially MLA, Gate, Expert and MoE. Inspected source; file revision b15f0dbbbe6a4bc403306175698439ef380f5fb5, dated 27 August 2025. The optimized absorb path stores kv_cache and pe_cache; the code also exposes a naive path. [Pinned implementation](#)

- **A02.** XiaomiMiMo. MiMo-V2.6-Flash-RL model implementation. Inspected at checkpoint revision 5711b268169967567844e1e560e8a3966da959b1. Relevant classes: MiMoV2MoEGate, MiMoV2MoE, MiMoV2Attention, MiMoV2DecoderLayer, vision patch/merger and audio classes. The router's noaux_tc branch explicitly raises when self.training is true. This establishes a limit of this implementation, not a claim that no other training system exists. [Pinned source](#)
- **A03.** Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, 2023. Primary definition and motivation for grouped-query sharing. The chapter does not reproduce the paper's speed/quality measurements as local results. [Paper](#)
- **A04.** DeepSeek-AI. DeepSeek-V3 Technical Report, first submitted 27 December 2024. Primary evidence for 671B total/37B active, MLA, MoE, 14.8T tokens and MTP. Historical reference, not a latest-release assertion. [Paper](#)
- **A05.** Dao et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, 2022. Supports distinction between exact IO-aware execution and changing the attention graph. [Paper](#)
- **A06.** Dao and Gu. Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality, 2024. Primary Mamba-2/SSD paper. [Paper](#)
- **A07.** State Spaces. Official Mamba-2 module. Inspected constructor, sequence path, step/cache handling and convolution/state structure. File revision 6b72c122713bb769cc82c6b8e6d019c53d27d6a1, dated 7 October 2024. No kernel was executed. [Pinned module](#)
- **A08.** Kimi Team. Kimi Linear: An Expressive, Efficient Attention Architecture, 2025. Primary KDA reference; a gated recurrent-memory explanation, not a promise to reproduce the reported throughput. [Paper](#)
- **A09.** Xie et al. mHC: Manifold-Constrained Hyper-Connections, version 2, 5 January 2026. Supports constrained mixing and Sinkhorn normalization. The two-scalar worked example in the book is original, not an experimental result from the paper. [Paper](#)
- **A10.** XiaomiMiMo. Official MiMo-V2.6-Flash-RL card and file listing. The card reports 309B total/15B active, while the dynamic Hub tensor summary reports approximately 311B. Card license label is MIT. The listing contains configuration, implementation, technical-report PDF, modality components and weights. Existence and metadata were verified; weight contents were not downloaded or independently counted. [Card](#) and [pinned files](#)
- **A11.** LLM-Core Xiaomi. MiMo-V2.6: Scaling Reinforcement Learning Towards Self-Improvement, technical-report PDF supplied by the official checkpoint repository. Downloaded as a document and text-extracted; architecture sections 2.1-2.4, pretraining/mid-training section 3, router-freezing discussion and post-training overview inspected. Report rounds Flash to 310B and Pro to 1.02T. It reports Flash 48T pretraining tokens (26T text stage +22T multimodal stage) and Pro 30T (27T+3T). The chapter's discussion of the report stays at the architecture/data-stage level. [Pinned technical report](#). Retrieved PDF SHA256: fb81e6e083801b3358f084ed6be953dc23b0d2e434690f4541d5eae03e01e7af
- **A12.** XiaomiMiMo. Official MiMo-V2.6-Pro-RL card and configuration. Checkpoint revision 73875d00b30a89ef8cc353a0b60b0e9f9561952d. Publisher architecture and rounded counts; code/config observations confirm layer pattern and head/expert geometry. [Card](#), [pinned configuration](#)
- **A13.** Z.ai. Official GLM-5.3-Flash card. Publisher evidence for newly trained multimodal base, 320B total/18B active, sparse/linear hybrid, mHC and 30T corpus. Its MIT card label and linked paper are recorded without assuming publication of all training data. The official family repository separately lists FP8 and BF16 variants. [Card](#), [family repository](#)
- **A14.** XiaomiMiMo. MiMo-V2.6-Flash-RL configuration, revision 5711b268169967567844e1e560e8a3966da959b1. Directly counted 39 local and 9 global layers in hybrid_layer_pattern; inspected dimensions, expert settings, modality configs, max positions and quantization fields. quant_method is fp8 and store_dtype is mxfp4; ignored tensors and mixed-precision components preclude an all-one-dtype byte claim. [Pinned configuration](#)
- **A15.** XiaomiMiMo. Separate DFlash draft configuration and implementation in the Flash-RL release. Five layers, window 1,024 and block size eight are in the separate config; it is not identified solely by the main model's MTP field. [Pinned draft configuration](#), [draft source](#)

- **A16.** XiaomiMiMo. MiMo-V2.6-Flash-MOPD card. Verified distinct post-training release and the card's teacher/prefix and repetition-mitigation description. Checkpoint revision 2479e2d0029eca9a34cc7e7f55a121925f81908e; retrieved metadata created 27 September 2026. Its Flash config was inspected alongside RL config. The card also links Pro-MOPD. [Card](#), [pinned config](#)
- **A17.** GLM-5 Team. GLM-5: from Vibe Coding to Agentic Engineering, arXiv 2602.15763. Verified as the report linked by the current official Flash card. Used only as a family reference, not as sole proof of the later Flash architecture. [Paper](#)
- **A18.** Z.ai. GLM-5.3-Flash configuration, revision eb9eb208eb0d988989d07a6a12d0fdeb5f52574a. Direct observations: 34 linear/11 sparse-attention layers, 3 dense/42 MoE FFNs, 288 routed experts/top8+1 shared, four mHC streams, KDA geometry, index settings and modality config. `indexer_types` all read full in this release, even though supporting code allows reuse. [Pinned configuration](#)
- **A19.** Hugging Face Transformers. `modeling_glm5_next.py`. Inspected actual KDA sequence/decode paths, FP32 recurrent-state storage, top-k router, mHC mixing, sparse indexer, compressed KV cache and expansion/reference attention path. File revision 0a896aa41bba78f92338db21cc468fe043888657, dated 2 October 2026 at 03:23:05 UTC, before this research. It is a current source inspection, not a tested installed package. [Pinned implementation](#)
- **A20.** Hugging Face Transformers. GLM-5.3-Flash model documentation. Direct statement that this implementation does not include an MTP layer. The checkpoint's saved `transformers_version` does not guarantee that any arbitrary installed release supports every component. [Documentation source](#)
- **A21.** Qwen. Official Qwen3.5-397B-A17B card. 397B/17B, 60-layer 3-to-1 hybrid layout, MoE geometry and native-versus-extended context. Used as a selected architecture comparison rather than a latest-Qwen claim. [Card](#)
- **A22.** Qwen. Qwen3.5-397B-A17B configuration. Model repository revision returned by metadata: 8472618112abcbd45acbcd58436aff4233c23f7. Config read directly; `layer_types` confirms 45 linear and 15 full-attention layers. [Pinned configuration](#)
- **A23.** NVIDIA. cuSPARSELt data types and supported sparsity formats. Primary source for pattern-specific sparse-kernel contracts. This is not a performance claim for an unspecified consumer RTX. [Documentation](#)
- **A24.** Rajbhandari et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, 2020 revision. Primary reference for reducing redundancy by sharding training states. The book does not extrapolate the paper's measured scaling to the user's hardware. [Paper](#)
- **A25.** PyTorch. FSDP/fully-shard documentation and NVIDIA Megatron Core parallelism documentation. Framework references for the distinction between model replicas and sharded parameters, tensor/pipeline/expert placement. [PyTorch FSDP](#), [Megatron Core](#). Verify runtime-specific meanings and support before implementing distributed work; no cluster recipe is promised in the book.

Training Lessons

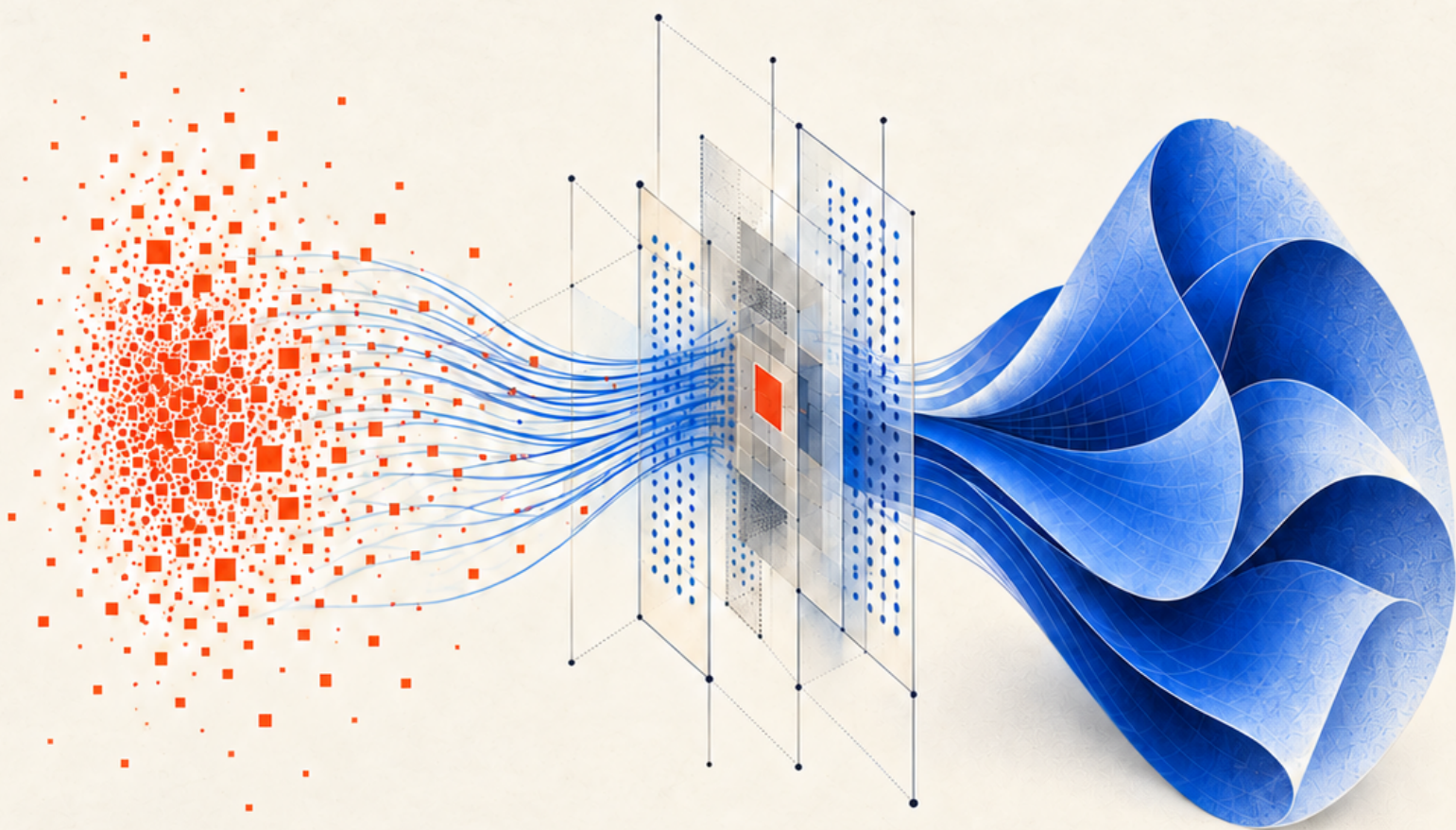
- **T01.** micrograd's scalar engine. Andrej Karpathy, [micrograd/engine.py](#). Inspected the full engine, including graph traversal, backward closures, repeated-use accumulation, and ReLU. Suitable before tensors become a distraction. Educational scalar operations are not a performance model for real training. Revision: 7bc720e951fe422b8f8814aa5aa1b64121d26b4c, default-branch commit 2026-08-03. Repository license: MIT. CPU learning exercise; no consumer-GPU benchmark claimed.
- **T02.** micrograd's correctness tests. [test/test_engine.py](#), same revision. Inspected both tests comparing outputs and derivatives against PyTorch, including expressions that reuse values. Teaches an independent reference check and numeric tolerance. Tests were read, not run here; PyTorch was not installed for this review.
- **T03.** micrograd video and corrected companion. Karpathy, [original YouTube lesson](#), published 2022-08-16; and [second-half lecture notebook](#). Access: author's YouTube description and chapter list read on the original YouTube page; substantive companion notebook code read. The notebook's `exp backward` method contains an explicit correction from assignment to accumulation. Useful author chapter markers: 08:08, 1:22:28, 2:01:12. Course repository: karpathy/nn-zero-to-hero, MIT; revision 73c3fcc741f0ec104ca850b1fb0df90e7e8d4cde, 2024-02-20. The course's correct repository name is nn-zero-to-hero, not neural-networks-zero-to-hero. [Official syllabus](#) states programming and introductory mathematics prerequisites; this book must teach that missing on-ramp itself.

- **T04.** an official visual explanation of gradient descent. Grant Sanderson, [Gradient descent, how neural networks learn](#), lesson dated 2017-10-16; official text adaptation by Josh Pullen. Read the prediction-function/cost-function distinction, local gradient direction, learning-rate explanation, and held-out generalization discussion. Use for intuition after one concrete update. No figures or extended prose from it are reproduced in this book; the official written adaptation was read, not video playback.
- **T05.** makemore's MLP lecture notebook. Karpathy, [makemore_part2_mlp.ipynb](#), course revision above. Read all code cells: whole-item train/dev/test partitioning, context construction, embedding lookup, minibatching, cross-entropy, manual updates, and sampling. [Original lecture](#) is linked by the creator's course README; its companion code, rather than captions, supplied the substantive evidence.
- **T06.** makemore as an executable educational trainer. [makemore README](#). Read data format, scope, model choices, usage, default tiny transformer, and source of the example names. It is a separate executable project from the evolving lecture notebooks. Revision `988aa59e4d8fef526d06f3b453ad116258398d4`, default-branch commit 2022-11-20; MIT. The README is evidence for intended educational/CPU use, not an independently measured runtime.
- **T07.** diagnosing activation and gradient scale. [makemore_part3_bn.ipynb](#), course revision above. Inspected training code, running statistics, gradient retention, histogram generation, saturation fraction, and update-to-weight statistics. Useful after a learner has a training loop to debug. The exercise's BatchNorm and tanh examples should not be generalized into a universal transformer architecture recipe.
- **T08.** corrected small GPT source for the video. [ng-video-lecture/gpt.py](#). Full file inspected. Its attention divides by the square root of the key/head dimension and masks future positions. It demonstrates a clear pedagogical decoder but is not a production serving or complete checkpointing system. Revision `52201428ed7b46804849dea0b3ccf0de9df1a5c3`, 2023-02-07. GitHub returned no license metadata for this repository; no reuse permission is inferred from its public visibility. This book links to it for study and uses independently written code. The final generation call is outside a no-grad/evaluation wrapper, so the book should retain its own explicit inference-mode hygiene rather than copying the script wholesale.
- **T09.** GPT video chapter markers and author corrections. Karpathy, [original GPT lesson](#), published 2023-01-17. Read the original description, exercises, chapter list, and corrections on the original YouTube page. No claim of watching the entire video or reading a caption transcript. Useful markers: [14:27](#) for batches; [47:11](#) for weighted aggregation; [1:02:00](#) for learned self-attention; [1:26:48](#) for residual connections. Author corrections concern future-token masking at 57:00 and head-dimension scaling at 1:20:05. Code and paper are the authority for the corrected operations.
- **T10.** attention notation and its visual interpretation. Sanderson, [Attention in transformers, step-by-step](#). Read its query/key/value explanation, notation warning, masking, dimensions, and illustrative-behavior caveat. Its column-oriented diagrams differ from the original paper's row-oriented convention. This is an excellent opportunity to teach shape annotations; attention visualizations should not be mistaken for a complete causal explanation of the trained model.
- **T11.** connect the implementation to the original Transformer. Vaswani et al., [Attention Is All You Need](#), HTML v7. Read sections 3.2.1-3.2.3 and 5.2 for scaled attention, projections, masking, and the eight-P100 training premise. Suggested just-in-time question: which operation in the reader's code corresponds to each part of the attention equation? This is a focused paper-reading bridge; the architecture chapter also discusses this paper.
- **T12.** inspect a complete small training process. [nanoGPT/train.py](#), revision `3adf61e154c3fe3fca428ad6bc3818b27a3b8291`, 2025-11-12; MIT. Inspected accumulation, mixed-precision scaling/clipping order, evaluation, checkpoint dictionary, and logging. Its readable structure is useful for process literacy. Its checkpoint fields do not establish complete deterministic replay; its final-microbatch log is not an exact accumulated-batch average.
- **T13.** deprecated examples are historical evidence. nanoGPT [README](#), same revision. Read the deprecation notice, toy-run/reproduction distinction, and original hardware premise. Its GPT-2 reproduction used eight A100 40 GB GPUs for about four days. Include only as a brief caution about transferring old hardware/API assumptions, not as a recommended current trainer or model survey. The small [character-model configuration](#) was inspected to confirm it was a distinct experiment.

- **T14.** Raschka's minimal training loop and license boundary. Sebastian Raschka, [gpt_train.py](#), [requirements](#), and [license](#). Read the standalone loop, evaluation mode, no-grad handling, token counter, split, small batch/context settings, and dependencies. Revision faa6205602f487c0382295c698ffa3d635af39, 2026-10-01. The LICENSE.txt is based on Apache 2.0 but modifies the definition of source to exclude books and related images; GitHub metadata returned NOASSERTION. Do not label the entire book/artwork corpus standard Apache-2.0 or reproduce it under the code's terms. Dependency lower bounds are not an exact environment lock.
- **T15.** make instruction-loss labels inspectable. [gpt_instruction_finetuning.py](#), same revision. Inspected dataset rendering and `custom_collate_fn`, especially shifted targets, padding masks, retained terminal token, and truncation. The collator does not mask all instruction positions by default. The transferable lesson is to inspect the actual objective before making an “assistant-only training” claim.
- **T16.** LoRA as a loading and export workflow. Hugging Face, [smol-course LoRA/PEFT lesson](#). Read the adapter-loading, merge, and trainer examples. Revision f445ae5d9dd83f355ce6a2b1c6c71b588c8e1541, 2026-09-17; Apache-2.0. A useful bridge after a learner understands which parameters receive gradients. The original [LoRA paper](#) was checked as a reading pointer; the LLM chapter supplies the detailed paper discussion.
- **T17.** an instructive tutorial-version mismatch. smol-course [SFT lesson](#), [hands-on chapter](#), and [root requirements](#). Inspected actual configuration, formatting, optional adapter, trainer, and upload blocks. The root lock names Torch 2.5.1, Transformers 4.46.3, and TRL 0.12.1; the lesson uses newer model/API examples. The ordinary trainer example's `args=config` differs from the nearby defined `training_config`. Its full-fine-tuning snippets are not verified 24 GB recipes.
- **T18.** compare against the declared trainer version. [TRL 0.12.1 SFT documentation](#). Read its `max_seq_length` configuration and completion-only collator discussion. This provides concrete evidence for checking API generation rather than combining contemporary examples with an old requirements file. Its token-context and EOS/padding cautions also show why a collator needs example-level inspection. Use the book's selected trainer version consistently.
- **T19.** embedding training as an evaluated retrieval process. Sentence Transformers, [NLI example](#) and [MS MARCO training example](#). Full scripts read. Revision 4a3b5cd6ec718e421f57e824a41ed3fd99595df6, 2026-09-21; Apache-2.0. Focus on initial evaluation, negative construction/filtering, duplicate avoidance, cached contrastive minibatches, and post-training evaluation. The MS MARCO script materializes corpus/query/teacher-score dictionaries in host RAM; caching does not eliminate those costs. Both inspected scripts attempt Hub upload despite “optional” comments. Their newer main-branch import layout should not be pasted into the embedding chapter's pinned v5.1.1 environment. [Sentence-BERT](#) is the theory bridge already covered in that chapter's source ledger.
- **T20.** toy denoising versus actual diffusion. Jonathan Whitaker and Hugging Face, [Diffusion Models from Scratch notebook](#), [creator's companion lesson](#), and [embedded original walkthrough](#). Notebook Markdown/code cells read, including the minimal UNet, uniform corruption, image-prediction objective, and comparison with Gaussian DDPM, noise prediction, timestep conditioning, and sampling. Video link verified as the companion page's embedded lesson; no video/transcript viewing claimed and no timestamp invented. Course revision 57b371aea6d477644726653c3318c3f36afe461c, 2026-09-17; Apache-2.0. The latest commit changes workflow maintenance, not necessarily the 2022-23 teaching content. Unpinned notebook installation cells are not a reproducible lock. The DDPM paper is already part of the diffusion bibliography.
- **T21.** follow audio representation through a training loop. [Diffusion for Audio notebook](#), same course revision. Read code/Markdown for resampling, random slicing, spectrogram conversion, batch construction, denoising objective, reconstruction, and upload. Its `loss.backward(loss)` call is a real source-code pitfall; use ordinary scalar backward unless an upstream-gradient weighting is intentional. Treat its pretrained model and music data as separately licensed inputs, not covered automatically by the course license. The generated model-card template's `license: mit` is not evidence that all resulting weights can legally receive that label.
- **T22.** check the mathematical meaning against framework code. PyTorch 2.8, [Tensor.backward](#), confirms the optional argument is an upstream gradient and gradients accumulate in leaves. Diffusers v0.40.0, [DDPM scheduler source](#), inspected `add_noise` and prediction-type branches. It uses square-root signal/noise coefficients and distinguishes epsilon, sample, and velocity prediction. These correct possible confusion between variance and standard-deviation notation in informal tutorials. The [Audio Diffusion API](#) confirms that this pipeline uses Mel-spectrogram representation and exposes reconstruction settings. It is a main-branch reference, not the book's environment lock.

- **T23.** `llm.c` teaches verification before low-level optimization. Karpathy and contributors, [llm.c README](#) and [LayerNorm Python reference](#). Inspected CPU/single-GPU distinctions, reference testing, and the full LayerNorm forward/backward study. Revision `f1e2ace651495b74ae22d45d1723443fd00ecd3a`, 2025-05-10; MIT. Useful after tensor training is understood. The CPU starter fine-tunes an existing GPT-2 checkpoint for 40 short-context steps; it is not a scratch-pretraining achievement.
- **T24.** `nanochat` gives an end-to-end process, with larger hardware defaults. [nanochat README](#) and [CPU/MPS script](#). README and complete small-run script inspected. Revision `92d63d4e8bb4df75c3b71618f31ddde2378b2bcd`, 2026-07-03; MIT according to README. End-to-end tokenizer/pretrain/SFT/evaluation/inference structure is a useful advanced map. The speedrun targets eight H100 GPUs; the README requires adjustment below 80 GB. Author-reported price and runtime are not current retail quotes or RTX measurements.
- **T25.** current checkpoint state and dependency separation. `nanochat` [checkpoint manager](#), [base training script](#), and [pyproject.toml](#), revision above. Read checkpoint serialization/loading, resume and data-loader-state plumbing, loop metadata, tokenizer compatibility check, and dependency declarations. Model and optimizer artifacts are separated; optimizer state is rank-specific for distributed runs. The inspected project pins Torch 2.9.1 and selects CPU or CUDA 12.8 wheels through mutually exclusive extras. This is its own environment, not a reason to mix its dependencies into the book's other projects. No deterministic cross-device resume or RTX fit claim is inferred.

Training Your Own Models



Vibe Authored by Dr.Puma